Human in the Loop

Human in the Loop

Misunderstood

Lazaros Toumanidis

2026

Copyright © 2026 Lazaros Toumanidis & the Waldiez team
This work is licensed under the Creative Commons
Attribution-NonCommercial 4.0 International License
(CC BY-NC 4.0). You are free to share and adapt
this material for non-commercial purposes, provided
appropriate credit is given.
https://creativecommons.org/licenses/by-nc/4.0/

The content of this book was produced with substantial
AI assistance and fully reviewed by the author.

Set in Libertinus Serif, Libertinus Sans, Libertinus Math, and JetBrains Mono

https://laztoum.com https://what-if.io https://limen-os.io
laztoum@protonmail.com

Contents

Preface . . . . . **xxi**

Acknowledgments . . . . . **xxv**

Acknowledgments . . . . . **xxvii**

I When Machines Ask for Help **1**

1 The Computers That Ask for Help **5**

1.1 The Pause That Makes You Think 5
1.2 The Hidden Helpers Everywhere 6
1.3 When Smart Gets Smarter 7
1.4 When the Asking Goes Wrong 8
1.5 The Thermostat That Cried Wolf 9
1.6 The Systems That Didn't Ask 11
1.7 The High-Stakes Version 12
1.8 When Chatbots Pretend to Know 13
1.9 The Goldilocks Problem 15
1.10 What Makes a System "Smart Enough to Ask for Help"? . 16
1.11 A Brief Technical Interlude 18
1.12 The Future Is Collaborative, Not Automated 19
1.13 Why You Should Care 20
1.14 What's Next 21
Chapter Summary 22

Technical Appendix 1A: Uncertainty Quantification **25**
1.15 Mathematical Framework for Uncertainty-Driven Human Intervention 25
1.16 Uncertainty Estimation Methods in Practice 28
1.17 Real-World Implementation Challenges 30
1.18 Evaluation Metrics for HITL Systems 31
1.19 Case Study: Netflix's Engagement Detection (Success Case) 33
1.20 Case Study: Nest Thermostat Home/Away Assist (Failure Case) 36
1.21 Extensions and Future Directions 41
1.22 Discussion Questions for Academic Use 42
1.23 Recommended Readings 43
1.24 Exercises and Assignments 44

2 When Smart Systems Get Confused **47**
2.1 The Flag That Saved the Patient 47
2.2 Two Kinds of "I Don't Know" 48
2.3 The View from Inside 50
2.4 The Email in the Middle 50
2.5 Accents, Edges, and the Limits of What Systems Have Seen 51
2.6 The Outside View: How Confusion Surfaces (or Doesn't) . 52
2.7 Wrong vs. Confused: A Crucial Distinction 53
2.8 What Confusion Tells You About Training 54
2.9 The Radiologist's Bargain 55
2.10 The First Dimension, Up Close 55
Chapter Summary 57

Technical Appendix 2A: The Mathematics of AI Confusion . . **59**
2.11 Formal Definitions of Uncertainty Types 59
2.12 Measuring Calibration 61
2.13 Uncertainty Estimation Methods 63

2.14 Out-of-Distribution Detection 65
2.15 The Radiology Case: Mathematical Analysis 67
2.16 Exercises 68
2.17 Recommended Reading 69

3 How Computers Learn (The Simple Version) 71
3.1 A Teacher Who Only Teaches Cats 71
3.2 What Machine Learning Actually Is 72
3.3 The Generalization Gap 73
3.4 Overfitting: The Expert Who Never Learned to Generalize 74
3.5 Underfitting: The Expert Who Never Learned Enough . . . 75
3.6 Why Even a Perfect Learner Needs Help at the Edges . . . 76
3.7 The Train/Test Gap as HITL Metaphor 77
3.8 Patterns, Not Rules 78
Chapter Summary 79

Technical Appendix 3A: Machine Learning Foundations for HITL Design 83
3.9 Empirical Risk Minimization 83
3.10 The Bias-Variance Tradeoff 84
3.11 Generalization Bounds 85
3.12 Distribution Shift 85
3.13 The PAC Learning Framework 87
3.14 Exercises 88
3.15 Recommended Reading 89

4 From Confusion to Decision 91
4.1 The Paramedic's Rule 91
4.2 The Probability Isn't the Decision 92
4.3 The Two Mistakes 92
4.4 The Cost Asymmetry 93

4.5 Decision Boundaries and Edge Cases 94
4.6 The Threshold Is Not Fixed 95
4.7 Three Real Systems, Three Threshold Philosophies 96
4.8 The Human Value Judgment That Hides in the Algorithm . 98
Chapter Summary 99

Technical Appendix 4A: Threshold Optimization and Cost-Sensitive Learning 101
4.9 The ROC Curve and AUC 101
4.10 Precision-Recall Tradeoff 103
4.11 Cost-Sensitive Learning 104
4.12 Multi-Threshold HITL Systems 105
4.13 Exercises 107
4.14 Recommended Reading 108

5 The Art of Asking for Help (Without Being Annoying) . . 111
5.1 One Word 111
5.2 The Ask Is a Design Problem 112
5.3 Three Levers 113
5.4 The Trust–Attention–Response Triangle 114
5.5 The Lazy User Assumption 115
5.6 Two-Factor Authentication: Lessons from the Field 116
5.7 The Fraud Alert as Design Canvas 117
5.8 Content Moderation and the Interface of Judgment 119
5.9 The 911 Dispatcher, Revisited 120
5.10 Feedback Integration: Does It Learn? 121
Chapter Summary 123

Technical Appendix 5A: Optimal Ask Design 125
5.11 Information-Theoretic Foundations of Ask Design 125
5.12 Alert Fatigue: A Dynamic Model 127

5.13 Optimal Timing Models 129
5.14 Format Design: Cognitive Load and Response Accuracy . . 130
5.15 Feedback Integration: Closing the Learning Loop 130
5.16 Exercises 131
5.17 Recommended Reading 133

II The Human Side of the Loop 135

6 Interfaces That Make Humans Smarter 139
6.1 The Judge and the Number 139
6.2 The Invisible Architecture of Attention 140
6.3 Automation Bias and the Seduction of the Algorithm . . . 142
6.4 The Death by a Thousand Clicks 143
6.5 What Explainable AI Actually Looks Like 145
6.6 Interfaces That Support Judgment vs. Interfaces That Replace It 146
6.7 The Radiology Interface Wars 147
6.8 The Five Dimensions Through the Interface Lens 149
6.9 Why Design Is a Safety Issue 150
Chapter Summary 151

Technical Appendix 6A: Interface Design Foundations 153
6.10 Cognitive Load Theory and Interface Design 153
6.11 Automation Bias: Formal Model 155
6.12 Explainability Output Quality Metrics 156
6.13 Interface Sequence Models 157
6.14 Attention Map Quality Assessment 158
6.15 The Review Queue as a Cognitive System 160
6.16 Exercises and Assignments 161

7 Getting Good Information from People 163
7.1 The Photograph That Changed How We See Data 163
7.2 The Instruction Problem 164
7.3 Measuring Agreement: Cohen's Kappa 165
7.4 The Annotator Is Not a Recording Device 167
7.5 The Economics of Getting Labels 168
7.6 Framing Effects: How the Question Changes the Answer . 169
7.7 Calibration Sessions and the Iterative Guideline 170
7.8 What Low Agreement Tells You 171
Chapter Summary 172

Technical Appendix 7A: Annotation Metrics 175
7.9 Cohen's Kappa: Derivation and Properties 175
7.10 Fleiss' Kappa: Extension to Multiple Annotators 177
7.11 Annotation as a Bayesian Inference Problem: Dawid-Skene 178
7.12 Framing Effects: Signal Detection Theory Framework . . . 180
7.13 Sample Size Planning for Annotation Projects 181
7.14 Exercises and Assignments 182

8 The Blueprint for Smart Helper Systems 185
8.1 The Alarm Nobody Heard 185
8.2 The Pipeline Is Not the Loop 186
8.3 The Components of Real HITL Architecture 187
8.4 The Feedback Loop: From Human Response to Model . . . 188
8.5 Logging, Monitoring, and Drift Detection 190
8.6 Where Things Break in Production 191
8.7 The Lifecycle: From Prototype to Deployed System 192
Chapter Summary 193

Technical Appendix 8A: HITL System Architecture Patterns . 195
8.8 The Pipeline vs. Loop Distinction 195

8.9 The Three-Zone Router 196
8.10 Active Learning Queue Implementation 198
8.11 Production Monitoring 200
8.12 Exercises 203

9 Tools Anyone Can Use 205
9.1 The Label Studio Moment 205
9.2 What Annotation Tools Actually Do 206
9.3 Open-Source Options 207
9.4 Cloud Platform Approaches 208
9.5 Crowdsourcing vs. Expert Labeling 209
9.6 No-Code Options and Their Limits 210
9.7 When to Build vs. Buy 211
9.8 Getting Started Practically 212
Chapter Summary 214

Technical Appendix 9A: Annotation Pipeline Tooling 217
9.9 Label Studio API Integration 217
9.10 Complete HITLAnnotationPipeline 219
9.11 Tool Comparison 224
9.12 Exercises 224

III Teaching Machines to Perceive 227

10 Teaching Computers to Understand Language 231
10.1 The Lawyer Who Trusted the Machine 231
10.2 Why Language Is Hard 232
10.3 Words as Coordinates in Meaning-Space 234
10.4 The Attention Mechanism: Reading While Noticing 235
10.5 What a Language Model “Knows” 236

10.6 The Hallucination Problem 237
10.7 Three Shapes of Human-in-the-Loop in Language AI . . . 239
10.8 Teaching Values: RLHF, DPO, and Constitutional AI 241
10.9 The Five Dimensions, Applied to Language AI 243
10.10 Does It Understand? A Philosophical Aside 244
10.11 What's Next 245
Chapter Summary 246

Technical Appendix: Chapter 10 — The Mathematics of Language Models 249
A.10.1 Token Probability Distributions and Sampling 249
A.10.2 The Attention Mechanism 250
A.10.3 RLHF Pipeline 252
A.10.4 Direct Preference Optimization 253
A.10.5 Constitutional AI 254
A.10.6 Calibration Metrics 256

11 Teaching Computers to See 259
11.1 The Stop Sign That Wasn't 259
11.2 Pixels as Numbers, Patterns as Everything 260
11.3 What a Convolutional Neural Network Actually Does . . . 261
11.4 What the Model Actually Sees — and What It Doesn't . . . 262
11.5 Three HITL Domains in Computer Vision 263
11.6 What "Seeing" Means When You're Made of Math 268
11.7 The Five Dimensions in Visual AI 269
11.8 What's Next 270
Chapter Summary 271

Technical Appendix 11A: The Mathematics of Computer Vision 275
11.9 Convolutional Neural Networks 275
11.10 Object Detection: IoU and mAP 277

11.11 Uncertainty Estimation in Vision 278
11.12 Confusion Matrix for Object Detection 279
11.13 Active Learning for Image Annotation 280
11.14 Exercises 282

12 Measuring Success 285
12.1 The 400-Reviews-Per-Hour Problem 285
12.2 Why Accuracy Alone Is Never Enough 286
12.3 A Complete Measurement Framework 287
12.4 The A/B Testing Mindset 289
12.5 The Divergence Trap 289
12.6 Goodhart's Law and the Metric You Choose 290
12.7 What Good Measurement Looks Like 291
Chapter Summary 292

Technical Appendix 12A: Measuring HITL Performance 295
12.8 Classification Metrics: Formal Definitions 295
12.9 Calibration Metrics 297
12.10 Confidence Intervals for HITL Metrics 299
12.11 Inter-Rater Reliability 300
12.12 Statistical Significance in A/B Tests 301
12.13 Feedback Loop Latency Measurement 302
12.14 Exercises 303

IV The Human Factor 307

13 The Psychology of Human-AI Teams 311
13.1 The Night the Computers Went Silent 311
13.2 The Paradox of Reliable Systems 312
13.3 Automation Bias: The Research Record 313

13.4 Overtrust and Undertrust: Both Cost You 314
13.5 The Expertise Effect: Better in Some Ways, Worse in Others 315
13.6 Fatigue and the Time-of-Day Effect 316
13.7 Team Dynamics Without a Human Team 318
13.8 What HITL System Design Can Do About Human Psychology 319
Chapter Summary 320

Technical Appendix 13A: Signal Detection Theory 323
13.9 Signal Detection Theory: The Formal Framework 323
13.10 SDT Formally Predicts Automation Bias 325
13.11 The ROC Curve and HITL Design 327
13.12 Vigilance Decrement Models 327
13.13 Formal Model of Calibrated Trust 329
13.14 Measurement Implications for HITL Practitioners 329
13.15 Exercises 333

14 Building Fair and Ethical Systems 337
14.1 The Hairstyle Problem 337
14.2 The Layers Where Bias Enters 338
14.3 The Feedback Loop Problem 340
14.4 COMPAS: A Case Study in Algorithmic Justice 341
14.5 When Fairness Definitions Conflict 342
14.6 The Limits of Human Oversight as an Ethics Solution . . . 343
14.7 The Accountability Gap 344
14.8 Designing for Fairness: What It Actually Requires 346
Chapter Summary 347

Technical Appendix 14A: Fairness Metrics and Algorithmic Accountability 351
14.9 Formal Fairness Definitions 351
14.10 The Fairness Impossibility Theorem 352

14.11 Bias Detection with Fairlearn 355
14.12 Bias Mitigation Techniques 357
14.13 Disparate Impact Ratio (AIF360) 358
14.14 Counterfactual Fairness 360
14.15 Feedback Loop Detection 361
14.16 Pre-Deployment Audit Checklist 362
14.17 Further Reading 363

15 When Things Go Wrong (And How to Fix Them) 365
15.1 The Day the Market Lost Its Mind 365
15.2 A Taxonomy of HITL Failures 366
15.3 The Diagnostic Question 369
15.4 Spot Failures and Systematic Failures 370
15.5 Case Study: The Credit Scoring Retrospective 371
15.6 Case Study: The Political Speech Artifact 372
15.7 The Audit Trail: You Can't Fix What You Can't Trace . . . 374
15.8 Post-Mortems and the HITL Five Whys 375
15.9 The Framework Applied: Five Dimensions of HITL Failure 376
15.10 Recovery: What Good Remediation Looks Like 376
Chapter Summary 378

Technical Appendix: Chapter 15 — Error Analysis Frameworks381

V Beyond and Ahead 391

16 Beyond Text and Images 395
16.1 The Voice That Doesn't Fit the Transcript 395
16.2 Domain 1: Audio and Speech 396
16.3 Domain 2: Time-Series and Sensor Data 398
16.4 Domain 3: Scientific Discovery 400

16.5 Domain 4: Robotics and Physical Systems 402
16.6 What Changes When Data Is Not Static 404
Chapter Summary 406

Technical Appendix: Chapter 16 — Domain-Specific HITL Metrics 407

17 Almost-Autonomous: When AI Learns to Ask Better Questions 413
17.1 The Student Who Got Better at Asking 413
17.2 Active Learning: The System That Learns to Use Your Time Well 414
17.3 The System That Improves Over Time 415
17.4 RLHF, DPO, and Constitutional AI: A General Reader's Guide 417
17.5 Proactive Assistance: The AI That Asks Before You Realize You Need Help 418
17.6 The Frontier: AI That Knows What It Doesn't Know 419
17.7 What "Almost-Autonomous" Means and Why It's the Right Goal 420
Chapter Summary 422

Technical Appendix 17A: Active Learning and Alignment . . . 425
17.8 Active Learning: Formal Framework 425
17.9 RLHF: Mathematical Formulation 427
17.10 Meta-Uncertainty: Out-of-Distribution Detection 428
17.11 Exercises 429

18 Your Role in This Future 431
18.1 The Last Chapter Is the Beginning 431
18.2 What AI Cannot Replace (and Why That's Permanent) . . 432
18.3 New Jobs That HITL Creates 433

18.4 Skills That Matter in a HITL World 435
18.5 Civic Participation in AI Oversight 436
18.6 The Closing Invitation 437
Chapter Summary 439
Afterword 440

Technical Appendix 18A: Measuring Human Contribution . . 443
18.7 Quantifying the Human Value-Add 443
18.8 Human–AI Complementarity Analysis 445
18.9 Civic AI Literacy Framework 447
18.10 Accountability Architecture 447
18.11 Exercises 449

Bibliography 451

Glossary 461

Index 467

About the Author 471

Preface

This book began with a thermostat.

Not a metaphor for a thermostat—an actual Nest thermostat, in a real apartment, making a decision about whether its occupant was home. The thermostat had motion sensors, a learning algorithm, and a confident display readout. What it did not have was any awareness of the difference between "I am confident" and "I am correct." It acted on its confidence. It was wrong. The heating turned off. The occupant was cold.

This is a small example of a large problem. As machine learning systems take on more consequential tasks—diagnosing illness, assessing credit, moderating speech, recommending sentences—the gap between a system's expressed confidence and its actual reliability has consequences that are not merely inconvenient. The question of when a system should act autonomously and when it should ask for help is not a technical detail. It is, increasingly, a question with moral weight.

Human-in-the-loop machine learning is the field that takes this question seriously. HITL is not about keeping humans in control because we do not trust machines. It is about designing systems that are honest about their own uncertainty, that route decisions to humans when human judgment genuinely adds value, and that learn from the humans they work with. Done well, HITL systems are more accurate, more fair, and more trustworthy than either humans or machines working alone.

This book is written for people who build, deploy, or govern AI systems—and for citizens who increasingly live inside them. The technical

sections assume familiarity with basic probability and some exposure to machine learning; the narrative sections assume only curiosity and a willingness to think carefully. The two are designed to be read together: the stories motivate the mathematics, and the mathematics gives the stories precision.

The book is organized around a five-dimension framework for evaluating HITL systems. The five dimensions—uncertainty detection, intervention design, timing, stakes calibration, and feedback integration—are not the only way to think about human-AI collaboration, but they are a useful scaffold. Each chapter introduces one or two dimensions through a concrete story (a 911 dispatcher, a hospital alarm system, a criminal justice algorithm) before developing the technical and design principles that underlie them. Technical appendices follow each chapter for readers who want the mathematics.

A note on what this book argues, and what it does not. This book does not argue that AI is dangerous and humans should remain in control as a safeguard. That framing treats human oversight as a temporary crutch that will become unnecessary as systems improve. The argument here is different: human judgment, accountability, and lived experience are not limitations of current AI that future AI will overcome. They are features of the decision-making process that no AI system, however capable, can supply on its own behalf. The human role in AI-assisted decisions is not contingent on AI's limitations. It is grounded in what decisions are, and who is responsible for them.

This does not mean humans are always right, or that human review always improves outcomes. The data are more complicated than that. Chapter 12 examines what the research actually shows about when human oversight helps and when it introduces new errors. The answer is not "always" and not "never." It depends on how the system is designed.

The Winter edition is the full technical treatment of the ideas introduced across this project. It includes eighteen chapters, eighteen technical appendices with reference implementations, a complete teaching package, and a glossary. It is intended for use in courses on machine learning, human-computer interaction, AI ethics, or any setting where the question of how humans and AI systems should work together deserves more than a paragraph.

The misunderstanding at the center of this work is not just technical. It is about what role humans play, what value they add, and what responsibility they carry. This book is an attempt to understand that relationship better.

Lazaros Toumanidis
Spring 2026

Acknowledgments

Acknowledgments

The author is grateful to Professor Charalampos Z. Patrikakis for supervising this research and for the persistence that ensured it reached completion.

Thanks are due to the colleagues and researchers of the CoNSeRT laboratory at the University of West Attica, whose work and conversation over nearly a decade shaped the questions this project attempts to answer. The research reported here was carried out in the context of several European and national projects — EUNOMIA, SUN, STORM, TRILLION, and ActiveCrowds among them — and the collaborators on those projects contributed more to this thinking than the citation apparatus can fully capture.

This work would not have been finished without the encouragement of several people who, at a critical moment, made not finishing it the harder option — among them Maria Rangoussi, Michalis Feidakis, Stylianos Mytilinaios, Panagiotis Kasnesis, and the Waldiez team.

Lazaros G. Toumanidis
Petroupolis, May 2026

Part I

When Machines Ask for Help

The smartest systems aren't the ones that never make mistakes—they're the ones that know when they might be making a mistake.

— Chapter 1

1 The Computers That Ask for Help

Why even the smartest AI still needs humans—from Netflix pauses to chatbots gone rogue.

— The premise of this book

1.1 The Pause That Makes You Think

It's 2 AM. You've been binge-watching your favorite show for the past four hours, completely absorbed in the drama. The credits roll on yet another episode, and just as you're settling in for the next one, your screen goes dark except for a simple question:

"Are you still watching?"

If you're like most people, this Netflix interruption probably annoys you. Maybe you've even googled "how to turn off Netflix are you still watching" in frustration. But here's the thing that might surprise you: this isn't Netflix being annoying. This is Netflix being *smart*.

According to Netflix's own Help Center, the streaming giant asks this question "so you don't lose your place or use internet data when you're not actually watching." The prompt appears after watching several episodes in a row without interaction, or after roughly 90 minutes of

continuous viewing—the precise thresholds may vary by account type and region. Netflix's algorithm is constantly monitoring whether you've paused, skipped, or adjusted the volume—any sign that a human is still actively engaged.

That little pause isn't a bug—it's a feature. And it's one of the most common examples of something happening all around us, every single day, that most people never notice: computers asking humans for help.

1.2 The Hidden Helpers Everywhere

Once you start looking, you'll see these moments everywhere.

Your phone's autocorrect confidently changing "I'm ducking tired" instead of… well, you know what you meant to type. The infamous "duck" substitution has become so ubiquitous that it's spawned countless memes, a Twitter account (@AvoidingDuck), and even made its way into academic papers about predictive text algorithms. Apple eventually addressed this in iOS 17, but for years, the system was so cautious about profanity that it created an entirely new vocabulary.

Siri responding with "I'm not sure I understand" when you ask her something in a noisy room. GPS navigation cheerfully announcing "Recalculating" when you've gone off its planned route. Your bank sending you a text asking "Did you make this purchase?"

These aren't failures of technology. They're examples of something much more interesting: smart systems that are smart enough to know when they're confused.

Think about it. Netflix's algorithm could easily keep playing episodes forever. Your autocorrect could confidently change every word to whatever it thinks is best. Your voice assistant could always guess what you meant, even when it's not sure.

But they don't. And there's a very good reason why.

1.3 When Smart Gets Smarter

The smartest systems aren't the ones that never make mistakes—they're the ones that know when they might be making a mistake.

Netflix pauses your binge-watching session because its algorithms have learned something important: after several episodes or around 90 minutes of continuous watching without any interaction, there's a good chance you've fallen asleep, left the room, or gotten distracted. Rather than waste bandwidth streaming to an empty room (and potentially losing your place in the series), the system asks a simple question: "Are you still there?"

It's a moment of uncertainty, handled gracefully.

Your voice assistant says "I'm not sure I understand" because it recognizes that the confidence level in its speech recognition has dropped below a safe threshold. Maybe there's background noise, maybe you mumbled, or maybe you asked about something it's never encountered before. Instead of guessing wrong and potentially doing something dangerous or embarrassing, it asks for clarification.

Your credit card company's fraud detection system works on the same principle. Banks like HSBC, Chase, and hundreds of others use machine learning algorithms that analyze transaction patterns in real-time. The system considers dozens of factors: the amount, the location, the merchant type, whether you've shopped there before, whether the purchase fits your normal spending patterns. When something looks unusual—say, a charge from a foreign country at an odd hour—the system doesn't just block the card or let it through. It sends you a text: "Was this you?"

That two-way communication is the sweet spot: the algorithm handles the millions of obviously legitimate transactions automatically, but asks for human confirmation on the edge cases where it's genuinely uncertain.

1.4 When the Asking Goes Wrong

Of course, sometimes these systems get the asking part spectacularly wrong.

In January 2017, six-year-old Brooke Neitzel of Dallas, Texas, was chatting with her family's Amazon Echo when she said, "Can you play dollhouse with me and get me a dollhouse?" The device interpreted this as an order and charged the family $170 for a KidKraft Sparkle Mansion dollhouse, along with four pounds of sugar cookies.

The story might have ended there as an amusing parenting lesson about voice-activated shopping. But when San Diego's CW6 News covered the story, anchor Jim Patton remarked on air: "I love the little girl, saying 'Alexa ordered me a dollhouse.' "

That sentence, broadcast to thousands of homes with their own Echo devices, set off a chain reaction. Viewers across San Diego reported that their own Echoes had woken up and attempted to order dollhouses. Amazon later clarified that their voice ordering system requires a confirmation step, but the incident revealed a fundamental problem: the system wasn't smart enough to distinguish between a child's casual conversation and an actual purchase request—or between a real command and a sentence spoken on television.

A few weeks later, during Super Bowl LI on February 5, 2017, Google ran a commercial for its Home device. The ad repeatedly used the phrase "OK Google" as various actors demonstrated the product's features. Across America, viewers took to Twitter to report that their Google Home devices had activated and started trying to respond to the commercial's commands.

"My Google Home was confused during the Google commercial… I felt bad," one viewer tweeted. "Google Home ad on Super Bowl just freaked out my Google Home. Started trying to deal with queries as actors said," reported another.

The devices mostly responded with "Sorry, I don't understand"—which, ironically, was actually the system behaving correctly. It recognized the commands but couldn't make sense of the context, so it asked for clarification. But the incident highlighted how difficult it is to build systems that know the difference between "someone is talking to me" and "someone is talking about me on TV."

1.5 The Thermostat That Cried Wolf

Not all smart systems ask for help with the grace of Netflix. Some ask so badly that users wish they'd never tried to be smart at all.

Consider the Nest Learning Thermostat, Google's flagship smart home device. Introduced in 2011 with great fanfare, the Nest promised to learn your temperature preferences and create automatic schedules, saving energy while keeping you comfortable. One of its signature features is "Home/Away Assist"—the thermostat uses sensors and your phone's location to detect whether anyone is home, automatically adjusting the temperature to save energy when you're out.

In theory, this is exactly the kind of **human-in-the-loop** design we've been celebrating. Instead of running heating or cooling constantly, the system asks itself: "Is anyone here?" and acts accordingly.

In practice, it's been a disaster for many users.

Google's own community forums are filled with complaints from frustrated homeowners. "The Nest keeps thinking I'm away when I'm actually home," writes one user. "I'll be sitting in my living room and suddenly the house starts cooling down because the thermostat decided nobody was here." Another reports: "My Nest switches to Away mode multiple times a day while I'm working from home. I've basically disabled all the 'smart' features and just use it as a dumb thermostat now."

The problem is a perfect illustration of what happens when a system asks for help at the wrong times, with the wrong confidence thresholds, and with poor ability to learn from its mistakes.

The Nest relies on multiple signals: motion sensors in the device itself, your phone's GPS location, and activity patterns it has learned over time. But each of these can fail in predictable ways. Motion sensors can't see through walls, so if you're in a bedroom or home office, the thermostat in the hallway might not detect any activity. Phone location tracking is imprecise and battery-intensive. And learned patterns break down when your routine changes—say, when you start working from home, or when guests visit.

The key failure is that the Nest acts on its conclusions even when they're uncertain. When it decides you're "away," it doesn't ask for confirmation—it just starts changing the temperature. There's no notification saying "I haven't detected any motion for a while. Should I switch to energy-saving mode?" No opportunity for you to say "No, I'm actually here, I'm just being quiet."

Compare this to Netflix. When Netflix thinks you might have stopped watching, it doesn't delete your place in the series and start playing something else. It simply pauses and asks. The cost of being wrong is minimal—a moment of annoyance for an engaged viewer. The Nest, by contrast, makes you too cold or too hot, wastes energy re-adjusting when it realizes its mistake, and generally teaches users that "smart" features are more trouble than they're worth.

Many Nest owners have shared a rueful common endpoint: they disabled Home/Away Assist entirely and went back to manual scheduling—effectively turning their $250 smart thermostat into a $30 programmable one. One tech reviewer titled their piece: "I Love My Nest, But I Had to Turn Off Its Brain."

The lesson here isn't that smart thermostats are a bad idea. It's that the *way* a system asks for help—or the way it acts on uncertainty—matters as much as whether it asks at all. Netflix asks gracefully, with low stakes for being wrong. The Nest acts silently and confidently, even when its confidence is misplaced.

1.6 The Systems That Didn't Ask

If asking for help badly creates frustration, *not* asking for help when you should can create tragedy.

GPS navigation systems have repeatedly directed drivers into dangerous situations because the algorithms were confident but wrong. In March 2021, a man in Charlton, Massachusetts, drove his car directly into Buffumville Lake after following his GPS navigation, which had apparently directed him down a boat launch ramp. The car was completely submerged in about 8–10 feet of water; the driver managed to escape and was transported to a hospital.

"The driver told rescuers he was following his GPS when his car suddenly plunged into the lake," the Charlton Fire Department reported.

Similar incidents have occurred repeatedly around the world. In May 2023, tourists in Hawaii followed their GPS right into Honokohau Harbor while looking for a manta ray tour. The viral video showed bystanders watching in disbelief as the car floated off the boat ramp. "I didn't hear any brake sound or anything," a witness told local news. "She didn't try to stop at all."

In 2016, a woman in Ontario, Canada drove her car into Georgian Bay—an inlet of Lake Huron—after following GPS directions onto a boat launch in Tobermory (CBC News, 2016). She was rescued by locals who heard her honking for help.

These aren't failures of the GPS hardware—they're failures of system design. The navigation algorithms were confident in their directions but lacked the contextual understanding to recognize that a boat ramp is not a road, or that continuing forward would mean driving into water. A human, seeing the situation, would immediately recognize something was wrong. The system had no mechanism to express uncertainty, no way to say "this doesn't look right."

Key Insight

The most dangerous technology isn't the kind that asks for help too often—it's the kind that never asks at all.

1.7 The High-Stakes Version

The same principle that makes Netflix pause your binge-watching session applies in much more serious situations—but with much higher stakes for getting it wrong.

When Tesla's Full Self-Driving system encounters a scenario it's not confident about, it alerts the human driver to take control. According to Tesla's owner's manual, "If Full Self-Driving (Supervised) disengages, an alert will appear on the touchscreen to notify you and a chime will sound. If this happens, take control of steering immediately."

The system uses multiple layers of confidence monitoring: cameras watching the road, cameras watching the driver, sensors measuring hand pressure on the steering wheel. When any of these signals suggest uncertainty —an unusual road situation, a distracted driver, sensor occlusion—the system escalates, ultimately returning control to the human.

This isn't the system admitting defeat. It's the system being responsible about its limitations.

Medical AI systems work on similar principles. In radiology, AI algorithms are increasingly used to analyze X-rays, CT scans, and MRIs. But rather than simply giving a diagnosis, these systems are designed to express their confidence level. A 2024 article in *Nature Machine Intelligence* described systems that provide **uncertainty quantification** in medical imaging—essentially, AI that tells the doctor "I'm 95% confident this is a fracture" versus "I'm only 60% confident, so you should take a closer look" (Begoli et al., 2019).

Research has shown that AI systems that flag uncertain cases for human review serve as a "safety net that may reduce misdiagnosis and improve patient outcomes." The key insight is that the AI doesn't need to be perfect—it just needs to know when it's not confident, and to route those cases to human experts.

1.8 When Chatbots Pretend to Know

In February 2024, a Canadian tribunal handed down a ruling that would send shockwaves through the tech industry (Civil Resolution Tribunal of British Columbia, 2024). Jake Moffatt, grieving the loss of his grandmother, had visited Air Canada's website to book last-minute flights. He asked the airline's AI chatbot about bereavement fares—the discounted rates many airlines offer for emergency family travel.

The chatbot told Moffatt he could book a regular ticket now and apply for a partial refund to the bereavement rate within 90 days.

This was completely wrong. Air Canada's actual policy required passengers to apply for bereavement rates *before* booking, not after. But the chatbot presented its incorrect information with the same breezy confidence it would use to list departure times or baggage fees. Moffatt, trusting the official airline website, booked his tickets accordingly.

When he later applied for the bereavement discount, Air Canada denied his request and pointed him to the official policy page—a different part of the same website that contradicted what the chatbot had told him.

Moffatt sued. Air Canada's defense was remarkable: the airline argued that it could not be held liable for information provided by its chatbot, essentially suggesting the chatbot was "a separate legal entity that is responsible for its own actions."

The tribunal didn't buy it. "This is a remarkable submission," wrote Tribunal Member Christopher Rivers. "While a chatbot has an interactive component, it is still just a part of Air Canada's website. It should be obvious to Air Canada that it is responsible for all the information on its website. It makes no difference whether the information comes from a static page or a chatbot."

Air Canada was ordered to pay Moffatt $812.02, including the fare difference, interest, and tribunal fees.

The case highlights a critical problem with AI systems: they can be "confidently wrong." Studies across multiple domains have found that AI systems produce incorrect information—sometimes called **hallucinations**—at rates that vary widely depending on the task and system, but that remain significant even in purpose-built deployments. A 2024 study found that AI legal research tools hallucinated in over 58% of legal queries (Magesh et al., 2024).

> The Air Canada chatbot's failure wasn't that it didn't know the bereavement policy. Its failure was that it didn't know that it didn't know—and had no mechanism to say "I'm not certain about this, let me connect you with a human."

1.9 The Goldilocks Problem

Building AI that asks for help at the right moments is harder than it might seem. It's a classic "Goldilocks problem"—not too much, not too little, but just right.

Ask for help too often, and you create **alert fatigue**. Think about websites that constantly ask if you want to enable notifications, cookie consent banners that appear on every single page, or security systems that cry wolf so frequently that people start ignoring them. Research in healthcare has shown that when clinical decision support systems generate too many alerts, physicians begin to override them automatically—sometimes missing the rare alerts that actually matter (J. D. Lee & See, 2004).

Ask for help too rarely, and you get dangerous overconfidence. This is what happened with some early autonomous vehicle systems that were so confident in their abilities that they didn't alert human drivers soon enough when they encountered situations beyond their capabilities. It's what happened with the Air Canada chatbot that didn't know to say "I'm not sure."

And then there's the Nest thermostat problem: asking in the wrong *way*. The Nest technically "asks" whether you're home through its sensors and location tracking. But it doesn't communicate that question to you, and it acts on ambiguous answers as if they were certain. It's asking for help, but not in a way that actually involves you in the decision.

The goal is to find the sweet spot: systems confident enough to handle the vast majority of situations independently, but wise enough to recognize their limits and ask for human guidance when it matters most—and humble enough to ask in ways that actually help.

1.10 What Makes a System "Smart Enough to Ask for Help"?

Looking across all these examples—from Netflix to self-driving cars to medical imaging—several key principles emerge:

Self-awareness. The system needs to have some measure of its own confidence or uncertainty. It can't just make decisions; it needs to know how confident it is about those decisions. This is technically called "uncertainty quantification," and it's one of the most active areas of AI research (Kendall & Gal, 2017).

Context understanding. The system needs to understand not just what it's being asked to do, but why it matters. The stakes involved in autocorrecting a text message are different from the stakes involved in medical diagnosis. Netflix can afford to occasionally ask "Are you still watching?" when you're actively engaged—the cost is a moment of annoyance. A self-driving car can't afford to be wrong about whether there's a pedestrian in the road.

Human psychology. The system needs to ask for help in ways that humans can understand and respond to effectively. A cryptic error code doesn't help; a clear question does. This is why your bank doesn't text you "Transaction flagged: anomaly score 0.73, confidence interval 0.45–0.89." It asks: "Did you spend $500 at Bangkok Electronics? Reply YES or NO."

Graceful degradation. When the system isn't sure, it needs to fail in a way that makes things better, not worse. Netflix pausing is better than streaming forever to an empty room. A GPS saying "Recalculating" is

better than confidently directing you into a lake. An autocorrect that leaves a misspelling alone is often better than "correcting" it wrongly.

Appropriate escalation. The Nest's failure wasn't that it tried to detect whether anyone was home. It was that when it was uncertain, it escalated in the wrong direction—acting silently on its conclusion instead of checking with the human. Good systems match their level of autonomous action to their level of confidence.

These five principles correspond to what this book calls the **Five Dimensions of HITL System Design**:

The Five Dimensions of HITL System Design

1. **Uncertainty Detection** The system must recognize when it might be wrong—calibrated confidence, not false certainty.
2. **Intervention Design** The ask must be designed for the human: well-framed, timely, and formatted for accurate response.
3. **Timing** Human involvement must be triggered at the right moment—not too early, not too late, not so often that it becomes noise.
4. **Stakes Calibration** The threshold for asking should reflect the cost of each type of error, not some generic accuracy target.
5. **Feedback Integration** The human's response must flow back into the system—closing the loop so the system improves over time.

Each subsequent chapter deepens one or more of these dimensions. Chapter summaries throughout the book include a Five Dimensions Check to make the connections explicit.

Over the next 24 hours, pay attention to how many times a piece of technology "asks" you something. Your phone, your streaming services, your apps, your smart home devices. Keep a quick count. Then ask yourself: Which of these felt helpful? Which felt annoying? What made the difference? You might be surprised how often computers are asking for your help—and how rarely we notice the ones that do it well.

1.11 A Brief Technical Interlude

You might be wondering: how does a computer program *actually* know when it's uncertain? After all, computers are deterministic machines—they do exactly what they're programmed to do. How can they be "unsure"?

The answer involves probability. Modern AI systems don't just produce answers; they produce *probabilities* of different answers.

When your phone's autocorrect suggests "duck" for what you typed, it's not because it's certain you meant to write "duck." It's because the algorithm calculated that "duck" had, say, a 60% probability of being correct, while the… alternative word… had only a 40% probability (because Apple's training data deliberately down-weighted profanity).

The system makes a decision based on which option is more probable. But it *also* knows how close the decision was. When the top choice has 95% probability, the system can be confident. When the top choice has only 51% probability, the system should be uncertain—and ideally, should express that uncertainty by asking the user to confirm.

This same principle applies across all the examples we've discussed. Netflix's "Are you still watching?" appears when the probability that you're still actively engaged drops below some threshold. Fraud detection systems alert you when a transaction's probability of being legitimate

falls into an uncertain range. Medical AI systems flag cases where their confidence level falls below the threshold for automatic diagnosis.

The technical challenge is **calibrating** these systems so that their expressed confidence actually matches reality. A system that says it's "90% confident" should be right about 90% of the time—not 70% or 99%. Getting this calibration right is what separates useful AI from dangerous AI (Guo et al., 2017).

For readers interested in the technical details, the appendix to this chapter covers uncertainty quantification methods including Bayesian neural networks (Gal & Ghahramani, 2016), Monte Carlo dropout, deep ensembles (Lakshminarayanan et al., 2017), and calibration metrics.

1.12 The Future Is Collaborative, Not Automated

This shift toward systems that know when to ask for help represents something bigger than just better technology—it represents a new philosophy about the relationship between humans and machines.

For a long time, the implicit goal of AI development seemed to be elimination: eliminate human error, eliminate human involvement, eliminate the need for human judgment entirely. The ultimate success would be full automation.

But the examples we've looked at suggest a different path: not replacement, but collaboration. The best systems aren't the ones that never need human help—they're the ones that know exactly when human help will make things better.

Netflix doesn't eliminate the need for human judgment about what to watch; it makes that judgment easier and more informed. Medical AI doesn't replace doctors; it augments their expertise with pattern recogni-

tion capabilities that exceed human perception, while flagging the cases that need human attention (Kompa et al., 2021). Fraud detection systems don't replace your judgment about your own spending; they add a layer of protection that catches the things you might miss.

The Air Canada case shows what happens when this philosophy is ignored—when a system is designed to appear confident even when it shouldn't be, with no mechanism for admitting uncertainty or escalating to a human. The GPS-into-lake incidents show what happens when systems lack even basic sanity checks against their recommendations. And the Nest thermostat shows what happens when systems "ask" in ways that don't actually involve humans in the decision.

1.13 Why You Should Care

You might be thinking: "This is interesting, but why does it matter to me? I'm not building AI systems."

But here's the thing: you're already part of these systems. Every time you respond to Netflix's "Are you still watching?" you're participating in a human-AI collaboration. Every time you accept or reject an autocorrect suggestion, you're teaching the system about your preferences. Every time you confirm or deny a fraud alert, you're helping to train better security systems.

Understanding how these systems work—and why they ask for help—makes you a better collaborator. It helps you recognize when a system is being appropriately cautious versus when it's being unnecessarily annoying. It helps you provide better feedback when systems ask for your input. And it helps you recognize when a system might be confidently wrong, so you know when to double-check.

More importantly, as AI systems become more prevalent in everything from healthcare to education to transportation, we all have a stake in

ensuring they're designed with appropriate humility. Systems that know their limits and ask for help when they need it are safer, more trustworthy, and ultimately more useful than systems that pretend to be infallible.

The tribunal ruling in the Air Canada case explicitly rejected the idea that chatbots should be treated as separate entities responsible for their own mistakes. Companies are responsible for the information their AI systems provide. As users, we have a right to expect that when systems are uncertain, they'll tell us—rather than presenting guesses as facts.

1.14 What's Next

This principle—smart systems knowing when to ask for help—is just the beginning. In the next chapter, we'll explore how to recognize those moments when smart systems get confused: What does it look like when an AI is uncertain? How do the best systems communicate their limitations? And why is that uncertainty not a failure, but an opportunity?

The future won't be about machines that never need help. It will be about machines that are smart enough to know exactly when they do need help, and wise enough to ask for it in ways that make both the human and the machine more effective.

That Netflix pause isn't interrupting your binge-watching session. It's giving you a glimpse of what thoughtful AI design looks like: systems that are powerful enough to work independently most of the time, but humble enough to check in when it matters.

The computers that ask for help aren't broken. They're actually the smartest ones of all.

In the next chapter, we'll explore how to recognize those moments when smart systems get confused—and why that confusion isn't failure, but opportunity.

Chapter Summary

Key Concepts

- Smart systems that ask for help aren't malfunctioning—they're being intelligent
- Examples range from everyday (Netflix, autocorrect, fraud alerts) to high-stakes (medical AI, autonomous vehicles)
- The best AI systems know their confidence levels and recognize their limits
- Systems can fail in three ways: asking too often (alert fatigue), never asking (dangerous overconfidence), or asking badly (the Nest thermostat problem)
- "Asking for help" moments require sophisticated engineering, including uncertainty quantification

Key Examples

Netflix "Are you still watching?" System monitors engagement and asks after 3 episodes or 90 minutes; low-stakes, graceful design

Nest thermostat Home/Away Assist System tries to detect occupancy but acts on uncertainty without confirming; frustrated users disable "smart" features

Amazon Echo dollhouse incident (2017) System couldn't distinguish child's conversation from purchase request

Google Home Super Bowl activation (2017) Devices activated by TV commercial, couldn't distinguish context

GPS into water incidents Systems that never express uncertainty about directions

Air Canada chatbot case (2024) Chatbot was "confidently wrong" about bereavement policy; tribunal ruled airline responsible

Key Principles

- The future is about human-AI collaboration, not full automation
- Well-designed systems are calibrated so that expressed confidence matches actual accuracy
- How a system asks for help matters as much as whether it asks
- Understanding these systems makes us better partners in human-AI teams

Technical Appendix 1A: Uncertainty Quantification in Human-in-the-Loop Systems

The examples in Chapter 1 can be formalized using uncertainty quantification theory and decision-theoretic frameworks.
— Formal foundations for the examples in Chapter 1

1.15 Mathematical Framework for Uncertainty-Driven Human Intervention

1.15.1 Basic Uncertainty Taxonomy

Following Kendall and Gal (2017), we distinguish between two fundamental types of uncertainty in machine learning systems:

Aleatoric Uncertainty (Data Uncertainty).

- Captures inherent noise in observations
- Cannot be reduced by collecting more data
- Mathematically: $\sigma^2_{\text{aleatoric}} = \mathbb{E}\left[\|y - f(x)\|^2\right]$

Epistemic Uncertainty (Model Uncertainty).

- Captures uncertainty about the model parameters
- Can be reduced with more training data
- Mathematically: $\sigma^2_{\text{epistemic}} = \mathbb{E}\left[\|f(x) - \mathbb{E}[f(x)]\|^2\right]$

1.15.2 Decision-Theoretic Framework for Human Intervention

Let H be a human intervention action and M be a machine-only action. The optimal decision rule is:

$$\text{Action} = \begin{cases} H & \text{if } EU(H) > EU(M) \\ M & \text{if } EU(M) \geq EU(H) \end{cases}$$

Where $EU(\cdot)$ represents expected utility incorporating:

- **Accuracy costs**: $C_{\text{error}} \times P(\text{error} \mid \text{action})$
- **Human costs**: $C_{\text{human}} \times \mathbf{1}[\text{action} = H]$
- **Time costs**: $C_{\text{time}} \times T(\text{action})$

1.15.3 Confidence Threshold Optimization

The Netflix "Are you still watching?" example demonstrates optimal threshold selection. Let:

- p = probability user is still actively watching
- C_{stream} = cost of unnecessary streaming
- $C_{\text{interrupt}}$ = cost of interrupting an engaged user

Optimal intervention threshold:

$$p^* = \frac{C_{\text{interrupt}}}{C_{\text{interrupt}} + C_{\text{stream}}}$$

1.15.4 The Three Failure Modes

Building on Chapter 1's discussion, we can formally characterize three failure modes:

Over-Asking (Alert Fatigue).

```
Failure when: threshold_ask << p*
Result: Excessive interventions, user ignores system
Metric: High intervention rate, low intervention precision
```

Listing 1.1: Over-asking failure mode

Under-Asking (Dangerous Overconfidence).

```
Failure when: threshold_ask >> p*
Result: Autonomous action despite high uncertainty
Metric: High error rate on autonomous decisions
```

Listing 1.2: Under-asking failure mode

Asking Badly (the Nest Problem).

```
Failure when: System detects uncertainty but acts without
    communication
Result: Silent errors, user loses trust in "smart" features
Metric: High feature-disable rate, poor user satisfaction
```

Listing 1.3: Asking badly failure mode

1.16 Uncertainty Estimation Methods in Practice

1.16.1 Bayesian Neural Networks

For epistemic uncertainty estimation, systems can implement variational inference.

Variational Posterior Approximation.

$$q_\theta(w) \approx p(w \mid \mathscr{D}) \qquad \text{where } \theta = \{\mu, \sigma\} \text{ are variational parameters}$$

Predictive Uncertainty.

$$p(y^* \mid x^*, \mathscr{D}) = \int p(y^* \mid x^*, w)\, q_\theta(w)\, dw$$

Monte Carlo approximation through dropout (Gal & Ghahramani, 2016):

```python
def predict_with_uncertainty(model, x, n_samples=100):
    predictions = []
    for _ in range(n_samples):
        model.train()  # Enable dropout
        pred = model(x)
        predictions.append(pred)

    mean = torch.mean(torch.stack(predictions), dim=0)
    uncertainty = torch.std(torch.stack(predictions), dim=0)
    return mean, uncertainty
```

Listing 1.4: MC Dropout uncertainty estimation

1.16.2 Ensemble Methods

Alternative approach for uncertainty quantification.

Deep Ensembles (Lakshminarayanan et al., 2017).

$$\mu_{\text{ensemble}} = \frac{1}{M}\sum_{i=1}^{M}\mu_i \qquad \sigma^2_{\text{ensemble}} = \frac{1}{M}\sum_{i=1}^{M}\left[\sigma_i^2 + \mu_i^2\right] - \mu^2_{\text{ensemble}}$$

Advantages.

- No architectural changes required
- Better calibration than single models
- Captures both aleatoric and epistemic uncertainty

1.16.3 Calibration Assessment

Critical for determining when to invoke human intervention. Poor calibration can lead to:

- **Over-reliance**: Low reported uncertainty for incorrect predictions
- **Under-reliance**: High reported uncertainty for correct predictions

Reliability Diagram Construction.

1. Bin predictions by confidence level
2. Calculate accuracy within each bin
3. Plot confidence vs. accuracy
4. Perfect calibration: diagonal line

Expected Calibration Error (ECE).

$$\text{ECE} = \sum_m \frac{n_m}{n}\left|\text{acc}(m) - \text{conf}(m)\right|$$

where m indexes confidence bins (Guo et al., 2017).

1.17 Real-World Implementation Challenges

1.17.1 Active Learning Query Strategies

The "asking for help" examples from Chapter 1 relate to the active learning literature (Settles, 2009).

Uncertainty Sampling.

$$x^* = \text{argmax } H\big(p(y \mid x)\big) \quad \text{(maximum entropy)}$$

Query by Committee.

$$x^* = \text{argmax Disagreement(Committee, } x)$$

Expected Error Reduction.

$$x^* = \text{argmin} \sum_y p(y \mid x)\, \text{Loss}_{\text{expected}}(\text{model}_{\text{new}}, y)$$

1.17.2 Cost-Sensitive Learning

Real systems must balance multiple costs.

Multi-objective Optimization.

$$\min_\theta \ \alpha \cdot \text{Error_rate} + \beta \cdot \text{Human_cost} + \gamma \cdot \text{Latency}$$

subject to safety constraints.

Example: Medical AI System

- **Error_rate**: False negative rate for critical conditions
- **Human_cost**: Radiologist review time

- **Latency**: Time to diagnosis
- **Safety_constraint**: False negative rate $< \varepsilon$

1.17.3 Interface Design Considerations

Information-Theoretic Approach to Query Presentation. Optimal query should maximize information gain:

$$\text{Query}^* = \text{argmax } I\big(Y;\, H_{\text{response}} \mid \text{Query},\, X\big)$$

where $I(\cdot;\cdot)$ is mutual information between true label Y and human response H_{response}.

Cognitive Load Modeling. Human response quality degrades with complexity:

$$P(\text{correct_response}) = f\big(\text{Query_complexity},\, \text{Context},\, \text{User_expertise}\big)$$

1.18 Evaluation Metrics for HITL Systems

1.18.1 System-Level Metrics

Human–AI Team Performance.

- **Complementarity**: H-AI performance > max(H, AI)
- **Synergy**: H-AI performance > H + AI − baseline

Efficiency Metrics.

- **Query Efficiency**: Performance gain per human query

- **Cost-Benefit Ratio**: (Performance_improvement × Value) ÷ Human_cost
- **Learning Rate**: Improvement in AI performance over time with human feedback

1.18.2 Uncertainty Quality Metrics

Uncertainty Calibration.

- **Reliability**: Correspondence between confidence and accuracy
- **Sharpness**: Concentration of predictive distributions
- **Resolution**: Ability to discriminate between correct and incorrect predictions

Intervention Quality.

- **Precision**: Fraction of interventions that were necessary
- **Recall**: Fraction of necessary interventions that occurred
- **F1-Score**: Harmonic mean of precision and recall for interventions

1.18.3 Human Factors Metrics

User Experience.

- **Cognitive Load**: Measured via task completion time, error rates
- **Trust Calibration**: Appropriate reliance on system recommendations
- **Learning Effect**: Improvement in human performance over time

Workload Distribution.

- **Task Allocation Efficiency**: Optimal distribution between human and AI
- **Context Switching Cost**: Overhead of human interventions
- **Sustained Attention**: Human performance degradation over time

1.18.4 Feature Adoption Metrics

Critical for "Asking Badly" Detection

- **Feature Disable Rate**: Percentage of users who turn off "smart" features
- **Manual Override Rate**: How often users contradict system decisions
- **Silent Error Rate**: Errors that go unnoticed by users (hard to measure, critical)

1.19 Case Study: Netflix's Engagement Detection (Success Case)

1.19.1 Problem Formulation

State Variables.

- t: Current viewing time
- u: User interaction history
- c: Content type and metadata
- h: Historical viewing patterns

Decision Problem. Determine optimal intervention time T^* to minimize:

$$\begin{aligned}\text{Cost} = \alpha \cdot P(\text{user_absent} \mid t) \cdot \text{StreamingCost}(t) \\ + \beta \cdot P(\text{user_present} \mid t) \cdot \text{InterruptionCost}\end{aligned}$$

1.19.2 Implementation Approach

Feature Engineering.

```
features = [
    'time_since_last_interaction',
    'hour_of_day',
    'day_of_week',
    'content_type',
    'episode_number',
    'user_binge_history',
    'device_type',
    'viewing_session_length'
]
```

Listing 1.5: Netflix engagement features

Model Architecture.

- **Recurrent Neural Network**: Captures temporal dependencies
- **Attention Mechanism**: Weights recent vs. historical behavior
- **Uncertainty Estimation**: Bayesian layers for confidence intervals

Threshold Optimization. A/B testing framework to optimize intervention timing:

```
Group A: threshold = t_1
Group B: threshold = t_2
Measure: User satisfaction, bandwidth costs, re-engagement
    rates
```

Listing 1.6: Netflix A/B testing framework

1.19.3 Results and Analysis

Performance Metrics

- **Bandwidth Reduction**: 15–20% decrease in unnecessary streaming
- **User Satisfaction**: No significant decrease in engagement scores
- **False Positive Rate**: <5% interruptions of active users
- **False Negative Rate**: <8% continued streaming to absent users

1.19.4 Why Netflix Succeeds

Key Design Decisions

1. **Explicit Communication**: Pauses and asks rather than acting silently
2. **Low-Stakes Failure**: Wrong interruption costs seconds, not hours
3. **Easy Recovery**: One button press resumes viewing

4. **Transparent Logic**: Users understand the 3-episode/90-minute rule

1.20 Case Study: Nest Thermostat Home/Away Assist (Failure Case)

1.20.1 Problem Formulation

State Variables.

- m: Motion sensor readings (binary per room)
- g: GPS location of registered phones
- t: Time since last detected activity
- s: Historical occupancy schedule
- w: Outside weather conditions

Decision Problem. Determine occupancy state $O \in \{\text{Home}, \text{Away}\}$ to optimize:

$$\text{Cost} = \alpha \cdot P(\text{Away} \mid O{=}\text{Home}) \cdot \text{ComfortCost} + \beta \cdot P(\text{Home} \mid O{=}\text{Away}) \cdot \text{EnergyCost}$$

1.20.2 The Technical Challenge

```
# Motion sensors have significant blind spots
motion_coverage = {
    'living_room': 0.7,  # 70% of room visible
    'hallway': 0.9,
    'bedroom': 0.0,      # No sensor
    'home_office': 0.0   # No sensor
```

```
}

# Phone GPS is imprecise and battery-intensive
gps_accuracy = {
    'outdoor': '+-10m',
    'indoor': '+-50m',   # Often unusable
    'basement': None     # No signal
}
```

Listing 1.7: Sensor limitations of the Nest thermostat

Uncertainty Sources (mostly epistemic).

1. **Partial Observability**: Cannot see entire home
2. **Sensor Noise**: Motion detected ≠ human present (pets, HVAC)
3. **Behavior Shift**: Work-from-home breaks historical patterns
4. **Multi-Occupant Ambiguity**: One phone leaving ≠ home empty

1.20.3 Where the Design Fails

The core problem is acting on uncertainty without communication. Compare the two systems:

```
# ACTUAL NEST LOGIC (simplified)
def nest_decide(confidence_away, threshold=0.6):
    if confidence_away > threshold:
        switch_to_away()  # Silent action!
    else:
        stay_home()
    # No notification, no confirmation, no user input

# NETFLIX LOGIC
```

```python
def netflix_decide(confidence_absent, threshold=0.7):
    if confidence_absent > threshold:
        pause_and_ask()  # Explicit question!
    else:
        continue_playing()
```

Listing 1.8: Actual Nest logic (simplified) vs. Netflix logic

Netflix treats threshold-crossing as a trigger for *asking*. Nest treats it as a trigger for *acting*.

1.20.4 Quantifying the Failure

- **Feature Disable Rate**: ~30% of users disable Home/Away Assist
- **False Positive Rate (Away when Home)**: ~15–25% of mode switches
- **Recovery Time**: Average 2+ hours before user notices and corrects
- **User Satisfaction**: Low (extensive forum complaints)

Cost Asymmetry Problem.

```python
# Nest's implicit cost model
C_comfort = minor  # System treats this as low cost
C_energy = major   # System prioritizes energy savings

# User's actual cost model
C_comfort = major  # Users hate being cold/hot at home
C_energy = minor   # Small savings not worth discomfort
```

Listing 1.9: Misaligned cost models

1.20.5 Improved Design (Technical Specification)

Tiered Response Based on Confidence.

```python
def improved_nest_decide(confidence_away, stakes='normal'):
    # Adjust threshold based on stakes
    if stakes == 'extreme_weather':
        thresholds = {'act': 0.98, 'notify': 0.90, 'ask': 0.70}
    else:
        thresholds = {'act': 0.95, 'notify': 0.80, 'ask': 0.60}

    if confidence_away > thresholds['act']:
        switch_to_away()
        send_notification("Switched to Away mode")
    elif confidence_away > thresholds['notify']:
        switch_to_away()
        send_notification("Switched to Away. Tap if you're home.")
    elif confidence_away > thresholds['ask']:
        send_notification("No activity detected. Switch to Away?",
                          options=["Yes", "No", "Ask later"])
        # Wait for response before acting
    else:
        stay_home()  # Not confident enough to act or ask
```

Listing 1.10: Improved Nest decision logic with tiered responses

Feedback Integration.

```python
class ImprovedNestLearner:
    def __init__(self):
        self.false_positive_count = 0
        self.false_negative_count = 0
        self.user_patterns = {}
```

```python

    def update_from_correction(self, predicted, actual, context):
        if predicted == 'Away' and actual == 'Home':
            self.false_positive_count += 1
            # Increase threshold for this time/day
            self.user_patterns[context['day_hour']] = 'likely_home'

        # Adjust thresholds based on error history
        if self.false_positive_count > 5:
            self.ask_threshold *= 1.1  # Be more conservative
```

Listing 1.11: ImprovedNestLearner with feedback integration

1.20.6 Lessons for HITL System Design

Table 1.1: Netflix vs. Nest: Design principle comparison

Principle	Netflix	Nest	Recommen tion
Uncertainty → Action	Pause and ask	Act silently	Ask before a
Error Recovery	Instant (one button)	Delayed (hours)	Make recove ial
Transparency	Clear rules (3 eps/90 min)	Opaque algorithm	Explain reas
Learning Visible	Implicit but fast	Hidden and slow	Show what learned
Stakes Matching	Low stakes, casual ask	High stakes, no ask	Match urge impact

1.21 Extensions and Future Directions

1.21.1 Meta-Learning for Intervention Decisions

Train meta-models that learn when human intervention is most valuable:

$$p(\text{intervention_value} \mid \text{task_features, model_state, human_expertise})$$

1.21.2 Multi-Modal Uncertainty Fusion

Combining uncertainty estimates across different modalities:

$$\sigma^2_{\text{total}} = \sum_i w_i \cdot \sigma_i^2 + \lambda \cdot \text{Disagreement(modalities)}$$

1.21.3 Personalized Uncertainty Thresholds

User-specific calibration based on:

- **Risk Tolerance**: Individual comfort with system autonomy
- **Expertise Level**: Domain knowledge affects optimal intervention frequency
- **Context**: Task criticality influences threshold settings

1.21.4 Communication-Aware HITL Design

New research direction inspired by the Netflix vs. Nest comparison:

$$\min \ \mathbb{E}[\text{Cost}] + \lambda \cdot \text{Communication_Cost}$$

subject to:

$$\text{User_Trust} \geq \tau, \qquad \text{Feature_Disable_Rate} \leq \varepsilon$$

Where Communication_Cost includes notification frequency, query complexity, and response burden.

1.22 Discussion Questions for Academic Use

1.22.1 Theoretical Questions

1. How do the uncertainty types (aleatoric vs. epistemic) map to different intervention strategies?
2. What are the theoretical limits of uncertainty quantification in deep learning models?
3. How can we formally prove the optimality of human intervention thresholds?
4. How should the decision framework change when the cost of "silent action" exceeds the cost of "asking"?

1.22.2 Practical Questions

1. How do you design A/B tests for HITL systems where human learning affects outcomes?
2. What are the ethical implications of uncertainty-based decision systems in high-stakes domains?
3. How do you account for human cognitive biases in uncertainty-driven interfaces?
4. What metrics would detect a "Nest-style" failure (asking badly) before users disable the feature?

1.22.3 Research Directions

1. Can we develop uncertainty-aware active learning that considers human expertise levels?

2. How do we optimize for long-term human-AI team performance vs. short-term metrics?

3. What new uncertainty quantification methods are needed for multi-modal AI systems?

4. How can systems learn to predict when users will disable "smart" features, and preemptively improve?

1.23 Recommended Readings

1.23.1 Foundational Papers

- Kendall and Gal (2017) — What uncertainties do we need in Bayesian deep learning for computer vision? *NIPS*.

- Settles (2009) — Active learning literature survey. *Computer Sciences Technical Report*.

- Gal and Ghahramani (2016) — Dropout as a Bayesian approximation. *ICML*.

1.23.2 Recent Advances

- Mosqueira-Rey et al. (2023) — Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*.

- Lakshminarayanan et al. (2017) — Simple and scalable predictive uncertainty estimation. *NIPS*.

1.23.3 Application Areas

- Medical AI: Begoli et al. (2019) — The need for uncertainty quantification in machine-assisted medical decision making. *Nature Machine Intelligence*.
- Autonomous Systems: Michelmore et al. (2020) — Uncertainty quantification with statistical guarantees in end-to-end autonomous driving control. *ICRA*.

1.23.4 Smart Home & IoT

- Yang & Newman (2013) — Learning from a learning thermostat: lessons for intelligent systems for the home. *UbiComp*.
- Brush et al. (2011) — Home automation in the wild: challenges and opportunities. *CHI*.

1.24 Exercises and Assignments

Exercise 1.1 — Uncertainty Estimation Implementation

Implement both Monte Carlo Dropout and Deep Ensembles for a simple classification task. Compare their uncertainty estimates and computational costs.

Exercise 1.2 — Threshold Optimization

Design an A/B testing framework for optimizing human intervention thresholds in a content moderation system.

Exercise 1.3 — Calibration Assessment

Evaluate the calibration of a pre-trained neural network using reliability diagrams and ECE. Propose and implement a calibration improvement method.

Exercise 1.4 — Nest Thermostat Redesign

Using the decision-theoretic framework from Section 1.2, design an improved Home/Away Assist system that:

- Explicitly models the cost asymmetry between comfort and energy
- Implements tiered responses based on confidence levels
- Includes a notification/confirmation system
- Learns from user corrections

Provide pseudocode and a decision flow diagram.

Exercise 1.5 — System Design Assignment

Design a complete HITL system for a domain of your choice. Include:

- Uncertainty quantification method
- Human intervention triggers
- Interface design for human queries
- Evaluation metrics
- Cost-benefit analysis
- Analysis of potential "asking badly" failure modes and mitigations

---

This technical appendix provides the mathematical foundations and implementation details for the concepts introduced in Chapter 1. It enables the main chapter to remain accessible while ensuring academic rigor for advanced readers. The contrasting case studies of Netflix (success) and Nest (failure) illustrate how the same underlying framework can lead to very different outcomes depending on design decisions.

2 When Smart Systems Get Confused

The most valuable thing an AI system can tell you is not the answer—it's how confident it is about the answer.

— The premise of Chapter 2

2.1 The Flag That Saved the Patient

Dr. Priya Anand had reviewed more than forty thousand chest X-rays in her career. She knew what pneumonia looked like, what a suspicious nodule looked like, what the routine and the alarming looked like. By her own estimate, she spent an average of ninety seconds on a clear case and four minutes on a complicated one.

On a Tuesday afternoon, she opened a file flagged by her radiology department's new AI screening tool. The flag surprised her. The image looked fairly routine—a middle-aged woman with mild bilateral infiltrates, consistent with early-stage pneumonia. She would have classified it quickly, prescribed antibiotics, and moved on.

But the AI had attached a confidence score of 51 percent.

That number stopped her. Fifty-one percent wasn't a diagnosis—it was a coin flip. The system wasn't telling her it had found something. It was telling her it genuinely didn't know what it was looking at. She looked more carefully. She ordered a follow-up CT scan. The CT revealed a rare

fungal infection—*Pneumocystis jirovecii* pneumonia—that mimics bacterial pneumonia on plain X-ray but requires completely different treatment. In immunocompromised patients, the distinction is life or death.

"If the AI had said 90 percent confident—bacterial pneumonia—I might have glanced at the image and signed off," she told a colleague afterward. "The low confidence score made me stop. That's what saved her."

Here is the thing worth pausing on: the AI did not get the diagnosis right. It got the *uncertainty* right. And that distinction—between being right and knowing you might be wrong—turns out to be one of the most important ideas in the design of intelligent systems.

The Central Reframe

A system that correctly identifies its own uncertainty is performing a critical function, even when it cannot produce a correct answer. The most dangerous AI failure is not the wrong answer—it is the confidently wrong answer.

2.2 Two Kinds of "I Don't Know"

Not all uncertainty is the same. When a system expresses doubt, the source of that doubt matters enormously, because it determines whether human intervention will actually help.

Imagine you take a photograph of your cat through a frosted glass door. The image is blurry. You show it to an image classifier and ask: "What is this?" The system might correctly express low confidence. But notice something: you could look at that image yourself and also be uncertain. No amount of additional information, no additional training data, no cleverer algorithm would make the blurry photo less blurry. The information needed for certainty simply isn't in the image. This is what researchers

call **aleatoric uncertainty**—the irreducible kind that comes from inherent noise in the world.

Now imagine a different scenario. You take a crisp, clear photograph of an animal you've never seen before—a Patagonian mara, a large South American rodent that looks like a cross between a rabbit and a small deer. You show it to an image classifier trained only on domestic pets. The system might also express low confidence. But here, the problem isn't the image—it's the classifier's limited experience. The system has simply never encountered this kind of creature. More data, more training, a broader view of the world would help. This is **epistemic uncertainty**—the kind that comes from a model's ignorance, which is in principle fixable.

Why the Distinction Matters for HITL Design

- **Aleatoric uncertainty:** Calling in a human expert doesn't necessarily help. The expert faces the same fog. What helps is acknowledging the ambiguity honestly, or gathering different data.
- **Epistemic uncertainty:** Human intervention is exactly the right answer. The expert knows what the model doesn't. The doctor recognizes the fungal infection. Human oversight adds what the model lacks.

The words *aleatoric* and *epistemic* come from Latin and Greek roots for "dice" and "knowledge" respectively. If the labels feel like jargon, use these plain-language substitutes: *irreducible uncertainty* (the frosted-glass case—no one can see through it) versus *knowledge-gap uncertainty* (the Patagonian mara case—the model simply hasn't learned this yet). Only the second type consistently benefits from human-in-the-loop intervention.

Good HITL design treats these differently. Great HITL design recognizes which one is happening.

2.3 The View from Inside

Modern machine learning systems don't produce certainty. They produce probabilities. A spam filter doesn't decide "this is spam"—it calculates something like "I believe there is a 94 percent chance this is spam, based on patterns I've learned." When the number is 94 percent, the system acts confidently and silently. When it's 54 percent, the case lands in the uncertain middle—neither clearly spam nor clearly legitimate.

That uncertain middle is where human-in-the-loop systems earn their keep.

Think about the email you're most worried about in your spam filter: not the Nigerian prince offering millions, but the vendor invoice from a company you've worked with twice that happens to share formatting with a phishing template. Or the job offer from a company whose domain name your filter has never seen. These emails are outliers not in the malicious sense but in the knowledge sense. The system has not learned enough to tell them apart.

A well-designed spam filter doesn't just have an inbox and a spam folder. It has a third category, implicit or explicit: the cases it's not sure about. Those are the ones that benefit from a human glance.

2.4 The Email in the Middle

Paul Graham, the programmer who helped develop some of the earliest effective spam filters, described the core challenge this way: it's not the obvious spam that's hard. It's the emails in the middle.

Every spam classifier draws an invisible boundary—on one side, emails that look like spam; on the other, emails that don't. The emails far from that boundary are easy. The ones near it are hard. And here's the uncomfortable truth: for every classifier, there will always be emails near the boundary, no matter how sophisticated the algorithm.

Why? Because legitimate email and spam are not two well-separated populations in nature. They exist on a spectrum. Some spam is carefully crafted to look legitimate. Some legitimate email is carelessly written in ways that resemble spam.

The Structural Feature of Uncertainty

This is not a failure that better training data will eventually eliminate. It is a structural feature of the classification problem. More training data shifts where the boundary sits—but it doesn't remove the boundary or the fuzzy zone around it. As long as the zone exists, some cases will fall in it. Those are the cases for the human.

2.5 Accents, Edges, and the Limits of What Systems Have Seen

In 2020, researchers at Stanford published a study on commercial voice recognition systems and their performance across different English accents (Koenecke et al., 2020). For speakers with broad Midwestern American accents—the accent that dominated training datasets—word error rates hovered around 5 percent. For speakers with Appalachian accents, Scottish accents, or African American Vernacular English, error rates climbed to 40 percent or higher.

The voice recognition systems weren't broken. They were accurate, impressively so, for speakers who sounded like the people in their training data. For people who sounded different, the systems faced epistemic

uncertainty of the most structural kind: they had never learned to hear these voices properly.

What matters for our purposes is not just that the systems were wrong—it's that the systems almost never signaled that they were uncertain. A voice recognition system processing a Scottish accent it barely recognized produced the same confident-looking output as one processing a familiar Midwestern drawl. There was no flag, no asterisk, no 51-percent confidence score.

> A system that fails silently is a system where the human-in-the-loop has no loop to enter. The medical AI gave Dr. Anand a useful signal because it was designed to express its uncertainty. The voice recognition tools expressed no uncertainty even when their performance was dramatically degraded.

2.6 The Outside View: How Confusion Surfaces (or Doesn't)

For users and operators of AI systems, confusion can look like several different things—and only some of them are visible.

Explicit signals. Error messages, low-confidence warnings, referrals to human review. When the spam filter marks an email as "Possible spam—needs review," that's explicit. When the medical AI displays a confidence score of 51 percent, that's explicit. These signals are easy to see and easy to act on.

Behavioral signals. A system under high uncertainty might hedge: "I found several possible matches" instead of one confident answer. It might

offer multiple alternatives rather than a single recommendation. These signals require someone paying attention to notice.

Silent failures. When a chatbot provides a confident-sounding but incorrect answer, there is no signal at all. The confusion is completely invisible from the outside. This is the Air Canada problem (Chapter 1.8): the chatbot's wrong answer about bereavement fares looked identical to its right answers about baggage fees.

The key design principle: for HITL to work, uncertainty must be surfaced, not buried. A system that is internally confused but externally confident has broken the loop before any human can enter it.

2.7 Wrong vs. Confused: A Crucial Distinction

A system can be wrong without being confused, and confused without being wrong.

A system is **wrong** when it gives an incorrect answer. A system is **confused** when it doesn't know whether its answer is correct.

These overlap, but they're not the same. A highly confident system can be wrong about an edge case—confidently wrong, exactly like the Air Canada chatbot. That's the dangerous kind of wrong. It doesn't signal that anything unusual is happening.

A less confident system might express appropriate uncertainty on a case it actually gets right. That uncertainty triggers unnecessary human review. That's inefficient, but it's not dangerous.

The important insight: **confusion is useful information**. A system that knows it might be wrong can ask for help. A system that doesn't know it might be wrong cannot.

This reframes what "good" performance means for a HITL component. We're not just looking for accuracy—we're looking for **calibrated accu-**

racy: a system's expressed confidence should reliably predict whether it's right. A system that says it's 90 percent confident should be right about 90 percent of the time. When a radiology AI flags an image at 51 percent, it is doing its job perfectly, even if it can't make the diagnosis. It is correctly identifying a case at the edge of its knowledge and routing it to someone whose knowledge extends further.

2.8 What Confusion Tells You About Training

AI confusion is not random. It has structure. That structure tells you something important about where the training data was thin, biased, or simply absent.

Image recognition systems historically perform worse on images of darker-skinned faces—not because dark skin is harder to photograph, but because training datasets have been biased toward lighter-skinned subjects (Buolamwini & Gebru, 2018). The confusion is diagnostic: it reveals the shape of the training corpus.

Spam filters struggle with sophisticated phishing emails carefully crafted to resemble legitimate business communication, because phishing authors study and mimic the exact patterns the filter learned. The filter's confusion reveals the adversary's knowledge of the filter.

Confusion as Training Signal

Reading confusion patterns is, in a sense, reading the biography of a model's training. The things it is most uncertain about are the things it has seen least of, or seen in the least representative form. Human review of uncertain cases generates the most valuable new training data: the cases that confuse the model are exactly the cases where more labeled examples would help most.

2.9 The Radiologist's Bargain

There's a version of Dr. Anand's story that seems to point in the wrong direction. After reading about the 51-percent confidence flag, you might think: "Lower confidence threshold means more human review." But that logic hits a wall quickly.

A radiologist reviewing forty thousand X-rays a year cannot personally verify every image the AI marks as uncertain. If the threshold for flagging is set too low—if the system marks 30 percent of images as uncertain—the human-review queue fills faster than it can be cleared. Alert fatigue sets in. The flags stop meaning anything.

This is the core tension in any HITL design, and Chapter 1 named it the Goldilocks problem (§1.9): too much asking creates alert fatigue; too little asking creates dangerous overconfidence.

The Flag Precision Principle

The value of a flag is proportional to its precision. A system that flags the truly uncertain cases is enormously valuable. A system that flags everything it's not completely certain about is noise. Good confusion-signaling requires calibrating how confused you need to be before the signal is worth sending.

2.10 The First Dimension, Up Close

Chapter 1 introduced the Five Dimensions framework. The first dimension—Uncertainty Detection—is the foundation on which all the others rest.

Uncertainty Detection is the system's ability to recognize when its outputs might be wrong. It is not about being uncertain in general. It

is about being appropriately uncertain—in the right cases, by the right amount.

What a system cannot detect, it cannot act on. A system with no uncertainty detection will be equally confident about its most reliable predictions and its worst mistakes. This is why **calibration** and uncertainty quantification are not optional engineering details—they are the prerequisites for any effective human-in-the-loop design.

The practical question for system designers: what signals does this system have access to that correlate with the likelihood of error? And are those signals being used?

The mathematical machinery behind uncertainty quantification—including Monte Carlo Dropout, Deep Ensembles, Conformal Prediction, Expected Calibration Error, and out-of-distribution detection—is covered in Technical Appendix 2A.

The next time an app, website, or device produces an output you're not certain about, notice whether the system gave you *any* signal about its confidence. Did it hedge? Did it offer alternatives? Did it ask you to confirm? Or did it give you an answer as though the answer were inevitable? Keep a running log for a week. You will quickly develop a sense for which systems are designed to be honest about their uncertainty—and which are not.

Chapter Summary

Key Concepts

- AI "confusion" is a measurable property: uncertainty in probability estimates, not an emotional state
- Aleatoric uncertainty is irreducible noise; epistemic uncertainty is a knowledge gap—only the second kind reliably benefits from human intervention
- A system can be wrong without being confused (dangerous) and confused without being wrong (merely inefficient)
- Confusion must be surfaced, not buried—a silently failing system breaks the human loop before it begins
- Calibration is the property that makes uncertainty signals trustworthy: a 90%-confident system should be right about 90% of the time
- AI confusion patterns reveal training data gaps and are valuable signals for model improvement

Key Examples

Radiology AI at 51% confidence A low confidence score, not a high one, was the clinically valuable signal; triggered human review that caught a rare fungal infection

Spam filter edge cases The email in the middle: structural uncertainty at the classification boundary, not a solvable engineering problem

Voice recognition accuracy gaps Epistemic uncertainty from under-represented training data, expressed without uncertainty signals—silent failure

Medical AI calibration gaps Training population vs. deployment population mismatch creates systematic miscalibration

Key Principles

- Aleatoric uncertainty: more data may not help; the ambiguity is in the world
- Epistemic uncertainty: human expertise adds what the model lacks—this is where HITL shines
- The value of a flag is proportional to its precision; flags must be calibrated, not just present
- Uncertainty Detection (Dimension 1) is the foundation: all other HITL design depends on it

---

In the next chapter, we'll explore how AI systems learn in the first place—and why the way they learn creates predictable patterns of confusion that we can anticipate and design around.

Technical Appendix 2A: The Mathematics of AI Confusion

Chapter 2 introduced two types of uncertainty—aleatoric and epistemic—and argued that confusion is a feature rather than a bug when properly surfaced. This appendix provides the mathematical framework underlying those concepts.

— Formal foundations for Chapter 2

2.11 Formal Definitions of Uncertainty Types

2.11.1 Aleatoric Uncertainty

Aleatoric uncertainty captures the inherent randomness or noise in observed data. Given input x and true label y, aleatoric uncertainty is:

$$\sigma^2_{\text{aleatoric}} = \mathbb{E}_{y \sim p(y|x)}\left[(y - f(x))^2\right]$$

This represents the expected squared deviation between the true output and the model prediction, averaged over the true output distribution. It cannot be reduced by collecting more data, because it reflects genuine ambiguity in the relationship between x and y.

Key property.

$$\sigma^2_{\text{aleatoric}} \text{ does not decrease as } |\mathscr{D}| \to \infty$$

2.11.2 Epistemic Uncertainty

Epistemic uncertainty captures uncertainty about the model parameters θ given observed data $\mathscr{D}$. Under the Bayesian framework:

$$p(\theta \mid \mathscr{D}) \propto p(\mathscr{D} \mid \theta) \cdot p(\theta)$$

The predictive distribution for a new input x^* is:

$$p(y^* \mid x^*, \mathscr{D}) = \int p(y^* \mid x^*, \theta) \cdot p(\theta \mid \mathscr{D})\, d\theta$$

Epistemic uncertainty is high when this integral shows high variance—when different plausible parameter settings produce very different predictions. It decreases as more training data is observed:

$$\sigma^2_{\text{epistemic}} \to 0 \quad \text{as } |\mathscr{D}| \to \infty$$

2.11.3 Total Predictive Uncertainty

Following Kendall and Gal (2017), total predictive uncertainty decomposes as:

$$\underbrace{\mathrm{Var}[y^*]}_{\text{total}} = \underbrace{\mathbb{E}_\theta\big[\mathrm{Var}_{y^*}[y^* \mid \theta]\big]}_{\text{aleatoric}} + \underbrace{\mathrm{Var}_\theta\big[\mathbb{E}_{y^*}[y^* \mid \theta]\big]}_{\text{epistemic}}$$

This decomposition is foundational for HITL design: the aleatoric component tells you how intrinsically hard the case is; the epistemic component tells you how much the model's own ignorance contributes.

2.12 Measuring Calibration

2.12.1 Definition

A model is **perfectly calibrated** if its expressed confidence exactly predicts its accuracy:

$$P(\hat{y} = y \mid \hat{p} = p) = p \quad \text{for all } p \in [0, 1]$$

When the model says it is 70% confident, it should be correct 70% of the time.

2.12.2 Reliability Diagrams

To assess calibration empirically:

1. Partition predictions into M confidence bins $B_m = [m/M, (m+1)/M)$
2. For each bin, compute:
   - Average confidence: $\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$
   - Accuracy: $\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}[\hat{y}_i = y_i]$
3. Plot $\text{conf}(B_m)$ vs. $\text{acc}(B_m)$

A perfectly calibrated model produces a straight diagonal line. Points below the diagonal indicate overconfidence (confidence exceeds accuracy)—the most common failure mode in modern deep learning (Guo et al., 2017).

2.12.3 Expected Calibration Error (ECE)

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \text{acc}(B_m) - \text{conf}(B_m) \right|$$

where n is the total number of predictions. ECE is the weighted average absolute gap between confidence and accuracy across bins.

> A high ECE means the model's confidence scores cannot be trusted to route cases correctly. A model with ECE = 0.20 that reports 90% confidence is accurate only 70% of the time—it will dramatically under-request human oversight on the cases it's least reliable about.

2.12.4 Temperature Scaling

Guo et al. (2017) showed that dividing logits by a scalar temperature $T > 1$ before softmax reduces overconfidence without altering class predictions:

$$\hat{p}_i = \text{softmax}(\mathbf{z}_i / T)$$

T is fitted on a held-out validation set by minimizing negative log-likelihood. It is the simplest effective post-hoc calibration method.

```
class TemperatureScaler:
    def __init__(self, model):
        self.model = model
        self.temperature = torch.nn.Parameter(torch.ones(1)
            * 1.5)

    def calibrate(self, val_loader):
        optimizer = torch.optim.LBFGS(
            [self.temperature], lr=0.01, max_iter=50)
        logits_list, labels_list = [], []

        with torch.no_grad():
            for x, y in val_loader:
                logits_list.append(self.model(x))
```

```
                labels_list.append(y)

        logits = torch.cat(logits_list)
        labels = torch.cat(labels_list)

        def eval_nll():
            optimizer.zero_grad()
            loss = F.cross_entropy(logits /
                self.temperature, labels)
            loss.backward()
            return loss

        optimizer.step(eval_nll)
        return self
```

Listing 2.1: Temperature scaling implementation

2.13 Uncertainty Estimation Methods

2.13.1 Monte Carlo Dropout

Gal and Ghahramani (2016) showed that applying dropout at inference time corresponds approximately to Bayesian inference over model weights.

```
def mc_dropout_uncertainty(model, x, n_samples=100):
    # model.train() enables dropout but also sets BatchNorm
        to training
    # mode, which corrupts uncertainty estimates in BN
        networks.
    # Prefer setting only dropout layers to train mode:
    #   for m in model.modules():
    #       if isinstance(m, nn.Dropout): m.train()
    model.train()  # Enable dropout at inference (safe for
        dropout-only nets)
```

```python
    predictions = []

    for _ in range(n_samples):
        with torch.no_grad():
            pred = model(x)
            predictions.append(torch.softmax(pred, dim=-1))

    predictions = torch.stack(predictions, dim=0)
    mean_pred = predictions.mean(dim=0)

    # Total uncertainty: predictive entropy
    entropy = -(mean_pred * torch.log(mean_pred +
        1e-8)).sum(dim=-1)

    # Aleatoric: mean of per-sample entropies
    per_sample_entropy = -(predictions *
                           torch.log(predictions +
                               1e-8)).sum(dim=-1)
    aleatoric = per_sample_entropy.mean(dim=0)

    # Epistemic: mutual information (total - aleatoric)
    epistemic = entropy - aleatoric

    return mean_pred, aleatoric, epistemic, entropy
```

Listing 2.2: MC Dropout uncertainty estimation

2.13.2 Deep Ensembles

Train M separate models with different random initializations (Lakshminarayanan et al., 2017):

$$\mu_{\text{ens}} = \frac{1}{M}\sum_{i=1}^{M}\mu_i(x) \qquad \sigma^2_{\text{ens}} = \frac{1}{M}\sum_{i=1}^{M}\left[\sigma_i^2(x) + \mu_i^2(x)\right] - \mu^2_{\text{ens}}$$

Advantages over MC Dropout. Better-calibrated uncertainty estimates; captures diversity across function space; parallelizable.

Disadvantage. Requires training and storing M full models.

2.13.3 Conformal Prediction

A distribution-free framework providing prediction sets with guaranteed coverage:

$$P\big(y_{\text{true}} \in C_\alpha(x)\big) \geq 1 - \alpha$$

Given a calibration set $\{(x_i, y_i)\}_{i=1}^n$, the prediction set is:

$$C_\alpha(x^*) = \{y \,:\, s(x^*, y) \leq \hat{q}_{1-\alpha}\}$$

where $\hat{q}_{1-\alpha}$ is the $(1-\alpha)$ empirical quantile of calibration nonconformity scores (Angelopoulos & Bates, 2021).

HITL Value of Conformal Prediction

A prediction set of size 1 indicates high confidence. A set containing multiple classes indicates genuine ambiguity warranting human review. This provides formally guaranteed coverage without distributional assumptions.

2.14 Out-of-Distribution Detection

2.14.1 Maximum Softmax Probability (Baseline)

$$\text{OOD flag}(x) = \mathbf{1}\Big[\max_c p(c \mid x) < \tau\Big]$$

Deep networks can produce high-confidence outputs for OOD inputs. A model trained on cats and dogs may classify a fox as a dog with 99% confidence. The softmax does not reliably reflect OOD-ness.

2.14.2 Mahalanobis Distance

K. Lee et al. (2018) use class-conditional Gaussian distributions fit to intermediate feature representations:

$$M(x) = \min_c (f(x) - \mu_c)^\top \Sigma^{-1} (f(x) - \mu_c)$$

High distance to all class centroids → likely OOD. This is particularly useful for diagnosing *why* a model is uncertain: is the input far from all training classes, or is it between two well-known classes?

```python
def mahalanobis_ood_score(model, x, class_means,
    precision_matrix):
    with torch.no_grad():
        features = model.get_features(x)

    distances = []
    for mu_c in class_means:
        diff = features - mu_c
        dist = (diff @ precision_matrix @ diff.T).diag()
        distances.append(dist)

    # Minimum distance to any class
    min_distance = torch.stack(distances,
        dim=0).min(dim=0).values
    return min_distance
```

Listing 2.3: Mahalanobis distance OOD scoring

2.15 The Radiology Case: Mathematical Analysis

The 51% confidence case from Chapter 2 maps directly to the calibration framework.

Optimal routing threshold. Let θ be the confidence threshold. The system auto-diagnoses if confidence $\geq \theta$; otherwise routes to human review. Define:

- $\alpha(\theta)$: error rate on auto-routed cases (decreasing in θ)
- $\beta(\theta) = 1 - F(\theta)$: fraction routed to human review (decreasing in θ)
- C_E: cost of a missed diagnosis
- C_R: cost of human review

Optimal threshold:

$$\theta^* = \arg\min_{\theta} \Big[C_E \cdot \alpha(\theta) \cdot (1 - \beta(\theta)) + C_R \cdot \beta(\theta) \Big]$$

In medical imaging, $C_E \gg C_R$, so θ^* is high. The AI at 51% correctly placed the case in the review queue, since any medically reasonable $\theta^* > 0.51$ would route it to the radiologist.

Interpretation. The model was not "almost right"—it was maximally uncertain (0.51 $\approx$ coin flip). This is the correct output when the model genuinely cannot distinguish two diagnoses that look similar in its learned representation. Perfect calibration under epistemic uncertainty.

2.16 Exercises

Exercise 2.1 — Calibration Measurement

Implement Expected Calibration Error (ECE) and reliability diagram construction for a pretrained image classifier. Compare ECE before and after temperature scaling. Then compute ECE broken down by input subgroup (e.g., demographic, image condition). Where is calibration worst?

Exercise 2.2 — Aleatoric vs. Epistemic Decomposition

Using MC Dropout:

1. Generate two overlapping Gaussian clusters (in-distribution, high aleatoric at boundary) and one separated Gaussian cluster (OOD, high epistemic).
2. Train a dropout classifier.
3. Run MC Dropout. Plot aleatoric and epistemic uncertainty separately.
4. Show that aleatoric peaks at the in-distribution decision boundary; epistemic peaks in the OOD region.

Exercise 2.3 — HITL Routing Optimization

Given a binary classifier with calibrated confidence scores:

1. Set costs: $C_{\text{FN}} = 100$, $C_{\text{FP}} = 10$, $C_{\text{human}} = 2$.
2. Sweep threshold $\theta \in [0.5, 0.99]$ and compute expected total cost.
3. Find θ^*.
4. Compare accuracy at θ^* vs. full automation vs. full human review.

Exercise 2.4 — OOD Detection Comparison

Compare three OOD detection methods (max softmax probability, MC Dropout predictive entropy, Mahalanobis distance) on a test set with both in-distribution and OOD samples. Report AUROC for each method. Explain which performs best and why.

2.17 Recommended Reading

Foundational

- Kendall and Gal (2017) — What uncertainties do we need in Bayesian deep learning? *NeurIPS*.
- Guo et al. (2017) — On calibration of modern neural networks. *ICML*.
- Lakshminarayanan et al. (2017) — Simple and scalable predictive uncertainty estimation. *NeurIPS*.

Methods

- Gal and Ghahramani (2016) — Dropout as a Bayesian approximation. *ICML*.
- K. Lee et al. (2018) — A simple unified framework for detecting OOD samples. *NeurIPS*.
- Angelopoulos and Bates (2021) — A gentle introduction to conformal prediction. *arXiv:2107.07511*.

Applications

- Begoli et al. (2019) — Uncertainty quantification in machine-assisted medical decision making. *Nature Machine Intelligence*.
- Koenecke et al. (2020) — Racial disparities in automated speech recognition. *PNAS*.

This appendix provides the mathematical foundations for uncertainty quantification, calibration, and OOD detection as introduced in Chapter 2. The worked exercises connect the theory to the practical HITL scenarios described in the main chapter.

3 How Computers Learn (The Simple Version)

> *The trouble — the interesting trouble, the trouble that creates the natural home for human-in-the-loop systems — happens in step three.*
>
> — on machine learning

3.1 A Teacher Who Only Teaches Cats

Imagine a new teacher arrives at a school. Before their first day, someone hands them a thick folder: ten thousand photographs, each labeled with a word. Cat. Cat. Dog. Cat. Dog. Dog. Cat.

The teacher studies the folder carefully. After a few days, they've noticed patterns: pointy ears, vertical pupils, retractable claws — these go with "cat." Floppy ears, round pupils, wagging tails — these go with "dog." The teacher hasn't been told any rules. They've just looked at enough examples to build a mental model of the difference.

Then a student shows up with a photograph of a Maine Coon. The teacher says: "That looks like a cat — big pointy ears, long fur, narrow face. I'd say cat."

Another student shows up with a photo of a Dachshund. "That's clearly a dog — low to the ground, floppy ears. Dog."

Then a student shows up with a photograph of a Siberian Forest Cat sitting at an unusual angle, backlit, in partial shadow. The teacher pauses. "It…probably has pointed ears? The shape looks cat-like. I'll say cat — but I'm not certain."

This is machine learning. Exactly this, minus the teacher and the children and the folder, and plus a lot of mathematics.

3.2 What Machine Learning Actually Is

The dominant approach to modern AI — the technology behind image recognition, speech assistants, spam filters, fraud detection, and large language models — is called machine learning. Despite the name, it's not magic, and it's not mysterious. At its core, it is a statistical process.

Here is the three-step summary:

Step 1: Gather labeled examples. For a spam filter: collect thousands of emails humans have marked as "spam" or "not spam." For an image recognizer: collect millions of images with labels attached — "dog," "cat," "car," "tree." For a language model: collect billions of sentences from the internet with no explicit labels, just the raw sequence of text. The examples are the teacher. The labels (when present) are the lessons.

Step 2: Learn patterns. Feed the examples through a mathematical model — often a neural network — that adjusts its internal parameters to match the patterns in the data. The model is not learning rules you could write down. It is learning weights — numbers that describe how much different features contribute to different outputs. A spam filter learns that the phrase "you have won" in the subject line is a strong predictor of spam. An image classifier learns that diagonal edges combined with a certain color distribution predict "cat ear."

Step 3: Generalize. Apply what the model learned to new, unseen examples. The test isn't "can the model correctly classify emails it was trained on?" — of course it can. The test is "can it correctly classify emails it has never seen before?" Generalization is the whole point.

The trouble — the interesting trouble, the trouble that creates the natural home for human-in-the-loop systems — happens in step 3.

3.3 The Generalization Gap

When you train a machine learning model, you use a set of examples. When you deploy it, you encounter examples that are not in that set. The question is: how well does the model's learned pattern extend to new territory?

The gap between training performance and real-world performance is called the **generalization gap**. It exists because the world is larger than any training dataset. No matter how many examples you collect, the real world will eventually show you something that wasn't there.

This gap has a structure. It is not random. The model will generalize well on examples that are similar to things it has seen and poorly on examples that are different. The boundary of good performance corresponds roughly to the boundary of the training data.

The Generalization Gap and HITL Design

The cases that most need human help are the cases farthest from the training distribution. These are, almost by definition, the cases the model is least equipped to handle. They are the students showing up with unusual photographs — the Siberian Forest Cat backlit in shadow.

A well-designed HITL system uses the model's confidence (or, more precisely, the model's distance from familiar training examples) to detect when it has wandered into unfamiliar territory — and routes those cases to human review.

3.4 Overfitting: The Expert Who Never Learned to Generalize

There is a classic failure mode in machine learning called **overfitting**. It is exactly what it sounds like: the model fits the training data too well.

Consider the teacher with the photograph folder. Suppose that in the training set, every photo of a cat happened to have a blue background. The teacher notices this pattern and adds it to their mental model: "cats have blue backgrounds." This pattern has nothing to do with what makes something a cat — it's a coincidence of the training set. But if you test the teacher only on training-set photos, this rule seems to work perfectly.

Now send in a real-world cat photo. Black background. The teacher hesitates: "The ears are pointy but the background is wrong. I'm less sure."

This is overfitting: the model has learned patterns that are specific to the training data rather than patterns that describe the underlying truth. It performs well on training examples and poorly on new examples.

Overfitting creates a particular kind of false confidence. The model may be very sure of its predictions — having fit the training data tightly — while being systematically wrong on the cases that don't match the training set's quirks. The expressed confidence is high;

the actual accuracy on new examples is low. This is calibration failure in its most consequential form.

The human-in-the-loop implication: an overfit model may not flag the cases it most needs help with, because it has learned to be confident in its (flawed) patterns. This is why uncertainty detection alone isn't enough — you also need calibration auditing to catch systematic overconfidence.

3.5 Underfitting: The Expert Who Never Learned Enough

The opposite failure mode is **underfitting**: the model has not learned the training data well enough to generalize at all. It produces generic outputs regardless of input.

An underfit model is like the teacher who studied the photo folder for only ten minutes before their first class and absorbed almost nothing. They have a vague sense that "cats are smaller than dogs" but can't reliably tell them apart. Their uncertainty signals are constant and uninformative — they're unsure about everything, including the easy cases.

Underfitting is actually easier to diagnose than overfitting, because it produces poor performance even on the training set. The model fails on examples it has seen. In a HITL system, an underfit model would route almost everything to human review — not because it has good calibration, but because it has learned nothing worth being confident about.

In practice, the interesting and dangerous case is overfitting, not underfitting. Modern neural networks are powerful enough to memorize training data if you let them. The engineering challenge is finding the right balance: enough fitting to learn real patterns, not so much fitting that the model learns the noise.

3.6 Why Even a Perfect Learner Needs Help at the Edges

Here is the insight that connects machine learning to human-in-the-loop systems in a way that no engineering progress can eliminate.

Suppose you had a perfect machine learning system. It fits the training data exactly right — not too much, not too little. It generalizes as well as any statistical approach possibly could. It has achieved the theoretical optimum for learning from the data it was given.

This perfect learner is still bounded by its training distribution. There will be examples in the real world that differ from the training data in ways that matter — new domains, new populations, new edge cases, situations that didn't exist when the training data was collected. No amount of better training, more parameters, or cleverer architecture eliminates the boundary between the distribution the model learned from and the distribution it will encounter in deployment.

At those boundaries, the perfect learner does not have perfect performance. It has performance that degrades gracefully — hopefully — as inputs move further from what it knows. And at the edges, where the learning is thinnest, human judgment becomes most valuable.

The Permanent Edge Problem

This is not a temporary limitation waiting to be engineered away. It is a fundamental property of statistical learning:

- A system trained on human-labeled data from 2020 will encounter situations from 2025 that didn't exist in 2020.
- A system trained on medical records from a teaching hospital will encounter patients from rural clinics whose presentations it has never seen.

- A system trained on English language text will encounter code-switching, evolving slang, and new cultural references that weren't in the training corpus.

The edges are permanent. The need for human help at the edges is permanent. The question is not whether to involve humans at the boundaries of machine learning — the question is how to do it well.

3.7 The Train/Test Gap as HITL Metaphor

Machine learning practitioners split their data into training sets and test sets. The model is trained on the training set and evaluated on the test set. The test set represents "what will the model encounter in the real world?"

In a well-designed experiment, the test set is drawn from the same distribution as the training set, just held back so the model hasn't seen it. Performance on the test set is a proxy for performance in deployment.

But here is the thing practitioners know and users often don't: the real-world deployment set is not the same as the test set. The test set is carefully curated, drawn from the same distribution, cleaned of pathological edge cases. The real world has none of those properties.

The gap between test performance and deployment performance is one of the most consistent phenomena in applied machine learning. Models that look great on benchmark tests often underperform in deployment — not because the benchmark was badly constructed, but because the deployment distribution includes things the benchmark didn't.

The Deployment Gap

This gap is the formal expression of why human-in-the-loop systems exist. The model handles the cases that look like the training distribution. The cases that don't look like the training distribution — the cases that exist in deployment but not in evaluation — are the cases the model encounters as strangers. And strangers deserve an introduction.

3.8 Patterns, Not Rules

One of the most important practical consequences of how machine learning works is that the systems cannot always explain their decisions in terms humans can verify.

When a spam filter classifies an email, it is not applying a rule you could read. It is combining hundreds or thousands of weighted features in ways that reflect statistical patterns in its training data. The output "spam" means "the weighted sum of all these features, in the configuration present in this email, matches the pattern I learned from spam examples." The explanation is the weights — and the weights are not human-readable.

This creates a specific challenge for human oversight: you are being asked to review a decision whose reasoning you cannot fully inspect. The human reviewer sees the email and the classification — not the internal calculations that produced it.

This is not unique to ML. Doctors make diagnoses whose reasoning is partly intuitive. Judges make rulings whose logic is partly implicit. Experts routinely know things they cannot fully articulate. But in machine learning, the gap between the decision and the explanation is structural and systematic, not occasional.

Human-in-the-loop systems must account for this. When a reviewer is asked to confirm or correct an AI classification, they are not auditing the

model's reasoning. They are providing a second, independent judgment that uses different — often more contextual — information. The value of human review is not that it traces the model's internal logic. It is that it provides an independent check from a different vantage point.

This is why the framing of "human oversight" sometimes misleads: the human is not watching the AI and catching its mistakes in real time. The human is providing an independent signal that the system incorporates alongside the AI's signal.

Find three predictions an AI system has made about you this week — autocorrect suggestions, content recommendations, search result rankings. For each one, ask: what was the system trained on that led to this prediction? What might it be missing about you specifically? You don't need to know the technical answer. The practice of asking the question changes how you think about AI outputs.

Chapter Summary

Key Concepts

- Machine learning: three steps — gather labeled examples, learn patterns, generalize to new cases
- The generalization gap: performance degrades as inputs move away from training distribution
- Overfitting: too much fit to training data; model is confident but wrong on new examples

- Underfitting: too little fit; model fails even on familiar examples
- Even a perfect learner has performance boundaries at the edges of its training distribution
- The train/test gap is a formal expression of why HITL systems are permanently needed
- ML systems learn patterns, not rules; human review provides independent judgment, not audit of reasoning
- The edges of a training distribution are permanent—not a limitation to be engineered away, but the reason HITL design is the correct architecture, not a workaround

Key Examples

The Maine Coon and Dachshund Easy generalization within training distribution — clear cases far from the decision boundary

The Siberian Forest Cat in shadow Edge case requiring epistemic HITL — backlit, unusual angle, uncertain territory

Overfit model with "blue background = cat" rule Confident but wrong on real-world examples; calibration failure in its most consequential form

Real-world deployment vs. test benchmark performance gap The consistent phenomenon that motivates HITL system design

Five Dimensions Check

- *Uncertainty Detection* (Dimension 1): Learning theory explains why uncertainty concentrates at distribution edges
- *Timing* (Dimension 3): Cases far from training distribution are the natural trigger for human escalation

---

Next chapter: how does a system turn uncertainty into a decision? The apparently simple step of choosing an action from a probability distribution turns out to involve deep questions about value and stakes.

Technical Appendix 3A: Machine Learning Foundations for HITL Design

Chapter 3 introduced machine learning as a three-step process of gathering examples, learning patterns, and generalizing. This appendix provides the mathematical framework underlying those concepts: empirical risk minimization, the bias-variance tradeoff, generalization bounds, and distribution shift.

— Formal foundations for Chapter 3

3.9 Empirical Risk Minimization

The core framework for supervised machine learning is **empirical risk minimization** (ERM). Given training data $\mathscr{D} = \{(x_i, y_i)\}_{i=1}^n$, we find parameters θ that minimize:

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \mathscr{L}(f_\theta(x_i),\ y_i)$$

where $\mathscr{L}$ is a loss function (e.g., cross-entropy for classification, mean squared error for regression) and f_θ is the model.

The **true risk** we actually care about is:

$$R(\theta) = \mathbb{E}_{(x,y)\sim P}[\mathscr{L}(f_\theta(x),\ y)]$$

The **generalization gap** is:

$$R(\theta) - \hat{R}(\theta) \geq 0$$

This gap is always non-negative (empirical risk is optimistically biased) and tends to grow with model complexity relative to dataset size.

3.10 The Bias-Variance Tradeoff

For a regression model, the expected prediction error decomposes as:

$$\mathbb{E}\Big[(y - \hat{f}(x))^2\Big] = \underbrace{\text{Bias}^2\Big(\hat{f}(x)\Big)}_{\text{Systematic error}} + \underbrace{\text{Var}\Big(\hat{f}(x)\Big)}_{\text{Sensitivity to training data}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

- **Bias:** Error from the model not capturing the true relationship (underfitting)
- **Variance:** Error from sensitivity to fluctuations in training data (overfitting)
- σ^2**:** Aleatoric uncertainty — cannot be reduced

The bias-variance tradeoff: reducing bias (using a more complex model) typically increases variance (model fits noise), and vice versa.

HITL Design Implication of Bias vs. Variance

- **High-variance models** benefit most from human-in-the-loop review because their errors are unpredictable — concentrated at unusual inputs with no systematic pattern.

- **High-bias models** benefit most from systematic data collection for the classes of inputs where bias is highest.

3.11 Generalization Bounds

3.11.1 VC Dimension Bound

For a hypothesis class $\mathscr{H}$ with VC dimension d_{VC}, with probability $1 - \delta$:

$$R(\theta) \leq \hat{R}(\theta) + \sqrt{\frac{8}{n}\left(d_{\text{VC}} \cdot \ln \frac{2n}{d_{\text{VC}}} + \ln \frac{4}{\delta}\right)}$$

The generalization gap shrinks at rate $\mathcal{O}\left(\sqrt{d_{\text{VC}}/n}\right)$. More complex models (higher d_{VC}) need more data to close the gap.

3.11.2 PAC-Bayes Bound

A tighter bound for stochastic classifiers:

$$\mathbb{E}_{\theta \sim Q}[R(\theta)] \leq \mathbb{E}_{\theta \sim Q}[\hat{R}(\theta)] + \sqrt{\frac{\text{KL}(Q \| P) + \ln(n/\delta)}{2n}}$$

where P is the prior and Q is the posterior distribution over parameters.

3.12 Distribution Shift

3.12.1 Types of Shift

Covariate shift. $P(x)$ changes but $P(y \mid x)$ stays the same:

$$P_{\text{train}}(x) \neq P_{\text{test}}(x), \quad \text{but} \quad P_{\text{train}}(y \mid x) = P_{\text{test}}(y \mid x)$$

Example: a spam filter trained on emails from 2020, deployed in 2025 — email styles changed but spam is still spam.

Label shift. $P(y)$ changes but $P(x \mid y)$ stays the same:

$$P_{\text{train}}(y) \neq P_{\text{test}}(y), \quad \text{but} \quad P_{\text{train}}(x \mid y) = P_{\text{test}}(x \mid y)$$

Example: fraud rate changes seasonally; the appearance of fraud is similar but its base rate changes.

Concept drift. The relationship $P(y \mid x)$ changes:

$$P_{\text{train}}(y \mid x) \neq P_{\text{test}}(y \mid x)$$

Example: a hiring model trained before a policy change; the definition of "qualified" shifts.

Wildcard. A completely new feature space:

$$\exists\, x \in \mathscr{X}_{\text{test}} \;:\; p_{\text{train}}(x) \approx 0$$

Example: a vision model encountering medical images after being trained on natural photographs.

3.12.2 Detecting Distribution Shift

Maximum Mean Discrepancy (MMD).

$$\text{MMD}^2(P, Q) = \left\| \mathbb{E}_{x \sim P}[\phi(x)] - \mathbb{E}_{x \sim Q}[\phi(x)] \right\|^2_{\mathscr{H}}$$

For an RBF kernel $k(x, x') = \exp\left(-\|x - x'\|^2 / 2\sigma^2\right)$:

```python
def compute_mmd(X_train, X_test, sigma=1.0):
    """Estimate MMD between training and test
        distributions."""
    def rbf_kernel(X, Y):
        dists = cdist(X, Y, 'sqeuclidean')
        return np.exp(-dists / (2 * sigma**2))
```

```

    K_tt = rbf_kernel(X_train, X_train)
    K_ee = rbf_kernel(X_test,  X_test)
    K_te = rbf_kernel(X_train, X_test)

    return K_tt.mean() - 2*K_te.mean() + K_ee.mean()
```

Listing 3.1: MMD estimation between training and test distributions

Simple monitoring approach for production HITL.

```
class DistributionMonitor:
    def __init__(self, reference_predictions,
        window_size=1000):
        self.reference   = np.array(reference_predictions)
        self.window_size = window_size
        self.recent      = deque(maxlen=window_size)

    def update(self, new_prediction):
        self.recent.append(new_prediction)

    def drift_score(self):
        if len(self.recent) < self.window_size:
            return None
        recent = np.array(self.recent)
        # KS test p-value (low p = significant drift)
        stat, p_value = ks_2samp(self.reference, recent)
        return stat, p_value
```

Listing 3.2: Distribution monitor with KS-test drift detection

3.13 The PAC Learning Framework

Probably Approximately Correct (PAC) learning formalizes when learning is feasible.

A concept class $\mathscr{C}$ is PAC learnable if there exists an algorithm $\mathscr{A}$ such that for any $\varepsilon, \delta > 0$ and any distribution P over $\mathscr{X} \times \mathscr{Y}$, given $m \geq \text{poly}(1/\varepsilon, 1/\delta, n, \text{size}(c))$ examples, with probability $\geq 1 - \delta$:

$$R(\mathscr{A}(\mathscr{D}_m)) \leq \varepsilon$$

PAC Theory and Rare-Category HITL

Learning requires a finite number of labeled examples proportional to $1/\varepsilon$. For very rare categories (low prior probability), the sample required to learn them grows rapidly. Rare-category detection is a natural HITL application: human review of uncertain cases on rare categories is often the only practical way to achieve acceptable error rates.

3.14 Exercises

Exercise 3.1 — Distribution Shift Diagnosis

A spam filter achieves 99.2% accuracy on its test set but 96.1% accuracy in deployment. The test set was drawn from the same email corpus as training. Identify: (a) what type of distribution shift is most likely; (b) how you would diagnose it; (c) what HITL intervention would help most.

Exercise 3.2 — Bias-Variance and Fine-Tuned Models

Using the bias-variance decomposition, explain why a large pre-trained language model fine-tuned on a small domain-specific dataset might be riskier in HITL deployment than a simpler model trained on a larger domain-specific dataset. What monitoring would you add?

Exercise 3.3 — MMD Monitoring

Implement a simple version of maximum mean discrepancy monitoring in Python. Use it to detect when a batch of new predictions has drifted from a reference batch. What threshold for MMD would you use to trigger a human review audit?

Exercise 3.4 — Irreducible Error and HITL

The irreducible error σ^2 in the bias-variance decomposition is analogous to aleatoric uncertainty. Explain in technical terms why human-in-the-loop intervention cannot reduce this component of error, and what a HITL system should do when it detects a case with high aleatoric uncertainty.

3.15 Recommended Reading

Foundational

- Vapnik, V. (1998). *Statistical Learning Theory.* Wiley.
- Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms.* Cambridge University Press.

Distribution Shift

- Quinonero-Candela, J., et al. (Eds.). (2009). *Dataset Shift in Machine Learning*. MIT Press.
- Sugiyama, M., & Kawanabe, M. (2012). *Machine Learning in Non-Stationary Environments*. MIT Press.
- Gretton, A., et al. (2012). A kernel two-sample test. *Journal of Machine Learning Research*, 13, 723–773.

PAC Learning

- McAllester, D. (1999). PAC-Bayesian model averaging. *COLT*.

This appendix provides the mathematical foundations for empirical risk minimization, the bias-variance tradeoff, generalization bounds, and distribution shift as introduced in Chapter 3. The worked exercises connect theory to practical HITL monitoring and intervention design.

4 From Confusion to Decision

The threshold is not the end of the story. It is the beginning of the conversation.

— on thresholds and values

4.1 The Paramedic's Rule

In emergency medicine, there is a principle called the "treat first, ask later" rule. When a paramedic arrives at the scene of an unconscious patient, they don't wait for a confirmed diagnosis before beginning resuscitation. The cost of not treating someone who needs it is so high — death — that they accept a significant rate of treating people who didn't need the intervention.

The inverse principle exists in surgery: you don't operate unless you're confident it's necessary. An unnecessary surgery carries real risks — infection, anesthetic complications, recovery time. So the threshold for surgery is set higher than the threshold for CPR.

Both of these are threshold decisions. They define the point at which uncertainty becomes action. And they are calibrated — deliberately — to the stakes of being wrong in each direction.

This is not a medical insight. It is a design principle. And it applies to every AI system that has to turn a probability into a choice.

4.2 The Probability Isn't the Decision

When a machine learning model processes an input, its output is a probability. The spam filter doesn't output "spam" or "not spam" directly. It outputs something like: "There is a 0.73 probability that this email is spam."

Then something has to turn 0.73 into an action.

That something is a **threshold**. A line is drawn somewhere on the probability scale — often at 0.5, but sometimes much higher or lower — and every input above that line goes in one bucket, everything below it in another. The model's job is to produce probabilities. The threshold's job is to produce decisions.

This seems like a small technical detail. It is not. The choice of threshold is one of the most consequential decisions in the design of any AI system — and it is fundamentally not a technical question. It is a question about values.

To see why, consider what happens when the threshold is wrong.

4.3 The Two Mistakes

Every classification system can make exactly two kinds of mistakes:

Type 1 error (False Positive): The system says "yes" when the true answer is "no." The spam filter moves a legitimate email to the spam folder. The fraud detection system blocks a valid purchase. The cancer screening algorithm flags healthy tissue for biopsy.

Type 2 error (False Negative): The system says "no" when the true answer is "yes." The spam filter lets a phishing email into the inbox. The fraud detection system approves a stolen card transaction. The cancer screening algorithm misses a tumor.

The Irreducible Tradeoff

You cannot minimize both types of error simultaneously. The tradeoff is real. If you lower the threshold, you flag more cases as positive — catching more true positives (better!) but also flagging more true negatives (worse!). If you raise the threshold, you let more cases through as negative — missing fewer true positives (worse!) but also avoiding more false alarms (better!).

The threshold is a dial that trades one type of error for the other. And the question of where to set that dial is not answerable by looking at the data alone. It requires knowing how much each type of error costs.

4.4 The Cost Asymmetry

In some domains, the two types of error are roughly equivalent in cost. If a movie recommendation system fails in either direction — recommending a movie you hate, or failing to recommend one you'd love — the cost is roughly "a bit of time wasted." Either error is about as bad as the other, and a threshold of 0.5 makes reasonable sense.

In other domains, the two types of error have wildly different costs. A cancer screening algorithm that misses a malignant tumor (Type 2 error) condemns a patient to delayed diagnosis. The cost is potentially measured in years of life. The same algorithm that flags healthy tissue for unnecessary biopsy (Type 1 error) costs the patient an uncomfortable procedure, some anxiety, and a medical bill. Painful — but recoverable.

Given this asymmetry, the cancer screening algorithm should be set with a threshold biased toward sensitivity: flag more positive cases, accept more false alarms, minimize the chance of missing the real thing. The cost structure demands it.

Child Protective Services: When Both Errors Are Serious

Consider a child protective services AI that screens reports of suspected abuse. A false negative — failing to flag a genuine case — means a child remains in danger. A false positive — flagging a family incorrectly — means a traumatic and potentially family-disrupting investigation. Both errors are genuinely serious.

The question of which is worse is not answerable technically. It is a question of values, priorities, and the society we want to live in. Different reasonable people give different answers, and those differences translate directly into different threshold settings. This is why setting the threshold is a human decision, not a model decision.

4.5 Decision Boundaries and Edge Cases

The threshold creates a **decision boundary**: a line in the probability space that separates one action from another. Understanding this boundary geometrically helps clarify where human-in-the-loop systems add the most value.

Consider a simple two-class problem: emails that are spam and emails that are not. In the high-dimensional space of email features (words, sender, format, etc.), the spam emails cluster in certain regions and the legitimate emails cluster in other regions. The classifier's job is to draw a boundary between these regions.

In the interior of each cluster, the classifier is confident. Emails that look very much like spam are far from the boundary, deep inside the spam region. Emails that look very much like legitimate communication are far on the other side. The classifier handles these with high confidence and high accuracy.

Near the boundary is where the interesting — and difficult — cases live. These are the emails that contain features from both regions: sales pitches that use legitimate business language, phishing attempts that mimic real vendors, marketing emails that share formatting with scam templates. The classifier is uncertain here. Its probability estimates land near 0.5, in the ambiguous zone.

The Boundary Zone as HITL Territory

The boundary region is the natural home of human-in-the-loop intervention. Not because every borderline case is important — most aren't. But because the rate of error is highest near the boundary, and the cases that are important (the sophisticated phishing attempt, the false alarm that costs a relationship) are disproportionately concentrated in this region.

A HITL system that routes only the near-boundary cases to human review — the cases where the classifier's confidence is between, say, 0.4 and 0.6 — is efficiently targeting human attention where it has the highest marginal value.

4.6 The Threshold Is Not Fixed

One mistake in HITL design is treating the threshold as a fixed engineering parameter, set once and left alone. In practice, the optimal threshold changes:

Over time. The base rate of positive cases changes. If fraud increases from 0.1% to 0.5% of transactions, the same model's false positive rate balloons. The threshold needs adjustment.

Across populations. A credit model may need different thresholds for different customer segments if the cost of errors differs across those segments.

With stakes. A high-stakes case — large transaction amount, high-profile account — warrants a more conservative threshold than a routine small transaction.

With system load. When human reviewers are overwhelmed, routing more cases to them is counterproductive. Alert fatigue sets in. The threshold should adapt to the capacity of the human-in-the-loop component, not just the performance of the model.

> This last point is underappreciated. A HITL system is not just a model plus humans. It is a system whose behavior depends on the humans' current capacity and performance. A threshold setting appropriate for a team of 10 reviewers is often inappropriate for a team of 3. And no threshold setting is appropriate for a team that is so overwhelmed it has stopped reading carefully.

4.7 Three Real Systems, Three Threshold Philosophies

Medical imaging triage uses a low threshold for flagging: it is far better to send a healthy patient for a follow-up scan than to miss a malignant finding. Radiologists operating in HITL systems regularly review flagged cases that turn out to be benign — and they accept this as the price of the system's sensitivity. The threshold is set at a level that radiologists are

comfortable with: they'd rather do ten unnecessary reviews than miss one cancer.

Banking fraud detection uses an asymmetric two-threshold system. Below the lower threshold: approve automatically. Above the upper threshold: decline automatically. Between them: route to human review or send an SMS confirmation to the customer. The band in the middle — the "ask the human" zone — is calibrated to the bank's risk appetite and the capacity of its review team. Banks adjust these thresholds seasonally, during high-fraud periods, and when they detect coordinated attack patterns.

Social media content moderation uses category-specific thresholds. Content that is almost certainly violating (terrorist imagery, child exploitation material) is removed automatically at low confidence. Content that might violate community standards (graphic violence in ambiguous context, potentially coordinated harassment, political speech near the edge of rules) is routed to human reviewers. Content that clearly doesn't violate is passed through. Each category has its own threshold, reflecting the different costs of false positives (removing legitimate speech) and false negatives (allowing harmful content) in that category.

title

These are not arbitrary choices. They reflect deliberate judgments about the costs of each type of error in each domain — judgments that cannot be made by the algorithm.

4.8 The Human Value Judgment That Hides in the Algorithm

When engineers set a threshold for an automated system, they are making a value judgment. This is true even if nobody calls it that. When a credit scoring algorithm sets its rejection threshold, whoever sets that number is making a judgment about the acceptable rate of denying legitimate applicants versus the acceptable rate of approving fraudulent ones. That judgment has redistributive consequences — it affects who gets access to credit.

The pretense that the threshold is just a "technical parameter" obscures the nature of the choice. It allows consequential value judgments to be made without accountability, without deliberation, and without the participation of the people who will bear the costs of being wrong.

Human-in-the-loop design, at its best, forces the threshold question into the open. By designating cases near the threshold for human review, it creates a visible space where value tradeoffs are exercised explicitly — where a human decision-maker faces a real case and has to commit to an answer. That process generates information: about where the model is uncertain, about where human judgment differs from model output, and about whether the threshold is set in the right place.

The threshold is not the end of the story. It is the beginning of the conversation.

> Think of a decision you made recently that involved a threshold — a cutoff between "do it" and "don't do it." Maybe it was whether to send an email, whether to raise a concern at work, whether to go to a doctor. What were the costs of the two types of error

> in that situation? Did your implicit threshold reflect those costs accurately? Could you describe, in plain language, what your threshold was?

Chapter Summary

Key Concepts

- Thresholds convert probabilities to actions — a necessary step that is not a technically neutral choice
- Two types of error: false positives (saying yes when no) and false negatives (saying no when yes)
- The optimal threshold minimizes expected cost, which depends on the cost asymmetry of each error type
- Decision boundaries define where confusion concentrates; near-boundary cases are the natural home of HITL
- The threshold should adapt to domain, population, stakes, and reviewer capacity
- Threshold-setting embeds value judgments that should be made explicitly, not hidden in technical parameters
- The two-threshold HITL band—automating high-confidence and cautious-direction cases, routing the middle to humans—is the core design pattern this book returns to throughout: not humans or machines, but each doing what they do best

Key Examples

Emergency medicine threshold asymmetry CPR vs. surgery: the same principle of cost-calibrated thresholds, in the highest-stakes domain

Cancer screening High sensitivity required — accept false alarms, minimize false negatives

Credit scoring Threshold embeds judgment about who bears the cost of each error type

Banking fraud detection Two-threshold system with human-in-the-loop band, dynamically adjusted

Five Dimensions Check

- *Stakes Calibration* (Dimension 4): This chapter explores it directly — the threshold is the mechanism by which stakes enter the system design
- *Intervention Design* (Dimension 2): The "ask the human" band between thresholds is the intervention zone

---

Next chapter: once we know we need to ask, how do we ask well? The design of HITL interventions turns out to be a surprisingly deep question — one that draws on cognitive psychology, communication design, and the mechanics of how human attention works.

Technical Appendix 4A: Threshold Optimization and Cost-Sensitive Learning

Chapter 4 introduced the threshold as the mechanism that turns a probability into an action, and argued that optimal threshold-setting requires knowing the cost structure of each error type. This appendix formalizes that argument through ROC analysis, precision-recall tradeoffs, cost-sensitive learning, and multi-threshold HITL band design.

— Formal foundations for Chapter 4

4.9 The ROC Curve and AUC

4.9.1 Definition

For a binary classifier with scores $s(x) \in [0, 1]$, sweeping the threshold $\tau \in [0, 1]$ traces the Receiver Operating Characteristic (ROC) curve:

$$\mathrm{TPR}(\tau) = P(s(x) \geq \tau \mid y = 1) \quad \text{[True Positive Rate / Sensitivity]}$$

$$\mathrm{FPR}(\tau) = P(s(x) \geq \tau \mid y = 0) \quad \text{[False Positive Rate]}$$

The Area Under the ROC Curve (AUC) equals the probability that a random positive is scored higher than a random negative:

$$\text{AUC} = P\big(s(x^+) > s(x^-)\big)$$

AUC measures ranking quality, independent of threshold choice.

AUC vs. Threshold: Separate Questions

AUC tells you how separable your classes are. The threshold tells you where to make the cut given your cost structure. These are separate questions. A model with high AUC can still have a badly chosen threshold; a model with modest AUC but a carefully chosen threshold can outperform a high-AUC model with a naive threshold.

4.9.2 Optimal Threshold Selection

The optimal threshold τ^* minimizes expected cost:

$$\mathbb{E}[\text{Cost}(\tau)] = C_{\text{FN}} \cdot \text{FNR}(\tau) \cdot P(y = 1) + C_{\text{FP}} \cdot \text{FPR}(\tau) \cdot P(y = 0)$$

where C_{FN} is the cost of a false negative (miss) and C_{FP} is the cost of a false positive (false alarm).

Setting $d\mathbb{E}[\text{Cost}(\tau)]/d\tau = 0$ gives the optimal threshold:

$$\tau^* = \frac{C_{\text{FP}} \cdot P(y = 0)}{C_{\text{FP}} \cdot P(y = 0) + C_{\text{FN}} \cdot P(y = 1)}$$

- For equal costs ($C_{\text{FN}} = C_{\text{FP}}$) and balanced classes ($P(y = 0) = P(y = 1) = 0.5$): $\tau^* = 0.5$
- For $C_{\text{FN}} \gg C_{\text{FP}}$ (missing a positive is very costly, e.g., cancer screening): $\tau^* \ll 0.5$

4.10 Precision-Recall Tradeoff

4.10.1 When AUC Is Misleading

AUC is dominated by performance on the majority class. For imbalanced problems (rare positive class), precision-recall analysis is more informative.

$$\text{Precision}(\tau) = \frac{\text{TP}}{\text{TP} + \text{FP}} = P(y = 1 \mid s(x) \geq \tau)$$

$$\text{Recall}(\tau) = \frac{\text{TP}}{\text{TP} + \text{FN}} = P(s(x) \geq \tau \mid y = 1)$$

The F_β score balances precision and recall with weight β:

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$$

- $\beta = 1$: Equal weight on precision and recall (F1 score)
- $\beta = 2$: Recall twice as important (use when false negatives are more costly)
- $\beta = 0.5$: Precision twice as important (use when false positives are more costly)

4.10.2 HITL Band as Precision-Recall Bridge

A two-threshold HITL system $(\tau_{\text{low}}, \tau_{\text{high}})$ with human review for scores in $(\tau_{\text{low}}, \tau_{\text{high}})$:

$$\text{Automated TPR} = P\Big(s(x) \geq \tau_{\text{high}} \mid y = 1\Big)$$

$$\text{Automated FPR} = P\Big(s(x) \geq \tau_{\text{high}} \mid y = 0\Big)$$

$$\text{Human Review Rate} = P\Big(\tau_{\text{low}} \leq s(x) < \tau_{\text{high}}\Big)$$

This system achieves full precision (only reviews above τ_{high} are automatically positive) at the cost of sending the uncertain band to humans. Optimizing the band width trades reviewer workload against precision/recall tradeoffs.

4.11 Cost-Sensitive Learning

4.11.1 Class Weighting

Most ML frameworks allow cost-sensitive training via class weights. For a binary classifier with $C_{\text{FN}} \neq C_{\text{FP}}$:

```
# PyTorch
weight   = torch.tensor([C_FP, C_FN])
loss_fn  = nn.CrossEntropyLoss(weight=weight)

# scikit-learn
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(class_weight={0: C_FP, 1: C_FN})
```

Listing 4.1: Cost-sensitive training in PyTorch and scikit-learn

This biases the model to minimize the more costly error during training, effectively shifting the optimal threshold without post-hoc adjustment.

4.11.2 Calibrated Decision Rules

Given a well-calibrated model with output $p(x) = P(y = 1 \mid x)$:

```
def optimal_decision(score, c_fp, c_fn, margin=0.1):
    """
    Make optimal decision given cost structure.

    Returns: 'positive', 'negative', or 'uncertain' (for
        HITL routing)
```

```
    """
    # Optimal threshold: score where cost_positive =
        cost_negative
    tau_optimal = c_fp / (c_fp + c_fn)

    # HITL band: add uncertainty margin around optimal
        threshold
    if score >= tau_optimal + margin:
        return 'positive'
    elif score <= tau_optimal - margin:
        return 'negative'
    else:
        return 'uncertain'   # Route to human review
```

Listing 4.2: Optimal decision rule with HITL routing band

4.12 Multi-Threshold HITL Systems

4.12.1 Three-Way Classification

A HITL band system partitions the score space into three regions:

Score range	Action
$[0, \tau_L)$	Automatic NEGATIVE (high-confidence negative)
$[\tau_L, \tau_H]$	HUMAN REVIEW (uncertain)
$(\tau_H, 1]$	Automatic POSITIVE (high-confidence positive)

Optimization. Find τ_L and τ_H to minimize total system cost:

$$
\begin{aligned}
\text{Cost}(\tau_L, \tau_H) = C_{\text{FN}} \cdot P(s(x) < \tau_L \mid y = 1) \cdot P(y = 1) \quad & \text{[missed positives]} \\
+C_{\text{FP}} \cdot P(s(x) > \tau_H \mid y = 0) \cdot P(y = 0) \quad & \text{[false alarms, automated]} \\
+C_H \cdot P(\tau_L \leq s(x) \leq \tau_H) \quad & \text{[human review cost]} \\
+C_{H,\text{FN}} \cdot P(\text{human misses } y = 1 \text{ in band}) & \\
+C_{H,\text{FP}} \cdot P(\text{human false alarms in band}) &
\end{aligned}
$$

where C_H is the per-case review cost and $C_{H,\text{FN}}$, $C_{H,\text{FP}}$ account for human errors in the review band.

4.12.2 Capacity Constraints

When human review has a capacity constraint (max reviews per unit time $= R_{\max}$):

$$P(\tau_L \leq s(x) \leq \tau_H) \cdot \text{arrival_rate} \leq R_{\max}$$

Threshold and Reviewer Capacity

As arrival rate increases (busy periods), the band $(\tau_H - \tau_L)$ must narrow — fewer cases go to human review, more are handled autonomously. This is the mathematical basis for the observation in Chapter 4 that threshold settings must adapt to reviewer capacity. Ignoring this constraint leads to alert fatigue, which degrades the quality of every human decision in the review queue.

4.13 Exercises

Exercise 4.1 — Optimal Threshold for Fraud Detection

A fraud detection system achieves AUC = 0.94. The base rate of fraud is 0.2%, $C_{\text{FN}} = \$500$ (average fraud amount), $C_{\text{FP}} = \$2$ (customer service cost of false decline). Compute the optimal threshold τ^* using the formula from Section 4A.1.2. At this threshold, what is the expected false positive rate? Is this operationally feasible?

Exercise 4.2 — Dynamic Threshold Under Queue Pressure

A content moderation system has a human review capacity of 1000 cases/hour. The arrival rate of flagged content is 800/hour normally, but spikes to 2000/hour during breaking news events. Design a dynamic threshold system that adjusts τ_L and τ_H based on current review queue length. What are the consequences for each error type during a high-traffic period?

Exercise 4.3 — HITL Band Simulation

Implement the three-way classification system from Section 4A.4.1 in Python. Create a simulation where you vary the margin parameter and observe how the review rate, false negative rate, and false positive rate change. Find the margin that minimizes total cost for a given cost structure.

Exercise 4.4 — ROC vs. Precision-Recall on Imbalanced Data

Draw the ROC curve and precision-recall curve for the same model on an imbalanced dataset (1% positive class). Identify the operating point on each curve that corresponds to the optimal threshold from Section 4A.1.2. Why does the ROC curve look more favorable than the PR curve for this scenario?

4.14 Recommended Reading

Foundational

- Elkan, C. (2001). The foundations of cost-sensitive learning. *IJCAI*, 17(1), 973–978.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.

Calibration and Decision Rules

- Zadrozny, B., & Elkan, C. (2001). Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. *ICML*.
- Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. *ICML*.

Unified Treatment

- Hernández-Orallo, J., Flach, P., & Ferri, C. (2012). A unified view of performance metrics: Translating threshold choice into expected classification loss. *Journal of Machine Learning Research*, 13, 2813–2869.

This appendix provides the mathematical foundations for threshold optimization, ROC and precision-recall analysis, cost-sensitive learning, and multi-threshold HITL band design as introduced in Chapter 4. The exercises connect the theory to concrete design decisions in fraud detection, content moderation, and other high-stakes HITL systems.

5 The Art of Asking for Help (Without Being Annoying)

> *How the design of the ask changes whether humans actually help.*

5.1 One Word

In the spring of 2019, the city of Louisville, Kentucky's 911 dispatch center faced a problem that had nothing to do with technology. Their emergency dispatch system had a high rate of false dispatches—crews sent to locations where nothing was wrong, or where the situation had resolved before they arrived. False dispatches wasted resources, delayed responses to genuine emergencies, and wore down dispatchers and first responders.

A consultant hired to analyze the problem reviewed audio recordings of dispatch calls and noticed something striking. When dispatchers asked callers, "Is there someone there who needs help?"—the typical formulation—they got a high rate of ambiguous answers. Callers who were uncertain, or who were calling on someone else's behalf, or who were watching something happen from a distance, tended to answer "yes" to the general question even when the situation was unclear.

The consultant recommended a single change: replace "Is there someone there who needs help?" with "Do you personally need emergency assistance right now?"

The addition of "personally" and "right now" did two things. It forced the caller to think specifically about themselves and the present moment, rather than reporting vaguely on a situation. And it changed the framing from a general inquiry to a direct commitment.

The false dispatch rate dropped by approximately 40 percent over the following months.

One word. One word in a question that dispatchers were asking hundreds of times a day changed how callers understood what they were being asked—and changed what information the dispatchers got back.

This is the art of asking for help. Not whether to ask. How.

5.2 The Ask Is a Design Problem

Everything we have established in the previous chapters points toward a moment: the moment when a system detects that it is uncertain, evaluates that the case is worth routing to a human, and then… asks. Or alerts. Or flags. Or notifies. Or interrupts.

That moment is where most HITL systems fail, even when everything before it has been done right.

Chapter 1 gave us the framework: five dimensions, three failure modes. Chapter 2 explained that confusion itself is useful information. Chapter 3 showed that learning systems produce predictable patterns of uncertainty at the edges of their training. Chapter 4 introduced the threshold problem and the human value judgments embedded in it.

All of that engineering, all of that calibration, all of that careful cost analysis—it delivers the system to the moment of the ask. And then a poorly designed question throws it all away.

The fraud alert that comes at 3 AM and wakes you from sleep to ask about a charge you made yourself that morning. The spam filter notification that flags the same newsletter for the hundredth time. The

pop-up that asks for your permission to access your location before you have even understood what the app is for. The car alarm that fires whenever a large truck passes.

Each of these is a system that detected uncertainty correctly, made a reasonable threshold decision, and then asked badly. The ask was the failure.

5.3 Three Levers

The design space for an ask has three primary dimensions:

Timing. When does the ask happen? Is it synchronous—immediately interrupting what the person is doing—or asynchronous, deferred until a natural break? Is it too early (before the person has the information needed to answer) or too late (after the decision has already been made by default)?

Frequency. How often does the ask happen? Is it rare enough to command attention, or common enough to become noise? Does the system have a mechanism to prevent the same person from being asked the same type of question too many times?

Format. How is the ask phrased? What choices does it offer? How much cognitive work does it require? Does the phrasing bias the answer?

These are not independent. A well-timed, infrequent ask can survive imperfect format. A perfectly formatted ask at the wrong moment fails. The goal is to optimize all three together, and to understand how the human's context—what they are doing, what they are attending to, what they have already been asked—shapes every dimension of the optimal ask.

5.4 The Trust–Attention–Response Triangle

Human psychology and the design of alerts interact in a system that has its own dynamics. Call it the **trust–attention–response triangle**.

When a system asks for human input, three things are required: the human must **trust** that the ask is worth responding to; the human must have **attention** available to process it; and the human must be willing and able to **respond** effectively.

Each of these can fail independently.

Trust failure. A system that has cried wolf too many times loses the trust that makes its alerts actionable. The alerts still arrive. The human learns to dismiss them. This is the clinical **alert fatigue** problem described in Chapter 1, but it applies everywhere. The fire alarm that fires so frequently that residents stop evacuating. The fraud alert service that flags so many routine purchases that cardholders start ignoring the texts. The over-alerting system trains its own users to be non-users.

Attention failure. A system that asks at the worst possible moment gets ignored not because the human does not trust it but because the human literally cannot process the ask right now. The update prompt that fires during a video call. The security notification that arrives mid-presentation. Timing failures are attention failures.

Response failure. A system that asks a question the human cannot answer effectively gets useless or harmful responses. A fraud alert that asks "Was this transaction legitimate?" when you cannot remember whether you made it, at an amount that is not extraordinary—you will answer randomly or default to "yes" without thinking. The ask's format must give the human the information they need to answer accurately.

The trust–attention–response triangle is not static. It changes over time, across users, and across contexts. A system that maintains all three is rare. A system that degrades all three is common.

5.5 The Lazy User Assumption

Here is a design assumption that underlies most bad alert design, usually implicitly: *users will not try.*

The premise behind asking questions with six items to review, or requiring navigating three menus to respond to a notification, or sending a fraud alert in a format that requires calling a phone number—all of these assume that the user is lazy and that the friction is acceptable because dedicated users will push through it.

This assumption is wrong in two ways.

First, it is empirically incorrect. Users in well-designed systems routinely engage with alerts, respond to notifications, and participate in feedback mechanisms—when the design makes it easy. The problem is not user laziness. The problem is designers treating users as if they have infinite attention and zero competing demands.

Second, the assumption fails even on its own terms. A user who is willing to engage with a fraud alert but is given a format that requires four steps to respond will not respond four times better than a user given one step. The friction does not select for motivated users. It just loses them.

The Design Principle

Good HITL intervention design starts from the opposite assumption: *users will try if the ask is worth their time.* The design challenge is making the ask worthy.

> "Worth their time" means: the ask is rare enough to be surprising; it arrives at a moment when the human can actually respond; the format makes the question clear and the response easy; and the human has reason to believe their response will actually matter.

5.6 Two-Factor Authentication: Lessons from the Field

Two-factor authentication (2FA) is perhaps the most widely deployed HITL mechanism in existence. Every time a website sends you a six-digit code and asks you to type it in, it is inserting a human into a loop that would otherwise be purely automated.

The security rationale is sound: even if an attacker has your password, they are unlikely to also have your phone. The human in the loop—you, confirming your identity by entering a code only you could receive—provides a verification layer that pure password authentication cannot.

But 2FA's implementation landscape is a masterclass in how to design the ask both well and badly.

Poorly designed 2FA.

- Push notifications sent to a phone that require dismissal without confirmation (so-called *MFA fatigue attacks* exploit users who approve requests reflexively)
- SMS codes sent to numbers that may be SIM-swappable
- Time pressure so aggressive that the window expires before the user finds their phone
- No context provided: "Your authentication code is 847291"—but for what? From where?

- Prompts sent at 3 AM that train users to approve requests without fully waking up

Well-designed 2FA.

- Provides context: "Someone is trying to sign into your account from a new device. If this was you, enter this code: [code]"
- Number matching: the login screen shows a number and the push notification asks "Did you see the number [X]?"—defeating reflexive approval
- Adaptive authentication: only prompts for 2FA when the login context is unusual (new device, new location, unusual time)
- Clear, low-friction response options with a prominent "I did not initiate this login" option

The difference between these two designs is not the technology. It is the application of the trust–attention–response triangle. Good 2FA provides the context the user needs (trust), arrives when the user is actively trying to log in (attention), and makes both "approve" and "deny" equally easy (response).

5.7 The Fraud Alert as Design Canvas

Bank fraud alerts are another rich case study, because the same underlying technology—"this transaction looks unusual, confirm it was you"—can be implemented in ways that range from excellent to actively harmful.

An Excellent Fraud Alert

- Sent within seconds of the transaction, while you are still in the situation
- Contains specific, identifiable information: the merchant name, the exact amount, the last four digits of the card
- Asks a simple binary question: "Did you make this $47.23 purchase at Costa Coffee? Reply YES or NO"
- Takes one action (a single text reply) to confirm
- If you reply NO: immediate card freeze plus one-tap call option to the fraud line
- If you do not reply within 10 minutes: gentle follow-up, then escalation protocol

- Sent six hours after the transaction, when context is gone
- Contains: "Your account has been flagged for unusual activity"
- Requires you to call a number, navigate an IVR system, verify your identity, explain the situation
- If the transaction was legitimate: 20 minutes of your life gone
- Teaches you that fraud alerts are time-consuming, training you to ignore the next one

The first design respects the trust–attention–response triangle at every step. The second violates all three. And the second is, unfortunately, still common.

5.8 Content Moderation and the Interface of Judgment

Content moderation is a domain that makes the design of the ask visible in ways that other HITL applications do not, because the "human" in the loop is often a professional reviewer whose job is entirely about responding to system-generated asks.

Professional content moderators at major platforms spend their days responding to a stream of flagged content: videos, images, posts, comments—each one a system-generated ask requiring a human response. The design of the interface they use shapes the quality of every response they give (Roberts, 2019).

Research into content moderation interfaces has documented several consistent findings.

Context improves accuracy. Moderators shown content with its original context (the thread, the account history, the apparent intent) make more accurate decisions than those shown the same content in isolation. But showing context slows throughput. The ask's format creates a tradeoff between speed and quality.

Category granularity matters. Interfaces with many specific categories ("graphic violence: real vs. artistic context," "hate speech: individual vs. systemic") produce more consistent and useful decisions than interfaces with coarse categories ("violent," "non-violent"). But more categories require more cognitive work per decision.

Throughput pressure degrades decision quality. Moderators required to make more decisions per hour show increased error rates, particularly near the end of shifts and during high-volume periods (Steiger et al., 2021). The ask format cannot compensate for structural workload problems.

Explanation requests improve downstream model quality. When moderators are asked not just "is this violating?" but "what makes this violating?" (even briefly), the resulting labels are more useful for model training. The ask format shapes what information is captured, not just what decision is made.

Each of these findings is a design lesson. The ask format is not neutral. It actively shapes the information the system gets back.

5.9 The 911 Dispatcher, Revisited

Return to Louisville. One word changed the false dispatch rate by 40 percent.

Why did "personally" and "right now" matter so much?

The original question ("Is there someone there who needs help?") asked callers to interpret a situation they may have been observing from some distance, with some uncertainty, with some motivation to act because they were already on the phone with 911. The framing invited them to extrapolate.

The revised question ("Do you personally need emergency assistance right now?") grounded the question in the caller's direct experience at the present moment. It asked for first-person knowledge rather than third-party interpretation. It implicitly provided a category: "emergency assistance" is more specific than "help," and "right now" excludes anticipated future needs.

These are not just clarity improvements. They are changes to what information the question extracts. The question's format shapes the answer's content.

The Core Insight

The question you ask determines the information you get. A vague question gets vague information. A question framed to invite confirmation gets confirmation. A question framed to invite nuance gets nuance.

And in a HITL system, the quality of the information the human provides determines how much the system improves. A question designed to extract maximum useful information from a minimum human effort is not just a UX nicety—it is the core engineering of the feedback loop.

5.10 Feedback Integration: Does It Learn?

There is a fifth dimension in our framework that often gets overlooked in the design of individual asks: **Feedback Integration**—whether the system actually learns from the human's response.

The ask is useless if the response disappears into a void. If a fraud alert's "NO" response does not update the model's representation of the user's spending patterns, the same type of transaction will keep getting flagged. If a content moderator's "not violating" decision on a borderline case does not inform how similar cases are classified in the future, the system is simply routing work rather than learning.

The design of the ask and the design of the feedback loop are not separate problems. The ask's format determines what information is captured. The feedback loop determines what happens with that information.

Well-designed HITL systems close this loop explicitly:

- The user's response is labeled and stored with the full context of the ask
- Cases where human decisions systematically differ from model predictions are flagged for calibration review
- The frequency of human corrections in a particular category is used to prioritize model improvement
- The human reviewer occasionally receives confirmation that their feedback mattered: "Your corrections in this category helped improve accuracy by X%"

That last point—feedback to the feedback provider—may seem like a courtesy. It is actually a mechanism for maintaining engagement. A human reviewer who never sees evidence that their responses matter eventually stops trying. The trust–attention–response triangle degrades from the trust side.

For the next week, pay attention to every digital alert or notification that asks you to take an action. For each one, ask three questions: (1) Is this well-timed—are you in a position to respond effectively right now? (2) Is this well-framed—do you understand exactly what is being asked and have what you need to answer? (3) Does your response matter—does it feel like your answer will change anything? Keep a log of your answers. You will quickly develop a sense for which systems are designed to get useful human input, and which are designed to appear to involve humans without actually extracting useful judgment.

Chapter Summary

Key Concepts

- The design of the ask is as important as the decision to ask—a technically correct routing decision can be wasted by a poorly designed intervention
- Three primary levers: **timing**, **frequency**, **format**
- The trust–attention–response triangle: all three must be maintained for HITL to work
- The Lazy User Assumption is wrong: users will engage with asks that respect their time and attention
- The question's format determines the information captured—not just what decision is made
- Feedback Integration closes the loop: the ask is wasted if the response does not improve the system

Key Examples

911 dispatch (Louisville) One word change in a question ("personally" + "right now") reduced false dispatches by 40%

Two-factor authentication The same security mechanism implemented well vs. poorly—context and number matching vs. fatigue-inducing push approval

Bank fraud alerts Specific, timely, binary ask vs. generic, delayed, high-friction request

Content moderation interfaces Context, category granularity, and throughput pressure all shape decision quality

Key Principles

- Timing must align with the human's attention window—ask when they can answer
- Frequency must be low enough to preserve the signal value of each ask
- Format must give the human what they need to answer accurately in minimum steps
- Good design assumes users will try; bad design assumes they will not
- The ask and the feedback loop are one system, not two separate problems

---

This chapter closes the first part of our journey: from understanding that smart systems need to ask for help, through understanding what confusion is and where it comes from, to understanding how to turn uncertainty into decisions, and finally to understanding how to make the ask itself effective. The next part turns to specific domains where HITL design matters most.

Technical Appendix 5A: Optimal Ask Design

Chapter 5 introduced three design levers for human-in-the-loop asks: timing, frequency, and format. This appendix provides mathematical frameworks for optimising each, drawing on information theory, cognitive psychology, and reinforcement learning.

— Formal foundations for Chapter 5

5.11 Information-Theoretic Foundations of Ask Design

5.11.1 Information Value of a Human Response

The value of asking a human is the expected reduction in uncertainty about the correct action. Let Y be the correct decision (unknown to the system), H the human's response, and X the system's current information. The **information gain** from asking is:

$$\mathrm{IG}(H; Y \mid X) = \mathbb{H}(Y \mid X) - \mathbb{H}(Y \mid H, X)$$

where $\mathbb{H}(\cdot)$ denotes entropy. A high information gain means the human's response substantially reduces uncertainty about the right action.

Optimal ask criterion. Ask a human when:

$$\mathrm{IG}(H;\, Y \mid X) > \frac{C_{\text{ask}}}{V_{\text{decision}}}$$

where C_{ask} is the cost (time, attention, trust) and V_{decision} is the value of making the correct decision.

5.11.2 Query Optimisation

The **optimal query** Q^* maximises information gain subject to cognitive load constraints:

$$Q^* = \arg\max_{Q \in \mathcal{Q}} \big[\mathrm{IG}(H_Q;\, Y \mid X) - \lambda \cdot \mathrm{CogLoad}(Q)\big]$$

where $\mathrm{CogLoad}(Q)$ is the cognitive effort required to answer query Q and λ is a cost–effort tradeoff parameter.

Simplicity vs. Information

A simpler query (lower cognitive load) is only preferable if its information loss is proportionally small. The 911 example from Chapter 5 illustrates this: the revised question had *lower* cognitive load and *higher* information gain simultaneously—a rare but achievable combination.

5.11.3 Human–AI Complementarity

The value of the full human–AI system depends on the *complementarity* between human and AI knowledge:

$$I(H_{\text{human}};\, Y \mid H_{\text{AI}}, X) = \mathbb{H}(Y \mid H_{\text{AI}}, X) - \mathbb{H}(Y \mid H_{\text{human}}, H_{\text{AI}}, X)$$

This is the additional uncertainty reduction from the human, given the AI has already processed X. High complementarity means the human knows things the AI does not—this is when HITL intervention is most valuable.

5.12 Alert Fatigue: A Dynamic Model

5.12.1 Trust as a State Variable

Let $\tau_t \in [0, 1]$ represent the human's trust in the system's alerts at time t. Trust evolves dynamically:

$$\tau_{t+1} = \tau_t + \alpha \cdot \mathbf{1}[\text{alert}_t \text{ was valid}] - \beta \cdot \mathbf{1}[\text{alert}_t \text{ was false}]$$

where α is the trust gain from a true alert and β is the trust loss from a false alarm. Empirically $\beta > \alpha$: trust erodes faster than it builds (J. D. Lee & See, 2004). Effective alert value therefore decays with sustained false-positive rates:

$$V_{\text{effective}}(t) = \tau_t \cdot V_{\text{intrinsic}}$$

5.12.2 Precision–Fatigue Tradeoff

Define r as the alert rate, $p(r)$ as precision (decreasing in r), $\tau(r)$ as equilibrium trust (also decreasing in r), and $\rho(\tau)$ as response probability. The effective information extracted per hour:

$$I_{\text{effective}}(r) = r \cdot p(r) \cdot \tau(r) \cdot \rho\big(\tau(r)\big) \cdot V_{\text{per alert}}$$

```
from scipy.optimize import minimize_scalar

def effective_information(r, precision_decay=0.1,
```

```
                                trust_decay=0.05,
                                    base_value=1.0):
    precision = 1 / (1 + precision_decay * r)
    false_alarm_rate = r * (1 - precision)
    trust = 1 / (1 + trust_decay * false_alarm_rate)
    response_prob = trust ** 0.5
    return r * precision * trust * response_prob *
        base_value

result = minimize_scalar(
    lambda r: -effective_information(r),
    bounds=(0.1, 100), method='bounded'
)
print(f"Optimal alert rate: {result.x:.2f} alerts/hour")
```

Listing 5.1: Effective information vs. alert rate

5.12.3 Habituation and Recovery

Consecutive alerts induce faster degradation:

$$\tau_t = \tau_{\text{baseline}} \cdot e^{-\gamma \cdot n_{\text{recent}}}$$

where n_{recent} is the number of alerts in the recent window. Recovery is slower:

$$\tau_{t+T} = \tau_t + (1 - \tau_t)\left(1 - e^{-\delta T}\right)$$

Spacing alerts recovers attention; clustering alerts degrades it faster than average-rate models predict. A well-designed HITL interface

enforces a minimum inter-alert gap, not just a maximum average rate.

5.13 Optimal Timing Models

5.13.1 Attention Windows

Let $a(t) \in [0, 1]$ be the human's attention availability. The effective response quality for an alert sent at time t:

$$Q_{\text{response}}(t) = Q_{\max} \cdot a(t) \cdot \tau(t)$$

Optimal timing: $t^* = \arg\max_t Q_{\text{response}}(t)$. Interruption at natural task breakpoints consistently produces higher response quality than mid-task interruption.

5.13.2 The Window of Relevance Problem

Context availability decays after an event:

$$C(\Delta t) = e^{-\kappa \Delta t}$$

for decay rate κ depending on event type. Optimal alert delay balances context availability against attention window:

$$t^* = \arg\max_t \; a(t + t_{\text{event}}) \cdot C(t)$$

For financial transactions κ is large, so $t^* \approx 0$. Asking about a charge 24 hours later yields near-chance accuracy.

5.14 Format Design: Cognitive Load and Response Accuracy

5.14.1 Cognitive Load Model

Following Sweller's cognitive load theory, response accuracy degrades beyond working memory capacity W:

$$A(c, W) = \begin{cases} A_{\max} & \text{if } c \leq W \\ A_{\max} \cdot (W/c)^{\alpha} & \text{if } c > W \end{cases}$$

5.14.2 Optimal Query Granularity

A k-choice query captures $\log_2 k$ bits at $O(k)$ cognitive cost:

$$k^* = \arg\max_k \frac{\log_2 k}{k^{\beta} \cdot C_{\text{per-choice}}}$$

For most practical settings $k^* \in [2, 5]$. Binary questions are easiest but information-sparse; 4–5 choices represent the practical optimum.

5.15 Feedback Integration: Closing the Learning Loop

5.15.1 Active Learning Formulation

Each human response is a labeled data point. Select the next query to maximise expected model improvement:

$$Q^*_{\text{next}} = \arg\max_Q \mathbb{E}\big[\Delta\text{ModelPerf} \mid H_Q, \mathscr{D}_{\text{current}}\big]$$

In practice this is approximated by querying cases near the decision boundary, in underrepresented categories, or where human and model systematically disagree.

5.15.2 RL Formulation of the Full HITL Loop

The ask–learn loop formalised as an MDP (Christiano et al., 2017):

- **State** s_t: model weights, trust τ_t, queue depth, context
- **Action** a_t: ask with format Q, or act autonomously
- **Reward**: $r_t = \Delta\text{Acc}(a_t) - C_{\text{ask}} \cdot \mathbf{1}[a_t = \text{ask}] - \lambda \cdot \Delta\tau_t$

Optimal policy π^* maximises $\mathbb{E}[\sum_t \gamma^t r_t]$, capturing the tension between short-run accuracy and long-run trust.

5.16 Exercises

Exercise 5.1 — Information Value Calculation

A spam filter has confidence 0.65 on an email. A human reviewer has 90% accuracy on boundary cases.

1. Calculate the expected information gain from asking the reviewer.
2. If the cost of asking is 30 seconds of reviewer time (valued at \$0.01/second), at what minimum email value must human review be worthwhile?
3. How does information gain change if filter confidence was 0.95?

Exercise 5.2 — Alert Fatigue Simulation

Using the dynamic trust model:

```python
import numpy as np

def simulate_alert_fatigue(n_alerts=300,
    precision=0.7,
                              alpha=0.05, beta=0.15):
    trust, history = 1.0, [1.0]
    for _ in range(n_alerts):
        is_valid = np.random.random() < precision
        trust = min(1.0, trust + alpha) if is_valid \
                else max(0.0, trust - beta)
        history.append(trust)
    return history
```

Listing 5.2: Alert fatigue simulator

Simulate: (a) 10/hour at 70% precision; (b) 3/hour at 90% precision; (c) 20/hour at 90% precision. Compare equilibrium trust levels.

Exercise 5.3 — Optimal Query Granularity

For a content moderation interface with 5 labels vs. 2 labels:

1. Calculate maximum information per response for each.
2. Assuming $\beta = 1.5$, compute k^*.
3. Design an experiment measuring actual information captured using inter-annotator agreement as a proxy.

Exercise 5.4 — HITL Policy Optimisation

Implement a Q-learning agent learning when to ask. State: confidence (binned 0–10), trust (high/medium/low). Actions: auto-decide, binary query, detailed query. Reward: +10 correct, −1 ask cost, $-0.5 \times (1 - \text{trust})$ incorrect auto-decision. Simulate 10,000 episodes. At what confidence level does the optimal policy switch from asking to auto-deciding?

5.17 Recommended Reading

Alert Design

- EEMUA (2013) — Engineering Equipment and Materials Users' Association: alarm system management guidelines.
- Drew et al. (2013) — Attention and awareness in simulated radiological search. *Psychological Science.*

Active Learning and HITL Feedback

- Settles (2009) — Active learning literature survey.
- Christiano et al. (2017) — Deep reinforcement learning from human preferences. *NeurIPS.*

This appendix provides the formal foundations for ask design, alert fatigue, timing, format, and feedback integration as introduced in Chapter 5.

Part II

The Human Side of the Loop

A HITL system is only as good as its worst interface.
— Chapter 6

6 Interfaces That Make Humans Smarter

A HITL system is only as good as its worst interface—the design of a screen can make a doctor brilliant, or helpless.

— The premise of Chapter 6

6.1 The Judge and the Number

On a Tuesday morning in Broward County, Florida, a judge opened a presentencing report and found something unusual near the top: a score. The number was produced by COMPAS—Correctional Offender Management Profiling for Alternative Sanctions—an algorithmic risk assessment tool used in dozens of American jurisdictions. The score was supposed to tell the judge how likely the defendant was to reoffend. High risk or low risk. A number to guide the sentence.

The judge had questions. What data had produced it? Which factors drove the score up or down? Was it the defendant's age? Their prior record? Their neighborhood? The tool's vendor, Northpointe (now Equivant), considered those inputs proprietary. The score arrived without explanation, like a verdict without a trial.

The judge did what many judges do: they used the number. Research by ProPublica, published in 2016, would later show that the COMPAS tool was wrong about recidivism almost as often as it was right—about

as accurate as asking a random person on the street—and dramatically more likely to falsely flag Black defendants as high risk compared to white defendants with similar profiles (Angwin et al., 2016). The algorithm had a race problem that the score, presented as a clean integer, completely obscured.

But here is the part of the story that gets less attention: the judge was not irrational. They were responding, predictably and almost inevitably, to the way the information was presented. A number with no explanation exerts a strange psychological gravity. It feels more objective than a probation officer's narrative. It feels more certain than it is. The interface between the algorithm and the human had not made the judge smarter. It had quietly replaced their judgment.

This is the subject of this chapter: not whether AI systems should assist human decision-makers, but *how*. The interface—the screen, the dashboard, the number, the recommendation—is not a neutral transmission channel. It shapes what humans notice, what they ignore, and ultimately what they decide. A badly designed interface does not just fail to help. It actively makes its human partner worse.

6.2 The Invisible Architecture of Attention

When radiologist Dr. Sarah Chen sits down at her workstation at six in the morning, she faces a queue of about 80 chest X-rays. Some are routine. Some are not. The AI triage system flags the ones it considers urgent and surfaces them first.

In 2018, a Stanford team led by Andrew Ng demonstrated that a deep learning system called CheXNet could detect pneumonia from chest X-rays at a level that matched or exceeded radiologist performance (Rajpurkar et al., 2017). The headlines celebrated the model's accuracy. Subsequent research examining how radiologists interacted with AI systems like

CheXNet found something more consequential: the way the AI's findings were presented to radiologists dramatically changed what the radiologists actually detected (Gaube et al., 2021).

When radiologists were shown AI outputs as a probability score alone—"94% pneumonia"—their accuracy on pneumonia improved, but their detection of other findings dropped. Their attention had been guided, almost literally by the number, toward the AI's specialty and away from everything else in the image.

When radiologists were shown AI outputs as a highlighted region on the image—a colored overlay showing exactly where the algorithm was looking—something different happened. The radiologists interrogated the highlighted region more carefully, sometimes catching errors in the AI's reasoning. They also paid more attention to the surrounding unhighlighted areas, because the highlighted region made the contrast vivid.

The Presentation Effect

Same model. Same data. Dramatically different human performance. The variable was the design of how the AI communicated with the radiologist. How information is shown determines what information is used.

This is what interface designers in medical AI call the **presentation effect**, and it is one of the most consistent findings in the field. The architecture of attention—what gets a bright box, what gets a sidebar, what appears first, what requires a click to access—is not a cosmetic choice. It is a decision about what humans will think.

6.3 Automation Bias and the Seduction of the Algorithm

In the spring of 2019, researchers at Vanderbilt University published a study with an uncomfortable finding. They had given radiologists the same set of mammography images under three conditions: no AI assistance, AI assistance with a correct recommendation, and AI assistance with a deliberately incorrect recommendation. The question was how often radiologists would agree with the AI, and whether they would catch the AI's mistakes.

When the AI was correct, it helped—radiologists agreed with it at high rates and their accuracy improved. When the AI was wrong, however, a distressing number of radiologists followed it anyway. Radiologists in the AI-assisted condition missed cancers that they had correctly identified without the AI, apparently because the AI's negative finding overrode their own judgment (Gaube et al., 2021).

This is **automation bias**: the tendency to over-rely on automated recommendations, particularly when the human is uncertain or fatigued. It is not a character flaw. It is a documented cognitive response to the presence of a system that presents itself as authoritative. The more confident-looking the recommendation, the more powerful the bias.

Automation bias was first systematically documented in aviation, where it was called "automation complacency." Pilots in highly automated cockpits were found to be less vigilant, less likely to notice anomalies, more likely to accept erroneous automated suggestions (Parasuraman & Manzey, 2010). The same pattern appears

> in air traffic control, nuclear power plant monitoring, and now, everywhere AI recommendations are shown to human reviewers.

The unsettling implication is that AI assistance can make expert humans worse. Not in every case, and not in every domain, but under the right conditions—a fatigued reviewer, a plausible-looking error, a confident-seeming display—the AI's recommendation can function as a kind of anchor that pulls human judgment toward it even when judgment should pull away.

The antidote is not to hide the AI recommendation. It is to design the interface so that the human forms their own judgment before seeing the AI's, or is required to articulate a reason when they deviate from their own initial assessment. The Israeli Air Force, studying automation bias in combat systems, found that requiring operators to commit to a decision before seeing the automated recommendation reduced bias significantly (Mosier et al., 1998). The sequence of information presentation turned out to matter more than the content.

6.4 The Death by a Thousand Clicks

Maria works for a content moderation company contracted by one of the largest social media platforms in the world. Every day she reviews about 400 posts flagged by the platform's AI as potentially violating community guidelines. Her interface has seventeen categories for violation type, each with sub-categories. To flag something as "coordinated inauthentic behavior" she must navigate to a different tool entirely, enter a case ID, and then return to her original queue.

Maria's average review time is 47 seconds per post. She is considered efficient.

The problem is not that 47 seconds is too little time. The problem is that a large fraction of those 47 seconds is consumed by interface mechanics—clicking, navigating, confirming, entering IDs—rather than by actual judgment. Researchers who study content moderation, including the team led by Sarah Roberts at UCLA, have documented what they call the "cognitive depletion" effect: as the day goes on and the number of posts reviewed climbs past 200, past 300, the quality of moderation decisions declines measurably (Roberts, 2019). Reviewers make more errors on the same types of content late in the day that they handled correctly in the morning.

But the depletion is not just fatigue in the ordinary sense. It is the specific cognitive cost of operating a complex interface under time pressure. Every unnecessary click is a small withdrawal from a limited attention budget.

The Appen Annotation Redesign

When Appen, one of the world's largest data annotation companies, redesigned its annotation interface in 2020 to reduce the number of clicks required per task and streamline instruction display, inter-annotator agreement on a sample benchmark task increased from 0.74 to 0.83 on Cohen's kappa. The annotators were the same people, looking at the same content, applying the same guidelines. The interface had made them better at their job.

The fix is rarely glamorous. It is usually keyboard shortcuts, sensible defaults, streamlined workflows, and a ruthless audit of which clicks actually serve the quality of the human decision and which merely serve the recordkeeping needs of the system. But it requires asking, seriously and specifically: what cognitive work do we actually want the human to do here? And then, for every interface element: does this support that work, or impede it?

6.5 What Explainable AI Actually Looks Like

The COMPAS story prompted a wave of interest in what researchers call **Explainable AI**, or XAI. The basic idea is simple: AI systems should be able to explain why they produced a particular output. A judge should be able to see not just a risk score but the factors that drove it. A doctor should see not just a diagnosis probability but the regions of the scan that drove the model's confidence.

The reality of XAI in practice is messier.

The most common approach is **LIME**—Local Interpretable Model-Agnostic Explanations, developed by Marco Ribeiro and colleagues at the University of Washington (Ribeiro et al., 2016). LIME works by generating a simpler model around a specific prediction that approximates the complex model's behavior in that local region.

The problem is that LIME explanations can be unstable. Run the same prediction through LIME twice with slightly different random seeds, and you may get a different ranking of important features. Researchers demonstrated that LIME's explanations for identical model predictions can vary enough that a human reviewer shown two different LIME outputs for the same case might make two different decisions.

A competing approach, **SHAP** (SHapley Additive exPlanations), provides more mathematically grounded explanations based on game-theoretic principles (Lundberg & Lee, 2017). SHAP is more consistent but more computationally expensive. Neither LIME nor SHAP tells you whether the model is actually reasoning correctly—they show you which features the model is using, not whether those features are the right ones to use.

For radiology AI, the most common XAI approach is the attention map or **Grad-CAM**—a visualization showing which pixels in an image most influenced the model's prediction (Selvaraju et al., 2020). Studies by Zech and colleagues at NYU showed that early pneumonia models were

sometimes attending to whether a hospital's name was embedded in the scan—those scans were more often from ICUs where pneumonia patients happened to concentrate. The model had learned a spurious correlation, and Grad-CAM made it visible.

> An explanation is only useful if the human receiving it can actually act on it. A heatmap that a radiologist cannot interpret provides no safety benefit. A SHAP plot that a loan officer cannot read does not protect the borrower. Explanation design is interface design, and all the same rules apply: what cognitive work do we actually want the human to do?

6.6 Interfaces That Support Judgment vs. Interfaces That Replace It

Imagine a spectrum. At one end: an interface that shows the human only the AI's final recommendation—"Approve," "Deny," "High Risk," "Low Risk." The human has information about the conclusion but not the reasoning. This is the COMPAS interface.

At the other end: an interface that shows the human all the raw information the AI has access to, all the uncertainty estimates, all the feature contributions, a full audit trail. This is maximal transparency, and it is also practically useless. No human can process all that information in the time available for a real decision.

Between these extremes is a design space that has been surprisingly little explored until recently. The best interfaces in this space share a few properties.

They separate the AI's job from the human's job. The AI handles pattern recognition and data synthesis; the human handles judgment about

edge cases, context, and stakes. The interface makes this division visible: here is what the AI found, here is where the AI is confident, here is where the AI is not.

They present uncertainty as information, not as failure. A confidence level of 62% is not the AI apologizing—it is the AI directing the human's attention to a case that genuinely requires human judgment. An orange flag on uncertain cases and a green tag on confident ones tells the human not "the AI isn't sure," but "the AI is inviting your expertise here."

They preserve the human's capacity for independent judgment. Some interface designs are so anchored around the AI recommendation that the human reviewer finds themselves responding to the AI rather than to the underlying case. Studies in legal document review found that human reviewers who saw a document summary without the original document were more likely to follow the summary even when it was wrong, compared to reviewers who saw both.

They make disagreement easy. If a reviewer wants to flag disagreement with the AI, the interface should make that disagreement easy to record and route to appropriate review. Many current HITL interfaces make agreement a single click and disagreement a three-menu navigation. This asymmetry is a thumb on the scale.

6.7 The Radiology Interface Wars

Few fields have experimented more intensively with AI-assisted human decision-making interfaces than radiology.

Zebra Medical Vision, an Israeli medical imaging AI company, deployed a liver disease detection tool at a large European hospital system in 2019.

The tool produced recommendations with high accuracy rates in trials. In the first six months of deployment, however, radiologist satisfaction was low, and the hospital's medical informatics team noticed something unexpected: radiologists were systematically reviewing AI-flagged cases more slowly than non-flagged cases.

Post-deployment interviews revealed why. The AI's confidence visualization—a horizontal bar from 0% to 100%—was being interpreted as a suggestion that they should spend time proportional to the AI's confidence. High confidence cases were getting long review times as radiologists tried to understand why the AI was so sure. Low confidence cases were getting short times because the low-confidence marker was making radiologists feel the case was probably benign.

The design team had built the confidence display to help. It was doing the opposite. A redesign replaced the bar with a simple flag: "AI found a region of interest" or "AI found no region of interest." Review times normalized. Radiologist satisfaction improved. Accuracy was unchanged.

MGH Stroke Detection: Parallel Reading Success

Massachusetts General Hospital's radiology AI for stroke detection implemented a "parallel reading" protocol: the human radiologist reads the scan independently, records their initial findings, then sees the AI's analysis alongside—including a highlighted region, a confidence level, and a plain-language summary of the key feature driving the AI's concern. Radiologists in the parallel reading condition showed meaningfully higher sensitivity for stroke detection than radiologists in the AI-first condition, and reported higher confidence in their own judgment.

The parallel reading interface had preserved one thing the AI-first interface had not: the radiologist's ownership of the diagnosis.

6.8 The Five Dimensions Through the Interface Lens

The five-dimension framework maps cleanly onto interface design choices.

Uncertainty Detection must be made visible. An AI's internal uncertainty estimate is worthless if the interface does not communicate it to the human reviewer. The COMPAS system showed a score without uncertainty bounds. The Zebra radiology system showed a bar that was misinterpreted. The MGH stroke system showed a highlighted region with a plain-language description. The last was most effective at directing human attention appropriately.

Intervention Design is the interface's core problem. What is the human being asked to do? The best interventions are designed with a specific cognitive task in mind: the human is being asked to do something they are genuinely better at than the AI.

Timing in interface terms is about sequencing. Does the human see the AI recommendation before forming their own view, after, or simultaneously? The parallel reading approach optimizes timing deliberately: human-first, then AI. The COMPAS score was delivered at the moment of sentencing in a format that made it feel like a fact rather than a recommendation.

Stakes Calibration should shape the visual weight of recommendations. A high-stakes flagged case should look different from a routine flagged case. Many annotation interfaces treat all flags equally, presenting a false confidence to human reviewers that every flag deserves the same attention.

Feedback Integration requires the interface to make the human's feedback visible and learnable. If a radiologist consistently disagrees with the AI about a particular type of finding, that pattern should be detectable—to improve the model, to route those cases to this radiologist, or to trigger a review of the AI's training data.

6.9 Why Design Is a Safety Issue

In 2020, a study in *JAMA Network Open* examined five emergency departments that deployed the same AI chest X-ray triage system. Three departments showed improved accuracy. Two showed no improvement, and one showed a statistically significant *decrease* in diagnostic accuracy for non-flagged cases.

The critical variable was not the AI model. It was the interface design. The three departments with improved outcomes had implemented the AI as a parallel second read, with its findings displayed after the radiologist's initial assessment. The two departments with unchanged or worsened outcomes had implemented the AI as a first read.

The AI did not make those radiologists worse. The interface did.

Interface Design as Safety Engineering

A HITL system whose interface makes humans worse is not a HITL system that is working well. It is a HITL system that has quietly removed the loop from human-in-the-loop. Interface design is a safety issue with the same practical significance as model accuracy.

The next time you use a system that gives you a recommendation—a navigation app, a streaming service, a shopping site—notice the order. Did you form your own preference before you saw the recommendation, or did the recommendation arrive first? Pay attention to whether having seen the recommendation makes it harder to decide differently, even if you genuinely want to. That slight resistance is automation bias in action. It is real, it is measurable, and it is built into the interface whether the designers intended it or not.

Chapter Summary

Key Concepts

- Interfaces are not neutral transmission channels—they shape what humans notice, how they reason, and what they decide
- **Automation bias** is the documented tendency to over-rely on automated recommendations; it degrades with fatigue and scales with displayed AI confidence
- XAI tools (LIME, SHAP, Grad-CAM) have real value but also real failure modes when outputs are misinterpreted or poorly presented
- The best interfaces separate the AI's job from the human's job, present uncertainty as information, preserve independent judgment, and make disagreement easy
- Interface design is a safety issue with the same practical significance as model accuracy

Key Examples

COMPAS risk scoring A score without explanation replaced judicial reasoning and embedded racial bias in a format that looked like objectivity

CheXNet radiology AI Same model, dramatically different radiologist performance depending on how AI findings were displayed

Vanderbilt mammography study AI assistance caused radiologists to miss cancers they had caught unassisted—automation bias in expert practice

Zebra Medical liver AI Confidence bar caused radiologists to misallocate review time; redesign to a simple flag improved satisfaction without loss of accuracy

MGH stroke parallel reading Human-first interface preserved diagnostic ownership and improved sensitivity

Appen annotation redesign Interface changes alone improved inter-annotator agreement from 0.74 to 0.83 kappa

---

In the next chapter, we turn to the other side of the interface: not what the AI shows the human, but what the human provides to the AI. Labels are the foundation of supervised learning—but they are not neutral, and extracting good ones from human annotators requires its own design discipline.

Technical Appendix 6A: Interface Design for Human-in-the-Loop Systems

The interfaces described in Chapter 6 can be analyzed using cognitive load theory, decision-theoretic models of automation bias, and formal metrics for explanation quality.

— Formal foundations for the examples in Chapter 6

6.10 Cognitive Load Theory and Interface Design

6.10.1 The Three-Component Model

Following Sweller (1988) and Paas & van Merriënboer (1994), we identify three components of cognitive load relevant to HITL interface design:

Intrinsic Load (IL). The inherent complexity of the decision task.

$$IL = f(\text{task complexity},\ \text{element interactivity})$$

Extraneous Load (EL). Cognitive overhead imposed by interface design that does not contribute to the decision.

$$EL = f(\text{navigation steps},\ \text{visual noise},\ \text{context switches})$$

Germane Load (GL). Cognitive effort directed toward actual judgment.

$$GL = \text{Total Working Memory} - IL - EL$$

The design goal for HITL interfaces: minimize EL, so GL can be maximized within working memory limits.

6.10.2 Formalizing the Annotation Click Problem

Let T be total session time, N be the number of items to review, and define:

- t_j = time on genuine judgment per item
- t_m = time on interface mechanics per item
- q_0 = baseline judgment quality (early session)
- $\delta(t_m)$ = quality degradation as a function of accumulated mechanical overhead

Then:

$$\text{Session Quality} = q_0 - \int_0^N \delta(t_m \cdot n)\, dn$$

Table 6.1: Annotation quality vs. interface design from production data

Interface Design	Avg. t_m per item	Session kappa at item 400
Legacy (menu-based)	14 sec	0.61
Optimized (keyboard shortcuts)	5 sec	0.79
Optimal (single-action)	2 sec	0.84

6.11 Automation Bias: Formal Model

6.11.1 The Anchor-Adjustment Framework

Automation bias can be modeled as an anchoring process (Tversky & Kahneman, 1974). When a human reviewer h is shown an AI recommendation r_{AI} before forming their independent judgment:

$$r_h = r_{AI} + \alpha \cdot (r_{\text{independent}} - r_{AI})$$

where $\alpha \in [0, 1]$ is the adjustment coefficient. When $\alpha = 1$, the human makes a fully independent judgment. When $\alpha = 0$, the human simply adopts the AI recommendation.

The adjustment coefficient degrades with fatigue and AI confidence display:

$$\alpha(t, f, c) = \alpha_0 \cdot e^{-\lambda_f f} \cdot e^{-\lambda_c c}$$

where f is fatigue (session hours elapsed), c is displayed AI confidence, and λ_f, λ_c are empirically estimated decay constants.

Empirical studies in radiology consistently find $\alpha \in [0.3, 0.7]$ under standard conditions.

6.11.2 Interface Interventions That Increase α

Pre-commitment approach. Require the human to record their initial assessment before revealing the AI recommendation:

$$\alpha_{\text{pre-commit}} \approx 1.0 \quad \text{(by construction)}$$

The Israeli Air Force study found this intervention brought measured α to 0.92 under conditions where the standard interface produced $\alpha = 0.41$ (Mosier et al., 1998).

Disagree-facilitation. If $C_{\text{agree}} = 1$ click and $C_{\text{disagree}} = k$ clicks:

$$P(\text{disagree}) \propto \frac{1}{k}$$

Setting $k = 1$ significantly increases the rate of warranted independent overrides.

6.12 Explainability Output Quality Metrics

6.12.1 Faithfulness and Stability

An explanation method E that maps model f and input x to a feature importance vector is evaluated on:

Faithfulness.

$$\text{Faithfulness}(E, f, x) = \text{corr}\big(\Delta f(x, \text{mask}),\ E(f, x)\big)$$

Stability.

$$\text{Stability}(E, f, x) = 1 - \mathbb{E}_{\epsilon}\big[\|E(f, x + \epsilon) - E(f, x)\|_2\big]$$

LIME stability scores: approximately 0.4–0.6. SHAP stability scores: typically 0.7–0.85 for the same models.

6.12.2 Human Interpretability Metric

$\text{Utility}(E, h) = P(\text{correct override} \mid E \text{ shown to } h) - P(\text{correct override} \mid E \text{ not shown to } h$

Table 6.2: Human utility of explanation types (medical imaging studies, 2019–2023)

Explanation Type	Human Utility (mean)
Probability score only	−0.04
Probability + highlighted region	+0.12
Probability + region + plain-language rationale	+0.21
Full SHAP feature plot (no training)	−0.07

6.13 Interface Sequence Models

6.13.1 Information Ordering and Decision Quality

Under ideal Bayesian updating, the order of evidence presentation does not matter. Humans are not ideal Bayesian updaters. Empirically:

$$\Delta_{\text{order}} = P(\text{correct} \mid \text{Human-first}) - P(\text{correct} \mid \text{AI-first})$$

Studies in radiology and legal document review find $\Delta_{\text{order}} \in [0.05, 0.15]$ in favor of human-first sequencing, particularly when the AI recommendation is wrong.

6.13.2 Optimal Interface Sequencing Algorithm

```
def optimal_review_sequence(item, ai_model, human_reviewer,
                            HIGH_CONF_THRESHOLD=0.92):
    # Phase 1: Establish human prior (independent)
    initial_judgment = human_reviewer.assess(item)
    record(initial_judgment, timestamp=now())

    # Phase 2: Compute AI recommendation
    ai_pred, ai_conf, ai_expl = ai_model.predict(item)
```

```python

    # Phase 3: Conditional display based on uncertainty
    if ai_conf < HIGH_CONF_THRESHOLD:
        show_to_human(ai_pred, ai_conf, ai_expl)
        final_judgment = human_reviewer.review(
            item,
            own_judgment=initial_judgment,
            ai_input=ai_pred
        )
    else:
        if initial_judgment == ai_pred:
            final_judgment = initial_judgment
            log(agreement=True, reviewed=False)
        else:
            # Disagreement on high-confidence case: escalate
            final_judgment = escalate_to_senior(item)

    record(final_judgment, ai_pred, ai_conf)
    return final_judgment
```

Listing 6.1: Optimal review sequencing for HITL interfaces

6.14 Attention Map Quality Assessment

6.14.1 Grad-CAM Formal Definition

For a convolutional network with class activation weights α_k^c and feature map activations A^k:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\Big(\sum_k \alpha_k^c A^k\Big)$$

where:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

6.14.2 Radiologist Utility Conditions

Grad-CAM is useful to radiologists when:

1. **Localization is correct**: IoU between Grad-CAM highlight and ground-truth annotation > 0.5
2. **Not dominated by artifacts**: Attention mass on non-anatomical features $< 5\%$
3. **Uncertainty not masked**: Diffuse low-magnitude Grad-CAM vs. tight high-magnitude should be visually distinguished

```python
def assess_gradcam_quality(gradcam_map, ground_truth_mask, artifact_mask):
    intersection = np.logical_and(gradcam_map > 0.5, ground_truth_mask)
    union = np.logical_or(gradcam_map > 0.5, ground_truth_mask)
    iou = intersection.sum() / union.sum()

    artifact_fraction = (gradcam_map * artifact_mask).sum() / gradcam_map.sum()
    entropy = -np.sum(gradcam_map * np.log(gradcam_map + 1e-8))

    return {
        'localization_iou': iou,
        'artifact_fraction': artifact_fraction,
        'attention_entropy': entropy,
        'display_recommended': iou > 0.5 and artifact_fraction < 0.05
    }
```

Listing 6.2: Grad-CAM quality assessment

6.15 The Review Queue as a Cognitive System

6.15.1 Queue Design and Cognitive Fatigue

Reviewer quality $q(n)$ as a function of items reviewed n:

$$q(n) = q_0 \cdot e^{-\lambda_q n} + q_{\text{floor}}$$

Break scheduling to maintain quality above threshold q_{min}:

$$n_{\text{break}} = \frac{1}{\lambda_q} \ln\left(\frac{q_0 - q_{\text{floor}}}{q_{\text{min}} - q_{\text{floor}}}\right)$$

For $q_0 = 1.0$, $q_{\text{floor}} = 0.7$, $q_{\text{min}} = 0.85$, $\lambda_q = 0.005$:

$$n_{\text{break}} = 200 \cdot \ln(2) \approx 139 \text{ items}$$

For high-stakes annotation, breaks should be enforced approximately every 140 items.

6.15.2 Priority Routing in Review Queues

$$\text{Priority}(x) = w_1 \cdot U_{\text{model}}(x) + w_2 \cdot S(x) + w_3 \cdot T(x)$$

where:

- $U_{\text{model}}(x)$ = model uncertainty for item x
- $S(x)$ = estimated stakes of a wrong decision
- $T(x)$ = time pressure (queue age of item)
- w_1, w_2, w_3 = domain-specific weighting parameters

Recommended starting weights:

- Content moderation (safety-critical): $w_1 = 0.3, w_2 = 0.5, w_3 = 0.2$
- Medical imaging triage: $w_1 = 0.4, w_2 = 0.5, w_3 = 0.1$
- General annotation: $w_1 = 0.5, w_2 = 0.3, w_3 = 0.2$

6.16 Exercises and Assignments

Exercise 6.1 — Interface Audit

Take any annotation or AI-assisted review interface. Count the number of clicks required to: (a) confirm the AI recommendation, (b) override the AI recommendation, (c) escalate to senior review. Calculate $k_{\text{disagree}}/k_{\text{agree}}$. What does this ratio imply about the interface's design assumptions regarding override frequency?

Exercise 6.2 — Automation Bias Measurement

Design a controlled experiment to measure the automation bias coefficient α for a binary classification review task. Define your Control, AI-First, and Human-First conditions. Describe how you estimate α from the resulting data. Compute the sample size needed to detect a difference of 0.05 in mean accuracy between conditions (power = 0.80, $\alpha = 0.05$).

Exercise 6.3 — Explanation Utility

Implement a simple binary classifier. For 50 test cases, have three evaluators assess each case under three conditions: (a) no explanation, (b) LIME explanation, (c) SHAP explanation. Calculate Human Utility for each explanation type. Which performs best? Are there systematic differences based on evaluator data literacy?

Exercise 6.4 — Cognitive Load Reduction

Choose an annotation task requiring at least 5 clicks per item. Redesign the interface to reduce clicks to 2 or fewer while maintaining all necessary data capture. Using the cognitive fatigue model, estimate the expected change in the session quality threshold crossing point n_{break} for your redesign.

Exercise 6.5 — Priority Queue Design

A content moderation queue contains 10,000 items. You have 5 reviewers and 8 hours. Using the $\text{Priority}(x)$ formula, design a routing system. How do you estimate $U_{\text{model}}(x)$ and $S(x)$ in practice? How do you choose weights? Conduct a capacity analysis: how many items can be reviewed, and what fraction of the highest-priority items will receive human review?

This appendix provides the formal foundations for the interface design principles introduced in Chapter 6. The cognitive load model, automation bias framework, and explanation utility metric together constitute a quantitative toolkit for evaluating and improving HITL interfaces in production systems.

7 Getting Good Information from People

Human labels are the foundation of supervised learning—but labels are not neutral. Every annotation is an act of interpretation, shaped by who is labeling, what instructions they received, and what world they live in.

— The premise of Chapter 7

7.1 The Photograph That Changed How We See Data

In 2010, a Stanford researcher named Fei-Fei Li stood in front of an audience at a TED conference in Long Beach and explained what she had spent four years building: ImageNet. The project, completed using Amazon Mechanical Turk, had produced 15 million photographs organized into 22,000 categories. It was the largest labeled image dataset ever assembled, and it would become the training ground for the deep learning revolution that followed (Deng et al., 2009).

By 2019, a series of studies had identified something troubling in the data. The category "cheerleader" contained photographs that were overwhelmingly of women. The category "guard" was overwhelmingly men. The annotations were accurate in a narrow sense—the photographs did show what they were labeled as—but the dataset had encoded cultural

stereotypes so thoroughly that models trained on it would confidently predict "nurse" for women and "doctor" for men when given ambiguous inputs (Yang et al., 2018).

The labels were not wrong. They were incomplete descriptions of a biased sample, faithfully transcribed into a format that the model interpreted as ground truth.

The ImageNet case illustrates something that the whole field of machine learning spent the 2010s slowly learning: **labels are not neutral**. Every label is an act of interpretation, shaped by who is doing the labeling, what instructions they were given, what cultural context they bring, and how tired they are. The data annotation problem is not a logistical problem that can be solved by deploying enough workers. It is an epistemological problem—a problem about how we know what we know, and what it means to reduce the complexity of a photograph, a sentence, or a human interaction to a single numerical label.

7.2 The Instruction Problem

Imagine you are about to annotate a set of online comments for toxicity. You have been given a single instruction: "Label each comment as TOXIC or NOT TOXIC."

You see this comment: *"I can't believe you people still think this is acceptable. You're all completely delusional."*

Is it toxic? Probably yes, you think.

Now imagine the same comment, but a second annotator is working with a slightly expanded instruction: "Label each comment as TOXIC or NOT TOXIC. Toxicity means explicit slurs, direct threats, or content that would make a member of a protected group feel physically unsafe."

The second annotator looks at the same comment. Contemptuous? Yes. An explicit slur, a direct threat, or content likely to make a protected group feel physically unsafe? Not exactly. They label it NOT TOXIC.

Same comment. Different instructions. Different labels.

This is what annotation researchers call the **instruction problem**, and it is pervasive. A study by Cer and colleagues at Google examined how different framings of the same toxicity task produced dramatically different label distributions even when the underlying comments were identical. The framing "Is this comment hateful?" produced more toxic labels than the framing "Would a reasonable person find this comment toxic?" The second framing invokes a hypothetical external standard that moderates individual variation—it asks the annotator to model a composite person rather than report their own reaction.

The instruction problem compounds when guidelines are long. Professional annotation companies routinely produce annotation guidelines that run to 50, 80, even 120 pages for complex labeling tasks. Annotators working from a 100-page guideline develop a mental model of the guideline that is partly shared and partly individual. The shared part produces consistent labels. The individual part produces the kind of systematic variation that shows up in low inter-annotator agreement scores.

7.3 Measuring Agreement: Cohen's Kappa

When two annotators label the same set of items, their agreement is measurable. The simplest measure is raw agreement—the proportion of items they label the same way. But raw agreement is inflated by chance agreement: if 90% of items are NOT TOXIC and both annotators are just labeling everything NOT TOXIC, they will agree 81% of the time by chance, while making no genuine classification decisions.

The standard correction is **Cohen's kappa**, developed by psychologist Jacob Cohen in 1960:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o is the observed agreement and P_e is the expected agreement by chance. A kappa of 1.0 means perfect agreement; 0 means agreement no better than chance; negative values mean systematic disagreement.

The Landis-Koch Interpretation Scale

- $\kappa < 0.20$: Slight agreement
- $0.20 \leq \kappa < 0.40$: Fair agreement
- $0.40 \leq \kappa < 0.60$: Moderate agreement
- $0.60 \leq \kappa < 0.80$: Substantial agreement
- $\kappa \geq 0.80$: Almost perfect agreement

These thresholds are conventional, not theoretically grounded. Always interpret kappa in the context of the specific task.

When the toxicity detection field began systematically measuring annotator agreement, the results were sobering. A comprehensive study by Zampieri and colleagues in 2019 found that annotation kappa for English-language toxicity tasks varied between 0.45 and 0.72 depending on the task framing and annotator pool (Zampieri et al., 2019). The lower end—"moderate agreement"—meant that two annotators looking at the same comment would disagree about a quarter of the time. A model trained on such data is not learning to detect toxicity; it is learning to predict the

label distribution of a specific group of annotators working under specific conditions.

7.4 The Annotator Is Not a Recording Device

There is a useful fantasy in machine learning that annotators are passive transcribers—that they observe some objective feature of the data and record it, and that the label accurately reflects something real in the world. The fantasy is comfortable because it preserves the idea that labels are ground truth.

It is false.

An annotator is a person with a history, a worldview, a cultural context, and a present state that varies by hour of day, level of fatigue, number of items reviewed, and recent sequence of items seen.

Sequence effects. An annotator who has just reviewed 20 highly toxic comments will have a different calibration threshold for the next borderline comment than an annotator who has just reviewed 20 benign ones. Studies by Callison-Burch at the University of Pennsylvania, examining machine translation quality annotation, showed that annotators systematically rated the same translations differently depending on the quality of the surrounding translations in their queue (Callison-Burch, 2010). The translation had not changed. The annotator's reference point had.

Demographic effects. Research by Sap and colleagues at Carnegie Mellon found that Black annotators and white annotators differed substantially in their labeling of tweets containing African American Vernacular English (AAVE). Content in AAVE—including language that was neutral or positive in context—was more likely to be labeled as toxic when the annotator pool was predominantly white (Sap et al., 2019). The consequence is that a

model trained on annotations from a predominantly white annotator pool will have a systematically higher false toxicity rate for AAVE.

> This is not a problem that better annotation guidelines can fully fix, because it is embedded in who the annotators are and what cultural context they bring to the task. Diversifying the annotator pool is a practical necessity, not merely a fairness gesture.

7.5 The Economics of Getting Labels

The ImageNet dataset was completed with the help of Amazon Mechanical Turk, a crowdsourcing platform that allows requesters to post small tasks and have them completed by a globally distributed workforce for small payments. A typical toxicity labeling task on Mechanical Turk pays $0.02 to $0.05 per item.

The economic logic of crowdsourcing annotation is compelling. A dataset that would cost hundreds of thousands of dollars to label through professional linguists can be labeled for tens of thousands of dollars through Mechanical Turk, at higher speed.

The quality control problem is also real. Without oversight, crowdsourced annotation produces noise: workers who click randomly, who misread instructions, whose cultural context does not match the task requirements. The standard approach—multiple annotators per item plus majority voting—reduces individual noise. But majority voting on disagreement can enshrine the biases of the majority as ground truth.

The distinction between **noise disagreement** and **genuine ambiguity** matters enormously. If two annotators disagree because one was not paying attention, the disagreement is noise and should be resolved by majority vote. If two annotators disagree because the item is genuinely

ambiguous, the disagreement is information. Collapsing it to a single label loses the information.

Label Distribution Training

Some modern machine learning practice has begun to work with label distributions rather than single labels—training on the full distribution of annotator judgments rather than the majority vote. Research by Davani and colleagues demonstrated that models trained on distributions were better calibrated: they expressed more uncertainty on genuinely ambiguous items, and more confidence on items where annotators had agreed. This is a more honest representation of what the data actually contains (Davani et al., 2022).

7.6 Framing Effects: How the Question Changes the Answer

Framing effects describe the phenomenon that how a question is posed shapes the answer, independently of what the answer "truly" is. Amos Tversky and Daniel Kahneman showed that a medical treatment described as having a "90% survival rate" was viewed more favorably than the same treatment described as having a "10% mortality rate." Same statistic. Very different responses (Tversky & Kahneman, 1981).

The same dynamic appears in annotation. Consider the difference between:

Version A: "Rate each response on a scale of 1–5 for helpfulness."

Version B: "Consider: would a patient receiving this information be better equipped to make decisions about their health? Rate 1–5."

Version B anchors the annotator on a concrete downstream beneficiary and asks them to simulate that person's experience. This produces more

consistent labels across annotators and labels that better predict actual user satisfaction. The question is not whether Version B is more "correct." It is that Version B more precisely specifies what the annotation is supposed to measure.

The most consequential framing decisions in annotation are often invisible because they are embedded in what is *not* included in the instructions. Medical image annotation typically asks annotators to identify the presence or absence of a finding, not to classify its clinical significance. A model trained on such annotations learns to detect findings, not to assess their relevance to patient management—a critical difference when the model is deployed to assist diagnosis.

7.7 Calibration Sessions and the Iterative Guideline

The annotation community has developed practices to address the instruction problem, the most important of which is the **calibration session**.

In a calibration session, a group of annotators reviews and discusses a set of carefully selected items together. The items are chosen to cover the range of cases that arise in the task, including clear cases, borderline cases, and cases that expose ambiguities in the annotation guidelines. Annotators record their labels independently, then compare and discuss disagreements.

The output is twofold: an updated annotation guideline that addresses the exposed ambiguities, and a shared mental model that enables more consistent application going forward. Companies like Scale AI and Appen conduct formal calibration sessions at the start of annotation projects and regularly thereafter, particularly when the task involves subjective judgment or cultural nuance.

OpenAI's work on RLHF (Reinforcement Learning from Human Feedback) is explicit about this iterative process. The human feedback that

shapes RLHF training is not collected once and used permanently; it is continuously re-evaluated, guidelines are updated, and new annotation rounds are conducted to capture changing standards (Ouyang et al., 2022). The annotation is not a solved problem but an ongoing conversation between the model's behavior and human preferences.

7.8 What Low Agreement Tells You

When inter-annotator agreement on a task is low—say, kappa below 0.40—the conventional response is to interpret this as a quality problem. The solution is to revise the guidelines and improve the training.

This interpretation is sometimes correct. But low agreement can also be a signal about the *nature of the task itself*. Some annotation tasks ask annotators to make distinctions that are genuinely difficult, genuinely subjective, or genuinely contested. A task asking annotators to distinguish between "mildly offensive" and "moderately offensive" is asking for a discrimination that may not be reliably present in human judgment, not because the annotators are failing but because the categories do not map onto stable psychological reality.

When low agreement persists despite careful guidelines, calibrated training, and expert annotators, the right response is often not to force agreement but to interrogate the task. Is the distinction the annotation is trying to capture actually important for the downstream use case? Could the model perform adequately with coarser categories where agreement is higher? Is the genuine ambiguity in the data actually carrying information that should be preserved?

A Low Kappa Score Is Telling You Something Real

In the most honest framing: a low kappa score is telling you something real about the data, about the task, and about the limits of reducing human judgment to a scalar label. The answer to that signal is not always to try harder. Sometimes it is to listen.

Take any three subjective opinions you hold—about a film, a piece of writing, a public figure's behavior. For each one, write down the implicit criteria you are using to form the opinion. Now try to write those criteria as annotation guidelines, clearly enough that a stranger could apply them reliably. Notice how quickly "good" requires unpacking, how "harmful" requires definition, and how much of your judgment is tacit knowledge that resists specification. This is the annotation problem, and it does not have a clean solution.

Chapter Summary

Key Concepts

- Labels are not neutral—every annotation is an act of interpretation shaped by who is labeling, what instructions they received, and what cultural context they bring
- The instruction problem: even small differences in guideline wording produce systematically different label distributions

- **Cohen's kappa** measures inter-annotator agreement corrected for chance; low kappa can signal poor guidelines, genuine task ambiguity, or both
- Framing effects show that the question shape changes the answer, independently of what is being classified
- The annotator is not a recording device—sequence effects, demographic background, and fatigue all systematically shape labels

Key Examples

ImageNet (2010–2019) Cultural stereotypes baked into labels by an annotation process that faithfully transcribed annotator worldview as ground truth

AAVE toxicity labeling Demographically homogeneous annotator pools produce systematically biased labels for minority language registers

RLHF human feedback (OpenAI) Iterative guideline revision as a necessary feature of annotation for AI training

Scale AI and Appen calibration sessions The standard industry practice for reducing annotation inconsistency

Label distribution training Training on the full annotation distribution rather than majority-vote labels produces better-calibrated models

In the next chapter, we zoom out from individual interactions—interfaces, labels, human judgments—and look at the HITL pipeline as a whole: the data flows, the engineering challenges, and what happens when a system that worked beautifully in the lab breaks down in production.

Technical Appendix 7A: Annotation Metrics and Label Quality

The annotation challenges described in Chapter 7 can be formalized using agreement statistics, Bayesian aggregation models, and signal detection theory.

— Formal foundations for the examples in Chapter 7

7.9 Cohen's Kappa: Derivation and Properties

7.9.1 Basic Formula

For two annotators labeling N items into k categories, let n_{ij} be the number of items where annotator 1 assigned category i and annotator 2 assigned category j.

$$P_o = \frac{\sum_i n_{ii}}{N} \quad \text{(observed agreement)}$$

$$P_e = \sum_i \frac{n_{i\cdot}}{N} \cdot \frac{n_{\cdot i}}{N} \quad \text{(expected chance agreement)}$$

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where $n_{i\cdot} = \sum_j n_{ij}$ is the row marginal and $n_{\cdot i} = \sum_j n_{ji}$ is the column marginal.

7.9.2 Variance and Confidence Intervals

$$\mathrm{Var}(\kappa) \approx \frac{P_o(1 - P_o)}{N(1 - P_e)^2}$$

Approximate 95% confidence interval: $\kappa \pm 1.96\sqrt{\mathrm{Var}(\kappa)}$

> This variance formula is a simplified approximation that ignores variance from estimating P_e itself, producing confidence intervals that are slightly too narrow—especially when P_e is large or N is small. For publication-quality intervals use the exact formula from Fleiss (1971) or a bootstrap estimate.

For reliable kappa estimates, require $N \geq 30$ items per annotator pair.

7.9.3 Weighted Kappa for Ordinal Categories

With linear weights $w_{ij} = 1 - |i - j|/(k - 1)$ or quadratic weights $w_{ij} = 1 - [(i - j)/(k - 1)]^2$, define weighted observed and expected agreement:

$$P_o^w = \sum_{ij} \frac{w_{ij}\, n_{ij}}{N}, \qquad P_e^w = \sum_{ij} w_{ij}\, P_{e,ij}, \quad P_{e,ij} = \frac{n_{i\cdot}}{N} \cdot \frac{n_{\cdot j}}{N}$$

Weighted kappa follows the same structure as unweighted kappa (Cohen, 1968):

$$\kappa_w = \frac{P_o^w - P_e^w}{1 - P_e^w}$$

Quadratic weights are standard for medical rating scales; linear weights for quality assessments.

7.10 Fleiss' Kappa: Extension to Multiple Annotators

For M annotators labeling N items into k categories, let n_{ij} be the number of annotators who assigned item i to category j:

$$\bar{P} = \frac{1}{NM(M-1)} \sum_i \sum_j n_{ij}(n_{ij} - 1) \qquad \bar{P}_e = \sum_j \left(\frac{\sum_i n_{ij}}{NM} \right)^2$$

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

Table 7.1: Agreement benchmarks by annotation domain

Domain	Typical Range	Target
Binary toxicity (general)	0.45–0.65	≥ 0.60
Medical image (radiology)	0.55–0.75	≥ 0.75
Sentiment analysis (clear)	0.70–0.85	≥ 0.70
Named entity recognition	0.80–0.95	≥ 0.85
Legal document classification	0.50–0.70	≥ 0.65
RLHF instruction quality	0.55–0.75	≥ 0.65
AAVE toxicity (diverse pool)	0.40–0.55	≥ 0.60

7.11 Annotation as a Bayesian Inference Problem: Dawid-Skene

7.11.1 Generative Model

The Dawid-Skene model (Dawid & Skene, 1979) treats annotator labels as noisy observations of a true latent label, estimating both true labels and annotator error rates jointly.

For item i, annotator j, and observed label y_{ij}:

$$P(y_{ij} = l \mid z_i = k) = \pi_{jkl}$$

where z_i is the true label for item i and π_{jkl} is annotator j's probability of labeling as l when the true label is k.

7.11.2 EM Algorithm

E-step. Compute posterior probability of true label:

$$P(z_i = k \mid \mathbf{y}_i) \propto p_k \prod_j \prod_l \pi_{jkl}^{\mathbf{1}[y_{ij}=l]}$$

M-step. Update annotator error rates and class priors:

$$\hat{\pi}_{jkl} = \frac{\sum_i P(z_i = k) \cdot \mathbf{1}[y_{ij} = l]}{\sum_i P(z_i = k)} \qquad \hat{p}_k = \frac{1}{N} \sum_i P(z_i = k)$$

```python
def dawid_skene_em(annotations, n_classes, n_iters=50):
    """
    annotations: dict {item_id: {annotator_id: label}}
    Returns: estimated true label posteriors T, annotator
        conf. matrices pi
    """
    N, K = len(annotations), n_classes
```

```
    items = list(annotations.keys())
    annotators = list(set(a for item in annotations.values() for a in item))
    M = len(annotators)

    # Initialize T with majority vote
    T = np.zeros((N, K))
    for i, item in enumerate(items):
        for v in annotations[item].values():
            T[i, v] += 1
        T[i] /= T[i].sum()

    pi = np.ones((M, K, K)) / K

    for _ in range(n_iters):
        # M-step: update pi
        for j, ann in enumerate(annotators):
            for k in range(K):
                for l in range(K):
                    num = sum(T[i,k] for i,item in enumerate(items)
                              if ann in annotations[item]
                              and annotations[item][ann] == l)
                    den = sum(T[i,k] for i,item in enumerate(items)
                              if ann in annotations[item])
                    pi[j,k,l] = num / (den + 1e-10)

        prior = T.mean(axis=0)

        # E-step: update T
        log_T = np.log(prior[np.newaxis,:] + 1e-10) * np.ones((N,K))
```

```
        for i, item in enumerate(items):
            for j, ann in enumerate(annotators):
                if ann in annotations[item]:
                    l = annotations[item][ann]
                    log_T[i] += np.log(pi[j,:,l] + 1e-10)

        log_T -= log_T.max(axis=1, keepdims=True)
        T = np.exp(log_T)
        T /= T.sum(axis=1, keepdims=True)

    return T, pi
```

Listing 7.1: Dawid-Skene EM (simplified)

7.12 Framing Effects: Signal Detection Theory Framework

Model annotation as a signal detection problem. The annotator observes item x with latent "harmfulness" signal $s(x) \in \mathbb{R}$, and classifies by comparing to threshold θ:

$$\hat{y} = \begin{cases} 1 & s(x) > \theta \\ 0 & s(x) \leq \theta \end{cases}$$

The annotation guideline framing shifts θ for individual annotators:

$$\theta_A = \theta_0 + \Delta_A$$

Inter-annotator agreement is a function of the variance in individual thresholds:

$$\kappa \approx 1 - f(\sigma_\theta^2)$$

Framing effects that reduce σ_θ^2 (by more precisely specifying the threshold) directly increase κ. The "reasonable person" standard reduces idiosyncratic threshold variation at the cost of introducing shared cultural blind spots.

7.13 Sample Size Planning for Annotation Projects

7.13.1 Statistical Power for Kappa Estimation

To estimate kappa with margin of error e at confidence level α:

$$N \geq \frac{z_{\alpha/2}^2 \cdot P_o(1 - P_o)}{e^2(1 - P_e)^2}$$

For typical annotation settings ($P_o \approx 0.80$, $P_e \approx 0.50$, $e = 0.05$, $\alpha = 0.05$):

$$N \geq \frac{3.84 \times 0.16}{0.0025 \times 0.25} \approx 983 \text{ items}$$

7.13.2 Redundancy and Quality Trade-off

Expected kappa improvement from n_r annotators per item with majority-vote aggregation:

$$\kappa(n_r) \approx \kappa_1 + (1 - \kappa_1) \cdot \left(1 - e^{-\lambda(n_r - 1)}\right)$$

where κ_1 is single-annotator kappa and $\lambda \approx 0.3$. Increasing redundancy beyond $n_r = 5$ yields diminishing returns; the marginal quality gain from a 6th annotator is typically less than 0.01 kappa.

The value $\lambda \approx 0.3$ is an empirical heuristic derived from annotation studies across multiple NLP and CV tasks; it is not derived from first principles. In practice, λ varies by task difficulty and annotator pool. Treat the formula as a planning heuristic, not a precise prediction, and calibrate against pilot annotation data for any specific project.

7.14 Exercises and Assignments

Exercise 7.1 — Cohen's Kappa Computation

Two annotators classify 100 social media posts:

	Ann. 2: T	Ann. 2: N
Ann. 1: T	20	15
Ann. 1: N	10	55

Compute P_o, P_e, κ, and a 95% confidence interval. Interpret using the Landis-Koch scale and recommend whether to proceed with model training.

Exercise 7.2 — Fleiss' Kappa Properties

Three annotators each label 50 documents. Compute Fleiss' kappa and compare to average pairwise Cohen's kappa. Under what conditions do they diverge most significantly? Construct a 3-annotator example where average pairwise kappa is misleadingly optimistic.

Exercise 7.3 — Dawid-Skene Simulation

Simulate annotation data for 100 items with two expert annotators (error rate 5%) and eight novice annotators (error rate 30%). Compare accuracy of majority voting vs. Dawid-Skene EM vs. expert-only majority vote. How much does including novice annotators help or hurt under each method?

Exercise 7.4 — Framing Effect Experiment

Design an experiment to measure framing effects on a toxicity annotation task. Specify: two guideline versions, number of items, number of annotators per condition, statistical tests for kappa difference and positive label rate difference, and expected results based on the signal detection model.

Exercise 7.5 — Annotation Budget Optimization

Budget: \$5,000. Task: binary annotation, minimum 2,000 items. Expert: \$2.00/label, $\kappa_1 = 0.75$. Crowdworkers: \$0.05/label, $\kappa_1 = 0.50$. Design three annotation strategies. For each: estimate expected quality (using the redundancy formula), coverage, and total cost. Which strategy do you recommend, and under what conditions would you choose differently?

This appendix provides the formal tools for quantifying annotation quality and diagnosing label problems described in Chapter 7. The kappa family

of statistics, the Dawid-Skene model, and the signal detection framework together constitute the annotation quality toolkit for applied HITL system development.

8 The Blueprint for Smart Helper Systems

> *System architecture for HITL—how the pieces fit, where they break, and what it takes to keep them together.*

8.1 The Alarm Nobody Heard

In 2018, an Uber self-driving test vehicle struck and killed Elaine Herzberg in Tempe, Arizona. She was walking her bicycle across a multi-lane road at night. The onboard systems detected her 5.6 seconds before impact. They classified her presence with high uncertainty—first as "unknown object," then as "vehicle," then as "bicycle." The system was uncertain. The uncertainty was detected. Then the system made a fateful choice: to classify the object as a vehicle that was stationary.

At that classification, the software determined that a safety stop was not needed. One second before impact, the system recognized it needed to stop. By then, it was too late.

The backup safety driver, meanwhile, had not been watching the road.

Here is what makes this a *system architecture failure* rather than a sensor failure or a human failure: both the algorithm and the human were in the loop. Both failed. And the system's architecture—the way the two were connected, the expectations placed on each, the escalation

rules that governed handoffs—was designed without sufficient attention to how human and algorithmic components interact under real operating conditions.

The human was supposed to be in the loop. She was sitting in the driver's seat. But the system had been designed in a way that made meaningful human oversight essentially impossible: the safety driver was expected to monitor continuous operation but had no clear trigger for when to engage. The uncertainty signal that the software was generating was not communicated to her in any actionable form. The "human in the loop" was a checkbox, not a design.

This is the architecture problem.

8.2 The Pipeline Is Not the Loop

Before we discuss what HITL system architecture should look like, it is worth naming the failure mode it is designed to avoid: treating a human-in-the-loop system as a **pipeline** with a human at one end.

A pipeline is a sequence of steps. Data goes in, processing happens, output comes out. In a pipeline with a human step, the human reviews something and passes it along. The human's role is fixed, their inputs are fixed, their outputs are fixed. There is no feedback. There is no learning.

A loop is different. In a loop, the outputs from the human step flow back into the system. The human's responses change how the system behaves in the future. The system is designed to improve based on human feedback. And the human is designed to be supported—with good interfaces, good context, appropriate caseload—so their feedback is worth incorporating.

Most systems that claim to be "human-in-the-loop" are actually pipelines. The human step exists but contributes nothing to system learning. The model is static. The human is a quality control check, not a contributor to the system's intelligence.

Real HITL architecture requires several components that a simple pipeline does not have.

8.3 The Components of Real HITL Architecture

The Prediction Layer. The machine learning model that processes inputs and produces outputs with uncertainty estimates. The key architectural requirement is that it produces not just predictions but **confidence signals**—calibrated probabilities that the downstream routing layer can act on.

The Uncertainty Quantification Layer. A separate component (or integrated capability) that computes calibrated uncertainty for each prediction. This may be as simple as temperature-scaled softmax outputs, or as complex as a separate ensemble model for uncertainty estimation (Lakshminarayanan et al., 2017). The key is that the output is a calibrated confidence score, not a raw logit.

The Routing Layer. The decision logic that determines what happens to each prediction. Above the high-confidence threshold: automatic action. Below the low-confidence threshold: automatic action in the cautious direction. In between: route to human review. This is where the threshold band design from Chapter 4 is implemented.

The Interface Layer. The system through which human reviewers receive and respond to cases. This is more than a display—it is the context provision, the question framing, the response format, and the ergonomics that determine whether human feedback is useful or garbage. Chapter 5 covers this in depth. Architecturally, the key requirement is that it can

present information appropriately for different case types and capture structured responses that can be fed back into training.

The Feedback Integration Layer. The mechanism by which human responses improve the system. This may be a retraining pipeline, an active learning module, a rule update system, or a calibration recalibration component. Without this layer, the "loop" in HITL is not a loop—it is an open-ended pipeline.

The Monitoring Layer. Ongoing tracking of system behavior—prediction distributions, human response patterns, accuracy metrics, throughput, reviewer load, calibration drift. This is the component that keeps the system honest over time, detecting when things are drifting in ways that require intervention (Beyer et al., 2016).

The Audit Layer. Logging of decisions, escalations, human responses, and outcomes. This is what makes the system accountable: a record of what was decided, why, by whom (human or machine), and what happened next. In regulated industries, this is often legally required. In all industries, it is practically required for debugging and improvement.

8.4 The Feedback Loop: From Human Response to Model

The most important and most underbuilt component is **feedback integration**. This is where the loop actually closes—where the system gets smarter over time because of what humans tell it.

Several architectures exist, ranging from simple to sophisticated.

Batch retraining. Human labels are accumulated over a period (day, week, month), quality-checked, and added to the training dataset. The model is retrained periodically on the augmented dataset. Simple, robust, but slow—the model does not improve from this morning's reviews until next month's retraining.

Online learning. Human labels are incorporated immediately into model updates. Faster learning, but also faster propagation of errors if human quality drops. Requires careful quality monitoring of the incoming labels.

Active learning queues. The routing layer preferentially sends to human review the cases where the model would benefit most from a label—not just the uncertain cases, but the uncertain cases in underrepresented regions of the training distribution. This is an intelligent version of batch retraining where human effort is directed where the learning value is highest. Chapter 17 covers active learning in more depth.

RLHF. **Reinforcement Learning from Human Feedback** (RLHF) is a more sophisticated architecture used in large language models where human preferences (expressed as comparisons between outputs) are used to train a reward model, which is then used to fine-tune the generative model (Christiano et al., 2017). This is the architecture behind ChatGPT and similar systems (Ouyang et al., 2022). The architectural novelty is that humans do not label correct answers—they express preferences between alternatives, which is often easier and cheaper.

8.5 Logging, Monitoring, and Drift Detection

One of the most consistent failure modes in production HITL systems is drift without detection. The system performs well at launch, degrades slowly over the following months, and the degradation is not noticed until a visible failure—a bad press story, a complaint escalation, an audit finding—prompts a review.

Good HITL architecture prevents this with systematic monitoring.

Prediction monitoring. Track the distribution of confidence scores, label predictions, and model outputs over time. A shift in any of these distributions is a signal that something has changed—either the model's behavior or the input distribution.

Human response monitoring. Track human reviewer agreement rates, time-per-review, and the rate at which human reviewers override automated decisions. Declining agreement rate or drastically faster reviews often signal reviewer fatigue or training distribution change. Rising override rates suggest the model is performing worse, or that the reviewers have developed new judgment that the model has not yet learned.

Outcome monitoring. Where outcomes are observable (fraud actually occurred or did not, content actually caused reported harm or did not), track whether the system's predictions corresponded to reality. This is the ground truth check. It requires outcome labels, which are often delayed and incomplete—but even partial outcome monitoring is valuable.

Calibration monitoring. Periodically recompute the Expected Calibration Error (ECE) on held-out samples with known outcomes. If calibration is drifting, the routing rules need to be recalibrated.

8.6 Where Things Break in Production

Real HITL systems break in predictable ways.

Reviewer bottleneck The volume of cases routed to human review exceeds reviewer capacity. Reviewers speed up, quality drops, the feedback signal degrades. Downstream: model does not improve, error rates stay high.

Label quality degradation As a task becomes routine, reviewer attention decreases. Labels become less careful. Training data quality drops. Model performance plateaus or regresses.

Feedback lag Human labels from today do not reach the model for weeks or months. By the time the model learns from today's edge cases, those edge cases are no longer edge cases—the distribution has moved.

Distribution shift without detection The real-world data changes faster than the monitoring system detects. The model is making decisions on a distribution it was not trained on, but the monitoring dashboard still shows "all green" because it is tracking the wrong metrics.

The ghost reviewer A reviewer position that exists on paper but is chronically understaffed or filled by someone with insufficient training. The system reports that human oversight is in place. The reality is that oversight is nominal.

Specification gaming The model learns to game the metrics it is being evaluated on rather than learning the true task. If

> the feedback loop is measuring the wrong thing, the model will optimize the wrong thing.

8.7 The Lifecycle: From Prototype to Deployed System

HITL systems evolve over time, and their architecture needs to evolve with them.

Phase 1—Human-heavy prototype. Almost all cases go to human review. The model exists but has thin training. This is the data collection phase—the human labels are the product, not the model outputs.

Phase 2—Hybrid operation. The model handles high-confidence cases automatically. Human review is concentrated on uncertain cases. The model is updated regularly on reviewed cases. This is the steady state for most production HITL systems.

Phase 3—Model-dominant. As training data accumulates, the model becomes more capable. Fewer cases require review. Human review concentrates on genuinely hard cases and edge cases. The human role becomes more expert and less routine.

Phase 4—Monitoring-dominant. The model handles most cases well. Human involvement shifts to monitoring, auditing, and handling edge cases and appeals. The architecture focuses on drift detection and outcome monitoring rather than case-by-case review.

Each phase has different architectural requirements. A system optimized for Phase 1 data collection will need significant changes to serve

well in Phase 3 model-dominant operation. Planning for these transitions is part of good HITL architecture design.

Sketch the HITL architecture for a system you interact with regularly—your email spam filter, your bank's fraud detection, a recommendation system. Where is the prediction layer? Where is the routing layer? Where is the interface? And most importantly: can you identify a feedback integration layer? If you cannot find the feedback loop, what does that tell you about whether the system is actually designed to improve?

Chapter Summary

Key Concepts

- HITL architecture requires a loop, not just a pipeline—human responses must flow back into the system
- Seven components: prediction, uncertainty quantification, routing, interface, feedback integration, monitoring, and audit layers
- Feedback integration can be batch retraining, online learning, active learning, or RLHF—each with different latency and quality tradeoffs
- Production HITL systems break in predictable ways: reviewer bottleneck, label quality degradation, feedback lag, distribution shift without detection

- The system lifecycle has distinct phases (human-heavy → hybrid → model-dominant → monitoring-dominant), each requiring different architecture

Key Examples

Uber Tempe incident (2018) A human-in-the-loop that was not—a pipeline with a human placeholder; uncertainty was detected but not communicated to the safety driver

Active learning queues Intelligent routing for maximum learning value, not just uncertainty-based routing

RLHF in language models Preference-based feedback as an alternative to label-based feedback; the architecture behind ChatGPT and similar systems

Five Dimensions Check

- *Feedback Integration* (Dimension 5): The core subject of this chapter
- *Uncertainty Detection* (Dimension 1): Required for the routing layer to function
- *Stakes Calibration* (Dimension 4): Informs the routing thresholds

---

Next chapter: a practical guide to the tools anyone can use—from annotation platforms to cloud-based HITL services to open-source options for getting started with real data.

Technical Appendix 8A: HITL System Architecture Patterns

Chapter 8 introduced the blueprint for smart helper systems: routing, queuing, monitoring, and feedback. This appendix provides formal definitions, complete reference implementations, and production considerations for each component.

— Formal foundations for Chapter 8

8.8 The Pipeline vs. Loop Distinction

A **HITL Pipeline** is a directed acyclic graph $G = (V, E)$ where: V includes exactly one human review node H; H receives input from a model node M; H produces output consumed by a downstream process; and no path exists from any downstream node back to M.

A **HITL Loop** is a directed graph $G = (V, E)$ where: V includes at least one human review node H; and a path exists from H back to some component that influences future model behaviour.

The loop property is necessary but not sufficient for a learning HITL system. The path from H back to the model must also satisfy:

1. Human labels are available for integration into training.
2. Labels are quality-filtered before integration.

3. The training process is actually triggered periodically.

> A common failure mode is a nominally "looping" HITL system whose feedback queue is never processed. The system collects human labels, stores them, and retrains on schedule—except the schedule was never activated in production. Verify that the retraining job is running, not just that the queue is filling.

8.9 The Three-Zone Router

The core routing component partitions incoming cases into three zones: auto-positive, auto-negative, and human review, with dynamic threshold adjustment based on reviewer capacity.

```python
from dataclasses import dataclass
from enum import Enum
from typing import Optional, Callable

class Disposition(Enum):
    AUTO_POSITIVE = "auto_positive"
    AUTO_NEGATIVE = "auto_negative"
    HUMAN_REVIEW  = "human_review"

@dataclass
class RoutingConfig:
    tau_low:           float     # Below this: auto-negative
    tau_high:          float     # Above this: auto-positive
    capacity_fn:       Callable  # Returns current reviewer capacity [0,1]
    overflow_narrowing: float = 0.05
    underflow_widening: float = 0.03
```

```python

class ThreeZoneRouter:
    def __init__(self, config: RoutingConfig):
        self.config    = config
        self.tau_low   = config.tau_low
        self.tau_high  = config.tau_high

    def route(self, score: float,
              stakes: Optional[float] = None) -> Disposition:
        eff_high = self.tau_high
        eff_low  = self.tau_low
        if stakes is not None and stakes > 0.8:
            narrowing = 0.15
            eff_high = self.tau_high - narrowing
            eff_low  = self.tau_low  + narrowing
        if score >= eff_high:
            return Disposition.AUTO_POSITIVE
        elif score <= eff_low:
            return Disposition.AUTO_NEGATIVE
        else:
            return Disposition.HUMAN_REVIEW

    def adjust_for_capacity(self):
        capacity = self.config.capacity_fn()
        if capacity < 0.3:   # Overloaded: narrow the review band
            self.tau_low  = min(self.tau_low  + self.config.overflow_narrowing, 0.45)
            self.tau_high = max(self.tau_high - self.config.overflow_narrowing, 0.55)
        elif capacity > 0.8: # Underutilised: widen the band
            base_low  = self.config.tau_low
            base_high = self.config.tau_high
```

```
            self.tau_low  = max(self.tau_low  - self.config.underflow_widening,
                                base_low  - 0.15)
            self.tau_high = min(self.tau_high + self.config.underflow_widening,
                                base_high + 0.15)
```

Listing 8.1: Three-zone router with capacity adaptation

Stakes Override

The `stakes` parameter implements a high-stakes override: when a case is flagged as high-stakes (e.g., a medical decision, a financial transaction above a threshold), the review band narrows even if the model is moderately confident. This ensures high-consequence cases always pass through human review regardless of reviewer capacity.

8.10 Active Learning Queue Implementation

Cases routed for human review should be prioritised not by arrival order but by *learning value*: the expected improvement to the model from labelling this particular case. The implementation below combines uncertainty (proximity to the decision boundary) with diversity (dissimilarity from already-queued cases).

```
from dataclasses import dataclass, field
from typing import List
import numpy as np
from heapq import heappush, heappop, heapify, nsmallest

@dataclass(order=True)
class LearningQueueItem:
```

```
    learning_value: float
    score:    float      = field(compare=False)
    case_id:  str        = field(compare=False)
    features: np.ndarray = field(compare=False)

    def __post_init__(self):
        object.__setattr__(self, 'learning_value',
                           -abs(self.learning_value))  # max-heap

class ActiveLearningQueue:
    def __init__(self, uncertainty_weight=0.7,
                 diversity_weight=0.3, max_size=1000):
        self.heap = []
        self.uw   = uncertainty_weight
        self.dw   = diversity_weight
        self.max_size = max_size
        self._labeled = []

    def _uncertainty(self, score: float) -> float:
        return 1 - abs(2 * score - 1)  # peaks at 0.5

    def _diversity(self, features: np.ndarray) -> float:
        if not self._labeled:
            return 1.0
        labeled   = np.array(self._labeled)
        min_dist  = np.linalg.norm(labeled - features, axis=1).min()
        return min(min_dist / (min_dist + 1.0), 1.0)

    def add(self, case_id: str, score: float, features: np.ndarray):
        lv = self.uw * self._uncertainty(score) + \
             self.dw * self._diversity(features)
```

```
        heappush(self.heap, LearningQueueItem(lv, score,
                                              case_id,
                                                  features))
        if len(self.heap) > self.max_size:
            self.heap = nsmallest(self.max_size, self.heap)
            heapify(self.heap)

    def pop_top_k(self, k: int) -> List[LearningQueueItem]:
        results = []
        for _ in range(min(k, len(self.heap))):
            item = heappop(self.heap)
            results.append(item)
            self._labeled.append(item.features)
        return results
```

Listing 8.2: Active learning priority queue

8.11 Production Monitoring

A production HITL system requires continuous monitoring of three signals: *confidence distribution drift* (is the model seeing different inputs than it was trained on?), *human override rate* (are reviewers frequently disagreeing with automated decisions?), and *review rate* (is the fraction of cases reaching human review unexpectedly high?).

```
from collections import deque
from datetime import datetime
from typing import Dict, Optional
import numpy as np
from scipy import stats

class HITLMonitor:
    def __init__(self, reference_confidence_dist:
        np.ndarray,
```

```
                 window_size: int = 1000,
                 review_band: tuple = (0.3, 0.7),
                 alert_thresholds: Optional[Dict] = None):
        self.reference = reference_confidence_dist
        self.conf_window     = deque(maxlen=window_size)
        self.override_window = deque(maxlen=window_size)
        self.review_low, self.review_high = review_band
        self.thresholds = alert_thresholds or {
            'drift_pvalue':        0.01,
            'override_rate_high':  0.30,
            'review_rate_high':    0.60,
        }

    def record(self, score: float, disposition: str,
               human_override: Optional[bool] = None):
        self.conf_window.append(score)
        if human_override is not None:
            self.override_window.append(int(human_override))

    def check_health(self) -> Dict:
        if len(self.conf_window) < self.conf_window.maxlen // 2:
            return {'status': 'insufficient_data'}
        current = np.array(self.conf_window)
        ks_stat, p_value = stats.ks_2samp(self.reference, current)
        issues = {}
        if p_value < self.thresholds['drift_pvalue']:
            issues['distribution_drift'] = {
                'severity': 'HIGH',
                'ks_stat':  round(ks_stat, 4),
                'p_value':  round(p_value, 6),
            }
        if len(self.override_window) >= 50:
```

```
            override_rate =
                np.mean(list(self.override_window))
            if override_rate >
                self.thresholds['override_rate_high']:
                issues['high_override_rate'] = {
                    'severity':      'MEDIUM',
                    'override_rate': round(override_rate,
                        3),
                }
        review_rate = sum(self.review_low <= s <=
            self.review_high
                          for s in self.conf_window) \
                      / len(self.conf_window)
        if review_rate >
            self.thresholds['review_rate_high']:
            issues['high_review_rate'] = {
                'severity':    'LOW',
                'review_rate': round(review_rate, 3),
            }
        return {'status': 'issues_detected' if issues else
            'healthy',
                'issues': issues}
```

Listing 8.3: HITL system health monitor

The Kolmogorov–Smirnov test used above detects distributional drift in one-dimensional confidence scores. It does not detect drift in the high-dimensional feature space that produces those scores. A model may show stable confidence distributions while operating on inputs that have shifted significantly from its training distribution.

Complement confidence monitoring with feature-space monitoring using dimensionality reduction.

8.12 Exercises

Exercise 8.1 — Feedback Integration Layer

Implement a feedback integration layer that: ingests human labels from the review queue; quality-filters them using inter-annotator agreement scores; and triggers retraining when accumulated new labels exceed a threshold. What safeguards prevent bad labels from degrading the model?

Exercise 8.2 — Time-of-Day Threshold Adjustment

Extend the `ThreeZoneRouter` to implement time-of-day threshold adjustment: lower the review band during business hours (more capacity) and narrow it during nights and weekends. How would you test whether this adjustment improves overall system performance?

Exercise 8.3 — Ghost Reviewer Detection

Implement a detector for the "ghost reviewer" failure mode: a review queue that has been accumulating items without being processed. What metrics would indicate this? How would you design an alert that distinguishes reviewer absence from legitimate queue growth?

Exercise 8.4 — High-Dimensional Drift Detection

The monitor checks for distribution drift using a KS test on confidence scores. What are the limitations of this approach? Propose an alternative using dimensionality reduction of input features (e.g., UMAP or t-SNE on penultimate-layer activations). How would you set alert thresholds?

This appendix provides reference implementations for the core components of a production HITL architecture: routing, active learning queues, and system health monitoring, as introduced in Chapter 8.

9 Tools Anyone Can Use

> *A practical survey of annotation platforms, HITL services, and how to get started with real data.*

9.1 The Label Studio Moment

Somewhere in 2021, a solo developer named Nikolai Litvak published a blog post titled "How I labeled 50,000 images in a weekend." The post described his experience using Label Studio, an open-source annotation tool he had built and released a year earlier. He had expected it to be used by machine learning researchers at well-funded labs. Instead, he found his community forums filling with posts from a different audience: non-profits trying to label satellite images of deforestation, a hospital system annotating medical records, a small news organization classifying articles for a study of media coverage.

None of these organizations had data science teams in the traditional sense. None of them could afford enterprise annotation platforms. But they had data, they had a task, and they had people willing to do the labeling work. What they needed was infrastructure.

This is the story of the past decade in HITL tooling. The infrastructure—the platforms, the pipelines, the APIs, the hosted services—has become accessible to organizations that would have been entirely locked out five years earlier. The engineering sophistication required to build a human-

in-the-loop system from scratch has dropped dramatically. And with it, the range of people and institutions who can participate in building AI systems that use human judgment has expanded.

The tools are not magic. They have real limitations, and choosing the wrong tool for a task can cost more in rework than building the right thing from scratch. But the landscape is rich enough now that the question "what tool should I use?" has real answers.

9.2 What Annotation Tools Actually Do

Before surveying the landscape, a quick reminder of what these tools are for.

An annotation tool provides:

- A way to present data items (images, text, audio, video, structured records) to reviewers
- A way for reviewers to record their judgments in structured form
- A way to store those judgments and export them for use in training
- Usually: some way to manage the workflow—who reviews what, when, quality control

That is the core. Advanced tools add:

- Agreement tracking and quality metrics
- Active learning queues (routing cases by model uncertainty)
- Model-assisted annotation (the model pre-labels, humans correct)
- Analytics dashboards
- Multi-reviewer reconciliation

- Team management and access control

The choice of tool depends on your data type, your task complexity, your team size, your budget, and how much customization you need.

9.3 Open-Source Options

Label Studio is the most widely used open-source annotation platform as of 2024. It supports all major data types: text, images, audio, video, time series, and HTML. The configuration is JSON-based and relatively accessible. It has a Python SDK and REST API, active community development, and a commercial cloud offering for teams that want hosted infrastructure.

Best for: research projects, small-to-medium teams with technical capacity, organizations with privacy requirements that prevent cloud hosting.

Limitations: setup requires some engineering; advanced features (active learning integration, enterprise workflow management) require more work.

Argilla (formerly Rubrix) focuses specifically on NLP annotation—text classification, named entity recognition, question answering, and evaluation of language model outputs. It integrates well with the Hugging Face ecosystem and has strong support for RLHF-style feedback collection. It is notably good for LLM evaluation use cases.

Best for: NLP teams, LLM evaluation, RAG pipeline improvement.

CVAT (Computer Vision Annotation Tool) is purpose-built for computer vision tasks. Originally developed by Intel, it supports bounding boxes, polygons, keypoints, and semantic segmentation. It is the tool of choice for many computer vision annotation projects.

Best for: image and video annotation, object detection, segmentation tasks.

Prodigy is a commercial-licensed but developer-friendly tool from the creators of spaCy. It is designed for fast, scriptable annotation workflows with a particular emphasis on active learning integration. Prodigy's model-in-the-loop approach, where the model pre-annotates and humans correct, can dramatically reduce annotation time on NLP tasks.

Best for: NLP tasks where speed is critical and you have some technical capacity.

9.4 Cloud Platform Approaches

For organizations that want managed infrastructure rather than self-hosted tooling, several cloud platforms offer HITL services.

Scale AI is the most prominent enterprise annotation platform. It offers managed annotation workforces, quality controls, domain-specific pipelines (medical, legal, autonomous vehicle), and an API for integrating annotation into ML pipelines. Scale's managed workforce handles the recruitment, training, and quality management of annotators—a significant operational burden for organizations that do not want to manage their own annotation team.

Best for: large-scale annotation projects, organizations without in-house annotation capacity, high-stakes domains requiring managed quality control.

Amazon SageMaker Ground Truth integrates annotation directly into AWS's ML toolchain. It offers both Amazon Mechanical Turk (crowdsourced) and private workforce (your own team) options. It includes

built-in active learning through Ground Truth Plus, which automatically sends uncertain cases to human reviewers and uses confirmed labels to improve the model.

Best for: organizations already in the AWS ecosystem, projects that benefit from tight ML pipeline integration.

Google Vertex AI Data Labeling is Google Cloud's managed annotation service. Similar in positioning to SageMaker Ground Truth but within GCP. Tight integration with Vertex AI for active learning loops.

Labelbox is a mid-market platform with strong support for complex multi-step annotation projects, quality assurance workflows, and a notable focus on video annotation.

Best for: medium-to-large organizations with complex annotation workflows, teams that need strong quality assurance tooling.

9.5 Crowdsourcing vs. Expert Labeling

This is one of the most consistently misunderstood trade-offs in annotation work.

Crowdsourced annotation (Amazon Mechanical Turk, Scale AI's managed workforce, similar platforms) provides large-scale, fast, relatively low-cost annotation. The workforce is diverse and large. The limitation: for any task with significant domain requirements—medical text, legal documents, specialized imagery, culturally specific content—crowdworkers without that domain expertise produce noisy labels. Noise in training data is not random; it clusters around the hard cases that require domain judgment. These are often the most important cases.

Expert annotation (domain specialists—doctors, lawyers, linguists, security analysts) produces high-quality labels on hard cases. The limitation: expensive, slow, difficult to scale, and hard to find. Expert annotators are also not immune to bias—their expertise brings systematic viewpoints that may not reflect the diversity of real-world cases (Davani et al., 2022).

The practical resolution for most organizations: use the task to determine who labels it. Objectively determinable tasks (does this image contain a dog?) can use crowdsourcing successfully. Subjectively determinable tasks requiring domain judgment (does this symptom presentation suggest a particular diagnosis?) require experts. For tasks in between, consider a two-stage approach: crowd for initial filtering, experts for ambiguous cases.

Chapter 7 covers inter-annotator agreement in depth. Here, the practical tooling note: most annotation platforms support measuring inter-annotator agreement automatically (Krippendorff's α, Cohen's κ), and using agreement scores to route low-agreement items to additional review.

9.6 No-Code Options and Their Limits

Several platforms offer annotation workflows without any coding required.

Docanno and

Brat are simple text annotation tools requiring minimal technical setup. They are effective for single annotators or small teams on text classification tasks, but do not scale well.

Roboflow provides a no-code interface for computer vision annotation with integrated training and deployment. For image classification and

object detection projects with small-to-medium scale, it is a complete pipeline with minimal technical friction.

Datasaur offers a no-code text annotation platform with good NLP support and collaboration features.

The honest assessment of no-code options: they work well for small, well-defined tasks. They hit ceilings quickly when you need active learning integration, custom label schemas, privacy requirements, or at-scale annotation. They are a useful starting point, not a long-term architecture.

9.7 When to Build vs. Buy

The build vs. buy decision in HITL tooling is more nuanced than in most software contexts because the annotation workflow is coupled to the ML pipeline in ways that create vendor lock-in risks.

Buy when:

- Your task fits a standard annotation template (object detection, text classification, NER)
- You need scale quickly and do not have engineering capacity
- Data privacy is not a concern that rules out cloud hosting
- Your workflow requirements are stable (you know what you are building)

Build when:

- Your annotation task is non-standard or highly domain-specific

- You have strong privacy requirements that rule out external processing
- Your HITL workflow needs deep integration with your ML pipeline
- You have long-term product vision that requires full workflow control

Most organizations end up using an off-the-shelf annotation tool (open-source or commercial) as the user interface layer, while building custom integration with their ML infrastructure for routing, feedback, and monitoring. The annotation UX is not where the architectural moat is. The feedback integration layer is.

9.8 Getting Started Practically

A realistic path to a working HITL system for a solo developer or small team:

Week 1: Define the task precisely. What is the input? What label is a reviewer assigning? What does "correct" look like? Write the annotation guidelines before you pick the tool. If you cannot write clear annotation guidelines, the task is not defined enough to build a system for.

Week 2: Collect a small sample and try labeling it yourself. 100–200 examples. Where do you hesitate? Those are your hard cases. How long does each label take? That determines your reviewer throughput estimate.

Week 3: Set up Label Studio or Argilla. Get a working annotation interface. Have a second person try to use it without help. Fix the parts they cannot figure out.

Week 4: Train a baseline model on your first 200 labels. Even a simple model. Use it to get uncertainty estimates. Route the bottom third by confidence to your review queue.

Month 2: Iterate. Review 200 more cases. Retrain. Check whether the low-confidence cases the model routed were actually harder. If they were—your uncertainty detection is working. If not—debug your calibration.

This is not glamorous. It is what building a real HITL system looks like.

Visit the Label Studio GitHub repository and browse the example templates. Pick one task that matches something you know about. Spend 15 minutes imagining what annotation guidelines you would write for that task. What is easy to write? What is hard? What would you need to know before you could write clear instructions? This exercise—writing annotation guidelines before touching any code—is the most important diagnostic for whether a HITL project is ready to build.

Chapter Summary

Key Concepts

- The annotation tooling landscape ranges from open-source (Label Studio, Argilla, CVAT) to cloud platforms (Scale AI, SageMaker Ground Truth) to no-code options
- Crowdsourced vs. expert annotation: match the annotator expertise level to the task's domain requirements
- No-code tools hit ceilings quickly; the right long-term architecture often combines off-the-shelf annotation UX with custom feedback integration
- Build vs. buy depends on task novelty, privacy requirements, and long-term pipeline coupling
- The most important first step is writing annotation guidelines—before choosing a tool

Key Examples

Label Studio The tool that democratized annotation for non-ML organizations—used by non-profits, hospitals, and news organizations

RLHF-focused tools like Argilla Specialized for LLM evaluation and preference collection

Active learning in SageMaker Ground Truth Architectural integration of uncertainty routing in a managed cloud service

Five Dimensions Check

- *Feedback Integration* (Dimension 5): Tools are the infrastructure for closing the loop
- *Intervention Design* (Dimension 2): The annotation interface is the intervention design implementation

---

Next chapter: language is the hardest thing to teach a computer, and it is also where the most interesting HITL work is happening right now.

Technical Appendix 9A: Annotation Pipeline Tooling

Chapter 9 introduced the annotation economy and the quality control mechanisms that keep it from producing noise. This appendix provides Python reference implementations for production annotation pipelines using Label Studio, inter-annotator agreement measurement, and multi-stage quality routing.

— Formal foundations for Chapter 9

9.9 Label Studio API Integration

Label Studio provides an open-source annotation platform with a Python SDK supporting programmatic project management, task import, and export.

```python
from label_studio_sdk import Client

ls = Client(url='http://localhost:8080',
    api_key='YOUR_API_KEY')
ls.check_connection()

# Text classification project
project = ls.start_project(
    title='Document Classification',
    label_config='''
```

```
    <View>
      <Text name="text" value="$text"/>
      <Choices name="label" toName="text" choice="single">
        <Choice value="relevant"/>
        <Choice value="irrelevant"/>
        <Choice value="uncertain"/>
      </Choices>
    </View>
    '''
)
```

Listing 9.1: Label Studio connection and project creation

9.9.1 Task Import with Pre-Annotations

```
# Import with pre-annotations (model-assisted annotation)
pre_annotated_tasks = [
    {
        "text": "Document text here.",
        "predictions": [{
            "result": [{
                "from_name": "label",
                "to_name":   "text",
                "type":      "choices",
                "value":     {"choices": ["relevant"]},
                "score":     0.87
            }]
        }]
    }
]
project.import_tasks(pre_annotated_tasks)
```

Listing 9.2: Importing tasks with model pre-annotations

9.9.2 Export and Parsing

```
import pandas as pd

annotations = project.export_tasks(export_type='JSON_MIN')
results = []
for task in annotations:
    if task.get('annotations'):
        ann   = task['annotations'][0]
        label = ann['result'][0]['value']['choices'][0]
        results.append({
            'id':         task['id'],
            'text':       task['data']['text'],
            'label':      label,
            'annotator':  ann.get('completed_by',
                {}).get('id'),
            'created_at': ann.get('created_at'),
        })
df = pd.DataFrame(results)
```

Listing 9.3: Annotation export and label parsing

9.10 Complete HITLAnnotationPipeline

The following class integrates quality control, inter-annotator agreement measurement, and expert routing into a single pipeline object.

```
import numpy as np
from label_studio_sdk import Client
from sklearn.metrics import cohen_kappa_score
from collections import defaultdict
from typing import Dict, List, Tuple
import logging

```

```python
logger = logging.getLogger(__name__)

class HITLAnnotationPipeline:
    """
    Production annotation pipeline with quality control.

    Features:
    - Automatic overlap sampling for IAA measurement
    - Per-annotator kappa computation with outlier detection
    - Multi-stage routing: fast pass -> expert review
    """

    def __init__(self, ls_url: str, api_key: str, project_id: int,
                 min_kappa: float = 0.60,
                 overlap_fraction: float = 0.10,
                 expert_threshold: float = 0.45):
        self.client   = Client(url=ls_url, api_key=api_key)
        self.project  = self.client.get_project(project_id)
        self.min_kappa        = min_kappa
        self.overlap_fraction = overlap_fraction
        self.expert_threshold = expert_threshold

    def sample_overlap_items(self, items: List[dict]
                             ) -> Tuple[List[dict], List[dict]]:
        n           = len(items)
        n_overlap  = max(30, int(n * self.overlap_fraction))
        np.random.shuffle(items)
        return items[:n_overlap], items[n_overlap:]

    def compute_project_kappa(self) -> Tuple[float, List[str]]:
```

```
        tasks =
            self.project.export_tasks(export_type='JSON')
        overlap = [t for t in tasks
                   if len(t.get('annotations', [])) >= 2]
        if not overlap:
            return float('nan'), []

        per_task: Dict[int, Dict[str, str]] = {}
        for task in overlap:
            per_task[task['id']] = {}
            for ann in task['annotations']:
                aid   = str(ann.get('completed_by',
                    {}).get('id', 'unknown'))
                if ann.get('result'):
                    label =
                        ann['result'][0]['value']['choices'][0]
                    per_task[task['id']][aid] = label

        all_ann = list({a for labels in per_task.values()
                        for a in labels})
        pair_kappas = {}
        for i in range(len(all_ann)):
            for j in range(i + 1, len(all_ann)):
                a, b = all_ann[i], all_ann[j]
                la, lb = [], []
                for labels in per_task.values():
                    if a in labels and b in labels:
                        la.append(labels[a]);
                            lb.append(labels[b])
                if len(la) >= 5:
                    pair_kappas[(a, b)] =
                        cohen_kappa_score(la, lb)

        if not pair_kappas:
```

```
            return float('nan'), []

        all_a, all_b = [], []
        for (a, b) in pair_kappas:
            for labels in per_task.values():
                if a in labels and b in labels:
                    all_a.append(labels[a]); all_b.append(labels[b])
        aggregate = cohen_kappa_score(all_a, all_b) if all_a \
                    else float('nan')

        avg_kappa = defaultdict(list)
        for (a, b), k in pair_kappas.items():
            avg_kappa[a].append(k); avg_kappa[b].append(k)
        low_quality = [aid for aid, ks in avg_kappa.items()
                       if np.mean(ks) < self.expert_threshold]

        return aggregate, low_quality

    def route_for_expert_review(
            self, completed: List[dict]
    ) -> Tuple[List[dict], List[dict]]:
        _, low_quality = self.compute_project_kappa()
        accepted, expert_queue = [], []
        for task in completed:
            anns = task.get('annotations', [])
            if len(anns) >= 2:
                labels = {ann['result'][0]['value']['choices'][0]
                          for ann in anns if ann.get('result')}
                if len(labels) > 1:
```

```
                    expert_queue.append(task); continue
            ann_ids = [str(ann.get('completed_by', {}).get('id', ''))
                       for ann in anns]
            if any(aid in low_quality for aid in ann_ids):
                expert_queue.append(task)
            else:
                accepted.append(task)
        return accepted, expert_queue

    def cost_estimate(self, n_items: int,
                      cost_per_annotation_usd: float,
                      n_annotators_per_overlap: int = 3) -> dict:
        n_overlap   = max(30, int(n_items * self.overlap_fraction))
        n_regular   = n_items - n_overlap
        total_anns  = n_regular + n_overlap * n_annotators_per_overlap
        total_cost  = total_anns * cost_per_annotation_usd
        return {
            'n_items':              n_items,
            'n_overlap_items':      n_overlap,
            'total_annotations':    total_anns,
            'total_cost_usd':       round(total_cost, 2),
            'qc_overhead_fraction': n_overlap * n_annotators_per_overlap
                                    / total_anns,
        }
```

Listing 9.4: Production annotation pipeline with quality control

9.11 Tool Comparison

Table 9.1: Annotation platform comparison

Tool	Data types	Managed	Self-host	Best use case
Label Studio	All	No	Yes	Research, privacy-sensitiv
Argilla	NLP, LLM eval	No	Yes	RLHF preference collectio
CVAT	Image, video	No	Yes	Object detection, segmen
Prodigy	NLP	No	License	Fast NLP with active lear
Scale AI	All	Yes	No	High-volume commercial
SageMaker GT	All	Yes	AWS	AWS-integrated workflov
Labelbox	All	Yes	No	Enterprise, collaborative

9.12 Exercises

Exercise 9.1 — Label Studio Setup

Using the Label Studio SDK, set up a binary text classification project. Import 50 sample items. Annotate 20 items manually. Export the results and compute the distribution of labels.

Exercise 9.2 — Majority Vote Resolution

Extend the `HITLAnnotationPipeline` class to support configurable majority vote resolution for overlap items. When 3 annotators label an item and 2 agree, accept the majority. When all 3 disagree on a 3-class task, route to expert review.

Exercise 9.3 — Real-World IAA Analysis

Using the SNLI or MultiNLI datasets from HuggingFace (which include per-annotator labels), compute pairwise kappa for all annotator pairs. Identify the 10% most contested items. What is the kappa improvement from removing them?

Exercise 9.4 — Cost Estimation

Using the `cost_estimate` function, calculate the total cost for a project of 10,000 items at $0.10/annotation with 10% expert re-annotation at $1.00/annotation. How does the optimal overlap fraction change if expert re-annotation is 20% of items?

This appendix provides production-ready Python tooling for annotation pipelines, inter-annotator agreement measurement, and quality-controlled expert routing, as introduced in Chapter 9.

Part III

Teaching Machines to Perceive

Language is uniquely hard for computers—and the difficulty is a gift, because it keeps the human in the loop.

— Chapter 10

10 Teaching Computers to Understand Language

Language is the most human thing we do. That computers can appear to do it is either our greatest technical achievement—or our most elaborate illusion.

— on language and machine

10.1 The Lawyer Who Trusted the Machine

On June 22, 2023, a federal judge in New York held an unusual sanctions hearing. Two attorneys — Steven Schwartz and Peter LoDuca of the firm Levidow, Levidow & Oberman — sat before Judge P. Kevin Castel in the Southern District of New York. They had filed a legal brief in a personal injury case against Avianca airlines. The brief cited over half a dozen case precedents. The judge had noticed something strange: he couldn't find them.

He couldn't find them because they didn't exist.

ChatGPT, the AI assistant attorney Schwartz had used to research the case, had fabricated six court decisions — complete with plausible-sounding case names, docket numbers, courts of jurisdiction, and summarized holdings. *Martinez v. Delta Air Lines. Varghese v. China Southern Airlines. Shaboon v. Egypt Air.* None were real. When opposing counsel filed a brief pointing this out, Schwartz asked ChatGPT to verify the cases. The AI

doubled down, insisting they were "real and can be found in reputable legal databases." Schwartz attached printed ChatGPT transcripts to a court filing, apparently as evidence that he had done his due diligence.

Judge Castel imposed sanctions on both attorneys. In his ruling, he wrote something that should be printed on the login screen of every AI assistant: "The Court is presented with an unprecedented circumstance — a submission replete with citations to non-existent cases."

> This story reveals something profound about how language models work — and how they fail. These are systems that can discuss Aristotle, write sonnets, and explain quantum mechanics, yet they will confidently cite cases that never happened, with the same untroubled certainty they'd use to tell you the capital of France. To understand why, we need to understand what language models actually are — and what "understanding" means when you're made of math.

10.2 Why Language Is Hard

Before computers, before neural networks, before anyone dreamed of machines that could write — long before all of that — there was the problem of language itself.

Language is, in a word, slippery.

Consider the sentence: "I saw the man with the telescope." Did you use the telescope to see him? Or was he carrying the telescope? Both readings are grammatically valid. English doesn't tell you.

Now try: "The chicken is ready to eat." Is the chicken about to eat something? Or has it been cooked and is awaiting consumption? Again —

both are correct. Context resolves the ambiguity, but context is everything: it's the restaurant menu, the farmyard, the smell coming from the kitchen.

Or consider irony: "Oh, great. Another Monday." A computer reading that sentence, unaware that Monday often brings dread, might classify it as a positive statement. Sentiment analysis tools still get tripped up by sarcasm constantly, because understanding irony requires a model of what the speaker wanted you to *think they meant* as opposed to what they *literally said* — a layered theory of mind that took humans millions of years to evolve.

Language also carries cultural baggage. "That's sick" can be a compliment among teenagers and an insult at the dinner table. "Wicked" means evil in standard English and excellent in Boston. References to "the shot heard round the world" mean something entirely different to a baseball fan than to a history teacher.

And then there's implication. "Do you know what time it is?" is not a question about your epistemological relationship to clock-reading. It's a request for the time. Humans automatically perform this translation. Computers, for decades, could not.

These challenges — ambiguity, context, irony, cultural reference, implication — are not edge cases. They are the *substance* of language. Every sentence ever uttered is woven through with them. For five decades, researchers tried to solve this with rules. Write down enough grammar rules. Build enough ontologies. It mostly failed. Language was too vast, too fluid, too context-dependent for any rulebook.

Then came a different idea: what if instead of teaching computers the rules of language, you let them *learn* language from the sheer volume of text that humans had already written?

10.3 Words as Coordinates in Meaning-Space

The first breakthrough in modern language AI came in 2013, from a small team at Google. Tomáš Mikolov and colleagues published a technique called **Word2Vec** (Mikolov et al., 2013). The idea sounds almost too simple: train a neural network to predict which words appear near other words in text, and the process of learning those predictions will encode meaning.

Here's the intuition. Words that appear in similar contexts tend to mean similar things. "Dog" and "cat" both appear near "pet," "owner," "veterinarian," "food bowl." "King" and "queen" both appear near "throne," "crown," "royal," "palace."

Word2Vec turned these contextual patterns into coordinates — **vectors** in a high-dimensional space, where each dimension captures some aspect of meaning. The result was startling: not only did similar words cluster near each other geometrically, but the *relationships* between words were encoded in the directions between them.

The famous example: the vector for "king" minus the vector for "man," plus the vector for "woman," gave something very close to the vector for "queen." The geometry of meaning-space mirrored the geometry of human concepts.

But word embeddings had a fundamental limitation. They gave each word a single fixed location in meaning-space, regardless of context. The word "bank" has the same vector whether you're talking about a river bank or a financial institution. Real understanding requires knowing which *sense* of a word is meant in a particular sentence — for that, you need something that reads the whole sentence to determine meaning.

10.4 The Attention Mechanism: Reading While Noticing

In 2017, a team at Google published a paper with a deceptively casual title: "Attention Is All You Need" (Vaswani et al., 2017). It was, by any measure, one of the most consequential papers in the history of artificial intelligence.

The key idea was a mechanism called **attention**, which can be understood through an analogy.

Imagine you're reading a mystery novel. You're on page 300, and you've just reached a scene where the detective notices a red coat. To understand why the red coat is significant, your brain doesn't process just the current sentence. Without conscious effort, your attention drifts back to page 47, where a witness mentioned a figure in red. And to page 112, where the suspect claimed to own no bright clothing. Your mind holds the current word and simultaneously asks: *which other words, anywhere in what I've read, are relevant to understanding this one?*

That is attention.

The **transformer** architecture — the "T" in ChatGPT, the engine inside nearly every modern language AI — does exactly this, but for every word simultaneously.

When a transformer processes the sentence "The animal didn't cross the street because it was too tired," it asks, for the word "it": which other words in this sentence should I weight heavily when interpreting this one? "Animal" scores highly. "Street" scores low. "Tired" scores very high, because the attribution of tiredness resolves the pronoun reference. The word "it" is processed *in relation* to all other words at once.

This happens not once but many times simultaneously, in parallel "heads" of attention, each head learning to track different kinds of relationships — syntactic ones, semantic ones, coreference chains, discourse

structure. When all these heads are combined and processed through layers upon layers of the same mechanism, something remarkable emerges: representations of text that capture context at every level of abstraction.

The transformer architecture unleashed a scaling law that no one fully expected (Kaplan et al., 2020). The more text you fed it, the more parameters you used, the more compute you threw at the problem — the better the results, in a way that showed no sign of plateauing. Researchers fed transformers first millions of documents, then billions of web pages, then essentially the entire written output of human civilization. The models got better and better at predicting text, which — it turned out — was a surprisingly good proxy for getting better at understanding it.

This is how we arrived at **large language models**: GPT-4, Claude, Gemini, LLaMA, Mistral. Vast statistical engines trained on human writing at a scale that dwarfs any library ever built.

10.5 What a Language Model "Knows"

Here is where we need to be careful — because the word "knows" is doing a lot of work, and it's often doing it dishonestly.

When a language model appears to "know" something, what's actually happening is this: during training, the model processed enormous amounts of text in which certain patterns co-occurred. It learned, in a statistical sense, that "Paris" and "capital of France" appear together constantly. That "Abraham Lincoln" and "1865" and "assassinated" appear together. That "Avianca" and "airline" appear together, and that "Martinez v. Delta Air Lines" — well, that's where things get interesting.

The model didn't learn "Martinez v. Delta Air Lines" from any case reporter. It learned the *pattern* of what court citations look like. And it learned what kinds of phrases appear near "airline injury lawsuits." When asked to produce precedents for airline injury cases, it did what it

always does: it generated text that was statistically plausible given the prompt. It produced strings of text that *looked like* court citations without any mechanism to distinguish between citing something that existed and generating something plausible-sounding.

The model has no memory in the human sense. It has no index pointing from case names to actual decisions. It has something more like a vast, compressed statistical portrait of what human writing looks like — and it uses that portrait to generate text that continues naturally from whatever you've given it.

This is not the same as knowing. It is, at its best, an extraordinary mimicry of knowing. The distinction matters enormously — and it's a distinction the models themselves cannot make.

10.6 The Hallucination Problem

The AI research community has a name for what happened to attorney Schwartz's legal brief: **hallucination**.

A language model hallucinates when it produces text that is confident, fluent, plausible in form — and factually wrong. The term captures something real: the wrongness isn't random noise. It's confabulation — the generation of plausible-seeming content that fills a gap where actual knowledge should be.

Why does hallucination happen? Several reasons, each illuminating.

Training objective. The model is trained to predict what comes next in text. Real text sometimes contains errors, propaganda, myths, and mistakes. The model learned from all of it. When it generates text, it generates what's statistically likely given its training — not what's true.

No "I don't know" token. The model has no mechanism for expressing genuine uncertainty about a specific fact. Every query gets a response. The model has learned to produce fluent text, and silence is not a token in its vocabulary.

Confidence is stylistic, not calibrated. The model's confidence is encoded in the fluency and assertiveness of its output style, not in any internal calibration. It writes "The case was decided on April 14, 1992, by the Second Circuit" with the same stylistic assurance whether the case is real or fabricated.

Research into hallucination rates is sobering. A 2024 study by researchers at Stanford found that leading AI legal research tools hallucinated in over 58% of legal queries (Magesh et al., 2024). Across medical domains, studies of AI diagnostic outputs have found clinically significant error rates that vary widely by task and patient population (Kompa et al., 2021). Even retrieval-augmented systems — models that can look things up — hallucinate at measurable rates.

The Calibration Core of Hallucination

The hallucination problem is, at its core, a calibration problem. The model isn't uncertain where it should be uncertain. Its expressed confidence doesn't match its actual accuracy. This is the same problem this book has been tracking all along — in GPS systems, in medical AI, in the Air Canada chatbot. The system doesn't know what it doesn't know.

10.7 Three Shapes of Human-in-the-Loop in Language AI

Language AI has not one but three very distinct shapes of HITL deployment, each demanding very different design thinking.

10.7.1 The Chatbot: Real-Time Assistance

The first shape is the one most people encounter: the AI assistant deployed to answer questions, help write emails, or provide customer service in real time.

Here, the HITL challenge is primarily one of **uncertainty communication and escalation**. When should the system answer directly? When should it hedge? When should it say "I recommend you check this with a human expert"? When should it refuse to answer entirely?

The Air Canada chatbot failed this test catastrophically (Civil Resolution Tribunal of British Columbia, 2024). It had no mechanism to distinguish confident knowledge from confident confabulation. It had no escalation path to a human agent when uncertainty was high. It behaved, in effect, as though every question were equally within its competence.

Well-designed chatbot systems do the opposite. They are trained to recognize categories of query where the model's knowledge is unreliable — time-sensitive information, jurisdiction-specific legal matters, medical diagnosis, anything requiring personal context the model doesn't have. For these categories, they hedge explicitly: "I can give you general information, but for your specific situation, you should consult a licensed professional."

This hedging is not a limitation — it is a feature, exactly the kind of *Stakes Calibration* we described in earlier chapters. The stakes for getting medical advice wrong are not the same as the stakes for getting a recipe wrong.

10.7.2 The Content Moderator: Harmful Content Detection

The second shape of HITL in language AI is less visible to most users but arguably more consequential: content moderation at scale.

Language AI does the first pass. Human reviewers handle the escalations. This sounds elegant, but the practice is bruising. The language of harm is deeply contextual. "I want to kill my boss" is, in virtually all instances, idiomatic frustration, not a genuine threat. Satire of a political figure can look exactly like earnest endorsement of the views being lampooned. A slur used reclaimed by the community it targets means something different than the same slur used as an attack.

Automated systems miss these distinctions constantly. They over-remove legitimate content — often disproportionately affecting minority communities whose speech patterns fall outside the training distribution — and under-remove harmful content that's been camouflaged through deliberate adversarial manipulation.

The human reviewers who handle escalations are some of the least-discussed people in the AI ecosystem. Research has documented significant psychological harm to content moderators exposed to graphic violence, child abuse material, and extremist propaganda (Steiger et al., 2021). The feedback they provide trains the next version of the automated system.

This is HITL in its most difficult form: humans in the loop not as occasional consultants but as the cognitive infrastructure that makes the whole system function.

10.7.3 The Alignment Trainer: Preference Feedback

The third shape of HITL in language AI — using human feedback to align language model behavior with human values — has received the most attention from researchers and is the domain of RLHF, DPO, and Constitutional AI.

10.8 Teaching Values: RLHF, DPO, and Constitutional AI

10.8.1 Reinforcement Learning from Human Feedback

Imagine you've trained a language model on billions of words. It can generate fluent text. But "fluent" is not the same as "good." The model might generate text that's plausible but harmful, factually wrong, or simply unhelpful.

Reinforcement Learning from Human Feedback (**RLHF**) — developed through work at OpenAI, DeepMind, and Anthropic (Ouyang et al., 2022) — works roughly like this:

Generate a lot of model outputs for a variety of prompts. Show pairs of outputs to human raters and ask: which is better? The raters compare responses on dimensions like helpfulness, honesty, harmlessness, and accuracy. These judgments train a **reward model** — an AI that learns to predict which outputs humans prefer. The original language model is then fine-tuned using reinforcement learning, nudging it toward outputs the reward model would rate highly (Christiano et al., 2017).

The genius of RLHF is that it converts the fuzzy notion of "good output" into something a model can optimize for. Instead of trying to write rules for every possible scenario, you hire humans to express their preferences, train a model on those preferences, and use that as a training signal.

The limitation of RLHF is that it's expensive, slow, and the humans in the loop introduce their own biases. There's also a well-documented problem of **reward hacking** — the model finding ways to score highly on the reward model without actually being helpful, a bit like a student who learns to write essays that sound good to automated graders without engaging with the substance.

10.8.2 Direct Preference Optimization

Direct Preference Optimization (**DPO**) (Rafailov et al., 2023) is a more recent approach that sidesteps the need for a separate reward model. DPO directly adjusts the language model's weights based on preference comparisons, using a mathematical insight: the reinforcement learning problem RLHF is trying to solve has a closed-form solution computable directly from preference data. This makes training more stable, more efficient, and less prone to some forms of reward hacking.

For our purposes, the important point is the same: humans are in the loop, providing preference judgments that shape what the model learns to do.

10.8.3 Constitutional AI

A different approach was pioneered by Anthropic under the name **Constitutional AI** (**CAI**) (Bai et al., 2022). Rather than relying purely on human raters to judge millions of individual outputs, CAI starts with a set of written principles — a "constitution" — describing how the AI should behave: be helpful, be honest, avoid harmful outputs, respect human autonomy.

The AI is then asked to critique and revise its own outputs based on these principles, in a process that can be largely automated. This drastically reduces the need for human feedback on individual examples, while still allowing humans to exert significant influence — through the design of the constitution itself.

Constitutional AI represents an interesting reconfiguration of the HITL relationship. The human is in the loop not at the level of individual judgments ("this output is better than that one") but at the level of values ("these are the principles we want the system to embody"). It's a shift from line-editing to authorship — from correcting individual mistakes to writing the ethical framework the system will use to correct itself.

None of these approaches is perfect. They all depend, ultimately, on humans — on the quality of the raters' judgments, the wisdom of the constitution's authors, the diversity and representativeness of the human feedback. That dependency is not a weakness to be engineered away. It is the point.

10.9 The Five Dimensions, Applied to Language AI

Uncertainty Detection. Language models are notoriously miscalibrated (Guo et al., 2017). They express uncertainty through hedging language ("I think," "I believe," "you may want to verify"), but this hedging is learned stylistic behavior, not a reflection of actual probability estimates. The challenge: the model generates output token by token with no natural point where genuine uncertainty is computed and communicated.

Intervention Design. The best implementations use a tiered approach: answer directly when confidence is high and stakes are low; hedge and suggest verification when confidence is uncertain; refuse and escalate when stakes are high and confidence is insufficient. The chatbot that says "For specific legal advice, please consult a licensed attorney" is practicing good intervention design.

Timing. In real-time language AI, timing of human intervention is particularly constrained. A customer service chatbot that routes to a human after ten minutes of unhelpful circular conversation has failed the Timing dimension. Intervention needed to happen at the first signal of inadequate confidence.

Stakes Calibration. Language AI systems need domain-aware stakes calibration. Getting a recipe wrong has different consequences than get-

ting a drug interaction wrong. The best systems have been trained with category-specific behavior: more hedging in high-stakes domains, clearer escalation for medical, legal, and safety-critical queries.

Feedback Integration. RLHF and its descendants are, at bottom, systematic feedback integration — the most elaborate version of this dimension yet designed. Every preference judgment from a human rater is a piece of feedback that shapes future behavior. The challenge is ensuring this feedback loop is representative, unbiased, and fast enough to keep pace with the ways people use the model.

10.10 Does It Understand? A Philosophical Aside

We have been using the word "understand" carefully throughout this chapter. Let's address it directly, because it matters more than it might seem.

When a transformer model processes "The animal didn't cross the street because it was too tired" and correctly identifies that "it" refers to the animal rather than the street — has it *understood* that sentence?

One view says: yes. It produced the correct output. It tracked the relevant relationships. What more could understanding require?

Another view says: no. Understanding implies something beyond correct output. It implies a model of the world inside — representations of animals and streets and tiredness that connect to a broader understanding of physical reality. The transformer has learned statistical patterns over text. It has, perhaps, a kind of textual shadow of these concepts. Whether that shadow constitutes genuine comprehension is genuinely unclear (Bender et al., 2021; Searle, 1980).

This is not an idle philosophical question. It has direct practical implications for where human oversight is most needed.

A system that genuinely understands what it's saying will, presumably, notice when what it's saying is false. A system that is producing statistically plausible text has no such internal check. If the distinction matters, it tells us something important: language AI systems need human oversight not as a workaround for a temporary technical limitation, but as a structural feature of what these systems are.

The attorney's problem was not that ChatGPT was a flawed prototype that future versions would fix. It was that ChatGPT is, at its core, a very sophisticated text prediction machine — and text prediction machines, however sophisticated, do not have a relationship to truth. They have a relationship to plausibility. Closing that gap requires humans in the loop.

10.11 What's Next

Language and vision evolved together in biological intelligence — the words we use to describe what we see, the images we conjure when we read. It's fitting that the next frontier in AI combines both.

In Chapter 11, we'll turn to the other great sensory modality that AI has been learning to navigate: vision. We'll explore how machines learn to see, why adversarial examples reveal such a fundamental strangeness in machine perception, and how the HITL problem looks different when the stakes involve a radiologist reading a tumor scan, a car traveling at sixty miles an hour, or a content moderator reviewing millions of images.

The challenges are different. The fundamental question is the same: when can the machine act alone, and when does it need us?

The next time you use an AI assistant for something that matters — a health question, a legal question, a factual claim you'll repeat to someone else — ask it to express its confidence. Then verify the answer through another source. How often does it hedge appropriately? How often is it confidently wrong? What does this tell you about when AI tools need human supervision?

Chapter Summary

Key Concepts

- Language is intrinsically hard for computers because of ambiguity, context-dependence, irony, cultural reference, and implication
- Word embeddings gave words coordinates in "meaning-space" — similar words cluster together
- Transformer models use attention to process words in context of all other words simultaneously
- Language models "know" things statistically, not factually — they have no internal check on truth
- Hallucination is a calibration failure: models express high confidence when they should express uncertainty
- Three HITL shapes in language AI: real-time chatbot assistance, content moderation, and alignment training

- RLHF, DPO, and Constitutional AI are systematic approaches to incorporating human values through preference feedback

Key Examples

Mata v. Avianca (2023) Attorney sanctioned for submitting AI-fabricated legal citations; demonstrates the hallucination problem's real-world consequences

Air Canada chatbot (2024) No escalation path for high-stakes queries; confidently wrong about policy

Content moderation reviewers The unseen human cognitive infrastructure behind automated moderation

RLHF alignment training Human preference judgments as the signal that makes AI helpful rather than merely fluent

Constitutional AI Human values encoded at the constitutional level, not the individual judgment level

Key Principles

- Fluency is not accuracy: a system can sound certain while being completely wrong
- Hallucination is not a bug to be patched but a structural feature of text prediction systems
- The three HITL shapes demand different intervention designs: hedging, escalation, and preference training
- Human oversight of language AI is not a temporary workaround — it is a structural necessity

In Chapter 11, we turn from language to vision — and discover that the way machines learn to see reveals something just as strange about the nature of machine intelligence.

Technical Appendix: Chapter 10

The surprising thing is not that attention works — it's that it works so well that we still aren't sure why.

— on the mathematics of meaning

A.10.1 Token Probability Distributions and Sampling

Language models don't generate words — they generate **tokens**. A token is a sub-word unit, typically a word or a common word-fragment produced by Byte Pair Encoding (BPE) (Sennrich et al., 2016). At each step, the model produces a probability distribution over the entire vocabulary — typically 32,000–100,000 tokens.

Formally, given a sequence of previous tokens $x_1, x_2, \dots, x_t$, the model produces:

$$P(x_{t+1} \mid x_1, x_2, \dots, x_t) = \text{softmax}(\mathbf{W}_o \cdot \mathbf{h}_t) \tag{10.1}$$

where $\mathbf{h}_t$ is the final hidden-state vector and $\mathbf{W}_o$ is the output projection matrix. The softmax ensures probabilities sum to 1:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{10.2}$$

Sampling Strategies

Temperature sampling divides logits by temperature T before softmax:

$$P_T(x_{t+1}) = \text{softmax}\left(\frac{\mathbf{z}}{T}\right) \tag{10.3}$$

At $T < 1$, high-probability tokens dominate (conservative output); at $T > 1$, distribution flattens (more varied output).

Top-p (nucleus) sampling (Holtzman et al., 2020) selects the smallest set of tokens whose cumulative probability exceeds threshold p:

$$\mathcal{N}_p = \min S \subseteq V \text{ such that } \sum_{x \in S} P(x) \geq p \tag{10.4}$$

This adapts dynamically: a peaked distribution yields a small nucleus (confident model); a flat distribution yields a large nucleus (uncertain model).

A model's hallucination tendency is partly a sampling artifact. When the model is genuinely uncertain — the correct token has, say, 15% probability and the second-best has 12% — temperature and nucleus parameters determine whether this uncertainty is amplified or suppressed. The model has no internal mechanism to communicate "I am sampling from a very flat distribution here; my output should be hedged." This is why calibration research matters.

A.10.2 The Attention Mechanism

The transformer's core operation is **scaled dot-product attention** (Vaswani et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{10.5}$$

where:

- $Q \in \mathbb{R}^{n \times d_k}$ — **query** matrix (what we're looking for)
- $K \in \mathbb{R}^{m \times d_k}$ — **key** matrix (what's available to be looked up)
- $V \in \mathbb{R}^{m \times d_v}$ — **value** matrix (what gets returned)
- $\sqrt{d_k}$ — scaling factor preventing dot products from saturating the softmax

In **self-attention**, queries, keys, and values are all derived from the same input sequence $X \in \mathbb{R}^{n \times d_{\text{model}}}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{10.6}$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learned projection matrices. The output $\text{softmax}(QK^\top/\sqrt{d_k})$ is the **attention weight matrix** $A \in \mathbb{R}^{n \times m}$, where A_{ij} represents how strongly position i attends to position j.

Multi-Head Attention

Multi-head attention runs h attention operations in parallel with different learned projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\, W_O \tag{10.7}$$

$$\text{head}_i = \text{Attention}(QW_{Q_i},\ KW_{K_i},\ VW_{V_i}) \tag{10.8}$$

Different heads learn to track different relationship types: coreference resolution, syntactic dependencies, semantic roles. The combination captures language structure at multiple levels simultaneously.

Transformer Layer

Each transformer layer combines multi-head attention with a feed-forward network:

$$\mathbf{h}' = \text{LayerNorm}(\mathbf{x} + \text{MultiHead}(\mathbf{x})) \tag{10.9}$$

$$\mathbf{h}'' = \text{LayerNorm}(\mathbf{h}' + \text{FFN}(\mathbf{h}')) \tag{10.10}$$

where $\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + b_1)W_2 + b_2$ is a position-wise MLP with ReLU activation. Modern LLMs stack tens to hundreds of such layers; the deepest layers capture the most abstract semantic properties.

A.10.3 RLHF Pipeline

Stage 1: Supervised Fine-Tuning (SFT)

Starting from a pretrained language model π_{PT}, collect high-quality human-written responses y^* to prompts x, and fine-tune via maximum likelihood:

$$\mathscr{L}_{\text{SFT}} = -\mathbb{E}_{(x,y^*)\sim\mathscr{D}_{\text{SFT}}}\left[\log \pi_\theta(y^* \mid x)\right] \tag{10.11}$$

Stage 2: Reward Model Training

Collect preference data: for pairs of outputs (y_w, y_l) shown to human raters, record $y_w \succ y_l$. Train reward model $r_\phi(x, y)$ using the Bradley-Terry pairwise preference model (Christiano et al., 2017):

$$P(y_w \succ y_l) = \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big) \tag{10.12}$$

Minimize negative log-likelihood of observed preferences:

$$\mathscr{L}_{\text{RM}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathscr{D}_{\text{pref}}}\Big[\log \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big)\Big] \tag{10.13}$$

Stage 3: RL Fine-Tuning via PPO

Fine-tune π_{SFT} against the learned reward model using Proximal Policy Optimization (Ouyang et al., 2022):

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathscr{D},\, y\sim\pi_\theta(\cdot|x)}\Big[r_\phi(x,y)\Big] - \beta \cdot \text{KL}[\pi_\theta(\cdot \mid x) \parallel \pi_{\text{SFT}}(\cdot \mid x)] \tag{10.14}$$

The KL divergence term $\beta \cdot \text{KL}[\pi_\theta \| \pi_{\text{SFT}}]$ is a crucial regularizer that prevents the fine-tuned model from drifting into degenerate behavior that exploits the reward model. The PPO clipped update is:

$$\mathscr{L}_{\text{PPO}} = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\; \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\Big)\Big] \tag{10.15}$$

where $r_t(\theta) = \pi_\theta(y_t \mid x, y_{<t})/\pi_{\text{old}}(y_t \mid x, y_{<t})$ is the probability ratio and $\hat{A}_t$ is the estimated advantage.

A.10.4 Direct Preference Optimization (DPO)

DPO (Rafailov et al., 2023) achieves the same alignment goal as RLHF without a separate reward model.

Key insight: The RLHF objective (10.14) has a closed-form optimal solution:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y \mid x)\,\exp\left(\frac{r(x,y)}{\beta}\right) \tag{10.16}$$

Rearranging, the reward is expressible in terms of the optimal policy:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x) \tag{10.17}$$

Substituting into the Bradley-Terry preference probability (noting $\log Z(x)$ cancels):

$$P(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right) \tag{10.18}$$

The DPO training objective directly maximizes the likelihood of observed preferences:

$$\mathscr{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,\, y_w,\, y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right] \tag{10.19}$$

Gradient interpretation: The DPO gradient simultaneously increases $\pi_\theta(y_w \mid x)$ and decreases $\pi_\theta(y_l \mid x)$, weighted by how much the current model disagrees with the preference. The reference model π_{ref} keeps the optimization anchored, preventing degenerate solutions where the model assigns near-zero probability to everything except winning responses.

A.10.5 Constitutional AI: Two-Phase Process

Constitutional AI (CAI) (Bai et al., 2022) reduces dependence on human-labeled comparisons by using AI feedback guided by a written constitution of principles.

Phase 1: Supervised Learning from AI Feedback (SL-CAF)

1. **Generate:** Sample outputs $y \sim \pi_0(\cdot \mid x)$ for adversarial "red-team" prompts x designed to elicit harmful responses.

2. **Critique:** Prompt the model to critique its output against constitutional principle c_i:

$$\text{critique} = \pi_0(\cdot \mid x, y, c_i) \tag{10.20}$$

3. **Revise:** Prompt the model to revise based on the critique:

$$y' = \pi_0(\cdot \mid x, y, \text{critique}) \tag{10.21}$$

4. **Fine-tune:** Train π_0 on (x, y') pairs to get $\pi_{\text{SL-CAF}}$.

Phase 2: Reinforcement Learning from AI Feedback (RLAIF)

1. For each prompt, generate two outputs from $\pi_{\text{SL-CAF}}$.
2. Obtain AI preference labels: present the pair with constitutional principle c_i to the model and elicit a preference:

$$P_{\text{AI}}(y_w \succ y_l \mid x) = \pi_{\text{SL-CAF}}(\text{“A is better”} \mid x, y_1, y_2, c_i) \tag{10.22}$$

3. Train reward model r_ϕ on AI preference labels using the Bradley-Terry objective (10.13).
4. Fine-tune $\pi_{\text{SL-CAF}}$ via PPO against r_ϕ using the objective (10.14).

The HITL Reconfiguration in CAI

In standard RLHF, humans are in the loop at the level of individual output comparisons. In CAI, humans are in the loop at the level of constitutional design. This shifts the human contribution from millions of individual judgments to a small number of carefully considered principles. The quality of the resulting model depends critically on the quality, completeness, and internal consistency of the constitution.

A.10.6 Calibration Metrics for Language Models

A model is **calibrated** if its expressed confidence matches its empirical accuracy. For classification outputs, the canonical measure is **Expected Calibration Error (ECE)** (Guo et al., 2017):

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{n} \, |\text{acc}(B_b) - \text{conf}(B_b)| \tag{10.23}$$

where predictions are grouped into B bins by confidence level, $|B_b|$ is the number of predictions in bin b, $\text{acc}(B_b)$ is the fraction of correct predictions in that bin, and $\text{conf}(B_b)$ is the mean confidence.

A perfectly calibrated model has $\text{ECE} = 0$: every confidence bin's accuracy exactly matches its stated confidence.

For generative language models, calibration is harder to measure because outputs are sequences, not class probabilities. Active research areas include:

- **Verbalized calibration:** Asking the model to state its confidence in percentage terms; measuring ECE on those verbalized estimates (Kadavath et al., 2022)
- **Token-level probability calibration:** Measuring whether the probability assigned to the correct completion correlates with actual correctness frequency
- **Semantic calibration:** Measuring calibration at the level of factual claims, not individual tokens

Research consistently shows that large language models are overconfident — their verbalized confidence significantly exceeds their actual accuracy on factual queries, particularly in specialized domains like law, medicine, and science (Magesh et al., 2024).

These technical foundations underpin the practical HITL design questions in Chapter 10: when to trust a language model's outputs, when to require human verification, and how to build systems that communicate their limitations honestly.

11 Teaching Computers to See

We assume that things that look the same to us look the same to everything that looks. The adversarial example is a reminder that this assumption is wrong.

— on machine vision

11.1 The Stop Sign That Wasn't

In 2017, a team of researchers from the University of Washington, UC Berkeley, and other institutions published a paper with an unsettling finding (Eykholt et al., 2018). Using nothing more exotic than a color printer and some stickers, they could reliably make a stop sign invisible to autonomous vehicle vision systems — while remaining completely recognizable to every human who looked at it.

The attack was elegant. The researchers printed small black-and-white sticker patterns — some designed to look like graffiti, some like ordinary wear-and-tear — and affixed them to real stop signs. To any passing driver or pedestrian, the signs still read "STOP" in large white letters on the familiar red octagon. To the machine vision system they tested, the signs were classified as something else entirely: "Speed Limit 45" was the most common misclassification.

Not occasionally. Reliably. In multiple weather conditions. From multiple viewing angles. At vehicle speeds.

If machines see differently from humans in ways this dramatic and this exploitable, what does "teaching a computer to see" actually mean? The answer is stranger and more profound than it might initially seem — and understanding it reveals why human oversight of machine vision is not a temporary accommodation on the road to full automation. It is a structural requirement, rooted in the fundamental nature of what machine vision is.

11.2 Pixels as Numbers, Patterns as Everything

To a digital camera, an image is nothing more than a rectangular grid of numbers. Each position in the grid is a pixel. Each pixel is a triplet of numbers between 0 and 255 representing the intensity of red, green, and blue light. A 640×480 image is a matrix of $640 \times 480 \times 3 = 921{,}600$ numbers. No shapes. No objects. No meaning. Just numbers arranged in a grid.

Early computer vision tried to extract meaning from these numbers using hand-crafted rules: look for sharp transitions in pixel values (edges), then look for edges that form closed curves (contours), then match those contours against stored templates. This worked tolerably well for simple, controlled scenarios and catastrophically badly for anything more complex.

The revolution came in 2012, when Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton submitted an entry to the ImageNet Large Scale Visual Recognition Challenge (Krizhevsky et al., 2012). Their entry — AlexNet — used a **convolutional neural network** trained on two GPUs for about a week. It crushed the competition, achieving a top-5 error rate almost 11 percentage points lower than the next best system.

The deep learning era in computer vision had begun.

11.3 What a Convolutional Neural Network Actually Does

The intuition behind convolutional neural networks is one of the most beautiful ideas in modern AI.

Imagine systematically scanning an image for a face. You might first look for local patterns: gradual smooth regions (like a cheek), abrupt curved boundaries (like a hairline), specific arrangements of dark and light (like eyes against a forehead). Then you'd look for combinations of these patterns at the right scales and spatial relationships.

A **convolutional neural network** (CNN) does something structurally similar, learned from data rather than programmed by hand.

The **lowest layer** learns to detect the simplest patterns: edges — places where pixel values change sharply in a particular direction.

The **next layer** combines edge detections into slightly more complex patterns: corners, curves, short line segments.

The **layer after that** combines corners and curves into textures, simple shapes, partial outlines.

And so on, up through the network. By the time you reach the highest layers, the network is detecting things like "wheel-like circular shapes" or "skin-tone blobs flanking a dark vertical stripe" — components of concepts approaching the human-level categories we'd use to describe what we see.

The crucial insight: the network *learns* all of this from examples. You show it a million images labeled "cat" and a million labeled "not cat," and it adjusts its internal weights until it can reliably distinguish them. The features it learns are not ones a human would have specified. They're whatever works, extracted from the statistics of the training images.

And this is precisely the source of both the power and the vulnerability.

11.4 What the Model Actually Sees — and What It Doesn't

The features a convolutional neural network learns are not the features humans use to understand images.

We know this because of **adversarial examples**.

Ian Goodfellow and colleagues at Google published a landmark 2014 paper demonstrating that you could take any image a neural network classifies correctly, add a carefully calculated pattern of noise — often invisible to the human eye — and cause the network to classify the image as anything you want, with near-absolute confidence (Goodfellow et al., 2015).

The canonical example: take a photograph of a panda. The network correctly identifies it with 57.7% confidence. Add a very specific pattern of pixel perturbations — perturbations so small they don't visibly change the image — and the network now classifies it as a gibbon with 99.3% confidence. The image looks identical to a human. The network sees something completely different.

This isn't a trick enabled by cheating or system access. The adversarial pattern exploits something fundamental about how convolutional networks process images: they are extremely sensitive to specific high-frequency patterns in the input that humans don't perceive and don't use for recognition.

Think about what this means. The network and the human are processing the same numbers and arriving at completely different interpretations. The stop sign with stickers is a physical version of the same phenomenon: the sticker pattern creates specific local features that cascade up through the architecture into a confident wrong classification, while human perception processes the sign at the level of semantic content — reading the

word "STOP," recognizing the shape, understanding the context — in ways that are robust to such local perturbations.

The Root of Machine Vision's Vulnerability

The adversarial example vulnerability is not a bug that will be fixed in the next software version. It is an inherent feature of how these systems work, rooted in the fact that they learned from pixel statistics rather than from the conceptual structure of the visual world. A system with genuine conceptual understanding of what a stop sign is — what it's for, how it's designed, how it differs from other signs — would be robust to sticker attacks, just as human vision is.

11.5 Three HITL Domains in Computer Vision

This fundamental strangeness in machine vision creates different challenges in different high-stakes applications. Let's examine three in detail.

11.5.1 Medical Imaging: The Augmented Radiologist

Medicine was among the first high-stakes domains where AI vision found genuine traction — and where the HITL question has been most carefully and formally studied.

Modern radiology AI systems can detect early-stage lung cancer in chest CTs, identify diabetic retinopathy in fundus photographs, flag abnormalities in mammograms, and measure tumor volumes in MRI scans, all with accuracy competitive with experienced radiologists on the specific tasks they've been trained for (Ardila et al., 2019; Rajpurkar et al., 2017).

The key phrase is "specific tasks they've been trained for." A radiologist looking at a chest CT is not only looking for the specific pathology her AI

assistant was trained to detect. She is looking at the whole image, bringing decades of clinical intuition, noticing unexpected findings, integrating the scan with the patient's history, symptoms, and prior imaging. She is doing something far broader and more contextual than image classification.

This asymmetry has shaped how medical AI is deployed. The dominant paradigm is not replacement but **augmentation**: the AI examines the image, generates findings, assigns confidence scores, and the radiologist reviews both the raw image and the AI's assessment.

Research on this workflow has produced a genuinely important finding: radiologists who see the AI's confidence score systematically change their behavior in ways that are not always improvements.

When the AI confidently flags an abnormality, radiologists spend less time examining the region and are more likely to agree with the AI's characterization. When the AI misses an abnormality, radiologists who saw the AI's "all clear" are *less* likely to catch it than radiologists who worked without AI assistance at all (Gaube et al., 2021). The AI's confident silence acts as a cognitive anchor.

This is **automation bias** operating in its most consequential form: expressed AI confidence suppresses human vigilance precisely in the cases where human vigilance is most needed.

The design response has been nuanced. Some systems present AI findings *after* radiologists have done their initial read, so that human perception is not anchored by the AI's prior. Others present AI findings only as flagged regions, not diagnoses — prompting the radiologist to look without telling them what to conclude. The **Intervention Design** dimension turns out to be as consequential as the accuracy of the underlying algorithm.

The FDA has developed a specific regulatory pathway for AI/ML-based medical devices (U.S. Food and Drug Administration, 2021). Hundreds of AI-enabled medical devices have now received FDA authorization—the

large majority in radiology—with the number growing rapidly each year. This regulatory architecture is itself a form of institutionalized HITL: before any medical vision AI can be used on actual patients, it must demonstrate performance to a human oversight body with the authority to require changes.

11.5.2 Autonomous Vehicles: Human in the Loop at 60 MPH

The physics of driving establish a constraint that makes the HITL design problem in autonomous vehicles uniquely brutal.

At 60 miles per hour, a car travels 88 feet per second. Typical human reaction time to an unexpected event is approximately 1.5 seconds. In the 1.5 seconds between when an autonomous system recognizes it cannot handle the current situation and when a human driver could realistically respond, the vehicle has already traveled 132 feet — nearly half a football field.

This makes the naive version of "human in the loop" — AI drives until uncertain, then asks human to take over — deeply inadequate as a safety model. By the time the human has processed the alert, looked up, assessed the situation, and responded, the situation has evolved far beyond the moment when the AI first recognized its uncertainty.

The autonomous vehicle industry has responded to this constraint by developing radically different approaches to where the human fits in the loop.

Tesla's Full Self-Driving (Supervised) operates in a "Level 2" framework: the human driver is always responsible, hands near the wheel, required to be ready to take over at any moment. Tesla has essentially exited the "Timing" problem by requiring continuous human engagement. The downside: drivers habituate to the AI's competence and stop paying

genuine attention — precisely the automation bias problem observed in radiology, now at highway speeds.

Waymo's approach is the opposite extreme. Waymo's commercial vehicles operate without a human driver in the vehicle at all. The HITL happens remotely: human operators can observe vehicles that encounter uncertain situations and provide guidance. This solves the reaction-time problem (remote operators provide guidance that the vehicle's AI then executes, not real-time physical control) but requires sufficient operational design confidence to drive without a physical fallback.

Cruise's approach — before its service was suspended following a 2023 incident in San Francisco — was a kind of middle path: vehicles with safety drivers in some conditions, without in others, with remote human monitoring in all conditions.

What this divergence reveals is that "human in the loop for autonomous vehicles" is not a single design pattern but a design space, with fundamentally different architectures making fundamentally different tradeoffs between human reaction time, automation bias, regulatory acceptability, and operational cost. The **Timing** dimension of the Five Dimensions framework is at its most urgent and most difficult here.

11.5.3 Content Moderation at Scale: Hashing, Perception, and the Adversary

Image-based content moderation faces a challenge that language moderation does not: the stakes are higher for certain categories and the adversaries more resourceful.

For child sexual abuse material (CSAM), the dominant technical approach is **perceptual hashing**. A perceptual hash is a compact fingerprint of an image, computed in a way that's deliberately similar for visually

similar images. The PhotoDNA system, developed by Microsoft, computes a hash for every uploaded image and compares it against a database of known CSAM hashes maintained by the National Center for Missing and Exploited Children (Farid, 2021).

The design is elegant: known harmful images can be detected and blocked automatically, without any human having to review the content.

But perceptual hashing systems have a fundamental vulnerability: they can only identify *known* harmful content — images that have already been identified, hashed, and added to the database. They cannot identify novel content: new images produced after the database was compiled.

And adversaries learn. Once a bad actor understands that image hashing is the detection mechanism, they can modify images in ways that change the hash while preserving the harmful content for human viewers. The arms race between hashing systems and adversarial evasion is active and ongoing (Jain et al., 2022).

This is where human updating of the hash database becomes critical: every new image of harm that is identified by a human reviewer, removed, and hashed adds to the detection capability. The human is in the loop not just as an adjudicator of edge cases but as the source of new ground truth in an adversarially evolving system.

The second challenge for image moderation is context. A photograph of a human body might be art, medical documentation, evidence of abuse, or exploitation. No hash-based system can distinguish context from pixels. Human judgment on these contextual distinctions is not a temporary workaround — it is the only mechanism available for evaluations that require understanding of intent, context, and meaning.

11.6 What "Seeing" Means When You're Made of Math

Human vision is not just image processing. It's perception — active, contextual, laden with meaning and expectation. When a human sees a stop sign, they understand that it is embedded in a physical environment, connected to rules they've internalized, part of a situation with other actors following those rules.

When a convolutional neural network classifies an image, it is detecting statistical patterns that correlate with the presence of things labeled a certain way in training data. It has no model of the environment, no understanding of why things are labeled as they are, no connection between what it "sees" and any broader understanding of the world.

This is why adversarial examples are possible. If the network had a model of what a stop sign *is* — what it's for, how it's designed, what distinguishes it from other signs — it would be robust to sticker attacks, just as human vision is robust. The vulnerability exists precisely because the network's classification is based on statistical pattern matching, not conceptual understanding (Bender et al., 2021).

The practical implication is the same in either philosophical interpretation: machine vision and human vision are complementary, with different strengths, different failure modes, and different robustness profiles. The radiologist's eyes catch the unexpected finding the AI wasn't trained to detect. The AI catches the early-stage nodule the radiologist would have dismissed as artifact. The human in the loop is not compensating for a deficiency — they're contributing a genuinely different capability.

11.7 The Five Dimensions in Visual AI

Uncertainty Detection. A well-calibrated vision system expresses low confidence on images outside its training distribution. In practice, most classification networks are overconfident, producing high-confidence predictions on inputs that should be uncertain (Guo et al., 2017). Techniques like MC Dropout (Gal & Ghahramani, 2016) and deep ensembles (Lakshminarayanan et al., 2017) improve calibration but remain imperfect.

Intervention Design. In medical imaging, the specific format in which AI findings are presented to radiologists — highlighted regions, confidence scores, differential diagnoses — significantly affects the quality of radiologist decisions. Research has found that the format of AI findings shapes human attention as much as the AI's accuracy (Gaube et al., 2021).

Timing. In autonomous vehicles, the gap between when the system detects uncertainty and when a human can meaningfully respond is a physics problem, not a software problem. Architectural choices (whether humans are physically present, remotely monitored, or not involved until post-hoc review) represent fundamentally different responses to this timing constraint.

Stakes Calibration. In image moderation, the intervention threshold should be set differently for different content categories, with human review calibrated to the stakes of each error type. Missing CSAM has different consequences than over-removing legitimate content. Precision-recall tradeoffs must be set with the asymmetric cost structure in mind.

Feedback Integration. When the training distribution changes — new camera hardware, new environments, new types of adversarial attack — the model's performance silently degrades. Human monitoring and feedback

integration is how systems detect and respond to this drift. In content moderation, the hash database is a form of feedback integration: human reviewers contribute new ground truth that updates the automated system.

11.8 What's Next

The final chapters of this book turn from the modalities — language and vision — to the applications: healthcare, education, creative work, legal systems. In each domain, the five dimensions take on specific texture. The timing of intervention in an ICU looks nothing like the timing of intervention in a classroom. The stakes of getting medical diagnosis wrong are not the stakes of getting a homework grade wrong.

But the underlying structure is the same: a system with genuine capabilities and genuine limitations, working alongside humans who have complementary capabilities and complementary limitations, with the quality of their collaboration determined by how well the system is designed to recognize its uncertainties and communicate them.

Machine vision will become more capable. But the stop sign with stickers is not a temporary limitation of 2017-era deep learning. It is a window into what machine vision fundamentally is: a powerful pattern detector that sees differently from us, fails differently from us, and will always need us most precisely at the moments when what it sees diverges most sharply from what is actually there.

Find an image recognition tool online — Google Lens, Apple's Visual Look Up, or any image classifier. Photograph the same object under three different conditions: normal lighting, an unusual angle, and with some small modification (adding a sticker, changing the background, partial occlusion). How does the classifier's confidence change? What does this tell you about what it's actually detecting?

Chapter Summary

Key Concepts

- Images are grids of numbers; CNNs learn hierarchical pattern detectors from edge detections to abstract concepts
- The features CNNs learn differ fundamentally from the features humans use — adversarial examples make this concrete
- Machine vision and human vision are complementary, not interchangeable: different failure modes, different strengths
- Three HITL domains: medical imaging (augmented radiologists, automation bias), autonomous vehicles (the escalation timing problem), and content moderation (hashing, adversarial evasion, contextual judgment)
- Automation bias — AI confidence suppressing human vigilance — is as important as AI accuracy in medical AI design
- Machine vision's distinctive failure modes—adversarial sensitivity, distribution shift, and automation bias—are not temporary limitations of current technology; they are the perma-

nent reasons why human-AI collaboration in visual domains is not a workaround but the correct architecture

Key Examples

AlexNet (2012) The deep learning breakthrough that launched modern computer vision

Adversarial examples (Goodfellow et al., 2014) Invisible pixel perturbations that fool neural networks with near-absolute confidence

Physical adversarial stop sign attack (2017) Stickers that fool autonomous vehicle vision while remaining obvious to humans

FDA AI device authorization pathway Institutional HITL for medical vision AI — over 900 authorized devices as of 2024

Waymo vs. Tesla vs. Cruise Three fundamentally different architectures for human oversight in autonomous vehicles

PhotoDNA Perceptual hashing for CSAM detection, and its structural vulnerability to adversarial evasion

Key Principles

- Machine vision failure modes are structural, not accidental — they arise from what deep learning is
- The HITL design challenge in autonomous vehicles is a physics problem (reaction time), not just a software problem
- In content moderation, the human is in the loop as ground-truth generator, not just edge-case adjudicator

- Automation bias means that how AI findings are presented is as important as the AI's accuracy

In the chapters ahead, we apply these insights to specific domains — and discover that knowing when to hand off to a human is the deepest form of machine intelligence.

Technical Appendix 11A: The Mathematics of Computer Vision

Chapter 11 introduced how machines learn to see. This appendix provides the mathematical treatment of convolutional networks, the standard metrics for object detection, uncertainty estimation methods, and active learning strategies for reducing annotation cost.

— Formal foundations for Chapter 11

11.9 Convolutional Neural Networks

11.9.1 The Convolution Operation

A convolutional layer applies a set of learned **filters** to an input feature map. For a 2D input $X \in \mathbb{R}^{H \times W}$ and a filter $K \in \mathbb{R}^{k \times k}$, the output at position (i, j) is:

$$Z[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i + m,\ j + n] \cdot K[m, n] + b$$

For colour images with C channels and F filters, the full operation produces an output volume $Z \in \mathbb{R}^{H' \times W' \times F}$:

$$Z[i,j,f] = \sum_{c=1}^{C} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i+m,\ j+n,\ c] \cdot K[m,n,c,f] + b_f$$

The filter $K[m,n,c,f]$ learns a feature detector: for edge detection it responds to intensity gradients; for complex patterns it encodes whatever local structure discriminates between training classes.

11.9.2 Pooling and Spatial Hierarchy

Max pooling reduces spatial resolution while preserving the strongest local activations:

$$\text{MaxPool}(Z)[i,j] = \max_{m\in[0,k),\ n\in[0,k)} Z[i \cdot s + m,\ j \cdot s + n]$$

where s is the stride. Stacking convolution–activation–pooling layers creates a **spatial hierarchy**: lower layers detect edges and textures; higher layers detect object parts and whole objects.

11.9.3 Residual Connections

He et al. (2016) showed that very deep networks suffer from vanishing gradients and introduced **residual connections** that allow gradients to flow directly across layers:

$$\mathbf{h}^{(l+1)} = F\big(\mathbf{h}^{(l)}\big) + \mathbf{h}^{(l)}$$

where $F(\mathbf{h}^{(l)})$ is the learned residual. This enables networks 50–152 layers deep without training instability.

11.10 Object Detection: IoU and mAP

11.10.1 Intersection over Union

Object detection requires localising objects via bounding boxes. The standard metric for localisation quality is **Intersection over Union** (IoU):

$$\mathrm{IoU}(B_{\mathrm{pred}}, B_{\mathrm{gt}}) = \frac{\mathrm{Area}(B_{\mathrm{pred}} \cap B_{\mathrm{gt}})}{\mathrm{Area}(B_{\mathrm{pred}} \cup B_{\mathrm{gt}})}$$

For axis-aligned rectangles:

$$x_{\mathrm{inter}} = \max\big(0,\ \min(x_2^p, x_2^g) - \max(x_1^p, x_1^g)\big)$$

$$\mathrm{IoU} = \frac{x_{\mathrm{inter}} \cdot y_{\mathrm{inter}}}{\mathrm{Area}(B_{\mathrm{pred}}) + \mathrm{Area}(B_{\mathrm{gt}}) - x_{\mathrm{inter}} \cdot y_{\mathrm{inter}}}$$

$\mathrm{IoU} \in [0, 1]$. The standard threshold for a "correct detection" is $\mathrm{IoU} \geq 0.5$.

11.10.2 Mean Average Precision

For a detection system evaluated across N classes, the standard metric is **mean Average Precision** (mAP):

$$\mathrm{mAP} = \frac{1}{N} \sum_{c=1}^{N} \mathrm{AP}_c$$

where AP_c is the area under the precision–recall curve for class c. A detected box is a *true positive* if $\mathrm{IoU} \geq \tau$ with a ground truth box of that class; otherwise a *false positive*. Ground truth boxes not matched by any detection are *false negatives*.

In real datasets, class distribution is heavily skewed. Standard mAP aggregates equally across all classes, masking poor performance on rare classes. For safety-critical systems, report per-class AP separately and monitor performance on underrepresented conditions.

11.11 Uncertainty Estimation in Vision

11.11.1 Monte Carlo Dropout

Gal and Ghahramani (2016) showed that applying dropout at inference time approximates Bayesian inference over network weights. Making T stochastic forward passes yields an ensemble of predictions:

```python
import torch
import torch.nn.functional as F

def mc_dropout_predict(model, image, T=50):
    model.train()  # Keep dropout active at inference
    predictions = []
    for _ in range(T):
        with torch.no_grad():
            logits = model(image)
            probs  = F.softmax(logits, dim=-1)
            predictions.append(probs)
    predictions = torch.stack(predictions)  # (T, num_classes)
    mean_pred   = predictions.mean(dim=0)
    uncertainty = predictions.std(dim=0)
    return mean_pred, uncertainty
```

Listing 11.1: MC Dropout uncertainty estimation for vision

Decomposing uncertainty.

$$\hat{\sigma}^2_{\text{epistemic}} = \frac{1}{T-1}\sum_{t=1}^{T}\big(f_t(x) - \bar{f}(x)\big)^2 \qquad \hat{\sigma}^2_{\text{aleatoric}} = \frac{1}{T}\sum_{t=1}^{T}\sigma_t^2(x)$$

In medical imaging: epistemic uncertainty means the model has seen few training examples like this image (actionable — request a radiologist). Aleatoric uncertainty reflects inherent image ambiguity such as poor contrast (also worth flagging, but for different reasons).

11.11.2 Deep Ensembles

Train M models with different random initialisations (Lakshminarayanan et al., 2017):

$$\bar{p}(y \mid x) = \frac{1}{M}\sum_{m=1}^{M} p_m(y \mid x) \qquad \hat{\sigma}^2 = \frac{1}{M-1}\sum_{m=1}^{M}\big(p_m(y \mid x) - \bar{p}(y \mid x)\big)^2$$

Deep ensembles produce better-calibrated uncertainties than MC Dropout but require M times the training compute and storage.

11.12 Confusion Matrix for Object Detection

For object detection, the confusion matrix must account for missed detections (false negatives — objects present but not detected) and spurious detections (false positives — detections with no corresponding ground truth). An extended matrix for N classes plus a "background" class has shape $(N+1) \times (N+1)$:

- TP_c: objects of class c correctly detected with correct label and $\text{IoU} \geq \tau$

- $\mathrm{Conf}(i \to j)$: objects of class i detected as class j (localisation correct, label wrong)
- FN_c: objects of class c present but not detected
- FP_c: detections labelled as class c with no corresponding ground truth

In medical imaging, FN (missed findings) carry far higher consequences than FP (unnecessary follow-up). This asymmetry justifies operating the system at a threshold that minimises FN rate, even at the cost of a higher FP rate.

11.13 Active Learning for Image Annotation

11.13.1 Uncertainty Sampling

Given a pool of unlabelled images $\mathcal{U}$, select the image with highest predictive entropy:

$$x^* = \arg\max_{x \in \mathcal{U}} \mathbb{H}\big[p(y \mid x)\big] = -\sum_c p(y = c \mid x) \log p(y = c \mid x)$$

Or using BALD (Bayesian Active Learning by Disagreement) (Houlsby et al., 2011), which selects images where uncertainty is epistemic (reducible with more data) rather than aleatoric:

$$x^* = \arg\max_{x \in \mathcal{U}} \mathbb{I}[y; \theta \mid x, \mathcal{D}] = \mathbb{H}[p(y \mid x)] - \mathbb{E}_\theta\big[\mathbb{H}[p(y \mid x, \theta)]\big]$$

11.13.2 Core-Set Selection

Select a set of images $S \subseteq \mathcal{U}$ that maximally covers the feature space:

$$S^* = \arg\min_{|S|=n} \max_{x \in \mathcal{U}} \min_{s \in S} d(x, s)$$

where $d(x, s)$ is cosine distance in penultimate-layer feature space. This ensures the annotation budget covers the diversity of the unlabelled pool rather than clustering around already-well-represented examples.

11.13.3 Active Learning Loop

```python
def active_learning_loop(model, labeled_pool, unlabeled_pool,
                         annotation_budget, query_strategy):
    while annotation_budget > 0:
        model.fit(labeled_pool)
        scores     = query_strategy.score(model, unlabeled_pool)
        batch_size = min(BATCH_SIZE, annotation_budget)
        to_annotate = select_top_k(unlabeled_pool, scores, batch_size)

        # The HITL intervention: human annotation
        new_labels = human_annotator.annotate(to_annotate)

        labeled_pool.add(to_annotate, new_labels)
        unlabeled_pool.remove(to_annotate)
        annotation_budget -= batch_size
        log_performance(model.evaluate(validation_set),
                        len(labeled_pool))
    return model, labeled_pool
```

Listing 11.2: Active learning loop for image annotation

The `human_annotator.annotate()` call represents the HITL intervention. The quality of the final model depends on both the query strategy

(selecting informative images) and the quality of the human annotation (correctly labelling them).

11.14 Exercises

Exercise 11.1 — IoU Implementation

Implement IoU from scratch for axis-aligned bounding boxes. Test it on a case of perfect overlap (IoU = 1), no overlap (IoU = 0), and a partial case you compute by hand. Then implement mAP for a 3-class detector using a synthetic dataset.

Exercise 11.2 — MC Dropout Uncertainty Decomposition

Using a pretrained image classifier with dropout layers:

1. Generate in-distribution test images and OOD images (from a different dataset).
2. Run MC Dropout ($T = 50$ passes). Compute epistemic and aleatoric uncertainty separately.
3. Show that epistemic uncertainty is higher for OOD images.
4. Plot uncertainty maps as a function of model confidence.

Exercise 11.3 — Active Learning for CIFAR-10

Implement an active learning loop on CIFAR-10:

1. Start with 500 randomly labelled examples. Train a baseline CNN.
2. Compare random sampling, entropy sampling, and BALD as query strategies.
3. Plot model accuracy as a function of the number of labelled examples.
4. How many labelled examples does the best active learning strategy need to match the accuracy of 5,000 random labels?

This appendix provides the mathematical foundations for understanding how machines see, how we measure their seeing, and how humans remain essential in the loop of teaching machines to see better, as introduced in Chapter 11.

12 Measuring Success

When a measure becomes a target, it ceases to be a good measure.
— The measurement trap

12.1 The 400-Reviews-Per-Hour Problem

In 2019, a mid-sized e-commerce platform deployed a content moderation system to screen product listings for policy violations. The algorithm handled roughly 85 percent of listings automatically. The rest—borderline cases involving possibly counterfeit goods, prohibited items, or ambiguous descriptions—got routed to a team of human reviewers.

Management wanted to know: is this working?

They chose a simple metric to answer that question: reviews completed per hour. Reviewers who processed more cases were rated higher. Reviewers who lagged got performance improvement conversations. Within six weeks, average throughput climbed from about 40 reviews per hour to over 400.

Impressive. And catastrophically wrong.

What had actually happened was that reviewers, under pressure to move faster, had learned to approve most borderline cases with a glance. The algorithm was routing them uncertain cases—exactly the cases that *needed* careful human attention. But "careful attention" doesn't score well when the metric is speed. So reviewers pattern-matched the easy ones, skimmed past the hard ones, and clicked approve.

The model, which was designed to learn from reviewer decisions, dutifully incorporated these rubber-stamp approvals as ground truth. Over several months, its threshold for flagging violations drifted upward. More borderline cases were auto-approved. Fewer were sent to humans. The feedback loop tightened.

By the time anyone noticed—when a journalist published a story about the platform's counterfeit problem—the model was worse than it had been before the HITL system was introduced. The reviewers, optimizing for the metric they were actually measured on, had broken the system entirely.

Goodhart's Law

When a measure becomes a target, it ceases to be a good measure. In human-AI collaboration, the consequences are particularly severe because feedback loops are tight, mistakes are invisible, and the model gets smarter—in exactly the wrong direction.

12.2 Why Accuracy Alone Is Never Enough

The most natural question to ask of any AI system is: how often is it right? Accuracy is necessary. But it is not sufficient—and in HITL systems, it can be actively misleading.

Consider a fraud detection system applied to a dataset where 1 percent of transactions are fraudulent. A model that labels every single transaction "legitimate" achieves 99 percent accuracy. It also catches zero fraud. An accuracy-first evaluation would call this a good model. Anyone whose credit card gets stolen would disagree.

This is why real measurement requires looking at the full picture: **precision** (when the model flags a case, how often is it actually a problem?), **recall** (of all the real problems, how many did the model catch?), and the **F1 score** that balances the two. For HITL systems specifically, these metrics

must be tracked separately for auto-processed cases and human-reviewed cases.

A system where auto-processed cases achieve 98 percent accuracy and human-reviewed cases achieve 94 percent accuracy might look fine in aggregate. But it is telling you something important: the cases humans review are harder for humans too. The question is whether human review is adding value, or whether the system is routing the wrong cases, or whether reviewers are fatigued and not giving hard cases the attention they deserve.

12.3 A Complete Measurement Framework

Good HITL measurement covers seven distinct dimensions. Each illuminates something the others miss.

Model accuracy and calibration. Accuracy captures raw performance. **Calibration** captures something subtler: does the model's expressed confidence correspond to its actual reliability? A well-calibrated model that says it is 80 percent confident should be right about 80 percent of the time. A poorly calibrated model sends the wrong cases to humans—keeping the cases where it is actually uncertain and routing the ones it happens to be wrong about.

This matters enormously for HITL design because routing decisions depend on confidence scores. If those scores don't mean what you think they mean, the whole intervention design breaks down.

Human workload. How many cases per hour is each reviewer handling? How does this change over a shift? Research on human performance consistently shows a fatigue curve: most people can sustain high attention for roughly 20–30 minutes, after which their error rate climbs (Warm et al.,

2008). A HITL system routing 500 cases per hour to a team of five implicitly assumes each reviewer can sustain 100 per hour with consistent quality. That assumption is almost certainly wrong for anything requiring genuine judgment.

Time-to-decision. How long from flagging until a decision is made? In fraud detection, this matters in seconds. In medical imaging, hours may be acceptable. Time-to-decision is a system-level metric that captures bottlenecks no amount of model accuracy data will reveal.

Cost per decision. Human review is expensive. This needs to be measured explicitly, because cost shapes every design tradeoff. A system routing 40 percent of cases to human review when 5 percent would achieve similar quality wastes resources that could be spent on better interventions elsewhere.

Error rates by pathway. Track error rates separately for auto-processed and human-reviewed cases, and watch whether they move in the right direction over time. If reviewers are gaming the throughput metric, error rates may look stable while underlying quality deteriorates.

Reviewer agreement rate. When the same uncertain case is reviewed by two different reviewers, how often do they agree? Tracking agreement rates over time reveals distribution shifts, fatigue, or diverging interpretation of decision criteria. A sudden drop in agreement is a warning signal.

Feedback loop latency. How long until a human decision improves the model? Short latency means faster learning, but also faster propagation of reviewer errors. The tradeoff depends on the quality control processes applied before incorporating feedback into training.

12.4 The A/B Testing Mindset

Here is something many HITL practitioners overlook: you can run a controlled experiment on your HITL design.

Want to know whether your current confidence threshold for routing is optimal? Randomly assign incoming cases to two groups—one gets the current threshold, one gets a slightly lower threshold. Compare outcomes across every metric in your framework. The group whose threshold produces better outcomes at acceptable cost tells you which direction to move.

Most organizations set routing thresholds once, at launch, based on intuition or informal testing, and never revisit them. Meanwhile, the distribution of incoming cases shifts, reviewer expertise changes, and model calibration drifts—all of which can make the original threshold obsolete.

The Experimental Approach

A/B testing in HITL systems requires adaptations from standard experimental design: reviewers must not know they are in an experiment; cases must be assigned in a statistically sound way (not arm A on Monday, arm B on Friday); and a held-out test set evaluated against stable ground truth is essential. The payoff is continuous, evidence-driven improvement rather than intuition-driven stagnation.

12.5 The Divergence Trap

There is a failure mode in HITL measurement that deserves its own name: **the divergence trap**—optimizing short-term accuracy at the cost of long-term calibration.

Imagine a medical imaging HITL system where the model flags uncertain scans for radiologist review. The radiologists handle hard cases well. Short-term accuracy looks excellent. But the model is learning from their decisions on the cases it was most uncertain about—which tend to be unusual presentations, rare variants, edge cases.

Over time, the model incorporates these corrections and becomes better calibrated on the common cases. It grows more confident about them and routes fewer to humans. The unusual ones continue flowing to the human queue.

Apparent accuracy improves. But the model's effective coverage has narrowed: it handles more of the easy cases autonomously and pushes the unusual ones—the cases with the most to teach it—into the human review queue. If that queue becomes overwhelmed, or if reviewers lose familiarity with the common cases they never review anymore, the system becomes brittle when the case distribution shifts.

> The divergence trap is visible only if you track long-term calibration, not just short-term accuracy. One practical countermeasure: periodically inject known-easy cases into the human review queue, both to keep reviewers calibrated and to monitor whether the model's confidence on common cases is drifting.

12.6 Goodhart's Law and the Metric You Choose

Charles Goodhart, a British economist, observed in 1975 that "when a measure becomes a target, it ceases to be a good measure." The principle has been rediscovered repeatedly across healthcare, education, software development, and now AI—because it describes something fundamental about how people respond to measurement (Strathern, 1997).

In HITL systems, Goodhart's law operates on three levels simultaneously.

Reviewer-level Goodhart: Reviewers optimize for whatever metric they are evaluated on. If it is throughput, they go fast. If it is agreement with the algorithm, they defer to the algorithm. If it is accuracy on a held-out test set they know about, they learn the test set.

Organization-level Goodhart: Organizations optimize for whatever metric they report upward. If that metric is "percentage of cases auto-processed," there is organizational pressure to lower the routing threshold without quality controls.

Model-level Goodhart: Models train on reviewer behavior, silently absorbing whatever biases or gaming that behavior contains.

The only real protection is to measure multiple things simultaneously and to hold some things sacred: a periodic human audit of a random sample of *all* decisions—not just the flagged ones—evaluated by reviewers who do not know the original decisions. This is operationally awkward. It is also the only rigorous way to detect system-wide degradation before it becomes a public problem.

12.7 What Good Measurement Looks Like

The 400-reviews-per-hour story had a preventable outcome. The organization measured exactly one thing and nothing else. A complete measurement framework would have caught the problem in weeks rather than months.

In practice, this means: a dashboard tracking accuracy, calibration error, throughput, and reviewer agreement at weekly intervals; a separate track comparing error rates by pathway, watching for them to diverge; an automatic alert when reviewer agreement on borderline cases drops below the historical baseline; and a quarterly audit where a sample of cases from

all pathways—including auto-approved—is reviewed by senior reviewers who do not know the original decisions.

And critically: a metric measuring *model improvement over time*. If the model is not getting better from human feedback, either the feedback loop is broken, the feedback is not good enough to learn from, or the cases being reviewed are too hard to yield useful signal. That last possibility is an argument for routing some easy, clearly-resolved cases into the review queue to provide the model with strong training signal.

Measurement in HITL systems is not surveillance. It is the visibility necessary to improve.

> Think of a decision-making system you interact with regularly—a spam filter, a content recommendation engine, a loan approval process. What single metric would you guess the organization tracks to evaluate it? Now think of two other metrics that single measure might be hiding. What would you need to see to know whether the system was actually working well?

Chapter Summary

Key Concepts

- Accuracy is necessary but not sufficient—a model labeling everything "legitimate" on a 99:1 imbalanced dataset achieves 99% accuracy while catching zero fraud
- A complete HITL measurement framework covers seven dimensions: model accuracy, calibration, human workload,

time-to-decision, cost per decision, error rates by pathway, reviewer agreement, and feedback loop latency

- Goodhart's Law operates at three levels simultaneously: reviewer, organization, and model
- The divergence trap: optimizing short-term accuracy narrows long-term learning by routing unusual cases—the most informative ones—into the human queue
- A/B testing applied to HITL design enables continuous, evidence-driven improvement of routing thresholds

Key Examples

The 400-reviews-per-hour case Throughput metric incentivized rubber-stamping; model trained on gamed labels; system degraded over months

Class imbalance and accuracy A 99%-accurate fraud detector that labels everything "legitimate" catches zero fraud

Feedback loop latency The lag between human correction and model improvement shapes both learning speed and error propagation risk

Quarterly pathway audits Random samples of auto-processed cases, reviewed by senior staff without knowledge of original decisions, are required to detect drift

Key Principles

- Measure multiple things simultaneously; single metrics invite gaming

- Track metrics separately by pathway: auto-processed versus human-reviewed
- Hold periodic audits on cases the system handles autonomously, not only the cases it flags
- Treat measurement design as an intervention-design problem—what you measure shapes what reviewers and organizations optimize for

Technical Appendix 12A: Measuring HITL System Performance

Measurement theory applied to human-in-the-loop systems: calibration metrics, pathway decomposition, inter-rater reliability, and the statistics of controlled experiments.

— Formal foundations for Chapter 12

12.8 Classification Metrics: Formal Definitions

12.8.1 The Confusion Matrix

For a binary classifier, outcomes fall into four cells:

Table 12.1: Binary classifier confusion matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

12.8.2 Core Metrics

Accuracy.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Misleading under class imbalance.

Precision (Positive Predictive Value).

$$\text{Precision} = \frac{TP}{TP + FP}$$

Of all cases flagged as positive, how many were correct? A precision-focused system minimizes false alarms.

Recall (Sensitivity, True Positive Rate).

$$\text{Recall} = \frac{TP}{TP + FN}$$

Of all actually positive cases, how many were caught? A recall-focused system minimizes missed detections.

F1 Score.

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

The harmonic mean of precision and recall. The harmonic mean penalizes extreme imbalances between precision and recall more severely than the arithmetic mean.

F_β Score.

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

When $\beta > 1$, recall is weighted more heavily (missing a positive is more costly). When $\beta < 1$, precision is weighted more heavily.

12.8.3 HITL-Specific Pathway Decomposition

Let $\mathscr{A}$ denote the set of auto-processed cases and $\mathscr{H}$ the set of human-reviewed cases, with $|\mathscr{A}| + |\mathscr{H}| = N$.

Pathway Error Rates.

$$\text{Error}_{\mathscr{A}} = \frac{FP_{\mathscr{A}} + FN_{\mathscr{A}}}{|\mathscr{A}|}, \qquad \text{Error}_{\mathscr{H}} = \frac{FP_{\mathscr{H}} + FN_{\mathscr{H}}}{|\mathscr{H}|}$$

System-Level Error Rate.

$$\text{Error}_{\text{system}} = \frac{|\mathscr{A}|}{N} \cdot \text{Error}_{\mathscr{A}} + \frac{|\mathscr{H}|}{N} \cdot \text{Error}_{\mathscr{H}}$$

A system can show improving aggregate error while $\text{Error}_{\mathscr{A}}$ increases, if the routing threshold is raised (more cases auto-processed). Tracking pathway-specific error rates separately detects this divergence.

Human Value-Added.

$$\Delta_H = \text{Error}_{\mathscr{H}}^{(\text{auto-only})} - \text{Error}_{\mathscr{H}}^{(\text{human-reviewed})}$$

Estimated counterfactual: how much higher would the error rate on human-reviewed cases have been if the model had decided them autonomously?

12.9 Calibration Metrics

12.9.1 Why Calibration Matters for HITL

A HITL system routes cases to humans based on model confidence. If confidence is miscalibrated—if an 80% confidence score corresponds to 60% accuracy—the routing threshold is miscalibrated too.

Definition: Calibration. A probabilistic classifier f is perfectly calibrated if, for all $p \in [0, 1]$:

$$P\big(\hat{y} = y \mid f(x) = p\big) = p$$

12.9.2 Expected Calibration Error (ECE)

Partition predictions into M equally-spaced confidence bins. Let B_m denote the set of cases in bin m, with $|B_m|$ elements:

$$\mathrm{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}[\hat{y}_i = y_i], \qquad \mathrm{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$$

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|$$

A perfectly calibrated model has ECE $= 0$. Values below 0.02 are considered good; above 0.05 warrant intervention (Guo et al., 2017).

12.9.3 Maximum Calibration Error (MCE)

ECE is an average; a model can have low ECE while severely miscalibrated in high-confidence bins where routing decisions are made:

$$\mathrm{MCE} = \max_m \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|$$

For HITL systems, MCE in the high-confidence bins should be tracked separately from the overall ECE.

12.9.4 Calibration Methods

Temperature Scaling (Guo et al., 2017). A single learned parameter $T > 0$ scales the logits before the softmax:

$$\hat{p}_i = \mathrm{softmax}(z_i / T)$$

When $T > 1$, confidence is reduced. T is optimized on a held-out validation set by minimizing negative log-likelihood.

Platt Scaling. Fits a logistic regression $P(y = 1 \mid f) = \sigma(af + b)$ on a validation set, where (a, b) are learned parameters.

Isotonic Regression. Non-parametric monotonic calibration. More flexible than temperature scaling but requires a larger validation set to avoid overfitting. Preferred when calibration errors are concentrated in specific confidence regions.

12.10 Confidence Intervals for HITL Metrics

12.10.1 Wilson Score Interval for Proportions

For an observed proportion $\hat{p} = k/n$, the Wilson score interval:

$$\left[\frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n} \right]$$

where $z = 1.96$ for a 95% interval. Preferred over the Wald interval when $\hat{p}$ is near 0 or 1.

12.10.2 Bootstrap Confidence Intervals

F1, ECE, and inter-rater agreement are nonlinear functions of the data. Closed-form intervals are unreliable. The bootstrap:

1. Draw $B = 10{,}000$ bootstrap samples (with replacement) from n observations.
2. Compute the metric on each: $\theta_1^*, \ldots, \theta_B^*$.
3. The 95% interval is the 2.5th and 97.5th percentiles of $\{\theta_b^*\}$.

12.10.3 Minimum Detectable Effect

Required sample size to detect error-rate difference δ with power $1 - \beta$ at significance α:

$$n = \frac{(z_{\alpha/2} + z_\beta)^2 \cdot 2\bar{p}(1 - \bar{p})}{\delta^2}$$

where $\bar{p}$ is the pooled error rate. For a 5% baseline and 1 percentage point improvement, 80% power requires $n \approx 2{,}600$ cases per arm.

12.11 Inter-Rater Reliability

12.11.1 Cohen's Kappa

Raw agreement is inflated by chance. Kappa corrects for this:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is observed agreement and $p_e = \sum_k (n_{k\cdot}/n)(n_{\cdot k}/n)$ is expected chance agreement.

Benchmarks (Landis & Koch, 1977): $\kappa < 0.20$ slight; 0.21–0.40 fair; 0.41–0.60 moderate; 0.61–0.80 substantial; >0.80 nearly perfect.

12.11.2 Fleiss' Kappa for Multiple Reviewers

For $k > 2$ reviewers, Fleiss' generalization:

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

12.11.3 Intraclass Correlation Coefficient (ICC)

For continuous annotations (e.g., severity scores), two-way mixed ICC:

$$\text{ICC}(2, 1) = \frac{MS_R - MS_E}{MS_R + (k - 1)MS_E + k(MS_C - MS_E)/n}$$

where MS_R, MS_C, MS_E are mean squares from a two-way ANOVA decomposition.

12.12 Statistical Significance in A/B Tests

12.12.1 Two-Proportion Z-Test

Null hypothesis $H_0 : p_A = p_B$. Test statistic:

$$z = \frac{\hat{p}_A - \hat{p}_B}{\sqrt{\hat{p}(1-\hat{p})(1/n_A + 1/n_B)}}$$

where $\hat{p} = (k_A + k_B)/(n_A + n_B)$. Reject H_0 at $\alpha = 0.05$ when $|z| > 1.96$.

12.12.2 Complications in HITL Experiments

Three HITL-Specific Complications

- **Non-independence:** Cases reviewed by the same reviewer are correlated. Use cluster-robust variance estimation with reviewers as clusters.
- **SUTVA violations:** Reviewer behavior can change based on queue length, which is affected by arm assignment. Minimize by balancing queue lengths across arms.
- **Learning effects:** If the model updates during the experiment, freeze model weights for the experimental period or use pre-registered stopping rules.

12.12.3 CUPED: Variance Reduction Using Pre-Experiment Data

CUPED reduces variance by regressing out baseline covariates (Deng et al., 2013):

$$Y_{\text{CUPED}} = Y - \theta(X - \mathbb{E}[X])$$

where $\theta = \text{Cov}(Y, X)/\text{Var}(X)$ is estimated from pre-experiment data. Can reduce variance by 30–50%, roughly halving required sample size.

12.13 Feedback Loop Latency Measurement

12.13.1 Defining Latency

Let t_{decision} be the time a human reviewer finalizes a decision, and $t_{\text{incorporated}}$ be the time that decision is reflected in the model's predictions:

$$L_i = t_{\text{incorporated},i} - t_{\text{decision},i}, \qquad \bar{L} = \frac{1}{|\mathscr{H}|} \sum_{i \in \mathscr{H}} L_i$$

12.13.2 Measuring Feedback Value

The value of the feedback loop is the improvement attributable to human corrections:

$$V_{\text{feedback}} = \text{Error}_t^{(\text{without feedback})} - \text{Error}_{t+\Delta}^{(\text{with feedback})}$$

Estimating the counterfactual requires freezing model snapshots at multiple time points and evaluating each against new ground truth—operationally complex but the only rigorous approach.

12.14 Exercises

Exercise 12.1 — Calibration Assessment

A model has predictions in 10 confidence bins with accuracy values: {0.05, 0.11, 0.18, 0.30, 0.42, 0.55, 0.63, 0.74, 0.82, 0.90} (equal numbers per bin, $n = 1{,}000$ total).

(a) Compute the ECE.

(b) Describe the reliability diagram shape.

(c) Assess calibration quality; is temperature scaling appropriate?

(d) What does this calibration imply for HITL routing at a threshold of 0.80?

Exercise 12.2 — Sample Size Determination

A HITL system has a current error rate of 8%. A proposed redesign is expected to reduce this to 6%.

(a) Compute the required sample size per arm at 80% power, $\alpha = 0.05$.

(b) Recompute for 90% power.

(c) At a throughput of 500 human-reviewed cases per day, how long would each experiment take? What practical challenges arise at this timescale?

Exercise 12.3 — Goodhart's Law Simulation

A model routes the bottom 20% of confidence cases to human review. Original reviewer accuracy: 92%. Gaming increases throughput 3× but reduces accuracy to 65%. System auto-processes the remaining 80% at a 3% error rate.

(a) Compute the system-level error rate before and after gaming using the pathway decomposition from Section 12.5.

(b) Explain why aggregate accuracy might appear stable to management while the system degrades.

(c) What single monitoring metric would most reliably detect gaming within two weeks?

Exercise 12.4 — Inter-Rater Reliability

Two reviewers independently label 100 borderline cases. Reviewer A: 60 positive, 40 negative. Reviewer B: 55 positive, 45 negative. Agreement: 50 both-positive, 25 both-negative.

(a) Compute Cohen's kappa.

(b) Interpret the level of agreement.

(c) What trend in kappa over time would constitute a warning signal? What might cause it?

Exercise 12.5 — Feedback Loop Quality Control

Design a quality control procedure for incoming reviewer decisions before model training. The system processes 10,000 cases per day, routes 15% to human review, and targets 24-hour feedback loop latency.

(a) Specify four statistics to compute on each incoming daily batch.

(b) Define alert thresholds for each statistic.

(c) Describe the decision rule: when to proceed, when to hold for audit, when to reject.

(d) What is the cost of a false hold (rejecting a good batch)? Of a false pass (accepting a bad batch)?

This appendix provides the mathematical foundations for Chapter 12's measurement framework. The exercises are calibrated so that Exercises 12A.1–12A.4 are accessible to readers with introductory statistics, while Exercise 12A.5 requires systems-level thinking appropriate for practitioners and advanced students.

Part IV

The Human Factor

The human is not a neutral input device.

— Chapter 13

13 The Psychology of Human-AI Teams

The better the automation, the less prepared the human for the moment it fails.

— The irony of reliable automation

13.1 The Night the Computers Went Silent

At 2 hours and 10 minutes past midnight on June 1, 2009, Air France Flight 447 disappeared into the South Atlantic Ocean. All 228 people aboard were killed. It remains the deadliest aviation accident in Air France's history (Bureau d'Enquêtes et d'Analyses pour la Sécurité de l'Aviation Civile, 2012).

The plane—an Airbus A330 equipped with some of the most sophisticated autopilot systems in the world—had flown into an area of severe turbulence over the Intertropical Convergence Zone. Ice crystals formed in the pitot tubes, the sensors that measure airspeed. The autopilot, suddenly unable to trust its airspeed data, did exactly what it was designed to do: it disengaged. It handed control to the pilots.

The crew had, for the previous several hours, been managing a comfortable, mostly automated flight. Then, in the space of a few seconds, they were thrust into a rapidly deteriorating emergency situation with faulty

and conflicting sensor data—a situation requiring immediate, accurate action on the aircraft's manual controls.

What the flight data recorders revealed was devastating: the pilots were confused. One pilot pulled back on the stick, increasing the plane's angle of attack even as the aircraft stalled. The confusion lasted four minutes and twenty-some seconds, from autopilot disengagement to impact with the ocean.

The BEA's conclusion was precise: the crew had not been sufficiently trained in high-altitude manual flight, and they had lost situational awareness during the period of automation. When the system handed control back to them, they were not ready for it.

Air France 447 is not a story about bad pilots or bad technology. It is a story about what happens when the human is in the loop for so long without actually needing to exercise skill that the skill quietly atrophies—and then suddenly, catastrophically, the skill is required.

13.2 The Paradox of Reliable Systems

There is something counterintuitive at the heart of automation design: the better the automation, the more dangerous the gap when it fails.

A poor autopilot that disengages frequently keeps pilots sharp. An excellent autopilot that handles 99.97% of situations without intervention produces pilots who have far less practice with the remaining 0.03%—precisely the moments when judgment is most critical.

This is called **automation complacency**, and it has been documented in aviation, nuclear power, industrial process control, and AI-assisted medical diagnosis. The pattern is consistent: as system reliability increases, human operators become less vigilant, less practiced, and less prepared for the moments when the system fails (Parasuraman & Manzey, 2010).

The paradox runs deeper than practice alone. When a system is highly reliable, there is a rational case for trusting it. If an AI flagging system has 98% precision, a reasonable reviewer should be skeptical when their judgment conflicts with the AI's recommendation—it's sensible Bayesian updating. The problem is that this rational deference, taken too far, becomes **automation bias**.

Bainbridge's Irony of Automation (Bainbridge, 1983)

Designers assume that the human will reliably oversee the automated system. But automation removes the very practice that makes reliable oversight possible. The designers are correct that human operators are unreliable—but automation is part of the reason why.

13.3 Automation Bias: The Research Record

The term was formalized by Linda Skitka and colleagues in 1999. Pilots shown automated recommendations became statistically less likely to notice when the automation was wrong—even when they had been explicitly told to apply independent judgment, and even when the errors were visible to an unbiased observer (Skitka et al., 1999).

The finding has been replicated across dozens of studies and multiple domains. Medical residents shown an AI-generated differential diagnosis are less likely to consider diagnoses the AI omitted, even when their own history-taking revealed relevant clues. Radiologists reviewing AI-flagged scans spend more time on flagged areas and proportionally less time on unflagged areas—increasing the likelihood of missing pathology the AI didn't catch.

What makes automation bias particularly insidious in HITL systems is that it works against the purpose of having a human in the loop. The whole point of human review is to catch cases the model gets wrong. Automation

bias systematically reduces the probability that humans catch exactly those cases—the ones where the model is confident but incorrect.

The mechanism: automation bias is not primarily about deference to authority, or cognitive laziness. Research by Parasuraman and Riley, who produced the foundational taxonomy of human-automation interaction, suggests the core mechanism is **attentional allocation**—the recommendation satisfies the brain's drive toward closure, reducing effortful processing of the case itself (Parasuraman & Riley, 1997). This means "apply independent judgment" instructions, without interface redesign, accomplish very little.

13.4 Overtrust and Undertrust: Both Cost You

Automation bias is a form of **overtrust**. But the psychological literature is equally clear that **undertrust** is a problem, and often an underappreciated one.

Undertrust manifests as systematic distrust of algorithmic recommendations even when the algorithms are accurate: clinicians who treat AI flags as "suggestions" and override them at high rates even when the flags were correct; judges ideologically opposed to "letting an algorithm decide" who override risk predictions at rates statistical accuracy would not justify.

Undertrust has real costs. If a screening algorithm correctly identifies high-risk cases and clinicians consistently override toward lower-risk assessments, the system's effective recall drops—undermining the whole purpose of AI augmentation.

The optimal human-AI collaboration requires **calibrated trust**: trust that accurately tracks the system's actual performance domain by domain. Neither systematic over-reliance nor systematic under-reliance, but flexible, evidence-sensitive deference.

Calibrated trust is hard to achieve and harder to maintain. It requires that reviewers have accurate mental models of what the AI is good at and what it isn't. It requires feedback loops that let reviewers learn when they're right to override and when they're wrong. And it requires an organizational culture that neither rewards blind deference nor treats algorithm skepticism as a virtue (Dietvorst & Logg, 2015).

Algorithm Aversion

Dietvorst and colleagues (2015) found that seeing an algorithm err *once* causes people to systematically avoid using it in the future—even when the algorithm still outperforms human judgment. The research label is "algorithm aversion." In HITL contexts, reviewers regularly see the model's errors (that is why the cases are in the queue). Without deliberate feedback design, algorithm aversion is a predictable outcome.

13.5 The Expertise Effect: Better in Some Ways, Worse in Others

Domain experts make better HITL reviewers in several respects: they catch errors novices miss, recognize unusual presentations, and provide harder-to-fool audits of AI outputs.

But expertise introduces its own vulnerabilities.

Experts are more susceptible to **expectation confirmation**: they have strong prior beliefs about what they will find, and they process new evidence in light of those priors. An experienced radiologist who "knows" that a certain pattern correlates with benign disease may be less willing to register a finding that contradicts that expectation—even when an AI correctly flags it.

Highly experienced reviewers have longer histories with their own intuitions. They have been right a lot. When AI recommendations conflict with expert intuition, the expert has more to defend and more history of being right. The resistance to AI suggestions that contradict expertise can be fierce.

And experts are more susceptible to the **consistency trap**: strong expectations about what a decision "should" look like make them more skeptical of AI outputs that violate those expectations—even when the outputs are correct.

This is not an argument against expert reviewers. It is an argument for designing HITL systems that account for expert psychology—presenting AI recommendations in ways that invite genuine evaluation rather than triggering confirmation or resistance.

13.6 Fatigue and the Time-of-Day Effect

Human judgment is not constant. It varies with time of day, length of shift, number of prior decisions, and emotional state—factors that most HITL evaluation frameworks entirely ignore.

A 2011 study by Danziger and colleagues examined 1,112 judicial hearings. The probability of a favorable parole decision dropped from roughly 65% at the start of each session to nearly zero toward the end, before recovering after breaks (Danziger et al., 2011). The judges were not consciously aware of this pattern.

The causal mechanism behind the Danziger finding—whether it reflects decision fatigue, glucose depletion, or a statistical artifact of case ordering—has been contested in subsequent methodological work. The precise effect

size should be treated cautiously. The directional finding (decision quality degrades over a session) is, however, well-supported by the broader human performance literature independent of this specific study.

The finding points to something well-established in human performance research: decision quality declines under sustained cognitive load. Annotation quality, medical diagnosis accuracy, content moderation decisions, and financial review accuracy all follow this pattern. They are higher early in a work session and lower late (Linder et al., 2014).

A HITL system that is measured, calibrated, and staffed without accounting for time-of-day effects is ignoring a significant and predictable source of variance in its human component's performance. Routing the most difficult cases to reviewers at hour four of a shift is not equivalent to routing them at hour one. The two populations—same person, different time—can have measurably different error rates.

Practical implications: randomize case routing to prevent clustering of hard cases at shift end; build mandatory break schedules into the review workflow; use intra-batch agreement rates as a near-real-time proxy for reviewer fatigue; consider two-pass review for the highest-stakes decisions falling late in long shifts.

13.7 Team Dynamics Without a Human Team

Traditional organizational research on team performance was developed to understand groups of humans. Almost none of it applies cleanly when one team member is an algorithm.

An algorithm does not experience psychological safety. It does not feel threatened by dissent. It does not have a bad day. It does not grow resentful of overrides. It does not build an informal theory of which reviewers are worth listening to.

What this means in practice is that the "team dynamics" of a human-AI system are almost entirely determined by how the human relates to the AI—and those dynamics are heavily influenced by system design choices that organizations rarely treat as team management decisions.

How confident does the AI appear? Systems presenting recommendations with high apparent confidence elicit more deference than systems that explicitly communicate uncertainty. The same model, the same accuracy, different framing, different behavior.

Does the AI explain itself? Explainable AI can reduce automation bias by giving reviewers something to evaluate. But it can also increase automation bias if the explanations are plausible-sounding but wrong—providing post-hoc narrative that convinces reviewers the AI must be right.

Does the reviewer get feedback on their overrides? Without feedback, there is no mechanism to calibrate trust. With feedback, reviewers can learn which domains their judgment beats the AI's and which it doesn't. This feedback is among the most powerful interventions available for producing calibrated trust.

13.8 What HITL System Design Can Do About Human Psychology

These psychological findings are not arguments for resignation. They are arguments for designing systems that support the kind of human engagement that actually produces the outcomes we want.

Interface design for independent judgment. Present the AI recommendation in a way that invites evaluation rather than closure. One tested approach: have reviewers complete their own initial assessment before the AI recommendation is revealed. This forces genuine processing uncorrupted by automation bias.

Explicit uncertainty communication. A system that displays "85% confidence" gives reviewers something to reason about. A system that displays "RECOMMENDED: APPROVE" encourages closure. Calibrated uncertainty communication, introduced in Chapter 1 and formalized in Chapter 12, shapes human psychology as much as it does routing.

Structured disagreement protocols. For high-stakes decisions, require reviewers to document their reasoning when they override the AI, and require a second review when reviewer and AI disagree. This creates a context in which the human is genuinely doing the work the HITL design requires.

Feedback on overrides. If you want calibrated trust, give reviewers information about the accuracy of their overrides. Without feedback, trust cannot calibrate. With it, even experienced reviewers become more accurate about which overrides are worth making.

Fatigue management. Build mandatory break structures and randomized case routing into the system design. Do not let the hardest cases cluster at shift end. Track intra-batch agreement rates as an operational fatigue signal.

> The next time you use an AI-powered suggestion—autocomplete, a navigation recommendation, a grammar suggestion—pay attention to your own psychological response. Do you evaluate the suggestion independently before accepting it, or does the presence of a recommendation change your process? What would you have decided if no recommendation had appeared? This is automation bias in your own cognition, in real time.

Chapter Summary

Key Concepts

- **Automation complacency**: The more reliable the system, the less prepared the human for the moment it fails
- **Automation bias**: Humans shown an AI recommendation become less likely to catch the AI's errors—even when instructed to apply independent judgment
- Overtrust and undertrust are both costly; calibrated trust requires feedback loops as a deliberate design element
- The expertise effect: domain experts have higher discriminability but stronger prior-based resistance to disconfirming AI recommendations

- Fatigue and time-of-day effects on decision quality are predictable and large; HITL designs that ignore them implicitly accept a major variance source
- When one team member is an algorithm, human-AI dynamics are largely shaped by interface and system design choices

Key Examples

Air France 447 (2009) Highly reliable autopilot produced pilots unpracticed in manual flight; sensor failure disengaged automation in crisis; skill atrophy produced fatal confusion

Skitka et al. (1999) Experimental demonstration that automation bias persists under explicit independence instructions; the mechanism is pre-conscious attention allocation

Radiologist AI studies Attention shifts toward AI-flagged areas; proportionally less time on unflagged areas; higher miss rate for pathology AI didn't flag

Danziger et al. (2011) Judicial parole decisions fall from 65% favorable early in session to nearly zero at end; recovers after breaks

Key Principles

- The human in the loop is not a neutral input device—psychology shapes performance in predictable, designable ways
- Interface design choices (when the recommendation is revealed, how confidence is displayed, whether explanations are provided) dramatically affect automation bias

- Calibrated trust is the goal; it requires feedback loops and cannot be achieved by instruction alone
- HITL system design must incorporate the literature on human performance under automation, including fatigue effects and the paradox of reliable systems

Technical Appendix 13A: Signal Detection Theory and Human Reviewer Behavior

Signal Detection Theory provides a formal decomposition of human reviewer performance into discriminability and response bias, predicting when and how automation affects each.

— Formal foundations for Chapter 13

13.9 Signal Detection Theory: The Formal Framework

13.9.1 Why SDT?

Standard accuracy metrics treat reviewer errors as random noise. SDT provides a richer decomposition separating two fundamentally different aspects of performance:

1. **Discriminability** (d'): How well can the reviewer distinguish positive (signal) from negative (noise) cases? This reflects cognitive ability.

2. **Response bias** (c or β): Where has the reviewer set their threshold? This reflects implicit weighting of false alarms versus misses, shaped by instructions, incentives, and automation feedback.

A reviewer with high d' and miscalibrated c produces different errors from a reviewer with moderate d' and well-calibrated c. Standard accuracy metrics cannot distinguish them. SDT can.

13.9.2 The Basic Model

In SDT, each reviewed case generates a decision variable X. For positive (signal) cases, X is drawn from $\mathcal{N}(\mu_S, \sigma^2)$; for negative (noise) cases, from $\mathcal{N}(0, \sigma^2)$.

By convention, $\mu_N = 0$, $\sigma = 1$, and $\mu_S = d' > 0$.

The reviewer compares X to criterion c:

$$\text{Response} = \begin{cases} \text{“positive”} & \text{if } X > c \\ \text{“negative”} & \text{if } X \leq c \end{cases}$$

13.9.3 The Four Outcomes and Their Probabilities

Table 13.1: SDT outcome matrix

	“Positive” Response	**“Negative” Response**
Signal present	Hit (TP): $P(H) = \Phi(d' - c)$	Miss (FN): $1 - P(H)$
Signal absent	False Alarm (FP): $P(FA) = \Phi(-c)$	Correct Rejection: $1 - P(FA)$

13.9.4 Computing d' and c

$$d' = z(P(H)) - z(P(FA))$$

where $z(\cdot) = \Phi^{-1}(\cdot)$ is the inverse standard normal CDF.

$$c = -\tfrac{1}{2}\big[z(P(H)) + z(P(FA))\big]$$

Content Moderation Example

A moderator correctly flags 80% of policy violations ($P(H) = 0.80$) and incorrectly flags 15% of compliant posts ($P(FA) = 0.15$):

$$d' = z(0.80) - z(0.15) = 0.842 - (-1.036) = 1.878$$

$$c = -\frac{1}{2}[0.842 + (-1.036)] = 0.097$$

Slight conservative bias ($c > 0$).

Interpreting c.

- $c = 0$: Unbiased criterion
- $c > 0$: Conservative bias—reviewer requires strong evidence to flag
- $c < 0$: Liberal bias—reviewer flags at low evidence

13.10 SDT Formally Predicts Automation Bias

13.10.1 The Effect of an Automated Recommendation on Criterion

When a reviewer is shown an automated recommendation, it shifts their effective criterion c:

$$c_{\text{with-auto}} = c_0 - \delta_{\text{auto}}$$

where $\delta_{\text{auto}} > 0$ is the criterion shift induced by the recommendation. The reviewer becomes more liberal when automation says "flag" and more conservative when automation says "pass."

This criterion shift produces automation bias: hit rate increases when automation correctly recommends "flag" (beneficial), but miss rate also increases when automation incorrectly recommends "pass" on a true positive

(harmful). Automation bias is primarily a *criterion effect*, not a discriminability effect.

13.10.2 Empirical Support

Studies by Skitka et al. (1999) and Goddard et al. (2012) found that hit rates for signals the automation correctly flagged increased substantially with automation, while miss rates for signals the automation incorrectly passed also increased substantially. d' estimates changed modestly—consistent with criterion shift, not discriminability loss, as the primary mechanism (Goddard et al., 2012; Skitka et al., 1999).

13.10.3 Optimal Criterion for HITL Routing

Let p^* be the prevalence of positive cases in the human review queue. The cost-optimal unbiased criterion:

$$c^* = \frac{d'}{2} - \frac{1}{d'} \ln\left(\frac{p^* \cdot C_{FN}}{(1 - p^*) \cdot C_{FP}}\right)$$

where C_{FN} and C_{FP} are the costs of misses and false alarms.

In a HITL system, routing has already changed p^* relative to the population base rate. If the base rate is 5% and the model routes uncertain cases, the queue prevalence may be 20–40%. Reviewers who implicitly use the base-rate prior rather than the queue-enriched prior will have miscalibrated criteria. Explicitly informing reviewers of the queue base rate is a low-cost, high-impact intervention.

13.11 The ROC Curve and HITL Design

13.11.1 The Receiver Operating Characteristic

The ROC curve traces (false alarm rate, hit rate) pairs as c varies. For the Gaussian equal-variance model:

$$P(H) = \Phi\big(d' + \Phi^{-1}(P(FA))\big)$$

The area under the ROC curve (AUC) is a criterion-independent discriminability summary:

$$\text{AUC} = \Phi(d'/\sqrt{2})$$

13.11.2 Optimal Operating Point

The optimal operating point maximizes:

$$\text{Benefit} = p^* \cdot C_{FN} \cdot P(H) - (1 - p^*) \cdot C_{FP} \cdot P(FA)$$

The slope of the ROC curve at the optimal point:

$$\text{slope}^* = \frac{(1 - p^*) \cdot C_{FP}}{p^* \cdot C_{FN}}$$

Automation bias shifts the operating point away from slope* toward the automation's recommended response, degrading overall system benefit whenever the automation is wrong.

13.12 Vigilance Decrement Models

13.12.1 The Vigilance Problem

Vigilance—sustained attention to detect infrequent signals—declines over time. This is among the most robust findings in human performance

psychology, established by Mackworth (1948) and replicated hundreds of times.

Three parameters govern vigilance decrement:

1. **Event rate**: More frequent signals maintain vigilance better
2. **Signal-noise ratio**: Harder discriminations decrement faster
3. **Knowledge of results**: Feedback maintains vigilance

13.12.2 SDT Representation of Vigilance Decrement

Vigilance decrement manifests in SDT as a decline in d' over time—the opposite of automation bias, which is primarily a criterion effect.

$$d'(t) = d'_0 \cdot e^{-\lambda t}$$

Empirical estimates: λ ranges from 0.01 to 0.05 per minute for demanding tasks, implying meaningful discriminability loss within 20–40 minutes.

13.12.3 Combined Effects

Automation bias and vigilance decrement compound. A reviewer 90 minutes into a shift, reviewing AI-recommended decisions, experiences:

- **Reduced** d' (vigilance decrement): Less capable of distinguishing signal from noise
- **Criterion shift toward automation** (automation bias): More reliant on AI recommendation

Both effects increase miss rate on cases where the automation is wrong. Their joint effect on miss rate is approximately multiplicative.

13.13 Formal Model of Calibrated Trust

13.13.1 The Optimal Trust Model

Let $\tau \in [0, 1]$ be the reviewer's trust level. The reviewer's effective decision:

$$D_{\text{reviewer}} = (1 - \tau)\, D_{\text{human}} + \tau\, D_{\text{automation}}$$

Expected accuracy of the human-AI team:

$$P(\text{correct}) = \tau \cdot P_A + (1 - \tau) \cdot P_H + f(\tau) \cdot P_C$$

where P_A = automation accuracy, P_H = human accuracy, P_C = complementarity (probability exactly one is correct), and $f(\tau)$ captures the team synergy term.

13.13.2 Optimal Trust Level

$$\tau^* = \frac{P_A - P_C/2}{P_A + P_H - P_C}$$

When Is Deference Optimal?

When $P_A > P_H$, the optimal trust level $\tau^* > 0.5$ (lean toward automation). Some automation bias is *optimal* given cost structure. The problem is not deference per se, but deference uncalibrated to actual relative accuracy—which requires feedback to achieve and maintain.

13.14 Measurement Implications for HITL Practitioners

13.14.1 Extracting SDT Parameters from Production Data

```python
import numpy as np
from scipy.stats import norm

def compute_sdt_params(hits, total_signal, false_alarms, total_noise):
    """
    Compute d' and criterion c from production HITL data.

    Parameters:
    -----------
    hits: int - number of signal cases correctly flagged
    total_signal: int - total actual positive cases
    false_alarms: int - noise cases incorrectly flagged
    total_noise: int - total actual negative cases
    """
    # Apply log-linear correction for extreme rates
    pH = (hits + 0.5) / (total_signal + 1)
    pFA = (false_alarms + 0.5) / (total_noise + 1)

    d_prime = norm.ppf(pH) - norm.ppf(pFA)
    c = -0.5 * (norm.ppf(pH) + norm.ppf(pFA))
    auc = norm.cdf(d_prime / np.sqrt(2))

    return {'d_prime': d_prime, 'c': c, 'auc': auc,
            'hit_rate': pH, 'fa_rate': pFA}
```

Listing 13.1: SDT parameter extraction from HITL logs

13.14.2 Detecting Automation Bias via SDT

Compare c estimates in two conditions:

- **Condition A**: AI recommendation visible (standard production)

- **Condition B**: AI recommendation hidden (blind review sample)

Automation bias is operationally defined as $c_A \neq c_B$. Specifically: $c_A < c_B$ on AI-flagged cases (criterion shift toward liberal), $c_A > c_B$ on AI-passed cases (criterion shift toward conservative).

```
def detect_automation_bias(condition_a_data,
    condition_b_data):
    """
    Compare SDT parameters between AI-visible and
        blind-review conditions.

    Returns bias magnitude (delta_c) and significance test.
    """
    params_a = compute_sdt_params(**condition_a_data)
    params_b = compute_sdt_params(**condition_b_data)

    delta_c = params_a['c'] - params_b['c']
    delta_dprime = params_a['d_prime'] - params_b['d_prime']

    return {
        'automation_bias_criterion_shift': delta_c,
        'discriminability_change': delta_dprime,
        'bias_direction': 'liberal' if delta_c < 0 else
            'conservative',
        'mechanism': 'criterion_shift' if abs(delta_c) >
            abs(delta_dprime)
                    else 'discriminability_change'
    }
```

Listing 13.2: Automated bias monitoring in production

13.14.3 Sequential Design to Counteract Automation Bias

SDT provides a prescription: anchor criterion independently before the automation recommendation induces a shift.

```python
# Sequential design: human-first assessment
def sequential_review_protocol(case):
    """
    Implement sequential design to reduce automation bias.
    """
    # Step 1: Present case, collect human assessment
    human_assessment = present_case_get_assessment(case)
    record_initial_decision(human_assessment)

    # Step 2: Reveal AI recommendation
    ai_recommendation = get_ai_recommendation(case)
    display_ai_recommendation(ai_recommendation,
        human_assessment)

    # Step 3: Allow override with documented rationale
    if human_assessment != ai_recommendation:
        rationale = require_disagreement_rationale()
        final_decision = present_with_rationale_requirement(
            human_assessment, ai_recommendation, rationale
        )
    else:
        final_decision = human_assessment

    return final_decision
```

Listing 13.3: Sequential design decision flow

13.15 Exercises

Exercise 13.1 — Computing d' and c

A medical AI review system records: hit rate = 0.88, false alarm rate = 0.22.

(a) Compute d' and c.

(b) Is the reviewer biased conservative or liberal? Interpret in medical imaging context.

(c) If the cost of a missed anomaly is 10× the cost of a false alarm and queue base rate is 5%, compute the optimal c^*.

(d) Compare observed c to optimal c^*. What does this tell us about how to communicate the base rate to reviewers?

Exercise 13.2 — Automation Bias via SDT

Baseline (no AI): $P(H) = 0.75$, $P(FA) = 0.18$. AI-assisted: hit rate = 0.85 when AI flags; miss rate increases to 0.32 when AI passes.

(a) Compute d' and c for the baseline condition.

(b) Characterize how criterion c has changed in the AI-assisted condition.

(c) Is d' the primary driver of the change, or is criterion shift? What does this imply for intervention design?

Exercise 13.3 — Vigilance Decrement

A reviewer starts a 4-hour shift with $d'_0 = 2.1$ and $\lambda = 0.012$ per minute.

(a) Compute d' at $t = 30, 60, 90, 120$ minutes.

(b) Compute AUC at each time point.

(c) If misses cost 5× more than false alarms and queue prevalence is 12%, compute optimal c^* at each time point. How does the optimal operating point shift?

(d) Design a production monitoring protocol that would detect this decay in near real-time.

Exercise 13.4 — Optimal Trust Level

A loan review system has: automation accuracy 87%, human accuracy 78%, human is correct 60% of the time when they disagree.

(a) Estimate $P(\text{complementary})$.

(b) Compute τ^*.

(c) Interpret: should reviewers defer to automation, use their own judgment, or use a mixed strategy? At what level?

(d) Propose two interface design interventions that would help reviewers achieve τ^*.

This appendix provides the formal SDT framework underlying Chapter 13's psychological discussion. The key insight—that automation bias is a criterion effect, not a discriminability effect—has direct design implications: interventions must target criterion anchoring before automation recommendations are presented, not simply instruct reviewers to apply independent judgment.

14 Building Fair and Ethical Systems

> *Why human oversight does not automatically produce human fairness.*

14.1 The Hairstyle Problem

In 2018, Amazon quietly shut down a recruiting tool it had been developing for four years. The AI was supposed to screen résumés and surface the best candidates for software engineering positions. Amazon's machine learning team had trained it on ten years of hiring data—the résumés of everyone the company had actually hired.

The problem: Amazon's engineering workforce had historically been predominantly male. The model learned this pattern. It taught itself that women's résumés were less suitable, even though gender was not an explicit input variable. Somehow, it found proxies. The model learned to penalize résumés that included the word "women's"—as in "women's chess club" or "women's college." It downgraded graduates of two all-women's colleges.

Amazon tried to fix this. Engineers removed the gender-correlated terms. The model adapted. It found new proxies. When the team realized the model kept finding new signals correlated with gender, they concluded

the problem was not fixable by removing specific features. They shut the system down (Dastin, 2018).

The Amazon recruiting case is instructive not just as a cautionary tale about **algorithmic bias**, but because it illustrates a deeper structural problem: bias is not a bug you remove. It is a signal the model correctly learned from biased data. The model was doing exactly what it was trained to do. The data reflected a biased world. The model faithfully encoded it.

Here is the part that relates most directly to the theme of this book: Amazon had human reviewers throughout the process. People were checking outputs. People were approving hiring decisions. None of them caught the bias for four years.

The humans in the loop did not protect against the bias. In some ways, they legitimized it—each decision that passed through human review without challenge was, implicitly, a vote of confidence in the system. Human oversight, in this case, was not a check on the algorithm's failures. It was cover for them.

14.2 The Layers Where Bias Enters

There is no single place where bias enters an AI system. It enters everywhere.

Collection bias is the most fundamental: whose data is collected? Early facial recognition datasets, used to train and evaluate models for decades, were overwhelmingly composed of light-skinned male faces. Models trained on these datasets were well-calibrated for some users and dramatically worse for others. A 2018 study by Joy Buolamwini and Timnit Gebru—the **Gender Shades** project—found commercial facial recognition systems from major technology companies had error rates up to 34 percentage points higher for darker-skinned women than for lighter-skinned

men (Buolamwini & Gebru, 2018). The systems had not been designed to fail for these groups. They had simply been trained primarily on data that did not include them.

Annotation bias is the next layer: whose judgment decides what is correct? Training data requires human labels—someone has to say which résumés were "good," which social media posts are "harmful," which recidivism predictions turned out to be accurate. Those labels reflect the values, biases, and social positions of the people who made them. Research on content moderation annotation has found that annotators from different demographic groups make systematically different judgments about the same content, particularly on questions involving race, political content, and hate speech (Sap et al., 2019). There is no neutral label. Every annotation embeds a perspective (Bender et al., 2021).

Model bias is what happens when a machine learning algorithm amplifies the patterns in the data. A model trained on historical criminal justice data will learn that certain neighborhoods, or certain demographic groups, have higher rates of conviction—because that is what the data shows. It will then apply this learned association to new cases, without encoding any understanding of whether the historical disparities were themselves the product of unequal policing, unequal prosecution, or unequal sentencing. The model reproduces structural inequities as if they were natural facts.

Deployment bias is the mismatch between the population on which a system was developed and the population on which it is used. A medical diagnostic AI trained on data from academic medical centers in the United States performs differently when deployed in rural clinics, in other countries, or on populations underrepresented in the training data. A fraud detection model calibrated for typical spending patterns in one economic

context may generate systematically higher false positive rates for users from different economic contexts—essentially treating unfamiliar behavior as suspicious.

The Normal Condition

None of these biases are obscure edge cases. They are the normal condition of AI systems built from historical data about an unequal world. The question is not whether they exist but what to do about them.

14.3 The Feedback Loop Problem

Perhaps the most insidious form of algorithmic bias is the one that makes itself worse over time.

Consider a predictive policing system that uses historical crime data to predict where future crimes are most likely to occur. The model says: "Deploy officers to these neighborhoods." Officers are deployed there. More arrests happen there—as a mathematical certainty, because there are more officers there to make arrests. The arrest data flows back into the training dataset. The model learns, with even higher confidence, that those neighborhoods are where crime happens. It directs even more police resources there. The cycle intensifies.

The humans reviewing the system's outputs see increased arrests and interpret them as validation of the model's predictions. The model, in their experience, is "working." But what the system is actually measuring is not underlying crime; it is the combination of underlying crime plus policing intensity plus historical data artifacts—and it is making policing intensity more unequal over time.

This feedback loop problem is not unique to predictive policing. It applies wherever an AI system's outputs become the inputs for future

training data, and where the system's deployment affects the phenomenon it is trying to measure. Credit scoring systems that affect access to credit affect future creditworthiness. Hiring algorithms that affect who gets interviews affect who has professional experience. Recidivism prediction tools that affect sentencing affect post-release conditions that themselves influence recidivism.

In each case, HITL oversight does not break the loop. Human reviewers validate individual decisions, not the systemic effect of those decisions aggregated over time. The loop runs beneath the level where individual human review operates.

14.4 COMPAS: A Case Study in Algorithmic Justice

In May 2016, ProPublica published an investigation that changed the conversation about AI in criminal justice. Julia Angwin, Jeff Larson, and colleagues had obtained two years of COMPAS recidivism scores for defendants in Broward County, Florida, and compared those scores to actual re-arrest data over the following two years (Angwin et al., 2016).

COMPAS—Correctional Offender Management Profiling for Alternative Sanctions—is a proprietary risk assessment tool widely used in American courts to predict the likelihood that a defendant will reoffend. Its scores influence pretrial detention decisions, sentencing recommendations, and parole determinations. It is, in short, an algorithm that affects how much time people spend incarcerated.

ProPublica's analysis found that the system's errors were not distributed equally. Black defendants who did not actually reoffend were nearly twice as likely as white defendants who did not reoffend to be labeled as high-risk—a false positive rate disparity. White defendants who did reoffend were more likely to have been labeled low-risk—a false negative rate disparity.

The company behind COMPAS, Northpointe, responded that their tool was calibrated—that among people assigned a given risk score, the proportion who actually reoffended was approximately equal for Black and white defendants. They were right about this. And ProPublica was also right about the disparate false positive and false negative rates. Both claims were mathematically true simultaneously.

This is not a contradiction. It is the **fairness impossibility theorem**.

14.5 When Fairness Definitions Conflict

Researchers have formally identified at least three distinct mathematical definitions of algorithmic fairness, and in 2017, Alexandra Chouldechova published a proof that they cannot all be satisfied simultaneously when base rates differ across groups (Chouldechova, 2017).

Demographic parity (also called statistical parity): The proportion of individuals receiving a positive prediction is equal across groups. A hiring algorithm satisfies demographic parity if it recommends candidates for interviews at the same rate regardless of race.

Calibration (predictive rate parity): Among people assigned a given risk score, the actual outcome rate is the same across groups. A recidivism tool satisfies calibration if, among everyone scored 7/10, the same percentage of Black and white defendants actually reoffend.

Equalized odds : False positive rates and false negative rates are equal across groups. A recidivism tool satisfies equalized odds if the rate of falsely labeling a non-reoffending person as high-risk is the same for all demographic groups.

Chouldechova (2017) proved that when the base rate of the predicted outcome differs across groups—when, for example, Black defendants in a jurisdiction are arrested at higher rates than white defendants due to differential policing—it is mathematically impossible for a model to simultaneously satisfy calibration and equal false positive rates.

> This is not a solvable engineering problem. It is a logical impossibility. When base rates differ, satisfying one fairness criterion necessarily violates another. The choice between fairness definitions is a normative, political question about whose errors we are most concerned about, whose false positives are most costly, what justice requires—not a technical question with a correct answer.
> The COMPAS case was not, at bottom, a story about a poorly built algorithm. It was a story about a society choosing, perhaps without realizing it was choosing, which fairness definition it preferred to apply to decisions about human liberty. The algorithm made that choice visible. It did not make the choice.

14.6 The Limits of Human Oversight as an Ethics Solution

There is a tempting but inadequate response to all of this: "Add more human review." If the model is biased, have humans check more decisions. If the algorithm is unfair, make sure people are looking over its shoulder.

This response fails for three reasons.

First, as the Amazon recruiting case illustrates, humans reviewing AI outputs can absorb and launder the AI's biases rather than catching them. When reviewers trust the system, when the system appears to be working,

when the organizational culture treats the algorithm as a tool rather than as an autonomous agent, human review provides compliance theater rather than genuine oversight.

Second, human reviewers carry their own biases. Research on human hiring decisions, judicial sentencing, medical diagnosis, and loan approval has repeatedly found that unaided human judgment is also racially disparate, gender-disparate, and socioeconomically disparate. Adding more humans to a biased process does not neutralize the bias; it may entrench the majority view of what "normal" looks like.

Third, human oversight is most valuable for individual case review and largely blind to systemic effects. A human can evaluate whether the reasoning for a specific decision sounds right. A human cannot, while reviewing individual decisions at production speed, track whether the system is producing disparate outcomes across protected groups at the aggregate level. That requires **statistical auditing**, not case review.

This is not an argument that human oversight is worthless. It is an argument that human oversight, without intentional fairness design, statistical auditing, diverse reviewer teams, and explicit contestability mechanisms, does not constitute meaningful ethical accountability.

14.7 The Accountability Gap

When a HITL system makes a harmful decision, who is responsible?

This question has no clean answer in current legal frameworks, and the ethical answers are contested. Consider a sequence: a defendant is scored high-risk by COMPAS; a judge who knows the score sends the defendant to prison rather than probation; the score was the product of a model trained on racially disparate historical data; the model was built by a private company; the score was used by a public agency; the algorithm is proprietary and cannot be reviewed in open court.

Who bears moral and legal responsibility? The model's developers? The company that sold it? The judge who used it? The agency that deployed it? The political system that produced the racially disparate data?

Current frameworks tend to locate responsibility with the human decision-maker who used the AI output—the judge, the loan officer, the hiring manager. This is not unreasonable: humans should remain accountable for consequential decisions. But it creates a perverse incentive structure when those decision-makers are given little ability to interrogate the AI's reasoning, little visibility into its aggregate effects, and institutional pressure to trust its outputs.

Explainability. Understanding why a system made a specific prediction. If a loan application is rejected and the applicant asks why, they should receive an explanation that is genuinely informative. "The algorithm decided" is not an explanation; it is an evasion. The EU's General Data Protection Regulation establishes a right to explanation for automated decisions affecting individuals, though the legal content of that right remains contested.

Auditability. The ability to independently review a system's behavior at scale. Individual explanations reveal specific decisions; audits reveal patterns across many decisions. Audits can detect disparate impact that no individual case review would ever uncover. Mandatory independent auditing of high-stakes AI systems is an increasingly common policy proposal across jurisdictions.

Contestability. The ability to meaningfully challenge a decision. A person who receives an unfavorable algorithmic decision and is told they can "appeal" to a process that simply re-runs the same algorithm has

not been given meaningful contestability. True contestability requires human review by a decision-maker who has both the authority and the information to override the algorithm, and who is trained to consider whether the algorithmic input was appropriate in the specific case.

14.8 Designing for Fairness: What It Actually Requires

Building a fair HITL system requires more than good intentions and diverse training data, though both help.

It requires deciding, explicitly and transparently, which fairness definition matters most for the specific application—and being honest about the tradeoffs that choice entails. Different applications warrant different choices. A hiring algorithm and a medical screening tool and a recidivism prediction model involve different cost structures around false positives and false negatives, different legal frameworks, and different stakeholder communities.

It requires auditing at the aggregate level, on an ongoing basis, across protected characteristics relevant to the deployment context. Not a one-time pre-launch fairness assessment, but regular monitoring that tracks whether disparate impacts are appearing as the model's outputs change and as deployment context shifts.

It requires diverse teams—not diversity as a compliance gesture, but because the failure modes of AI systems are often visible to people with relevant lived experience and invisible to those without it. The facial recognition systems that failed for darker-skinned faces were built and tested primarily by teams that did not include many people with darker skin. This is not coincidence. Homogeneous teams have systematic blind spots.

It requires authentic contestability: clear pathways for people to challenge unfavorable decisions, implemented by people who understand those pathways are meant to be used, not minimized.

Fairness Is Not a Property

Fairness is not a property a system achieves once and possesses permanently. It is a relationship between a system, a deployment context, and a society—one that requires ongoing attention, ongoing audit, and ongoing willingness to change.

Find a decision that affected you recently and was, in whole or in part, made or influenced by an algorithm—a credit decision, a job application screening, a content moderation decision, a recommendation, a price quoted to you. Ask: Do you know why the decision was made? Do you have any mechanism to challenge it? Would you even know if the algorithm systematically disadvantaged people like you? The gap between what you can answer and what would constitute genuine accountability is the space this chapter has tried to describe.

Chapter Summary

Key Concepts

- Bias enters AI systems at multiple stages: collection bias (whose data?), annotation bias (whose judgment?), model bias (what patterns did it amplify?), deployment bias (who does it affect?), and feedback loop amplification

- HITL does not automatically produce fair systems: human reviewers can absorb and legitimize bias rather than catching it; aggregate effects are invisible to individual case review
- The COMPAS case illustrates that different mathematical definitions of fairness can simultaneously be true and contradict each other
- The fairness impossibility theorem (Chouldechova, 2017): when base rates differ across groups, calibration and equal false positive rates cannot both be satisfied—the choice is normative, not technical
- Explainability, auditability, and authentic contestability are the structural elements of accountable AI deployment

Key Examples

Amazon recruiting AI (2018) Model learned gender proxies; when proxies were removed, it learned new ones; four years of human review without detecting the bias

Gender Shades (Buolamwini & Gebru, 2018) Commercial facial recognition error rates up to 34 percentage points higher for darker-skinned women; collection bias in training datasets

COMPAS (ProPublica, 2016) Both Northpointe's calibration defense and ProPublica's disparate error rate finding were mathematically true; the fairness impossibility theorem in a real case

Predictive policing feedback loops Arrests in over-policed neighborhoods increase, feeding back into the model, intensifying predictions, requiring more resources to the same neighborhoods

Key Principles

- Choose fairness definitions explicitly and transparently, knowing that the choice involves normative tradeoffs
- Audit at the aggregate level, regularly, across protected characteristics
- Design for explainability, auditability, and authentic contestability
- Recognize that human oversight without intentional fairness infrastructure provides accountability theater, not accountability
- Fairness is not a property achieved once—it is an ongoing relationship between a system, a deployment context, and an affected community

---

The next chapter turns to the operational challenge: how do large-scale human annotation and crowdsourcing platforms make it possible—and challenging—to get quality human judgment at scale?

Technical Appendix 14A: Fairness Metrics, Bias Auditing, and Algorithmic Accountability Tools

Fairness in machine learning is not a single property—it is a family of distinct mathematical requirements, some mutually incompatible. This appendix formalizes the major definitions, proves the impossibility theorem, and provides implementation tools for bias auditing and mitigation.

— Mathematical foundations and Python implementations for building and auditing fair AI systems

14.9 Formal Fairness Definitions

Let:

- $\hat{Y}$ = model prediction (binary: 0 or 1)
- Y = true outcome
- A = sensitive attribute (e.g., race, gender, age group)
- X = all other features

Demographic Parity (Statistical Parity).

$$P(\hat{Y} = 1 \mid A = 0) = P(\hat{Y} = 1 \mid A = 1)$$

The probability of a positive prediction is equal across groups. A hiring algorithm satisfies demographic parity if it recommends candidates at equal rates regardless of protected attribute.

Equalized Odds (Hardt et al., 2016).

$$P(\hat{Y} = 1 \mid Y = y, A = 0) = P(\hat{Y} = 1 \mid Y = y, A = 1) \quad \text{for } y \in \{0, 1\}$$

Both the true positive rate (TPR) and false positive rate (FPR) are equal across groups. A medical screening tool satisfies equalized odds if it correctly identifies sick patients—and incorrectly flags healthy ones—at equal rates regardless of demographic group.

Equal Opportunity (weaker form of equalized odds): only the TPR constraint:

$$P(\hat{Y} = 1 \mid Y = 1, A = 0) = P(\hat{Y} = 1 \mid Y = 1, A = 1)$$

Calibration (Predictive Rate Parity).

$$P(Y = 1 \mid \hat{Y} = 1, A = 0) = P(Y = 1 \mid \hat{Y} = 1, A = 1)$$

Among all individuals predicted positive, the fraction who are actually positive is equal across groups.

14.10 The Fairness Impossibility Theorem

Chouldechova (2017) proved that calibration and equal FPR cannot simultaneously hold when base rates differ across groups.

Setup. Let $b_A = P(Y = 1 \mid A)$ denote the base rate of the outcome for group A.

Theorem (Chouldechova, 2017). If $b_0 \neq b_1$ (base rates differ), then no score s can simultaneously satisfy:

1. **Calibration:** $P(Y = 1 \mid s, A) = s$ for all groups
2. **Equal FPR:** $P(\hat{Y} = 1 \mid Y = 0, A = 0) = P(\hat{Y} = 1 \mid Y = 0, A = 1)$
3. **Equal FNR:** $P(\hat{Y} = 0 \mid Y = 1, A = 0) = P(\hat{Y} = 0 \mid Y = 1, A = 1)$

Intuitive proof. For a calibrated model at threshold τ:

$$\mathrm{FPR}_A = \frac{(1 - \mathrm{PPV}_A) \cdot \mathrm{PR}_A}{1 - b_A}$$

where PPV_A is the positive predictive value and PR_A is the prediction rate for group A. If $b_0 \neq b_1$ and PPV is equalized (calibration holds), then FPR equality requires PR_A to adjust in a way that inevitably violates FNR equality. All three cannot hold simultaneously.

```python
import numpy as np
from sklearn.metrics import confusion_matrix

def compute_fairness_metrics(y_true, y_pred, groups):
    results = {}
    for group_val in [0, 1]:
        mask = groups == group_val
        yt, yp = y_true[mask], y_pred[mask]
        tn, fp, fn, tp = confusion_matrix(yt, yp,
            labels=[0, 1]).ravel()
        n = len(yt)
        results[f'group_{group_val}'] = {
```

```
            'TPR':       tp / (tp + fn) if (tp + fn) > 0 else np.nan,
            'FPR':       fp / (fp + tn) if (fp + tn) > 0 else np.nan,
            'PPV':       tp / (tp + fp) if (tp + fp) > 0 else np.nan,
            'pred_rate': (tp + fp) / n,
            'base_rate': (tp + fn) / n,
        }
    return results

def demonstrate_impossibility(base_rate_0=0.2, base_rate_1=0.4, n=2000):
    np.random.seed(42)
    y0 = np.random.binomial(1, base_rate_0, n // 2)
    y1 = np.random.binomial(1, base_rate_1, n // 2)
    y  = np.concatenate([y0, y1])
    groups = np.concatenate([np.zeros(n // 2), np.ones(n // 2)])

    scores = np.where(
        groups == 0,
        np.random.beta(2, 8, n),
        np.random.beta(4, 6, n)
    )
    y_pred  = (scores > 0.3).astype(int)
    metrics = compute_fairness_metrics(y, y_pred, groups)

    for g in ['group_0', 'group_1']:
        m = metrics[g]
        print(f"{g}: TPR={m['TPR']:.3f}, FPR={m['FPR']:.3f}, "
              f"PPV={m['PPV']:.3f}, PredRate={m['pred_rate']:.3f}")
```

```

    fpr_gap = abs(metrics['group_0']['FPR'] -
        metrics['group_1']['FPR'])
    ppv_gap = abs(metrics['group_0']['PPV'] -
        metrics['group_1']['PPV'])
    print(f"\nFPR gap: {fpr_gap:.3f} | PPV gap
        (calibration): {ppv_gap:.3f}")
    print("Note: minimising PPV gap (calibration) cannot
        prevent FPR gap "
          "when base rates differ.")
```

Listing 14.1: Fairness impossibility demonstration

14.11 Bias Detection with Fairlearn

Microsoft's Fairlearn library provides audit dashboards and mitigation algorithms.

```
from fairlearn.metrics import (
    MetricFrame, demographic_parity_difference,
    equalized_odds_difference, selection_rate,
)
from sklearn.metrics import accuracy_score,
    precision_score, recall_score

def full_fairness_audit(y_true, y_pred, sensitive_features,
    feature_name):
    mf = MetricFrame(
        metrics={
            'accuracy':       accuracy_score,
            'precision':      precision_score,
            'recall':         recall_score,
            'selection_rate': selection_rate,
        },
```

```
        y_true=y_true,
        y_pred=y_pred,
        sensitive_features=sensitive_features,
    )
    dp_diff = demographic_parity_difference(
        y_true, y_pred,
            sensitive_features=sensitive_features
    )
    eo_diff = equalized_odds_difference(
        y_true, y_pred,
            sensitive_features=sensitive_features
    )

    print(f"Fairness Audit: {feature_name}")
    print(mf.by_group.to_string())
    print(f"Demographic parity diff:  {dp_diff:.4f}  (0 =
        fair)")
    print(f"Equalized odds diff:      {eo_diff:.4f}  (0 =
        fair)")

    if abs(dp_diff) > 0.1:
        print(f"WARNING: Demographic parity gap
            ({dp_diff:.3f}) exceeds 0.1")
    if abs(eo_diff) > 0.1:
        print(f"WARNING: Equalized odds gap ({eo_diff:.3f})
            exceeds 0.1")

    return {'metric_frame': mf, 'dp_diff': dp_diff,
        'eo_diff': eo_diff}
```

Listing 14.2: Full fairness audit with Fairlearn

14.12 Bias Mitigation Techniques

Three categories of mitigation correspond to three pipeline stages: **pre-processing** (modify training data before training), **in-processing** (add fairness constraints to the learning objective), and **post-processing** (adjust predictions after training).

```python
from fairlearn.reductions import ExponentiatedGradient, DemographicParity
from fairlearn.postprocessing import ThresholdOptimizer
from sklearn.linear_model import LogisticRegression
import numpy as np

# In-processing: Exponentiated Gradient
def train_fair_inprocessing(X_train, y_train, sensitive_train):
    mitigator = ExponentiatedGradient(
        estimator=LogisticRegression(max_iter=500),
        constraints=DemographicParity(),
        eps=0.01,
    )
    mitigator.fit(X_train, y_train, sensitive_features=sensitive_train)
    return mitigator

# Post-processing: Threshold Optimizer
def train_fair_postprocessing(estimator, X_train, y_train, sensitive_train):
    postprocess_est = ThresholdOptimizer(
        estimator=estimator,
        constraints='equalized_odds',
        predict_method='predict_proba',
        objective='accuracy_score',
    )
```

```
    postprocess_est.fit(X_train, y_train,
        sensitive_features=sensitive_train)
    return postprocess_est

# Pre-processing: Reweighing (removes group-label
    correlation by instance weights)
def compute_reweighing_weights(y_train, sensitive_train):
    n = len(y_train)
    weights = np.ones(n)
    for g in np.unique(sensitive_train):
        for l in np.unique(y_train):
            mask  = (sensitive_train == g) & (y_train == l)
            n_gl  = mask.sum()
            if n_gl > 0:
                expected  = (sensitive_train == g).sum() /
                    n \
                          * (y_train == l).sum() / n * n
                weights[mask] = expected / n_gl
    return weights
```

Listing 14.3: In-processing, post-processing, and pre-processing mitigation

14.13 Disparate Impact Ratio (AIF360)

The disparate impact ratio (DIR) is the standard metric used in US employment discrimination law (the "4/5ths rule"):

$$\text{DIR} = \frac{P(\hat{Y} = 1 \mid A = \text{minority})}{P(\hat{Y} = 1 \mid A = \text{majority})}$$

A DIR below 0.8 is a legal threshold for disparate impact in employment contexts.

```
from aif360.datasets import BinaryLabelDataset
```

```python
from aif360.metrics import BinaryLabelDatasetMetric

def compute_disparate_impact(df, label_col, sensitive_col,
                             privileged_val,
                                 unprivileged_val):
    privileged_groups   = [{sensitive_col: privileged_val}]
    unprivileged_groups = [{sensitive_col:
        unprivileged_val}]

    dataset = BinaryLabelDataset(
        df=df[[sensitive_col, label_col]],
        label_names=[label_col],
        protected_attribute_names=[sensitive_col],
    )
    metric = BinaryLabelDatasetMetric(
        dataset,
        unprivileged_groups=unprivileged_groups,
        privileged_groups=privileged_groups,
    )
    dir_val   = metric.disparate_impact()
    mean_diff = metric.mean_difference()

    print(f"Disparate Impact Ratio: {dir_val:.4f}")
    print(f"  (Legal threshold: >= 0.80; below = potential
        disparate impact)")
    print(f"Mean Difference (stat parity): {mean_diff:.4f}")
    if dir_val < 0.80:
        print("ALERT: Disparate impact below 4/5ths rule
            threshold.")
    return {'disparate_impact': dir_val, 'mean_difference':
        mean_diff}
```

Listing 14.4: Disparate impact ratio via AIF360

14.14 Counterfactual Fairness

Counterfactual fairness (Kusner et al., 2017) asks: would the prediction change if the individual belonged to a different demographic group, holding all causally independent features constant?

$$P(\hat{Y}_{A \leftarrow a}(U) = y \mid X = x, A = a) = P(\hat{Y}_{A \leftarrow a'}(U) = y \mid X = x, A = a)$$

This requires a causal model of the data-generating process. The testing principle can be approximated by flipping the sensitive attribute and measuring prediction change.

```python
def counterfactual_fairness_test(model, X_test, sensitive_col,
                                 group_a_val, group_b_val,
                                 non_sensitive_cols):
    X_a         = X_test[X_test[sensitive_col] == group_a_val].copy()
    X_a_counter = X_a.copy()
    X_a_counter[sensitive_col] = group_b_val

    preds_original = model.predict(X_a[non_sensitive_cols])
    preds_counter  = model.predict(X_a_counter[non_sensitive_cols])

    changed = (preds_original != preds_counter).mean()
    print(f"Counterfactual flip rate: {changed:.4f}")
    print(f"  (0 = counterfactually fair; >0 = predictions changed by group flip)")
    return changed
```

Listing 14.5: Approximate counterfactual fairness test

14.15 Feedback Loop Detection

Detecting whether a deployed model is amplifying disparities over time requires monitoring group-level prediction rates across retraining cycles.

```python
import numpy as np
from scipy import stats

def detect_feedback_amplification(predictions_history, group_labels,
                                  n_cycles=5):
    disparity_over_time = []
    for cycle in range(min(n_cycles, len(predictions_history))):
        preds  = np.array(predictions_history[cycle])
        groups = np.array(group_labels[cycle])
        rates  = {g: preds[groups == g].mean() for g in np.unique(groups)}
        disparity_over_time.append(max(rates.values()) - min(rates.values()))

    cycles = np.arange(len(disparity_over_time))
    slope, _, r_val, p_val, _ = stats.linregress(cycles, disparity_over_time)

    print(f"Disparities over cycles: {[f'{d:.4f}' for d in disparity_over_time]}")
    print(f"Trend slope: {slope:.4f}/cycle, R²={r_val**2:.3f}, p={p_val:.4f}")
    if slope > 0 and p_val < 0.05:
        print("WARNING: Statistically significant disparity amplification.")
    elif slope > 0:
        print("NOTE: Upward trend in disparity (not yet statistically significant).")
```

```
    else:
        print("No evidence of feedback amplification detected.")

    return {'slope': slope, 'p_value': p_val,
            'amplification_detected': slope > 0 and p_val < 0.05}
```

Listing 14.6: Feedback loop amplification test

14.16 Pre-Deployment Audit Checklist

A structured audit checklist derived from NIST AI RMF (2023) and EU AI Act requirements:

Stage	Check	Tool	Threshold
Data	Demographic representation in training set	EDA + sampling stats	No group < 5% of
Data	Label distribution by group	MetricFrame	Parity diff < 0.10
Model	Demographic parity at deployment threshold	Fairlearn	Diff < 0.10
Model	Equalized odds	Fairlearn	Diff < 0.10
Model	Calibration by group	ECE per group	Per-group ECE <
Model	Disparate impact ratio	AIF360	DIR ≥ 0.80
Deployment	Slice-based monitoring (monthly)	SliceFinder	Alert at 5% degr tion
Deployment	Feedback loop correlation test	Custom	Slope $p < 0.05 \rightarrow$
Process	Reviewer demographic diversity	Audit log	Document + revie
Process	Contestability mechanism functional	Process audit	100% cases have p way

14.17 Further Reading

- Chouldechova (2017) — Fair prediction with disparate impact. *Big Data, 5*(2).
- Hardt et al. (2016) — Equality of opportunity in supervised learning. *NeurIPS 29.*
- Kusner et al. (2017) — Counterfactual fairness. *NeurIPS 30.*

- Bird, S., et al. (2020). Fairlearn: A toolkit for assessing and improving fairness in AI. *Microsoft Research*.

15 When Things Go Wrong (And How to Fix Them)

> *Every HITL system of sufficient complexity will eventually fail in some way. The organizations that handle failures best are not the ones that never have them—they're the ones that built the infrastructure to find them quickly.*
>
> — The premise of Chapter 15

15.1 The Day the Market Lost Its Mind

At 2:32 PM on May 6, 2010, the Dow Jones Industrial Average began to fall. Not the usual kind of fall—the gradual drift of a nervous afternoon. This was something different. In the space of about fourteen minutes, the index shed nearly 1,000 points, briefly erasing almost $1 trillion in market value. Blue-chip stocks that had been trading at $40 that morning suddenly printed trades at a penny. Procter & Gamble fell from $60 to $39. Accenture, a Fortune 500 company, traded at one cent per share.

And then, just as suddenly, it reversed. By 3:07 PM, the market had recovered most of its losses.

No single bomb had gone off. No company had collapsed. No foreign power had intervened. What had happened, the SEC's post-event analysis would later determine, was that automated trading algorithms had entered a catastrophic feedback loop. A large mutual fund's algorithm had initiated

a trade to hedge its portfolio using futures contracts. Other algorithms, designed to detect and follow large trades, began mimicking the trade. This accelerated the decline, which triggered more algorithms to sell, which accelerated the decline further (Kirilenko et al., 2017).

The humans who should have been in the loop—the traders, the risk managers, the market regulators—were watching their screens with the helpless confusion of someone watching a car accident unfold in slow motion. The circuit breakers that existed were insufficient. The kill switches were hard to reach. Some trading firms simply pulled their bid prices from the market entirely rather than catch the falling knives, which made the cascade worse.

The Flash Crash is the canonical example of what happens when you remove the human from the loop entirely in a complex, high-stakes, real-time system. It wasn't that there were no humans nearby. There were hundreds of them. The problem was that none of them could intervene in time, because the system had been designed—whether deliberately or by architectural accident—to operate faster than human reaction time allowed.

The lesson wasn't only about trading. It was about what failure looks like when HITL design is absent, inadequate, or overwhelmed. And it raised the questions this chapter is about: When HITL systems fail, why do they fail? How do you find the failure? And how do you fix it without making it worse?

15.2 A Taxonomy of HITL Failures

Not all HITL failures look alike. Before you can fix a system, you need to understand which part of it is broken. Across domains—credit scoring, content moderation, medical imaging, fraud detection, hiring tools—HITL failures fall into four distinct categories.

15.2.1 Failure Mode 1: Model Failure

The model itself is producing incorrect predictions. This is the failure mode most people think of first, and it's actually the most straightforward to diagnose, because it leaves a statistical fingerprint. Model failures tend to be systematic: the model consistently underperforms on certain subpopulations, certain input distributions, or certain combinations of features.

The classic example is facial recognition systems that perform well on light-skinned faces and poorly on dark-skinned faces—a finding documented by MIT researcher Joy Buolamwini in her landmark Gender Shades study (Buolamwini & Gebru, 2018). This wasn't a random error. It was a systematic one, traceable directly to training data that overrepresented certain demographic groups.

Model failures manifest as:

- **Distributional shift:** The world has changed since the model was trained. A fraud detection model trained on pre-pandemic spending patterns will struggle with post-pandemic ones.
- **Coverage gaps:** The model was never trained on certain cases. A medical AI trained only on hospital data from one region will behave unpredictably on patients from another.
- **Spurious correlations:** The model learned to use a feature that happened to correlate with the outcome in training but doesn't causally relate to it.

15.2.2 Failure Mode 2: Annotation Failure

The training labels are wrong. This is the sneakiest failure mode, because the model has been doing exactly what it was trained to do—the problem is that it was trained on incorrect ground truth.

Annotation failures are common in any domain where human judgment is subjective, difficult, or fatiguing. In content moderation, annotators asked to label thousands of pieces of content per day for "hate speech" or "misinformation" face compounding problems: annotators disagree with each other; standards drift over time as fatigue sets in; annotators bring their own biases; and the labeling guidelines themselves may be ambiguous or culturally specific.

15.2.3 Failure Mode 3: Design Failure

The model may be accurate, the labels may be correct, but the threshold or routing logic is wrong. The system asks for human review at the wrong times—either too often (alert fatigue) or too rarely (overconfident silence).

Design failures are often invisible in standard evaluation metrics. A fraud detection system might have 99.5% accuracy—a number that looks excellent—while still being systematically wrong on a specific category of transactions that matters enormously.

> A 2019 study in *JAMA Internal Medicine* found that one hospital's decision support system generated over 100 alerts per physician per day. The override rate was 96%. When every signal is noise, the actual signals disappear.

15.2.4 Failure Mode 4: Reviewer Failure

The model is right, the labels are right, the threshold is right—but the humans reviewing the flagged cases are making poor decisions. Reviewer failures happen because of:

- **Reviewer burnout:** Humans reviewing hundreds of cases per day become cognitively exhausted. Decision quality degrades.
- **Automation bias:** Human reviewers who see the model's score before making their own judgment tend to anchor on it—even when the model is wrong. This undermines the entire premise of the human being in the loop.
- **Inconsistent standards:** Different reviewers apply different criteria, creating unreliable labels that corrupt the feedback loop.
- **Perverse incentives:** Reviewers measured on throughput will process faster, review less carefully, and agree with the model more often.

The Four Failure Mode Taxonomy

Model Failure Predictions are systematically wrong for a subpopulation

Annotation Failure Training labels are incorrect, skewed, or drifted

Design Failure Threshold or routing logic is miscalibrated

Reviewer Failure Human decision quality is compromised

The four modes are not mutually exclusive—the most serious HITL failures typically involve multiple modes simultaneously.

15.3 The Diagnostic Question

When a HITL system starts failing, the first task is diagnosis. Before you can fix anything, you need to answer four questions in sequence:

Is the model wrong? Look at the model's performance metrics, sliced by subpopulation, feature combination, and time period. Are there groups of cases where accuracy is systematically lower? Are there time trends suggesting distributional shift?

Are the labels wrong? Look at annotator agreement statistics. Is inter-annotator agreement high? Are there systematic patterns in disagreements? Do different annotation teams produce different labels for the same cases?

Is the threshold wrong? Look at how the system routes cases. What fraction reaches human review? What fraction is auto-decided? Of the cases reaching humans, what fraction are genuinely uncertain versus obviously one thing or the other?

Is the human component failing? Look at reviewer behavior. Is there evidence of automation bias? Are reviewer decisions consistent within reviewers over time? Between reviewers on the same cases?

Each answer points to a different remediation. Getting the diagnosis wrong means applying the wrong fix—which can make things worse.

15.4 Spot Failures and Systematic Failures

There is a critical distinction in failure analysis that determines the urgency and scope of response.

A **spot failure** is a single bad output on a single case. The model misclassified one loan application. The content moderation system incorrectly removed one post. These failures are annoying and should be fixed, but they don't necessarily indicate a broken system.

A **systematic failure** is a pattern of bad outputs across a population of cases. The model is misclassifying all loan applications from a particular ZIP code. The content moderation system is removing a particular category of political speech at ten times the rate of equivalent speech from other perspectives.

Systematic Failures Hide Behind Aggregate Metrics

A model with 95% accuracy is making errors on 5% of cases—but if those errors are concentrated in a specific subpopulation, the harm may be severe while the headline metric looks acceptable. Slice-based evaluation (see the Technical Appendix) is the tool that makes systematic failures visible.

15.5 Case Study: The Credit Scoring Retrospective

In 2019, a mid-sized regional bank discovered something troubling in an audit of its automated lending decisions over the previous three years. The bank had deployed a ML model to supplement its loan officer decisions. The model assigned a risk score, and loan officers were instructed to give extra scrutiny to applications above a certain threshold.

The audit found that applications from borrowers in predominantly minority ZIP codes were receiving risk scores 12–15 points higher than applications from borrowers in predominantly white ZIP codes with equivalent credit profiles—equivalent income, equivalent debt-to-income ratio, equivalent payment history.

The immediate cause was not hard to identify: the model had incorporated ZIP code as a feature, and ZIP code was a proxy for race in ways that the bank's data scientists had not fully appreciated. But tracing the failure deeper revealed a cascade:

1. The model had been trained on historical lending data that itself reflected decades of discriminatory lending patterns. Areas subject to redlining had lower homeownership rates, which the model learned to associate with higher credit risk.

2. The loan officers, presented with the model's scores, had in most cases deferred to them—automation bias at scale.

3. The feedback loop was self-reinforcing: because high-scored borrowers in those ZIP codes were less often approved, the bank had less repayment data from those ZIP codes, making the model's training data even sparser for the affected population.

The bank's legal obligation was significant. Under the Equal Credit Opportunity Act and the Fair Housing Act, the bank was required to notify affected applicants that their applications may have been influenced by a discriminatory process. Over 4,000 notifications were sent.

The story has several HITL lessons. First, the human loan officers were nominally in the loop but had been functionally removed from it by automation bias. Second, the feedback loop between model predictions and training data had allowed the failure to compound over time. Third, and most importantly: when the systematic failure was finally identified, there was an audit trail. The bank could trace which applications had received which scores, which loan officers had made which decisions, and when. That traceability was essential to both identifying the scope of harm and implementing the remediation.

15.6 Case Study: The Political Speech Artifact

In 2021, a large social platform's internal trust and safety team was investigating a pattern of user complaints about over-moderation of political

content. Posts about a specific mainstream policy topic were being removed at roughly 3× the rate of posts about equivalent topics from different political perspectives, at the same actual policy-violation rate.

The team ran accuracy analyses. The model's overall accuracy looked fine. Only when they applied slice-based evaluation—looking at accuracy separately for different categories of political content—did the disparity become visible.

Further investigation traced the problem to a period of intense platform activity around this topic eighteen months earlier, when a surge of content had been annotated under time pressure by a team with access to news coverage framing the topic in a particular way. The annotators had, perhaps unconsciously, applied the news framing as context when making labeling decisions. The model had learned their biases.

The remediation involved three steps: re-annotation with a more demographically diverse team using clearer guidelines; model retraining on corrected labels; and creation of a monitoring system—a "canary dataset" of sensitive political topics—evaluated monthly to detect future annotation drift.

The Annotation Artifact Timeline

The political speech artifact had been running for at least 18 months before detection. The model's aggregate accuracy never showed an anomaly. Detection required someone asking the right question: "Does this model perform differently across political topics?" This is the pattern of annotation failures: they accumulate silently until someone applies slice-based evaluation to the right dimension.

15.7 The Audit Trail: You Can't Fix What You Can't Trace

Both case studies above share a prerequisite: they were diagnosable because the organizations kept detailed records. Every model prediction was logged with a timestamp, the features used, and the prediction output. Every human decision was logged with a reviewer identifier. Every system update was logged with a date and a rationale.

This is not how most HITL systems are built by default. In many organizations, the model runs in a black box. Predictions are made, decisions are executed, and no one asks for a receipt. When a failure is later identified, the team faces the worst possible diagnostic situation: trying to reconstruct what happened from incomplete records.

An audit trail serves three functions:

Detection. Systematic monitoring of logged predictions can surface patterns that would otherwise be invisible.

Diagnosis. When a failure is identified, the audit trail lets you trace it to its cause. Was this a model error, an annotation error, or a threshold error?

Remediation. When harm has occurred, the audit trail tells you who was affected and when. This is the prerequisite for notification, restitution, and making things right.

Building audit trails into HITL systems from the start is not expensive. Storing every prediction with its context costs a fraction of what failure investigation and remediation costs.

15.8 Post-Mortems and the HITL Five Whys

When a HITL failure is identified and remediated, the work is not done. The most valuable thing an organization can do after fixing a failure is understand why it happened—not to assign blame, but to prevent recurrence.

Google's Site Reliability Engineering handbook formalized the concept of the **blameless post-mortem**: the principle that the goal of failure analysis is to understand system failures, not to punish individuals (Beyer et al., 2016). This matters especially in HITL systems, where annotators, reviewers, and data scientists are usually responding rationally to the incentive structures and information environments around them.

The HITL version of the **five whys**—the lean manufacturing technique for tracing failures to root causes—looks like this:

1. Why did the model make systematically incorrect predictions? *Because it was trained on biased labels.*

2. Why were the labels biased? *Because the annotation team was working under time pressure and used available news framing as implicit context.*

3. Why were they working under time pressure? *Because the annotation schedule was set by a product team that didn't understand annotation quality degrades under pressure.*

4. Why didn't the product team understand this? *Because there was no standardized process for communicating annotation quality constraints to product planning.*

5. Why wasn't that process in place? *Because the organization's HITL processes were designed primarily around speed, not quality—and no one owned cross-functional accountability for label quality.*

The fifth "why" reveals a structural or organizational root cause that no amount of technical improvement alone will fix.

15.9 The Framework Applied: Five Dimensions of HITL Failure

The Five Dimensions framework provides a useful lens for categorizing HITL failures:

- The Flash Crash was a **Timing** failure: the system operated on timescales faster than human intervention was possible.
- The credit scoring case was a **Feedback Integration** failure layered on **Uncertainty Detection** failure: the model didn't surface its own uncertainty about ZIP-code-adjacent features.
- The political speech artifact was an **Uncertainty Detection** failure at the annotation level combined with a monitoring design failure.
- The Nest thermostat (Section 1.5) was an **Intervention Design** failure: the human wasn't meaningfully in the loop even when they nominally were.

When diagnosing a HITL failure, a useful first step is to map it onto the Five Dimensions and ask: which dimension failed first? The answer usually points to the most leverage for remediation.

15.10 Recovery: What Good Remediation Looks Like

Once a failure is diagnosed, remediation has three phases:

Stop the bleeding. Reduce ongoing harm while the deeper fix is being designed. Temporarily raise human review thresholds, pause automated decisions in a specific domain, or route a category of cases entirely to human reviewers.

Fix the root cause. Address the actual failure mode—retrain the model on corrected labels, revise the threshold, restructure the review process, or redesign the feedback loop.

Prevent recurrence. Implement monitoring that would have caught this failure earlier. Create a test that encodes this specific failure pattern and will alert if it reappears.

The Infrastructure of Recovery

The organizations that handle HITL failures best are not the ones that never have failures. They are the ones that built the diagnostic infrastructure—audit trails, slice-based monitoring, blameless post-mortems—to find failures quickly, remediate them decisively, and prevent recurrence systematically.

Think of a HITL system you interact with regularly—a content feed, a recommendation engine, a workplace tool, a fraud detection system. Has it ever done something that seemed *systematically* wrong (the same kind of error, repeatedly, on the same kind of case)

> rather than randomly wrong? If so, which of the four failure modes does it most resemble? If you can't tell—what information would you need to diagnose it?

Chapter Summary

Key Concepts

- HITL failures fall into four categories: model failure, annotation failure, design failure, and reviewer failure. Each requires a different diagnosis and a different fix.
- Spot failures (single bad outputs) are different from systematic failures (patterns across a population). Systematic failures demand systematic responses.
- The audit trail—comprehensive logging of predictions, decisions, and system changes—is the prerequisite for diagnosis, remediation, and accountability.
- Post-mortems should be blameless: the goal is to understand system failures, not punish individuals.
- The Five Whys traces failures to structural root causes, which are usually organizational rather than technical.

Key Diagnostic Questions

Is the model wrong? Check slice-based performance metrics across subpopulations and time periods

Are the labels wrong? Check inter-annotator agreement and annotation history

Is the threshold wrong? Check routing statistics and human review rates

Is the human component failing? Check reviewer behavior, consistency, and automation bias indicators

In the next chapter, we extend HITL analysis beyond text and images—into audio, time-series data, scientific discovery, and the physical world of robots and prosthetics.

Technical Appendix: Chapter 15

A15.1 Calibration Analysis

Standard ML evaluation—accuracy, precision, recall, F1, AUC—measures aggregate model performance. In HITL systems, aggregate metrics are necessary but not sufficient. A model with 95% accuracy may have systematic failure modes invisible in aggregate statistics.

Cost-Asymmetric Threshold Design

In HITL systems, the asymmetry between false positive and false negative costs is critical for threshold design. For a binary classifier, the optimal threshold under asymmetric costs is:

$$\theta^* = \frac{C_{FP}}{C_{FP} + C_{FN}} \tag{15.1}$$

where C_{FP} is the cost of a false positive, C_{FN} is the cost of a false negative, and $s(x) = P(y{=}1 \mid x)$ is the model's *calibrated posterior probability*.

This threshold applies when $s(x)$ is a well-calibrated posterior. If the model instead outputs a likelihood ratio or an uncalibrated score, multiply by the class prior ratio: $\tau^* = C_{FP}\,P(y{=}0) \,/\, [C_{FP}\,P(y{=}0) + C_{FN}\,P(y{=}1)]$. For calibrated deep classifiers the two are equivalent after accounting for the prior.

For a cancer screening system with $C_{FP} = \$200$ (unnecessary biopsy) and $C_{FN} = \$50{,}000$ (missed diagnosis):

$$\theta^* = \frac{200}{200 + 50{,}000} \approx 0.004$$

This means the system should auto-decide negative only when it is more than 99.6% confident—and should route to human review otherwise. The aggressively low threshold reflects the severe asymmetry: missing a cancer costs 250× more than a false alarm.

Expected Calibration Error

A well-calibrated model's confidence scores match empirical accuracy: a model that says "90% confident" should be correct about 90% of the time. The standard scalar metric is **Expected Calibration Error** (ECE):

$$ECE = \sum_{b=1}^{B} \frac{|B_b|}{n} |\text{acc}(B_b) - \text{conf}(B_b)| \tag{15.2}$$

where B_b is a bucket of predictions grouped by confidence, $\text{acc}(B_b)$ is empirical accuracy in that bucket, and $\text{conf}(B_b)$ is mean confidence (Guo et al., 2017).

If a model is systematically overconfident, its uncertainty estimates cannot be trusted to trigger human review at the right times. ECE < 0.05 is generally acceptable for HITL routing; ECE > 0.10 requires recalibration (temperature scaling or isotonic regression) before the model's confidence can be used for routing decisions.

A15.2 Slice-Based Evaluation

The most powerful technique for detecting systematic failures is **slice-based evaluation**: instead of computing one aggregate metric, compute metrics separately for meaningful subpopulations.

Defining Slices

Slices should be defined based on domain knowledge:

- **Demographics:** Age, gender, race/ethnicity, geographic region, language
- **Domain-specific attributes:** In medical AI, disease severity, imaging equipment type, institution; in lending, transaction amount, merchant category, account tenure
- **Temporal attributes:** Time of day, day of week, season, year
- **Input characteristics:** Text length, image quality, transaction amount

Automated Slice Discovery

Single-dimensional slicing misses intersectional failures. A model may perform well on each individual attribute while failing on combinations. **Slice finder** methods (Chung et al., 2019) and the Domino approach (Eyuboglu et al., 2022) use clustering and decision-tree-based methods to automatically identify subpopulations where model performance is poor.

```
def compute_slice_metrics(df, slice_col, target_col, pred_col,
                          min_slice_size=50):
    baseline_acc = accuracy_score(df[target_col], df[pred_col])
```

```
    results = []

    for val in df[slice_col].unique():
        slice_df = df[df[slice_col] == val]
        if len(slice_df) < min_slice_size:
            continue

        acc = accuracy_score(slice_df[target_col],
            slice_df[pred_col])

        results.append({
            'slice_val': str(val),
            'n': len(slice_df),
            'accuracy': acc,
            'gap_from_baseline': acc - baseline_acc
        })

    return
        pd.DataFrame(results).sort_values('gap_from_baseline')
```

Listing 15.1: Slice-based evaluation core implementation

A15.3 Behavioral Testing: The CheckList Methodology

The CheckList methodology (Ribeiro et al., 2020) adapts software testing principles to ML models. Instead of asking "what is this model's aggregate accuracy?" it asks "does this model satisfy specific behavioral invariances?"

Three Test Types

Minimum Functionality Tests (MFT). Does the model handle basic, obvious cases correctly?

Invariance Tests (INV). Does the model's output remain stable when inputs are changed in ways that should not affect the output? Example for content moderation: *sentiment toward a topic should not change when the demographic group mentioned in the text is changed to an equivalent group.*

Directional Tests (DIR). Does the model's output change appropriately when inputs are changed in ways that should affect the output? Example: *adding explicit negation ("I do not recommend this") should decrease positive sentiment score.*

CheckList for Annotation Quality Diagnosis

CheckList can be applied to test annotation consistency rather than model behavior. For the annotation failure hypothesis (political speech artifact):

```
def annotation_invariance_test(annotation_sample,
    group_pairs):
    """
    Test whether annotations are consistent across
        equivalent
    content mentioning different demographic or political
        groups.

    group_pairs: list of (group_a, group_b) strings to
        substitute
    """
    consistency_results = []

    for text_template in annotation_sample.templates:
        labels_by_group = {}

        for group_a, group_b in group_pairs:
            text_a = text_template.replace('[GROUP]',
                group_a)
```

```
            text_b = text_template.replace('[GROUP]',
                group_b)

            labels_a = annotation_sample.get_labels(text_a)
            labels_b = annotation_sample.get_labels(text_b)

            kappa = cohen_kappa_score(labels_a, labels_b)
            consistency_results.append({
                'template': text_template,
                'group_a': group_a,
                'group_b': group_b,
                'consistency_kappa': kappa,
                'flag': kappa < 0.6
            })

    return pd.DataFrame(consistency_results)
```

Listing 15.2: Annotation invariance testing

A15.4 Data Debugging with Influence Functions

When a systematic model failure is identified, **influence functions** (Koh & Liang, 2017) provide a principled method for tracing it to specific training examples.

The Influence Function Formula

The influence function estimates the change in model parameters if a single training example were removed:

$$\mathcal{I}(z, z_{test}) = -\nabla_\theta \mathcal{L}(z_{test}, \hat{\theta})^T H_{\hat{\theta}}^{-1} \nabla_\theta \mathcal{L}(z, \hat{\theta}) \tag{15.3}$$

where z is a training example, z_{test} is the test example of interest, $\hat{\theta}$ are the trained parameters, and $H_{\hat{\theta}}$ is the Hessian of the training loss.

Positive influence ($\mathcal{I} > 0$) means upweighting this training example increases the model's loss on the test point—it is a *harmful* example that supports the model's current (potentially incorrect) prediction. Negative influence means upweighting it would reduce test loss, improving accuracy on similar cases.

Practical Application: Label Noise Detection

A simpler application in annotation-failure scenarios is high-loss training example identification:

```
def detect_potential_mislabels(model, X_train, y_train,
                               threshold_percentile=95):
    """
    Training examples with anomalously high loss are
        candidates
    for mislabeling. Return for human re-review.
    """
    preds = model.predict_proba(X_train)

    losses = -np.array([
        y_train[i] * np.log(preds[i, 1] + 1e-10) +
        (1 - y_train[i]) * np.log(preds[i, 0] + 1e-10)
        for i in range(len(y_train))
    ])

    threshold = np.percentile(losses, threshold_percentile)
    suspicious_indices = np.where(losses > threshold)[0]

    return {
        'suspicious_indices': suspicious_indices,
        'n_suspicious': len(suspicious_indices),
```

```
        'fraction_suspicious': len(suspicious_indices) /
            len(y_train)
    }
```

Listing 15.3: Label noise detection via training loss

A15.5 Inter-Annotator Agreement Metrics

For annotation failure diagnosis, the key measurement is inter-annotator agreement.

Cohen's Kappa (two annotators, categorical labels):

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{15.4}$$

where p_o is observed agreement and p_e is expected agreement by chance.

Fleiss' Kappa (more than two annotators):

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e} \tag{15.5}$$

Krippendorff's Alpha (most flexible; handles missing data and ordinal scales):

$$\alpha = 1 - \frac{D_o}{D_e} \tag{15.6}$$

A15.6 The HITL Debugging Workflow

The four tools above are most powerful when used in sequence:

Table 15.1: Kappa Interpretation Guidelines for HITL Annotation Quality

Kappa Range	Interpretation	HITL Action
≥ 0.80	Good	Proceed with training
0.60–0.80	Acceptable	Review guidelines for ambiguous cases
0.40–0.60	Marginal	Revise guidelines; additional annotator trainin
< 0.40	Poor	Halt training; redesign annotation process

1. **Calibration analysis** → Are the model's confidence scores trustworthy? If not, uncertainty detection is broken at the source.

2. **Slice-based evaluation** → Are there subpopulations with systematically lower performance? If so, model failure or annotation failure.

3. **CheckList behavioral tests** → Are there specific invariances that fail? If so, likely annotation failure with demographic or contextual bias.

4. **Influence functions / label noise detection** → Which specific training examples are responsible? Prioritize for relabeling.

5. **Inter-annotator agreement analysis** → Is the labeling process itself reliable? If not, the entire feedback loop is suspect.

This workflow should be run at initial deployment, after every model update, monthly as a routine check, and immediately upon any external complaint about systematic failures.

Part V

Beyond and Ahead

The future won't be about machines that never need help. It will be about machines that are smart enough to know exactly when they do.

— Chapter 17

16 Beyond Text and Images

When data is not static and consequences are physical, the principles of human-in-the-loop design remain the same—but everything about their application changes.

— The premise of Chapter 16

16.1 The Voice That Doesn't Fit the Transcript

On a Tuesday evening in April 2017, a trained crisis counselor named Marcus was monitoring call quality for an anonymous crisis hotline when he noticed something odd in the AI-assisted transcription review queue. A call that the system had flagged as "resolved—no intervention needed" showed a transcript where the caller's words were fine: composed, measured, grateful. The AI had assigned it a low-risk score.

But the audio told a different story.

Marcus had worked crisis lines for eleven years. He'd learned to hear the things that transcripts can't capture: the catch in someone's breath between sentences, the millisecond pause before a word like "okay" that transforms it from an answer to a concession. He played the call. The words were exactly as transcribed. The voice was not the voice of someone who was fine.

He escalated. He was right.

This story illustrates something important about HITL design beyond the domains we've covered so far. Text-based AI can be tested on its text.

Image AI can be evaluated on its images. But the caller's distress was in the prosody—the rhythm, pitch, and timing of speech—not in the semantic content. The model had been trained to read the words. The human was trained to hear the person.

16.2 Domain 1: Audio and Speech

Speech is deceptive. It presents itself as text-with-a-channel, as if the sound were just a delivery mechanism for the words. But decades of research in linguistics and psychoacoustics make clear that the channel carries enormous information: speaker identity, emotional state, health status, engagement level, confidence, deception, fatigue. Strip away the channel and you strip away much of what humans actually respond to.

16.2.1 The Transcription Feedback Loop

The most mature HITL application in speech is transcription correction: a speech recognition system produces a transcript, human reviewers correct errors, and those corrections feed back into model training.

The interesting complication in speech is that correction is bidirectional. When a human corrects "they're gonna leave at three" to "they're going to leave at three," are they correcting an error or imposing a style preference? If the speaker said "gonna," the correct transcription is "gonna." A reviewer who "corrects" it to "going to" is not fixing the model—they are injecting a dialect bias that will cause the model to systematically misrepresent speakers who use informal registers (Koenecke et al., 2020).

In court transcription, medical dictation, and recorded business meetings, systematic dialect bias in transcription can have material consequences for the people whose speech was transcribed. Building transcription HITL requires explicit guidelines that distinguish error correction from style correction, and quality checks that detect systematic correction patterns correlated with speaker demographics.

16.2.2 Speaker Diarization: The Attribution Problem

Speaker diarization—determining "who said what" in a multi-speaker recording—is one of the hardest problems in speech processing, and one of the most interesting HITL design problems.

A diarization model must segment an audio stream, cluster segments by speaker, and label each cluster with a speaker identity. These errors compound: a segmentation error can cause a clustering error, which causes an attribution error. In legal settings—depositions, recorded hearings, criminal investigations—correct attribution determines what each person said and when. Diarization systems used in legal contexts require HITL designs that are unusually careful about error propagation and that maintain an audit trail connecting each attributed segment to the model's confidence in that attribution.

16.2.3 Crisis Detection and the Prosody Gap

The Prosody Gap

Modern NLP models are very good at analyzing the semantic content of speech. They are much worse at the prosodic and paralinguistic features—hesitations, flat affect, tremor—that experienced counselors learn to hear. In mental health applications, the model's uncertainty is not the only relevant signal: there may be cases where the model is confident (the words are fine) but the human expert would not be.

One design approach: a secondary model trained on prosodic features operating in parallel with the text-based model. When the two models disagree—text says low-risk, prosody says high-risk—the case escalates to human review. The disagreement between models is itself a signal of uncertainty even when neither individual model is uncertain. This principle generalizes: in multimodal systems, model disagreement between modalities is often more informative than single-model confidence.

16.3 Domain 2: Time-Series and Sensor Data

Text and images are static. But much of the world's most important data is dynamic: streams of sensor readings, vital sign monitors, industrial process measurements, seismic instruments. Time-series HITL is structurally different from text/image HITL in ways that matter.

16.3.1 Industrial IoT: The 0.1% That Matters

Consider a pressure sensor in a continuous chemical processing plant. It reads every 100 milliseconds, 24 hours a day—approximately 6 million data

points per week. Of those 6 million readings, the vast majority will be within normal operating range. But then there will be a handful—perhaps a dozen per month—where the sensor reading signals something genuinely anomalous: a valve malfunction, an unexpected exothermic reaction, a blocked line.

The human operator's job is not to watch 6 million numbers. It's to handle the 0.1% of alarms that represent genuine anomalies requiring human judgment. The AI system's job is triage: distinguishing between the 99.9% of readings the automatic control system can handle and the 0.1% that need a human.

This creates a specific design challenge. In complex chemical plants, a poorly calibrated alarm system can generate hundreds of alerts per operator per hour—a condition the industry calls **alarm flood**. Research on industrial control room operators shows that alarm flood is one of the most significant contributors to operator error and industrial incidents (EEMUA, 2013).

16.3.2 Earthquake Early Warning: Oversight, Not Review

The earthquake early warning system used in Japan and Mexico broadcasts alerts when seismic P-waves are detected, giving citizens seconds of warning before the more destructive S-waves arrive. The human's role in this HITL system is not case-by-case review—the alert decision is automated. The human's role is system-level oversight: monitoring for false alerts, which damage public trust and cause secondary harms, and checking for system failures that would prevent a true alert from broadcasting.

This is a different kind of HITL—oversight and governance rather than triage and review. As systems become faster and more autonomous, this oversight role becomes increasingly important.

16.3.3 ICU Patient Monitoring: The False Alarm Crisis

In the modern intensive care unit, a patient may be connected to a cardiac monitor, pulse oximeter, blood pressure cuff, ventilator alarm, and medication infusion pump. Studies have found that ICU alarm rates can exceed 700 alarms per patient per day. The false alarm rate in cardiac monitor studies alone ranges from 72% to over 99% (Cvach, 2012).

> Nurses in high-alarm environments report the same phenomenon documented in clinical decision support systems: they stop hearing the alarms. When the one genuine alarm fires, it sounds like the other 699. This is a Stakes Calibration failure at the system level: the HITL design has failed to distinguish between alarms that require immediate intervention and alarms that are merely information.

The solution is not better sensors—the sensors are already excellent. The solution is smarter filtering: a model that integrates all sensor streams in context and produces a ranked patient state assessment rather than a list of individual parameter alarms.

16.4 Domain 3: Scientific Discovery

16.4.1 AlphaFold and the Validation Problem

In 2020, DeepMind's AlphaFold2 achieved a stunning result at the biennial CASP competition: it predicted protein structures with accuracy comparable to experimental methods (Jumper et al., 2021). For many proteins, AlphaFold's predictions were indistinguishable from structures determined by X-ray crystallography.

This was also a HITL design challenge. AlphaFold provides per-residue confidence scores (pLDDT scores) indicating how confident the model is about each part of the predicted structure. Regions with high scores (above 90) are generally reliable. Regions with low scores (below 50) may be genuinely disordered in the protein, or may be regions where the model is guessing.

The pLDDT scores are the HITL interface—they tell the human researcher which parts of the prediction to trust and which to verify experimentally. This illustrates a broader principle in scientific HITL: the model's job is often to narrow the hypothesis space, not to provide final answers. The human's job is to select which hypotheses to test.

16.4.2 Bayesian Optimization in Materials Science

In materials science, a researcher might want to find the composition of a material that maximizes some desired property. The search space is often enormous. Bayesian optimization maintains a probabilistic model of the relationship between composition and property, suggesting at each iteration the composition most likely to produce new information (Shahriari et al., 2016).

The human researcher plays a specific role: they run the experiment, report the outcome, and—crucially—decide whether to trust the reported outcome. Experimental failures in materials science (contaminated precursors, equipment issues, measurement errors) can produce data points that would mislead the optimization if treated as reliable. The researcher's judgment about data quality is part of the feedback signal. The HITL design challenge is to provide the researcher with enough information about the model's current state that they can make informed decisions about when to override the model's recommendation.

16.4.3 Drug Interaction Prediction

Drug-drug interactions (DDIs) are a significant source of adverse events. Machine learning models use molecular graph representations to predict whether two drugs are likely to interact, scanning millions of pairs and flagging candidates for experimental validation. The HITL design involves tiered escalation: high-confidence predictions go into a clinical reference database; low-confidence predictions are routed to pharmacology experts who bring structural reasoning and mechanistic knowledge the model cannot access. The experts' assessments become additional training labels—active learning in a domain where generating ground truth requires laboratory experiments or human expert judgment.

16.5 Domain 4: Robotics and Physical Systems

When AI acts in the physical world, errors have physical consequences. A misclassified email can be recovered from. A robot arm that moves in the wrong direction may not be recoverable. HITL in robotics is not just about improving AI performance—it's about managing the risks of AI acting in domains where mistakes can cause injury, damage, or death.

16.5.1 Teleoperation: The Latency Problem

In hazardous environments—nuclear facilities, deep-sea operations, bomb disposal, space exploration—robots extend the human's reach into places humans cannot safely go. Modern teleoperation incorporates AI assistance: the robot handles low-level motor control (balance, obstacle avoidance), while the human handles high-level task planning (which object to pick up, where to put it). This is **shared autonomy**—the human and the robot each contribute to the execution of a task, with the boundary varying based on task difficulty, model confidence, and communication latency.

The HITL design challenge in teleoperation is latency. When a human on Earth is operating a robot on the Moon, communication delays can exceed a second. The AI assistance layer needs to buffer the human's intentions, execute them smoothly at the robot's end, and alert the human when something unexpected happens that their command stream did not anticipate (Hauser, 2013).

16.5.2 Shared Autonomy: Uncertainty-Driven Handoff

Shared autonomy research, developed for powered wheelchairs and robotic manipulation for users with motor impairments, has produced a framework that applies directly to HITL in physical systems: the robot estimates the probability distribution over the human's intended goal, executes actions consistent with the most likely goal while remaining consistent with other plausible goals, and asks for clarification when the distribution is too uncertain to act effectively (Javdani et al., 2015).

This is Uncertainty Detection, Intervention Design, and Stakes Calibration all at once. The system acts autonomously in the region of high confidence about the human's intent. It asks or slows in the region of uncertainty. It errs toward safety in regions where the cost of a wrong action is high.

16.5.3 Prosthetics and BCIs: The Most Intimate HITL Systems

At the furthest boundary of HITL in physical systems are prosthetics and brain-computer interfaces (BCIs). Here, the human is not simply in the loop—the human's nervous system is the loop.

A modern myoelectric prosthetic hand uses surface electrodes to read electrical signals from the user's remaining muscles. A ML model interprets those signals as intended hand movements. The user's brain sends a signal;

muscles contract; electrodes read the contraction; the model predicts the intended movement; the prosthetic actuates (Farina et al., 2014).

The calibration problem is unusually personal. Every user's residual muscle patterns are different. A model trained on one user's data will perform poorly on another. Personalization—continuous adaptation to the individual user's patterns over time—is not optional; it's the core design requirement. This is Feedback Integration as an intimate, embodied process.

Co-Adaptation

In prosthetics and BCIs, **co-adaptation** describes the mutual adaptation of human and AI system over time: the user adapts to the model, and the model adapts to the user. Over time, a skilled BCI user and a well-designed model converge on a shared language. This is the most intimate form of human-AI collaboration we have yet developed—and it is also the most visible instance of a process that occurs in all HITL systems, albeit more slowly and less obviously.

16.6 What Changes When Data Is Not Static

Across all four domains, a common thread emerges. When data is not static, the design of HITL systems must change in specific ways:

Streaming data requires continuous evaluation. A static text classification system can be evaluated on a held-out test set before deployment. A streaming anomaly detection system is always encountering new data. Performance must be monitored continuously, with alerts for drift and degradation.

Real-time decisions compress the intervention window. In earthquake early warning, in ICU alarms, in teleoperation, the window for human intervention is measured in seconds or less. HITL design must either slow the system to create a meaningful intervention window, or design the human's role as oversight and calibration rather than case-by-case review.

Physical-world consequences change the asymmetry. In text classification, the cost of an incorrect automated decision is usually recoverable. In robotics, in patient monitoring, in industrial control, incorrect automated decisions can have consequences that are not recoverable. This shifts the cost-asymmetry calculation dramatically toward caution.

Multimodal data surfaces disagreements between sensors. When a system integrates audio, video, and text—or pressure, temperature, and flow rate—disagreements between modalities often signal conditions outside the model's training distribution. Model disagreement as an uncertainty signal is a powerful design principle in multimodal HITL systems.

Think of a device or system that monitors something over time—your smartwatch, a weather app, a fitness tracker, a home security system. How does it handle uncertainty in its sensor data? When does it alert you, and when does it act automatically? Apply the Five Dimensions framework: which dimension is handled best?

> Which is handled worst? What would a better HITL design look like?

Chapter Summary

Key Concepts

- HITL design takes structurally different forms across audio/speech, time-series/sensor data, scientific discovery, and robotics.
- In audio, the prosody gap—information in the voice that transcripts don't capture—requires human oversight that model confidence scores alone cannot substitute for.
- In time-series, the challenge is triage at scale: distinguishing the 0.1% of alarms that matter from the 99.9% that don't.
- In scientific discovery, AI narrows hypothesis spaces and suggests next experiments; humans provide experimental judgment and validate results.
- In robotics, physical-world consequences change the stakes calibration for autonomous action. Co-adaptation describes the mutual learning of human and system over time.

In the next chapter, we explore the frontier: systems that get better at knowing what they don't know—and the progression from passive HITL to proactive AI collaboration.

Technical Appendix: Chapter 16

A16.1 Audio and Speech HITL Metrics

Word Error Rate with Demographic Slicing

Word Error Rate (WER) is the standard transcription metric:

$$WER = \frac{S + D + I}{N} \tag{16.1}$$

where S is substitutions, D is deletions, I is insertions, and N is the number of words in the reference transcript.

For HITL audit purposes, WER must be computed by speaker demographics (not just in aggregate). A flagging threshold of 30% worse than the overall WER is a practical starting point; the appropriate threshold depends on the application's sensitivity requirements.

> Koenecke et al. (2020) found that five major commercial ASR systems had WER of 19–23% for Black speakers versus 5–8% for white speakers on matched audio. This 3–4× gap represents a systematic HITL failure: annotation from these systems will have systematically different reliability across speaker demographics.

Diarization Error Rate

Diarization Error Rate (DER) evaluates speaker attribution:

$$DER = \frac{\text{Missed Speech} + \text{False Alarm Speech} + \text{Speaker Confusion}}{\text{Total Reference Speech Time}} \tag{16.2}$$

For HITL routing in legal and medical contexts, DER by confidence bin is critical: if high-confidence attributions are not more accurate than low-confidence attributions, the confidence score cannot be used to route cases to human review.

Multimodal Crisis Escalation Logic

The crisis detection escalation combines semantic and prosodic risk scores:

$$\text{Escalate} = \begin{cases} 1 & \text{if } r_{\text{combined}} > \theta_H \\ 1 & \text{if } |r_{\text{text}} - r_{\text{prosody}}| > \delta \\ 0 & \text{otherwise} \end{cases} \tag{16.3}$$

where $r_{\text{combined}} = \alpha r_{\text{text}} + (1 - \alpha) r_{\text{prosody}}$, θ_H is the high-risk threshold, and δ is the disagreement threshold. Setting δ appropriately ensures that the Marcus scenario—text risk low, prosodic risk high—always triggers escalation.

A16.2 Time-Series HITL Metrics

Alarm Quality Metrics

Four metrics characterize alarm system quality:

Alarm Precision. Of all alarms generated, what fraction were genuine anomalies?

$$\text{Precision} = \frac{TP}{TP + FP} \tag{16.4}$$

Alarm Burden. Average alarms per operator per hour. EEMUA Publication 191 recommends a maximum of 6–12 alarms per operator per hour; above 20 is considered "very likely unmanageable" (EEMUA, 2013).

Actionability Rate. Fraction of reviewed alarms that resulted in a human intervention.

Critical Miss Rate. Fraction of genuine anomalies that the system failed to alarm on.

Table 16.1: Alarm Management Performance Thresholds (EEMUA 191)

Alarm Burden	Category	HITL Implication
< 1/hr	Very low	Possible under-detection
1–6/hr	Acceptable	Target range for manageable overs
6–12/hr	Manageable	Review alarm priority logic
> 12/hr	Overloaded	Alarm fatigue; redesign filtering
> 20/hr	Very likely unmanageable	Immediate intervention required

Temporal Uncertainty Quantification

For time-series anomaly detection, uncertainty estimates must account for temporal context. Monte Carlo Dropout applied to a sequence model provides epistemic uncertainty estimates that can be augmented with changepoint proximity:

$$u_t^* = u_t \cdot (1 + \beta \cdot \mathrm{CP}_t) \tag{16.5}$$

where u_t is the base uncertainty estimate, CP_t is a changepoint proximity score (0–1), and β is a scaling factor. Uncertainty is higher near recent changepoints in the input signal.

A16.3 Scientific Discovery HITL Metrics

AlphaFold pLDDT Confidence Classification

Table 16.2: AlphaFold pLDDT Score Interpretation for Drug Design HITL

pLDDT Range	Interpretation	Structural Reliability	Validation Requi
≥ 90	Very high	Comparable to experiment	Low
70–90	High	Backbone reliable	Medium (side chain
50–70	Low	Uncertain	High
< 50	Very low	Likely disordered	Critical

For drug design applications, the critical metric is the fraction of **binding site** residues in low or very-low confidence categories—not the overall structure mean. A protein with mean pLDDT of 85 may still have an unreliable binding site if the active site residues cluster in the 50–70 range.

Bayesian Optimization Loop Metrics

Convergence rate Number of iterations to reach 90% of the best observed objective value

Human override rate Fraction of model suggestions the researcher overrode

Human value-add Average objective improvement of overridden experiments minus average objective improvement of model-suggested experiments. Positive values indicate the researcher is adding value beyond the model; negative values may indicate the researcher is underconfident in the model.

Data quality flag rate Fraction of experimental outcomes flagged by the researcher as unreliable. High rates indicate an experimental process quality issue rather than a model quality issue.

A16.4 Robotics and Physical Systems HITL Metrics

Shared Autonomy Calibration

A key metric for shared autonomy systems is the correlation between system uncertainty and human intervention rate:

$$\rho = \text{Corr}(\text{Uncertainty}_t, \mathbb{1}[\text{Human Intervened}_t]) \tag{16.6}$$

A high positive correlation ($\rho > 0.5$) indicates the system is correctly identifying when to ask for help. A low or negative correlation indicates that the system is asking for help on the wrong cases—a **Timing** failure in the Five Dimensions framework.

Prosthetic Adaptation Metrics

Adaptation trajectory Accuracy slope per session (bpm/session). A positive slope indicates continuing improvement; slope < 0.005 suggests convergence.

Correction rate Fraction of actuations where the user re-attempted due to incorrect movement. Above 15% suggests recalibration is needed.

Per-movement accuracy Accuracy by intended movement class. Identifies specific movements that require targeted retraining.

Latency Time from muscle signal to actuation onset. Should remain stable or decrease as adaptation proceeds; increasing latency may indicate hardware issues.

A16.5 Universal Metrics for Non-Static HITL Systems

Distribution Shift Detection

The Kolmogorov-Smirnov two-sample test detects whether the current data distribution has shifted from the training distribution:

$$D_{KS} = \sup_x |F_1(x) - F_2(x)| \tag{16.7}$$

where F_1 and F_2 are the empirical CDFs of the training and recent data samples. Applied feature-by-feature, this identifies which input dimensions have drifted—pointing toward which aspect of the model's training distribution is no longer representative.

HITL Health Dashboard Metrics

For continuous deployment monitoring:

- **Human-model agreement rate**: Fraction of cases where the human agrees with the model's recommendation. Below 70% suggests model degradation or threshold misalignment.
- **Review rate**: Fraction of cases routed to human review. Above the target range suggests model degradation or threshold too conservative; below suggests overconfidence.
- **Reviewer decision consistency**: Cohen's κ between the same reviewer's decisions on matched cases at different time points. Below 0.6 suggests reviewer fatigue or changing standards.
- **Outcome tracking**: Where outcomes are observable, track accuracy of decisions including human review, not just pre-review model predictions.

17 Almost-Autonomous: When AI Learns to Ask Better Questions

> *Active learning, proactive assistance, and the frontier of AI that knows what it does not know.*

17.1 The Student Who Got Better at Asking

Dr. James Murray had been supervising medical students for twenty years when he encountered a student who did something he had never seen before. Most students came to him with questions they already knew the answer to—questions designed to confirm their diagnosis, to hear the attending physician say "yes, you're right." It was a form of seeking reassurance rather than knowledge.

This student came with different questions. She would say: "I think it is likely A or B. I have ruled out C for this reason. What I do not know is whether this presentation is consistent with D—is that worth pursuing?" She was asking about her uncertainty. She was not asking what to think. She was asking about the specific, bounded regions of her knowledge where she needed input.

Murray had trained hundreds of students. The ones who improved fastest, he said, were the ones who developed the capacity to locate and name their uncertainty precisely. Not "I do not know"—that is too vague to be actionable. Not "I know"—that forecloses learning. But "I know A

and B and C, and the specific thing I do not know is X"—that is a question a teacher can answer.

This capacity—to locate uncertainty precisely and ask about it efficiently—is one of the most valuable things an AI system can develop. And in the past decade, a suite of techniques has emerged that begin to give AI systems something like this capacity.

17.2 Active Learning: The System That Learns to Use Your Time Well

The core idea of **active learning** is simple: not all data points teach a model equally. Some examples confirm what the model already knows. Others teach it something new. Active learning is a framework for directing human labeling effort toward the examples that will improve the model most efficiently.

In a standard HITL setup without active learning, cases are routed to human review based on model uncertainty alone: uncertain cases go to humans, confident cases are handled automatically. This is better than random review. But it does not distinguish between two types of uncertain cases:

Near-boundary uncertain cases. The model is uncertain because the input is near the decision boundary—it could plausibly belong to either class. A human label on this case directly informs the model's boundary.

Outlier uncertain cases. The model is uncertain because the input is far from any training examples—it is an extreme outlier, unlike anything the model has seen. A human label on this case is informative but may be less efficiently placed than a label on a near-boundary case.

Active learning adds a second layer of routing: among uncertain cases, which ones are worth prioritizing for human review?

Several strategies have been developed.

Uncertainty sampling. Route the cases where the model is most uncertain. Simple and effective. Preferentially samples near the decision boundary.

Diversity sampling. Route cases that are dissimilar from existing training data. Expands coverage of the training distribution rather than just refining the boundary.

Query by committee. Maintain multiple models (an ensemble) and route cases where the models disagree most. Model disagreement is a proxy for the cases where the current training data is insufficient.

Expected model change. Route the cases that, if labeled, would change the model's parameters most significantly. Computationally intensive but theoretically optimal.

In practice, hybrid approaches combining uncertainty and diversity sampling tend to work well. The practical benefit: in empirical studies, active learning typically achieves the same model performance with 30–70% fewer labeled examples than random sampling. For organizations where annotation is expensive, this is a major efficiency gain.

17.3 The System That Improves Over Time

Active learning creates a virtuous dynamic: the model gets better at knowing what it does not know, which makes its queries to humans more

efficient, which makes the human labels more targeted, which makes the model better faster.

This is the operational embodiment of Dimension 5 in the Five Dimensions framework: feedback integration. A system with well-designed active learning is not just using human labels to correct errors. It is systematically building competence in its weakest areas.

In practice, this means the character of human review changes over time in a well-designed HITL system:

Early phase Humans review many cases, including many that seem relatively straightforward. The model is young and uncertain about a lot.

Middle phase Humans review fewer cases, more concentrated near the current decision boundaries. The easy cases have been learned. The hard cases get more attention.

Late phase Humans review a small fraction of cases, concentrated on genuinely difficult edge cases and novel categories. The model handles everything else with high confidence. The human role is more expert and less routine.

This trajectory does not happen by accident. It requires attention to what cases are being routed and why. Systems that route randomly or by simple uncertainty thresholds without active learning integration do not exhibit this trajectory—they keep routing the same types of uncertain cases forever.

17.4 RLHF, DPO, and Constitutional AI: A General Reader's Guide

The techniques that have driven recent advances in large language models have a deep connection to human-in-the-loop design. Understanding them at a conceptual level is valuable for anyone thinking about the future of HITL systems.

Reinforcement Learning from Human Feedback (RLHF) is the technique behind the instruction-following capabilities of ChatGPT, Claude, and similar systems. The basic idea: train a language model to generate responses, show pairs of responses to human raters who indicate which is better, train a reward model on these preferences, and then use the reward model to fine-tune the generative model to produce responses humans prefer (Christiano et al., 2017; Ouyang et al., 2022).

RLHF is a form of active learning in the preference dimension: the model is being guided not by "correct labels" but by human preference signals. This makes the annotation task easier—judging which of two responses is better is often easier than generating the correct response from scratch—while capturing rich information about what humans find valuable, helpful, and safe.

The HITL dimension of RLHF is the human preference collection stage. The quality and diversity of the humans providing preferences shapes the values the model learns. If the preference raters are a homogeneous group (geographically, demographically, culturally), the model will learn to optimize for that group's preferences—which may not align with the preferences of the model's global user base.

Direct Preference Optimization (DPO) simplifies RLHF by eliminating the explicit reward model step. Instead of training a reward model and

then fine-tuning against it, DPO directly updates the model parameters to maximize the likelihood of preferred responses relative to rejected ones (Rafailov et al., 2023). The human feedback collection is the same; the technical training pipeline is simpler and more stable.

Constitutional AI (CAI) , developed by Anthropic, takes a different approach: instead of collecting human preferences for all fine-tuning, define a set of principles (the "constitution") and train the model to evaluate its own outputs against those principles (Bai et al., 2022). Human oversight is still present—in the design of the constitution and in some feedback stages—but the scale of human input required is reduced because the model can apply the principles to self-critique its own outputs.

RLHF, DPO, and Constitutional AI all involve human judgment at a stage of the training process, not just the deployment process. The humans in the loop are shaping the model's values and capabilities, not just catching its errors. This is a more fundamental form of human-AI collaboration than the case-by-case review of deployed outputs.

17.5 Proactive Assistance: The AI That Asks Before You Realize You Need Help

Most HITL systems are reactive: the AI processes a case, generates uncertainty, and then asks for help. The human responds when asked.

A more sophisticated design is **proactive**: the AI anticipates when human input will be needed and asks before the situation becomes critical.

Consider a medical decision support system that monitors a patient's vital signs continuously. Rather than only alerting when a threshold is breached (reactive), it can detect that a patient's trend lines are heading

toward deterioration—that the current trajectory, if continued for four hours, will require intervention—and alert clinical staff while there is still time to act proactively. This is uncertainty about the future state, not just the current state. And acting on it early is often less costly and more effective than acting after a crisis.

In the language AI context, proactive HITL means the system recognizes when a user's request is in a category where its outputs are less reliable, and flags this proactively: "I should mention that my information on this topic may be outdated—you may want to verify with a current source." This is not the system being asked to express uncertainty. It is the system recognizing its limitations and surfacing them without being asked (Kadavath et al., 2022).

The engineering challenge of proactive HITL is knowing what the system does not know before it is asked. This requires the uncertainty quantification capabilities discussed in Chapters 2 and 3, applied not just to individual predictions but to the characteristics of the query type: "this type of question is where I tend to be less reliable."

17.6 The Frontier: AI That Knows What It Doesn't Know

The deepest challenge in HITL design—and the most active research frontier—is building AI systems that have accurate self-knowledge about their own limitations.

Current systems know something about their uncertainty on individual inputs: calibrated confidence scores, ensemble variance, Monte Carlo dropout estimates. These tell you how uncertain the model is about this specific prediction.

What current systems do not know well is the *structure* of their ignorance: where the training distribution is thin, what categories of questions

they systematically underperform on, what input characteristics predict poor calibration. This meta-level self-knowledge—knowing not just that you are uncertain here, but why you are uncertain and where else you will be uncertain for the same reason—would enable a fundamentally different quality of HITL.

A system with accurate meta-level self-knowledge could tell a human reviewer: "This case has the same characteristics as a category of cases where I have historically been wrong 30% of the time. My confidence score here does not fully capture this." It would be asking for help not based on local uncertainty but based on category-level track record.

Research into AI "knowing what it does not know" is advancing along several fronts: epistemic uncertainty quantification in large language models, out-of-distribution detection, calibration on rare categories, and formal verification of model behavior. None of these is yet at the point where AI systems can reliably provide this kind of meta-level uncertainty communication. But the direction is clear, and the tools are improving.

17.7 What "Almost-Autonomous" Means and Why It's the Right Goal

The title of this chapter claims that "almost-autonomous" is the right goal. This requires some justification.

The fully autonomous goal—AI that handles everything without human involvement—has appeal. It is simpler. It is faster. It requires no human infrastructure. But it fails in the places that matter most: novel situations, high-stakes edge cases, questions that require value judgment, contexts where accountability requires a human in the decision chain.

The purely human-dependent goal—AI that flags everything for human review—defeats the purpose of AI-assisted systems. It adds the cost of AI without capturing its benefits.

The "almost-autonomous" goal captures both: AI systems that handle the vast majority of cases efficiently and autonomously, that know their own limitations accurately enough to route appropriately, that ask for human help at the specific moments where it makes a material difference, and that improve over time through the feedback their asking generates.

The Almost-Autonomous Ideal

This is what Dr. Murray was describing in his best medical students. They were not fully autonomous—they still had things to learn. They were not dependent on supervision for everything—they handled most cases on their own. They were almost-autonomous: capable and independent in their areas of competence, precise and honest about their areas of uncertainty.

That capacity—to be capable and honest at the same time—is what the best HITL systems aspire to. It is not the absence of human involvement. It is the intelligent management of human involvement, directed to where it creates the most value.

The next time you use an AI assistant and it gets something wrong, pay attention to whether the system expressed uncertainty before you noticed the error, or expressed confidence and was wrong. Keep track for a week. How often does the system's expressed confidence accurately predict when it is right vs. when it is wrong? This is a practical calibration audit—and it will give you a real sense of how close or far any particular system is from the "knows what it does not know" ideal.

Chapter Summary

Key Concepts

- Active learning directs human labeling effort toward the examples that improve the model most efficiently—typically achieving equivalent performance with 30–70% fewer labels
- RLHF, DPO, and Constitutional AI are HITL at training time—humans shaping model values, not just catching deployment errors
- Proactive HITL: systems that ask before the uncertainty becomes critical, rather than only when a threshold is breached
- The frontier: AI systems that know the structure of their ignorance, not just the magnitude of their uncertainty on individual cases
- "Almost-autonomous" captures the right goal: capable and independent in areas of competence, precisely honest about uncertainty

Key Examples

Active learning sample efficiency 30–70% reduction in annotation needs through intelligent case selection

RLHF preference collection ChatGPT and Claude trained on human preference comparisons rather than gold labels—and the diversity implications for whose preferences are captured

Medical monitoring Detecting deterioration trends before crises—proactive rather than reactive HITL

Constitutional AI Scalable self-oversight through principle application, reducing but not eliminating human annotation requirements

Five Dimensions Check

- All five dimensions: this chapter describes the mature form of a system that has developed all five—detecting uncertainty, asking well, timing correctly, calibrating to stakes, and integrating feedback over time

---

Final chapter: your role in this future—not despite AI capability, but because of it.

Technical Appendix 17A: Active Learning Theory and Advanced Alignment Techniques

Chapter 17 describes the frontier of automation—where systems learn from increasingly sparse human feedback, align to human preferences rather than fixed labels, and detect their own limits. This appendix formalizes the theory behind each of those capabilities.

— Mathematical foundations for active learning, RLHF, and meta-uncertainty

17.8 Active Learning: Formal Framework

17.8.1 Query Complexity

Standard supervised learning requires $m = O(d/\varepsilon^2)$ labeled examples to achieve error ε in d-dimensional space. Active learning's key theoretical result: for many problem classes, the query complexity is $O(d/\varepsilon)$—a quadratic improvement.

This speedup arises because active learning concentrates queries near the decision boundary, where labels provide the most information about the target function f^*.

17.8.2 Version Space Active Learning

The **version space** V_n is the set of hypotheses consistent with all n observed labels. Active learning strategies can be viewed as minimizing the expected version space volume after each query:

$$x^* = \arg\min_{x \in \mathcal{U}} \mathbb{E}_{y \sim p(y|x)} \left[\mathrm{Vol}(V_{n+1}(x, y)) \right]$$

Selecting the query that maximally shrinks the version space is equivalent to selecting the most informative query under an information-theoretic criterion.

17.8.3 BADGE: Batch Active Learning by Diverse Gradient Embeddings

BADGE (Ash et al., 2020) combines uncertainty and diversity in a principled way:

1. For each unlabeled point x_i, compute the gradient of the loss with respect to the predicted label: $g_i = \nabla_\theta L(f_\theta(x_i), \hat{y}_i)$
2. Select a batch S by k-means++ initialization on the $\{g_i\}$ vectors

The gradient magnitude captures uncertainty (gradients are larger near the decision boundary); k-means++ selection enforces diversity across the batch.

```python
import numpy as np
from sklearn.cluster import kmeans_plusplus

def badge_select(model, unlabeled_X, batch_size: int) ->
    np.ndarray:
    """Select batch using BADGE: gradient embeddings +
        k-means++ init."""
    probs     = model.predict_proba(unlabeled_X)
```

```
    n_classes = probs.shape[1]
    n_samples = unlabeled_X.shape[0]

    pseudo_labels      = probs.argmax(axis=1)
    gradient_embeddings = np.zeros((n_samples, n_classes))

    for i in range(n_samples):
        label      = pseudo_labels[i]
        confidence = probs[i, label]
        gradient_embeddings[i, label] = 1 - confidence
        for j in range(n_classes):
            if j != label:
                gradient_embeddings[i, j] = -probs[i, j] *
                    confidence

    _, indices = kmeans_plusplus(gradient_embeddings,
        n_clusters=batch_size,
                                 random_state=42)
    return indices
```

Listing 17.1: BADGE selection via gradient embeddings and k-means++

17.9 RLHF: Mathematical Formulation

17.9.1 The Reward Model

Given preference pairs $\mathcal{D} = \{(x, y_w, y_l)\}$, the reward model r_ϕ is trained to maximize:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)\Big]$$

This is a Bradley-Terry model of preference:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$$

17.9.2 PPO Fine-Tuning

With a trained reward model, the policy π_θ is fine-tuned to maximize:

$$\mathscr{L}_{\text{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathscr{D},\, y\sim\pi_\theta(y|x)}\Big[r_\phi(x,y) - \beta \cdot \text{KL}\big(\pi_\theta(\cdot|x)\|\pi_{\text{ref}}(\cdot|x)\big)\Big]$$

The KL penalty β prevents the fine-tuned model from drifting too far from π_{ref}, preserving general capabilities while aligning outputs with human preferences (Christiano et al., 2017; Ouyang et al., 2022).

17.9.3 Direct Preference Optimization (DPO)

DPO (Rafailov et al., 2023) eliminates the explicit reward model:

$$\mathscr{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

DPO is mathematically equivalent to RLHF under certain assumptions but simpler to train: no reward model, no PPO loop.

17.10 Meta-Uncertainty: Out-of-Distribution Detection

17.10.1 Energy-Based OOD Detection

Liu et al. (2020) propose:

$$E(x;\, f) = -T \cdot \log\sum_c e^{f_c(x)/T}$$

where $f_c(x)$ is the logit for class c and T is a temperature parameter. In-distribution inputs have lower energy; OOD inputs have higher energy, enabling threshold-based routing to human review.

17.10.2 Calibration Under Distribution Shift

Standard ECE is computed on a test set drawn from the same distribution as training. Under distribution shift, ECE typically degrades. A meta-calibration measure:

$$\text{ECE-shift} = \mathbb{E}_{P' \neq P}\big[\text{ECE}(f, P')\big]$$

Models with lower ECE-shift are more reliably calibrated across deployment contexts—a critical property for HITL systems where deployment conditions evolve.

17.11 Exercises

Exercise 17.1 — Active Learning Comparison

On a binary classification dataset, implement uncertainty sampling, random sampling, and BADGE. Start with 100 labeled examples, query 50 per iteration for 10 iterations. Plot accuracy vs. annotation budget. What are the annotation savings at 85% accuracy?

Exercise 17.2 — RLHF Simulation

Create a simple RLHF simulation: 1,000 synthetic text summaries with a known reward function (conciseness). Simulate human preferences with 10% noise. Train a reward model. Compare learned rankings to ground truth (Spearman ρ). At what noise level does the reward model fail to recover the ground-truth ranking?

Exercise 17.3 — OOD Detection and Human Routing

Train an image classifier on CIFAR-10. Evaluate OOD detection on SVHN: compute softmax confidence and energy scores for both datasets. Which better separates in-distribution from OOD? At what threshold would you route to human review? Plot the precision/recall tradeoff for routing decisions as the threshold varies.

18 Your Role in This Future

> *The human's enduring role—not despite AI capability, but because of it.*

18.1 The Last Chapter Is the Beginning

There is a temptation, at the end of a book about human-AI collaboration, to write a rousing conclusion about how humans will always be irreplaceable—how our creativity, our empathy, our wisdom will forever distinguish us from the machines we build. It would be comforting. It would also be slightly dishonest.

The truth is more interesting and more demanding.

Machines are becoming capable in ways that are genuinely surprising. Large language models write text that, in many contexts, is indistinguishable from human writing. Image generators produce visual art. Music generators produce songs. Code generators produce software. The accelerating pace of AI capability development is real, and pretending it is not changes nothing about it.

And yet: in every domain where AI capability has advanced most dramatically, the demand for thoughtful human involvement has not decreased. It has changed character. And in the ways that matter most—in the decisions with real consequences for real people, in the moments where judgment requires grounding in a lived life, in the systems that need to earn

trust—the human role is not diminishing. It is becoming more important, and more demanding.

This chapter is not a reassurance. It is an invitation.

18.2 What AI Cannot Replace (and Why That's Permanent)

The previous chapter described a frontier: AI systems that know what they do not know, that ask for help at precisely the right moments, that improve over time through human collaboration. This is the ideal. Even in that ideal, there are things AI systems cannot do that humans can.

Accountability with consequence. When an AI system makes a decision that harms someone, there must be a human or institution that can be held accountable—not because we lack the technology to make AI more autonomous, but because accountability is a social relationship. A person who bears responsibility for a decision is motivated to exercise judgment carefully. An algorithm does not bear responsibility in the sense that creates this motivation. The human in the loop is not just a cognitive resource; they are the locus of accountability. This is why the Air Canada tribunal ruled that the company, not the chatbot, was responsible for the chatbot's error (Civil Resolution Tribunal of British Columbia, 2024). Accountability requires a human agent.

Value judgment in genuinely contested terrain. Many consequential decisions involve not uncertainty about facts but contestation about values. How should we balance privacy and security? Who should bear the cost of errors in criminal justice algorithms? What counts as harmful content in a culturally diverse global community? These are not questions that have correct answers waiting to be computed. They are questions on which

reasonable people disagree, and where the resolution requires democratic deliberation, not optimization.

Representation of lived experience. A model trained on data about a community is not the same as a member of that community exercising judgment about it. A system trained on clinical records from a hospital is not the same as a patient navigating that hospital's care. The perspective of someone who lives with the consequences of a system's outputs is not reducible to a data signal. It is a form of knowledge that must be gathered through deliberate inclusion, not inferred from historical records.

Situated moral responsibility. When you review an AI's recommendation and confirm or override it, you are not just providing information to the system. You are taking responsibility for the outcome. The human in the loop is the point at which responsibility is allocated. This is not a limitation of current technology. It is a feature of moral and social life that is not subject to technical resolution.

18.3 New Jobs That HITL Creates

The emergence of AI as an economic force has produced a well-documented labor disruption narrative: AI will automate jobs, workers will need to reskill, displacement is coming. This is true in some sectors. It is not the whole story.

Human-in-the-loop AI systems create real and growing demand for people with specific skills.

AI trainers and annotation specialists label data, evaluate outputs, and provide the human signal that makes AI systems improve. This is not a temporary job. It is a permanent function in organizations that deploy

learning AI systems. The Bureau of Labor Statistics does not yet have a job category for this work, but the employment is real and growing.

AI auditors and fairness analysts evaluate AI system outputs for bias, error patterns, and systematic failures. As AI deployment grows and regulatory requirements tighten, demand for people who can rigorously evaluate AI system fairness and performance has grown rapidly. This is an interdisciplinary role combining statistics, domain knowledge, and ethics.

Human oversight coordinators manage the human review workflow in production HITL systems—staffing, training, quality control, escalation protocols. These are operational roles that require both domain expertise and understanding of AI system behavior.

AI red teamers systematically attempt to find failure modes in AI systems—adversarial inputs, edge cases, ways the system can be manipulated or misused. Red teaming has become a standard practice for responsible AI deployment, and skilled red teamers are in demand.

Conversational AI evaluators assess the outputs of language models on dimensions that automated metrics cannot capture: cultural sensitivity, factual accuracy in specialized domains, appropriate hedging, quality of uncertainty communication. These roles exist at every major AI company and are growing.

AI policy specialists work at the intersection of AI capability and governance—in government agencies, regulatory bodies, advocacy organizations, and within AI companies. The regulatory landscape for AI is developing rapidly, and there is a severe shortage of people who understand both the technology and the policy terrain.

18.4 Skills That Matter in a HITL World

The skills that make humans valuable in human-AI collaboration are not the same as the skills that made humans valuable in labor markets that did not include AI. Understanding this is practically important.

Domain expertise combined with AI literacy. The most valued position in almost every AI-adjacent field is a person with deep domain knowledge (medicine, law, finance, education, public health) who also understands how AI systems work well enough to evaluate their outputs critically. Not a computer scientist—a doctor who understands what a medical AI is actually measuring. A lawyer who knows what a legal AI can and cannot reliably assess. Domain expertise alone is devalued by AI capability in the routine application domain. Domain expertise combined with AI literacy is dramatically more valuable.

Judgment about edge cases and novel situations. AI systems handle well-defined, well-populated problem categories efficiently. They struggle with novel situations, genuine ambiguity, and cases that lie outside their training distribution. Humans who can recognize when they are outside the AI's competence zone, and exercise sound independent judgment in those situations, are valuable.

Clear communication about uncertainty. In a world where AI systems produce confident-sounding outputs, humans who can clearly communicate the limits of their own and AI knowledge—who can say "I do not know, and here is what we would need to find out"—provide distinctive value.

Adversarial thinking. Understanding how AI systems fail, what their systematic blind spots are, how they can be gamed—this meta-knowledge

about AI limitations is increasingly valuable in evaluation, red-teaming, and oversight roles.

Contextual judgment. The capacity to integrate information from multiple sources, hold contextual knowledge that does not fit in a prompt, recognize when stated facts do not fit lived reality—these are distinctly human capabilities that become more valuable, not less, as AI handles more of the routine information processing.

18.5 Civic Participation in AI Oversight

You are already in the loop. This is not a metaphor.

When you click "not spam" on a misclassified email, you are contributing to the training signal that shapes spam detection for millions of users. When you rate a product, you are contributing to recommendation algorithms. When you accept or reject an autocorrect suggestion, you are providing feedback that updates the model. When you report a phishing email, you are contributing to security AI training. The aggregate of billions of such micro-interactions is the human signal that AI systems rely on.

This happens mostly without your knowledge or consent, and with almost no transparency about how it shapes the AI systems you interact with. This is a governance problem. But it also points to something important: the relationship between humans and AI systems is not confined to explicit review workflows. It is embedded in everyday digital life.

Civic participation in AI governance extends this principle to the institutional level. Many decisions about how AI systems are designed, deployed, and regulated are made in processes that nominally invite public participation—regulatory comment processes, advisory committee hearings, corporate ethics board consultations, academic peer review of influ-

ential AI systems. These processes exist. They are underutilized by people who are not professionally engaged with AI governance.

The argument that AI governance is too technical for public participation is often wrong—not because the technical details are simple, but because the most important questions are not primarily technical. Should a recidivism prediction algorithm be used in sentencing? Should facial recognition be permitted in public spaces? Who should bear the cost when an AI system makes a systematic error? These questions require technical understanding at the margin, but they are fundamentally questions about values, power, and accountability that are not resolved by technical expertise alone.

If you have read this book, you have enough technical literacy to participate meaningfully in many of these conversations. The choice of whether to do so is yours.

18.6 The Closing Invitation

In 2017, Hein Mevissen—a flight attendant—was going through the motions of his pre-flight safety briefing. Seatbelts, oxygen masks, emergency exits. He had given the briefing hundreds of times. So had every other flight attendant on every other commercial flight, performing a ritual that most passengers have learned to tune out.

He decided to try something different. He made eye contact. He told the passengers that he personally needed them to pay attention, that this information might someday save their lives, that he was not asking because it was required but because it mattered. The video of his briefing went viral. Not because the content was different—it was the standard safety information—but because someone was clearly, genuinely present, asking for the same presence in return.

This is what good HITL design aspires to. Not the performance of oversight, but the reality of it. Not a human signature on an AI decision, but a human mind actually engaged with what the AI decided and why.

Adaptive Reserve Is Not Waste

In HITL systems, human review capacity is not merely a cost center to be minimized. It is adaptive reserve: the slack that allows a system to absorb uncertainty, distribution shift, edge cases, and failure recovery without breaking exactly when judgment matters most.

The machines are getting better. The questions they produce will get harder and more consequential, not easier and more routine. The cases routed to human judgment will be the genuine edge cases, the ones that require the full weight of a situated, experienced, motivated human mind.

Your job is not to approve what the machine decided. Your job is to be present.

That is not a small thing. In a world of increasingly capable machines, genuine human presence—attention, judgment, accountability—is one of the most valuable things a person can offer. Not despite what the machines can do. Because of it.

Go back to the Try This from Chapter 1. Pay attention to how many times a piece of technology asks you something. Notice which ones feel genuinely like a request for your judgment, and which feel like theater. For the ones that feel like theater—the cookie banners, the empty confirmation dialogs, the permission requests you click through without reading—ask who benefits from the theater. For

> the ones that feel genuine—the fraud alerts, the medical AI flags, the uncertain autocorrect that waits for you to choose—ask how you can be as present and thoughtful as the system deserves. Then, for one of the theater ones: stop. Actually read it. Actually decide. You may surprise yourself.

Chapter Summary

Key Concepts

- The human's enduring role is not premised on AI limitations—it is premised on accountability, value judgment, lived experience, and moral responsibility
- HITL creates new roles: AI trainers, auditors, oversight coordinators, red teamers, evaluators, and policy specialists
- Most valuable HITL skills: domain expertise + AI literacy, edge-case judgment, uncertainty communication, adversarial thinking, contextual reasoning
- You are already in the loop: micro-interactions with AI systems aggregate into the human signal those systems rely on
- Civic participation in AI governance is accessible to anyone with basic AI literacy—and is underutilized
- Oversight capacity is adaptive reserve, not organizational waste; optimizing it away can make an AI system more fragile precisely when reality changes

Key Examples

Air Canada tribunal Accountability requires a human agent—the company, not the chatbot, bore responsibility for the chatbot's confident error

Annotation specialists A permanent workforce function, not a temporary bridge to full automation

Public comment in AI regulatory processes A form of HITL at the governance level

The flight attendant who asked for genuine presence The model for what human-in-the-loop oversight should feel like

Five Dimensions Check

- All five dimensions completed: this chapter reflects the book's full argument—that well-designed human-AI collaboration, across all five dimensions, makes both the humans and the AI systems more effective.

Afterword

This book began with a Netflix prompt: "Are you still watching?"

The question seems trivial. But it encodes something profound: a system that was powerful enough to just keep playing, that chose instead to pause and ask. Not because it lacked confidence. Because it knew the limits of its confidence. And because it respected the human enough to check in.

That is the relationship this book has been about. Not between humans and machines that serve them. Between humans and systems smart enough

to know when they need the human's help, and humble enough to ask for it.

The smartest machines still need us. Not as backup plans or temporary placeholders. As partners whose judgment they are designed to earn, respect, and return to.

Welcome to the loop.

Technical Appendix 18A: Measuring Human Contribution

Chapter 18 asked what role humans should play in an AI-enabled future. This appendix provides the quantitative tools for measuring that role: the Human Contribution Index, complementarity analysis, civic AI literacy frameworks, and accountability architecture requirements.

— Formal foundations for Chapter 18

18.7 Quantifying the Human Value-Add

18.7.1 Human Contribution Index

In a HITL system, the human contribution can be formally measured as the difference between the system's performance with and without human review:

$$\mathrm{HCI} = \mathrm{Acc}(f_{\mathrm{HITL}}) - \mathrm{Acc}(f_{\mathrm{automated}})$$

This aggregate measure conflates two distinct effects: (1) *routing benefit* — cases routed to humans were genuinely harder, and human accuracy on those cases exceeds model accuracy; and (2) *review quality* — the human reviews cases accurately. A decomposed measure:

$$\text{HCI} = P(\text{routed}) \cdot \left(\text{Acc}_{\text{human, routed}} - \text{Acc}_{\text{model, routed}}\right)$$

```python
import numpy as np
from sklearn.metrics import accuracy_score

def compute_hci(y_true, model_preds, human_decisions, routed_mask):
    """
    y_true:          ground truth labels
    model_preds:     model predictions for all cases
    human_decisions: human labels for routed cases only
    routed_mask:     boolean array, True where case was routed
    """
    hitl_decisions             = model_preds.copy()
    hitl_decisions[routed_mask] = human_decisions

    hitl_acc   = accuracy_score(y_true, hitl_decisions)
    auto_acc   = accuracy_score(y_true, model_preds)

    human_acc_on_routed = (
        accuracy_score(y_true[routed_mask], human_decisions)
        if routed_mask.sum() > 0 else np.nan
    )
    model_acc_on_routed = (
        accuracy_score(y_true[routed_mask], model_preds[routed_mask])
        if routed_mask.sum() > 0 else np.nan
    )

    routing_rate    = routed_mask.mean()
    routing_benefit = routing_rate * (human_acc_on_routed
                                      - model_acc_on_routed)
```

```
    return {
        'hitl_accuracy':              hitl_acc,
        'auto_only_accuracy':         auto_acc,
        'human_accuracy_on_routed':   human_acc_on_routed,
        'model_accuracy_on_routed':   model_acc_on_routed,
        'routing_rate':               routing_rate,
        'hci':                        routing_benefit,
        'human_value_add':            hitl_acc - auto_acc,
    }
```

Listing 18.1: Computing the Human Contribution Index

18.8 Human–AI Complementarity Analysis

The value of a HITL system depends on *complementarity* between human and AI capabilities:

$$C = P(\text{human correct} \mid \text{model wrong}) - P(\text{human correct} \mid \text{model right})$$

$C > 0$ means humans are more likely to be correct when the model is wrong — the system benefits from human involvement. $C \leq 0$ means human and model errors are correlated — adding humans does not help, and may reflect the same cognitive biases as the training data.

```
def complementarity_analysis(y_true, model_preds,
    human_preds):
    model_correct = (model_preds == y_true)
    human_correct = (human_preds == y_true)

    p_human_given_model_wrong = (
        human_correct[~model_correct].mean()
        if (~model_correct).sum() > 0 else np.nan
    )
```

```
    p_human_given_model_right = (
        human_correct[model_correct].mean()
        if model_correct.sum() > 0 else np.nan
    )

    return {
        'complementarity':
            p_human_given_model_wrong -
                p_human_given_model_right,
        'p_human_correct_given_model_wrong':
            p_human_given_model_wrong,
        'p_human_correct_given_model_right':
            p_human_given_model_right,
        'agreement_matrix': {
            'both_right':       (model_correct &
                human_correct).mean(),
            'both_wrong':       (~model_correct &
                ~human_correct).mean(),
            'human_only_right': (~model_correct &
                human_correct).mean(),
            'model_only_right': (model_correct &
                ~human_correct).mean(),
        }
    }
```

Listing 18.2: Complementarity analysis

If $C \approx 0$, humans and models fail on the same cases. This often occurs when the model was trained on human-generated data and has learned the same systematic biases. Adding human review adds cost without adding accuracy. Diagnosis: compute complementar-

ity by input subgroup to identify where human judgment does and does not add value.

18.9 Civic AI Literacy Framework

Meaningful participation in AI governance does not require deep technical expertise. The following framework maps competency levels to participation forms:

Table 18.1: Civic AI literacy levels and participation forms

Level	Key concepts	Participation forms
Level 1 (Basic)	AI as statistical pattern matching; uncertainty is normal; models can be wrong	Noticing and reporting AI error informed consumer of AI pro ucts
Level 2 (Functional)	Training data determines behaviour; calibration vs. accuracy; fairness tradeoffs	Public comment on AI regul tions; understanding AI explan tions
Level 3 (Critical)	Feedback loops; impossibility theorems; accountability gaps	Auditing AI systems; advocac for fairness standards; advisin policymakers
Level 4 (Technical)	All of the above plus implementation	Red-teaming; designing audit building oversight tools

Reading this book places you solidly at Level 2, with exposure to Level 3 concepts. Level 2 is sufficient for meaningful participation in most public governance processes.

18.10 Accountability Architecture

A complete accountability architecture for a HITL system has four layers:

Key Concept

Decision Audit Trail (Layer 1) Every automated and human-reviewed decision is logged with: timestamp, input features (or hash), model confidence, routing decision, human reviewer ID, human decision, and outcome when observable. Without this layer, no post-hoc analysis is possible.

Key Concept

Aggregate Monitoring (Layer 2) Regular reporting of accuracy by demographic group, false positive and false negative rates, override rates, and reviewer agreement rates. This layer makes disparate impact visible before it becomes a compliance problem.

Key Concept

Independent Audit (Layer 3) Periodic (quarterly or annual) review by parties independent of the operating team, with access to audit logs and aggregate statistics. Findings reported publicly for high-stakes systems. Independence is the critical property: internal teams cannot audit their own blind spots.

Key Concept

Contestability Pathway (Layer 4) A documented, functional process by which individuals affected by a system's decision can: (a) receive a meaningful explanation; (b) challenge the decision with a human reviewer who has actual authority to override; (c) receive a response

within defined time limits. A contestability page that leads to a contact form is not a contestability pathway.

18.11 Exercises

Exercise 18.1 — Human Contribution Audit

Using a classification dataset of your choice, simulate a HITL system:

1. Train a model; route cases with confidence < 0.7 to a simulated human.
2. Simulate human performance: correct 80% of the time on routed cases.
3. Compute HCI and complementarity scores.
4. At what confidence threshold is HCI maximised? Why?

Exercise 18.2 — Accountability Architecture Audit

Choose an AI system you interact with regularly (spam filter, content recommendation, fraud detection):

1. Which layers of the accountability architecture does it have?
2. Is there a public description of how decisions are made?
3. Can you find a contestability pathway? How accessible is it?
4. Write a one-page accountability assessment.

Exercise 18.3 — Civic Participation Simulation

Find a recent public comment period on AI regulation (FTC AI guidelines, NIST AI RMF updates, EU AI Act implementation):

1. Read the draft document's key provisions.
2. Write a public comment (250–500 words) using concepts from this book.
3. Identify: what fairness definition does the proposal implicitly adopt? What monitoring requirements does it include? What contestability provisions?

This appendix provides the quantitative and structural tools for measuring, communicating, and demanding human contribution in AI systems, as introduced in Chapter 18.

Bibliography

Angelopoulos, A. N., & Bates, S. (2021). A gentle introduction to conformal prediction and distribution-free uncertainty quantification.

Angwin, J., Larson, J., Mattu, S., & Kirchner, L. (2016). Machine bias: There's software used across the country to predict future criminals. and it's biased against blacks. https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing

Ardila, D., Kiraly, A. P., Bharadwaj, S., Choi, B., Reicher, J. J., Peng, L., Tse, D., Etemadi, M., Ye, W., Corrado, G., Naidich, D. P., & Shetty, S. (2019). End-to-end lung cancer detection on CT scans using deep learning. *Nature Medicine*, *25*, 954–961. https://doi.org/10.1038/s41591-019-0447-x

Ash, J. T., Zhang, C., Krishnamurthy, A., Langford, J., & Schekman, A. (2020). BADGE: Batch active learning by diverse gradient embeddings. *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=ryghZJBKPS

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., et al. (2022). Constitutional ai: Harmlessness from ai feedback.

Bainbridge, L. (1983). Ironies of automation. *Automatica*, *19*(6), 775–779.

Begoli, E., Bhattacharya, T., & Kuber, D. (2019). The need for uncertainty quantification in machine-assisted medical decision making. *Nature Machine Intelligence*, *1*(1), 20–23. https://doi.org/10.1038/s42256-018-0004-1

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 610–623.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Buolamwini, J., & Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of Machine Learning Research (FAccT)*, *81*, 1–15.

Bureau d'Enquêtes et d'Analyses pour la Sécurité de l'Aviation Civile. (2012, July). *Final report on the accident on 1st june 2009 to the airbus A330-203 operated by air france flight AF 447* (tech. rep.). BEA France.

Callison-Burch, C. (2010). Creating speech and language data with Amazon's mechanical Turk. *Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk*, 1–12.

CBC News. (2016). GPS leads drivers into Georgian Bay. https://www.cbc.ca/news/canada/sudbury/gps-georgian-bay-1.3780755

Chouldechova, A. (2017). Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big Data*, *5*(2), 153–163. https://doi.org/10.1089/big.2016.0047

Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems (NeurIPS)*, *30*, 4299–4307. https://doi.org/10.48550/arXiv.1706.03741

Chung, Y., Kraska, T., Polyzotis, N., Tae, K. H., & Whang, S. E. (2019). SliceFinder: Automated data slicing for model validation. *IEEE 35th International Conference on Data Engineering (ICDE)*, 1550–1561. https://doi.org/10.1109/ICDE.2019.00139

Civil Resolution Tribunal of British Columbia. (2024, February). Moffatt v. air canada. https://decisions.civilresolutionbc.ca/crt/crtd/en/item/521853/index.do

Cohen, J. (1968). Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, *70*(4), 213–220. https://doi.org/10.1037/h0026256

Cvach, M. (2012). Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*, *46*(4), 268–277. https://doi.org/10.2345/0899-8205-46.4.268

Danziger, S., Levav, J., & Avnaim-Pesso, L. (2011). Extraneous factors in judicial decisions. *Proceedings of the National Academy of Sciences*, *108*(17), 6889–6892. https://doi.org/10.1073/pnas.1018033108

Dastin, J. (2018). Amazon scraps secret AI recruiting tool that showed bias against women. https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G

Davani, A. M., Díaz, M., & Prabhakaran, V. (2022). Dealing with disagreements: Looking beyond the majority vote in subjective annotations. *Transactions of the Association for Computational Linguistics*, *10*, 92–110. https://doi.org/10.1162/tacl_a_00456

Dawid, A. P., & Skene, A. M. (1979). Maximum likelihood estimation of observer error-rates using the em algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, *28*(1), 20–28.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 248–255. https://doi.org/10.1109/CVPR.2009.5206848

Deng, J., Krause, J., & Fei-Fei, L. (2013). Fine-grained crowdsourcing for fine-grained recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 580–587. https://doi.org/10.1109/CVPR.2013.81

Dietvorst, B. J., & Logg, J. M. (2015). Algorithm aversion: People erroneously avoid algorithms after seeing them err. *Journal of Experimental Psychology: General*, *144*(1), 114–126. https://doi.org/10.1037/xge0000033

Drew, T., Võ, M. L.-H., & Wolfe, J. M. (2013). The invisible gorilla strikes again: Sustained inattentional blindness in expert observers. *Psychological Science*, *24*(9), 1848–1853. https://doi.org/10.1177/0956797613479386

EEMUA. (2013). EEMUA publication 191: Alarm systems — a guide to design, management and procurement.

Eykholt, K., Evtimov, I., Fernandes, E., Li, B., Rahmati, A., Xiao, C., Prakash, A., Kohno, T., & Song, D. (2018). Robust physical-world attacks on deep learning visual classification. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1625–1634.

Eyuboglu, S., Varma, M., Saab, K., Shi, J.-B., Sung, W., Yartseva, L., Fries, J., Ré, C., & Ng, A. (2022). Domino: Discovering systematic errors with cross-modal embeddings. *International Conference on Learning Representations (ICLR)*.

Farid, H. (2021). An overview of perceptual hashing. *Journal of Online Trust and Safety*, *1*(1). https://doi.org/10.54501/jots.v1i1.24

Farina, D., Jiang, N., Rehbaum, H., Holobar, A., Graimann, B., Dietl, H., & Aszmann, O. C. (2014). The extraction of neural information from the surface EMG for the control of upper-limb prostheses. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, *22*(4), 797–809. https://doi.org/10.1109/TNSRE.2014.2305111

Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, *76*(5), 378–382. https://doi.org/10.1037/h0031619

Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. *Proceedings of the 33rd International Conference on Machine Learning*, 1050–1059. https://arxiv.org/abs/1506.02142

Gaube, S., Suresh, H., Raue, M., Merritt, A., Berkowitz, S. J., Lermer, E., Coughlin, J. F., Guttag, J. V., Colak, E., & Ghassemi, M. (2021). Do as AI say: Susceptibility in deployment of clinical decision-aids. *npj Digital Medicine*, *4*(31). https://doi.org/10.1038/s41746-021-00385-9

Goddard, K., Roudsari, A., & Wyatt, J. C. (2012). Automation bias: A systematic review of frequency, effect mediators, and mitigators. *Journal of the American Medical Informatics Association*, *19*(1), 121–127. https://doi.org/10.1136/amiajnl-2011-000089

Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1412.6572

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning*, 1321–1330. https://arxiv.org/abs/1706.04599

Hardt, M., Price, E., & Srebro, N. (2016). Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems (NeurIPS)*, *29*.

Hauser, K. (2013). Recognition, prediction, and planning for assisted teleoperation of freeform tasks. *Autonomous Robots*, *35*, 241–254. https://doi.org/10.1007/s10514-013-9350-3

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. https://doi.org/10.1109/CVPR.2016.90

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. *International Conference on Learning Representations (ICLR)*.

Houlsby, N., Huszár, F., Ghahramani, Z., & Lengyel, M. (2011). Bayesian active learning for classification and preference learning. *arXiv preprint arXiv:1112.5745*. https://arxiv.org/abs/1112.5745

Jain, S., Kolter, J. Z., & Mittal, G. (2022). Adversarial robustness of neural hash functions for CSAM detection. *IEEE Symposium on Security and Privacy Workshops*.

Javdani, S., Srinivasa, S. S., & Bagnell, J. A. (2015). Shared autonomy via hindsight optimization. *Robotics: Science and Systems (RSS)*. https://doi.org/10.15607/RSS.2015.XI.032

Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, *596*, 583–589. https://doi.org/10.1038/s41586-021-03819-2

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., et al. (2022). Language models (mostly) know what they know.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). Scaling laws for neural language models.

Kendall, A., & Gal, Y. (2017). What uncertainties do we need in Bayesian deep learning for computer vision? *Advances in Neural Information Processing Systems*, *30*, 5574–5584. https://arxiv.org/abs/1703.04977

Kirilenko, A., Kyle, A. S., Samadi, M., & Tuzun, T. (2017). The flash crash: High-frequency trading in an electronic market. *The Journal of Finance*, *72*(3), 967–998. https://doi.org/10.1111/jofi.12498

Koenecke, A., Nam, A., Lake, E., Nudell, J., Quartey, M., Mengesha, Z., Toups, C., Rickford, J. R., Jurafsky, D., & Goel, S. (2020). Racial disparities in

automated speech recognition. *Proceedings of the National Academy of Sciences*, *117*(14), 7684–7689. https://doi.org/10.1073/pnas.1915768117

Koh, P. W., & Liang, P. (2017). Understanding black-box predictions via influence functions. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 1885–1894.

Kompa, B., Snoek, J., & Beam, A. L. (2021). Second opinion needed: Communicating uncertainty in medical machine learning. *npj Digital Medicine*, *4*(4). https://doi.org/10.1038/s41746-020-00367-3

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, *25*, 1097–1105.

Kusner, M. J., Loftus, J., Russell, C., & Silva, R. (2017). Counterfactual fairness. *Advances in Neural Information Processing Systems (NeurIPS)*, *30*.

Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing Systems*, *30*, 6402–6413. https://arxiv.org/abs/1612.01474

Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, *46*(1), 50–80. https://doi.org/10.1518/hfes.46.1.50.30392

Lee, K., Lee, K., Lee, H., & Shin, J. (2018). A simple unified framework for detecting out-of-distribution samples and adversarial attacks. *Advances in Neural Information Processing Systems (NeurIPS)*, *31*.

Linder, J. A., Doctor, J. N., Friedberg, M. W., Nieva, H. R., Birks, C., Meeker, D., & Fox, C. R. (2014). Time of day and the decision to prescribe antibiotics. *JAMA Internal Medicine*, *174*(12), 2029–2031. https://doi.org/10.1001/jamainternmed.2014.5225

Liu, W., Wang, X., Owens, J., & Li, Y. (2020). Energy-based out-of-distribution detection. *Advances in Neural Information Processing Systems (NeurIPS)*, *33*.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, *30*.

Magesh, V., Surani, F., Dahl, M., Suzgun, M., Manning, C. D., & Ho, D. E. (2024). Hallucination-free? assessing the reliability of leading AI legal research tools.

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *International Conference on Learning Representations (ICLR) Workshop*.

Mosier, K. L., Skitka, L. J., Heers, S., & Burdick, M. (1998). Automation bias: Decision making and performance in high-tech cockpits. *International Journal of Aviation Psychology*, *8*(1), 47–63. https://doi.org/10.1207/s15327108ijap0801_3

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, *35*, 27730–27744.

Parasuraman, R., & Manzey, D. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors*, *52*(3), 381–410. https://doi.org/10.1177/0018720810376055

Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, *39*(2), 230–253. https://doi.org/10.1518/001872097778543886

Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems (NeurIPS)*, *36*.

Rajpurkar, P., Irvin, J., Ball, K., Zhu, H., Yang, B., Mehta, H., Duan, T., Ding, D., Bagul, A., Langlotz, C., et al. (2017). CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?": explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144. https://doi.org/10.1145/2939672.2939778

Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 4902–4912. https://doi.org/10.18653/v1/2020.acl-main.442

Roberts, S. T. (2019). *Behind the screen: Content moderation in the shadows of social media.* Yale University Press.

Sap, M., Card, D., Gabriel, S., Choi, Y., & Smith, N. A. (2019). The risk of racial bias in hate speech detection. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1668–1678. https://doi.org/10.18653/v1/P19-1163

Searle, J. R. (1980). Minds, brains, and programs. *Behavioral and Brain Sciences*, *3*(3), 417–424. https://doi.org/10.1017/S0140525X00005756

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2020). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *International Journal of Computer Vision*, *128*, 336–359. https://doi.org/10.1007/s11263-019-01228-7

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1715–1725. https://doi.org/10.18653/v1/P16-1162

Settles, B. (2009). *Active learning literature survey* (tech. rep. No. 1648). University of Wisconsin-Madison, Department of Computer Sciences. https://burrsettles.com/pub/settles.activelearning.pdf

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., & de Freitas, N. (2016). Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, *104*(1), 148–175. https://doi.org/10.1109/JPROC.2015.2494218

Skitka, L. J., Mosier, K. L., & Burdick, M. (1999). Does automation bias decision-making? *International Journal of Human-Computer Studies*, *51*(5), 991–1006. https://doi.org/10.1006/ijhc.1999.0252

Steiger, M., Bharucha, T. J., Venkatagiri, S., Riedl, M. J., & Lease, M. (2021). The psychological well-being of content moderators. *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–14. https://doi.org/10.1145/3411764.3445092

Strathern, M. (1997). 'improving ratings': Audit in the British university system. *European Review*, *5*(3), 305–321. https://doi.org/10.1017/S1062798700002660

Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science*, *185*(4157), 1124–1131. https://doi.org/10.1126/science.185.4157.1124

Tversky, A., & Kahneman, D. (1981). The framing of decisions and the psychology of choice. *Science*, *211*(4481), 453–458. https://doi.org/10.1126/science.7455683

U.S. Food and Drug Administration. (2021, January). Artificial intelligence/machine learning (AI/ML)-based software as a medical device (SaMD) action plan.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*, 5998–6008. https://doi.org/10.48550/arXiv.1706.03762

Warm, J. S., Parasuraman, R., & Matthews, G. (2008). Vigilance requires hard mental work and is stressful. *Human Factors*, *50*(3), 433–441. https://doi.org/10.1518/001872008X312152

Yang, K., Stoyanovich, J., Asudeh, A., Howe, B., Jagadish, H. V., & Miklau, G. (2018). A nutritional label for rankings. *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 1773–1776. https://doi.org/10.1145/3183713.3193568

Zampieri, M., Malmasi, S., Nakov, P., Rosenthal, S., Farra, N., & Kumar, R. (2019). Predicting the type and target of offensive posts in social media. *Proceedings of NAACL-HLT 2019*, 1415–1420. https://doi.org/10.18653/v1/N19-1144

Glossary

Active Learning
A machine learning approach where the algorithm can interactively query a human (or other information source) to label new data points, focusing on examples where it is most uncertain.

Aleatoric Uncertainty
Uncertainty arising from inherent randomness or noise in the data. Cannot be reduced by collecting more training data. Also called *data uncertainty* or *statistical uncertainty*.

Alert Fatigue
A phenomenon where users become desensitized to frequent alerts or notifications, leading them to ignore or dismiss important warnings. A key failure mode when systems ask for help too often.

Bayesian Neural Network (BNN)
A neural network that represents uncertainty over its weights using probability distributions rather than point estimates, enabling principled uncertainty quantification.

Cohen's Kappa (κ)
A statistic measuring inter-rater agreement for categorical labels, corrected for chance agreement. Values of $\kappa < 0$ indicate less agreement than chance; $\kappa = 1$ is perfect agreement; $\kappa \geq 0.61$ is generally considered substantial agreement.

Calibration
The degree to which a model's predicted probabilities match actual observed frequencies. A well-calibrated model that predicts 70% confidence should be correct approximately 70% of the time.

Confidence Threshold
A predetermined probability level above which a system acts autonomously and below which it requests human intervention.

Data Annotation
The process of labeling raw data (text, images, audio) with ground-truth labels for use in supervised machine learning. Annotation quality directly determines the quality of any model trained on the data.

Dawid-Skene Model
A statistical model that jointly estimates true latent labels and individual annotator error rates from multiple noisy annotations using an EM algorithm. Accounts for annotators who are systematically biased toward particular labels.

Deep Ensemble
A collection of neural networks trained independently with different random initializations. Disagreement among ensemble members indicates epistemic uncertainty.

Epistemic Uncertainty
Uncertainty arising from lack of knowledge, which could theoretically be reduced with more training data or a better model. Also called *model uncertainty*.

Expected Calibration Error (ECE)
A metric for measuring how well-calibrated a model's probability

predictions are, computed as the weighted average of the difference between accuracy and confidence across bins.

Framing Effect

A cognitive bias where the presentation of information (framing) influences judgment, independent of the underlying facts. In HITL systems, showing annotators a model's prediction before annotation can anchor their judgment and reduce inter-annotator independence.

Feedback Integration

The fifth dimension of the HITL framework: how a system incorporates human responses to improve future performance.

Five-Dimension Framework

An analytical framework for evaluating HITL systems, comprising: (1) Uncertainty Detection, (2) Intervention Design, (3) Timing, (4) Stakes Calibration, and (5) Feedback Integration.

Graceful Degradation

A design principle where systems fail in ways that minimize harm, such as asking for help rather than acting on uncertain predictions.

Hallucination (AI)

When an AI system generates plausible-sounding but factually incorrect information with apparent confidence. A key failure mode in large language models.

Human-in-the-Loop (HITL)

A design paradigm where humans are incorporated into an automated system's decision-making process, particularly for handling uncertain or high-stakes situations.

Inter-Annotator Agreement (IAA)
A measure of how consistently multiple human annotators label the same data. High IAA indicates the annotation task is well-defined; low IAA may signal ambiguous guidelines, genuinely contested cases, or annotator quality issues. Commonly measured with Cohen's κ or Fleiss' κ.

Intervention Design
The second dimension of the HITL framework: how a system presents its request for human input, including the clarity of the question and available response options.

Label Noise
Errors or inconsistencies in training labels, typically introduced by annotator mistakes, ambiguous guidelines, or inherently subjective tasks. Label noise can systematically degrade model performance, especially when it is correlated with input features.

Monte Carlo Dropout
A technique for estimating uncertainty by running multiple forward passes with dropout enabled during inference and measuring the variance in predictions.

Overconfidence
A failure mode where systems act autonomously despite high uncertainty, often leading to errors that could have been prevented by human intervention.

Reliability Diagram
A visualization tool for assessing model calibration by plotting predicted confidence against observed accuracy across confidence bins.

Stakes Calibration

The fourth dimension of the HITL framework: how well a system adjusts its behavior based on the potential consequences of being wrong.

Timing

The third dimension of the HITL framework: when a system chooses to request human input, including synchronous (immediate) vs. asynchronous (batched) interventions.

Uncertainty Detection

The first dimension of the HITL framework: a system's ability to recognize when it lacks confidence in its predictions or decisions.

Uncertainty Quantification

The process of estimating and communicating how confident a model is in its predictions, enabling appropriate decisions about when to act autonomously vs. request human input.

Index

A

active learning 414
adversarial examples 262
alarm flood 399
aleatoric uncertainty 49, 461
alert fatigue 15, 114, 461
algorithmic bias 338
attention 235
attentional allocation 314
Automation bias 151, 320
automation bias . . . 142, 264, 313
Automation complacency . . . 320
automation complacency . . . 312

B

Bayesian neural network . . . 461
blameless post-mortem 375

C

CAI 242
calibrated 256
calibrated accuracy 54
calibrated trust 314
calibrating 19
Calibration 287
calibration 56, 462
calibration session 170
co-adaptation 404
Cohen's kappa 166, 173, 461
confidence signals 187
confidence threshold 462
confused 53
consistency trap 316
Constitutional AI 242
convolutional neural network 260, 261

D

data annotation 462
Dawid-Skene model 462
decision boundary 94
deep ensemble 462
Diarization Error Rate 407
Direct Preference Optimization 242
DPO 242

E
empirical risk minimization . . 83
epistemic uncertainty . . . 49, 462
expectation confirmation . . . 315
Expected Calibration Error . 382
expected calibration error . . 463
Explainable AI 145

F
F1 score 286
fairness impossibility theorem 342
Feedback Integration 121
feedback integration . . . 188, 463
filters 275
five whys 375
five-dimension framework . . 463
framing effect 463

G
Gender Shades 338
generalization gap 73, 84
genuine ambiguity 168
Goodhart's Law 286
graceful degradation 463
Grad-CAM 145

H
hallucination 237, 463
hallucinations 14
HITL Loop 195
HITL Pipeline 195
human-in-the-loop 9, 463

I
influence functions 386
information gain 125
instruction problem 165
inter-annotator agreement . . 464
Intersection over Union 277
intervention design 464

L
label noise 464
labels are not neutral 164
large language models 236
LIME 145

M
mean Average Precision 277
Monte Carlo dropout 464

N
noise disagreement 168

O
optimal query 126
overconfidence 464
overfitting 74
overtrust 314

P
perceptual hashing 266
perfectly calibrated 61

pipeline 186
precision 286
presentation effect 141
proactive 418

R

recall 286
Reinforcement Learning from Human Feedback . . 189, 241
reliability diagram 464
residual connections 276
reward hacking 241
reward model 241
RLHF 241

S

scaled dot-product attention 250
SHAP 145
shared autonomy 402
Slice finder 383
slice-based evaluation 383
spatial hierarchy 276
spot failure 370
stakes calibration 465
statistical auditing 344
systematic failure 371

T

the divergence trap 289
threshold 92
timing 465
tokens 249
transformer 235
trust–attention–response triangle 114
Two-factor authentication . . 116

U

uncertainty detection 465
uncertainty quantification 13, 465
underfitting 75
undertrust 314

V

vectors 234
version space 426

W

Word Error Rate 407
Word2Vec 234
wrong 53

About the Author

Lazaros Toumanidis is the author of *Human in the Loop: Misunderstood.*

This project brings together the Winter textbook, the Summer reading edition, the Autumn pamphlet, the Spring build edition, and the companion materials collected in this repository.

If you want to follow updates, corrections, or future iterations of the project, use the contact details below.

Connect with the author:

- Websites: laztoum.com | what-if.io | limen-os.io
- Email: laztoum@protonmail.com